Advanced Angular Applications



André de Sousa

Advanced Angular Applications

Code, build, test, package, deploy, monitor and scale Angular applications in enterprise environments.

André de Sousa

Brief Contents

1 Web Development with Angular

2 Angular Framework, Style Guide and Best Practices

3 Architecting Angular Applications

4 Testing Angular Applications

5 Automating Software Delivery

6 Web Infrastructure, Operations and Security

7 Web Application Performance

Table of Contents

Preface *17*

1 Web Development with Angular *19*

1.1 Overview **19**

1.2 Web Development **20**

1.3 Web Technologies **22**

1.4 HTML **29**

1.5 TypeScript **29**

1.5.1 Fundamentals 29

1.5.2 Project Configuration 31

1.5.3 Object-Oriented Programming 32

1.5.4 Functional Programming 33

1.5.5 Style Guide 33

1.5.6 Best Practices 34

1.6 CSS **40**

1.6.1 CSS Evolution 40

1.6.2 Sass or SCSS 41

1.6.3 Best Practices 41

1.7 Browser APIs **47**

1.8 Web Components **50**

1.8.1 Definition 50

1.8.2 Angular Elements 51

1.8.3 Lit 51

1.9 Dependency Management **52**

1.9.1 Management Tasks 52

1.9.2 Package Managers 53

1.9.3 Dependency Security 54

1.9.4 Dependency Governance 55

1.10 Version Control **55**

1.10.1 Version Control Systems 55

1.10.2 Git 56

1.10.3 Conventional commits 59

1.10.4 Changelog 60

1.11 Code Quality 60
1.11.1 Linters 60
1.11.2 ESLint + Prettier 61
1.11.3 IDE Integration 62
1.11.4 Git Hooks and CI 62
1.11.5 SonarQube 63

1.12 User Experience 64
1.12.1 Fundamentals 64
1.12.2 Design Systems 65
1.12.3 Design Tools 65
1.12.4 Best Practices 66

1.13 Artificial Intelligence 68

1.14 Final Remarks 69

2 Angular Framework, Style Guide and Best Practices 71

2.1 Overview 71

2.2 Angular Framework 72
2.2.1 Ecosystem 72
2.2.2 Features & Benefits 73
2.2.3 Architecture 74
2.2.4 OOP vs FP style 75
2.2.5 Dependency Injection 75

2.3 Angular Core Concepts 76
2.3.1 Components 76
2.3.2 Services 77
2.3.3 Directives 78
2.3.4 Pipes 79
2.3.5 Forms 79
2.3.6 Guards & Interceptors 80
2.3.7 Routing 81
2.3.8 Modules 82

2.4 RxJS 82
2.4.1 Why RxJS? 82
2.4.2 Fundamentals 83
2.4.3 Promises vs Observables 83
2.4.4 Best Practices 84

2.5 Signals 85

2.5.1 Why Signals? 85
2.5.2 Fundamentals 86
2.5.3 Signals vs Observables 87
2.5.4 Best Practices 87

2.6 Angular Advanced Concepts 88
2.6.1 CDK 88
2.6.2 Animations 89
2.6.3 User Access Control 90
2.6.4 Error Handling 91
2.6.5 Internationalization 91
2.6.6 Accessibility 92
2.6.7 Analytics 92
2.6.8 SEO 93
2.6.9 PWA 94
2.6.10 WebMCP 94

2.7 Development Workspace 95
2.7.1 Project Structure 95
2.7.2 Project Documentation 96
2.7.3 Build Environments 97
2.7.4 Build Configuration 97
2.7.5 Proxy Configuration 98
2.7.6 Template Type Checking 99
2.7.7 Source Maps 99
2.7.8 Schematics 100
2.7.9 Debugging 100

2.8 Style Guide 101
2.8.1 Naming 101
2.8.2 Coding Conventions 102
2.8.3 Application Structure 102
2.8.4 Components 103
2.8.5 Directives 104
2.8.6 Services 104

2.9 Best Practices 105

2.10 Final Remarks 110

3 Architecting Angular Applications 111

3.1 Overview 111

3.2 Fundamentals 112
3.2.1 Design Principles 112
3.2.2 Design Patterns 115
3.2.3 Architectural Patterns 115

3.3 Angular-Based Architectures 119
3.3.1 Clean Architecture 119
3.3.2 Hexagonal Architecture 120
3.3.3 Layered Frontend Architecture 122
3.3.4 Feature-Based Architecture 123
3.3.5 Modular Architecture 124
3.3.6 Micro-Frontends Architecture 125

3.4 Data Flow Architecture 126
3.4.1 Reactive Architecture 126
3.4.2 Component-Based Architecture 127
3.4.3 Event-Driven Architecture 128
3.4.4 Signal-Based Architecture 129

3.5 State Management Architecture 130
3.5.1 Application State Management 130
3.5.2 Solutions for State Management 132
3.5.3 Best Practices 136

3.6 API Layer Architecture 138
3.6.1 Service Organization 138
3.6.2 Client Code Generation 139
3.6.3 REST API Integration 140
3.6.4 GraphQL Integration 141
3.6.5 REST vs GraphQL 141
3.6.6 WebSocket Integration 142
3.6.7 Server-Sent Events Integration 143
3.6.8 WebSocket vs SSE 143

3.7 CSS Architecture 144
3.7.1 Fundamentals 144
3.7.2 CSS Methodologies 144
3.7.3 CSS Styling Patterns 145
3.7.4 CSS Project Structure 146
3.7.5 Responsive Strategies 147
3.7.6 Perceived Responsiveness 149
3.7.7 Resilient CSS 150

3.8 Workspace Architecture 152
3.8.1 Standalone 152
3.8.2 Monorepo 153
3.8.3 Multi-Repo 154

3.9 Angular Rendering Strategies 154
3.9.1 Client-Side Rendering 154
3.9.2 Server-Side Rendering 155
3.9.3 Static Site Generation 156
3.9.4 Deferrable Views 156
3.9.5 Incremental Hydration 157

3.10 Software Architecture Documentation 157
3.10.1 C4 Model 157
3.10.2 arc42 158
3.10.3 Architecture Decision Records 159
3.10.4 Documentation as Code 160

3.11 Final Remarks 161

4 Testing Angular Applications 163

4.1 Overview 163

4.2 Software Development Process 164
4.2.1 Software Development Life Cycle 164
4.2.2 Behavior-Driven Development 164
4.2.3 Domain-Driven Design 165
4.2.4 Spec-Driven Development 166
4.2.5 Test-Driven Development 166
4.2.6 Acceptance Test-Driven Development 167

4.3 Testing Types 167

4.4 Testing Pyramid 168

4.5 Testing Tools 169

4.6 Unit Tests 171
4.6.1 Fundamentals 171
4.6.2 Isolated Tests 172
4.6.3 Shallow Tests 173
4.6.4 Snapshot Tests 175
4.6.5 Marble Tests 176
4.6.6 Asynchronous Tests 178

4.6.7 Code Coverage 179
4.6.8 Test Reporters 180
4.6.9 Debugging 181
4.6.10 Best Practices 181

4.7 Integration Tests 187

4.8 Functional Tests 188
4.8.1 Fundamentals 188
4.8.2 Browser Interaction 189
4.8.3 Auto-Waiting 190
4.8.4 Mocking APIs 191
4.8.5 Mocking Clock 191
4.8.6 Page Objects 192
4.8.7 Helper Functions 193
4.8.8 Flaky Tests 193
4.8.9 Test Reporters 194
4.8.10 Debugging 194
4.8.11 Best Practices 195

4.9 Visual Tests 197
4.9.1 Fundamentals 197
4.9.2 Screenshot Tests 198
4.9.3 Best Practices 199

4.10 Other Tests 200
4.10.1 Accessibility Testing 200
4.10.2 Cross Browser Testing 201
4.10.3 Smoke Testing 201
4.10.4 Monkey Testing 201
4.10.5 Performance Testing 201
4.10.6 Security Testing 201

4.11 Gherkin 202
4.11.1 Fundamentals 202
4.11.2 Jira + Xray 203
4.11.3 Cucumber + Playwright 204
4.11.4 AI + BDD 205
4.11.5 Best Practices 206

4.12 Testing Suites 208

4.13 Final Remarks 209

5 Automating Software Delivery 211

5.1 Overview 211

5.2 Continuous Testing 212

5.2.1 Definition 212
5.2.2 Test Automation 213
5.2.3 Cross Browser Testing 213
5.2.4 Mobile Testing 215
5.2.5 Selenium Grid 216
5.2.6 Testcontainers 219
5.2.7 Best Practices 220

5.3 Continuous Integration 223

5.3.1 Definition 223
5.3.2 GitHub Workflows 224
5.3.3 Jenkins 229
5.3.4 Notifications 233
5.3.5 Best Practices 233

5.4 Continuous Delivery 236

5.4.1 Definition 236
5.4.2 GitHub Workflows 237
5.4.3 GoCD 238
5.4.4 Delivery Artifacts 239
5.4.5 Artifact Repository 240
5.4.6 Feature Flags 242
5.4.7 Best Practices 242

5.5 CI vs CD vs CD 245

5.5.1 Overview 245
5.5.2 Continuous Workflow 246
5.5.3 Application Deployment 247
5.5.4 Deployment Strategies 248

5.6 Continuous Automation 249

5.6.1 Definition 249
5.6.2 GitOps 250
5.6.3 GitHub Workflows 250
5.6.4 Argo CD 251
5.6.5 Best Practices 251

5.7 Final Remarks 255

6 Web Infrastructure, Operations and Security 257

6.1 Overview 257

6.2 Cloud Providers 258

6.3 Cloud Hosting 258

6.3.1 Static Files Hosting 258
6.3.2 Serverless Containers 259
6.3.3 Kubernetes Cluster 260

6.4 Infrastructure Components 261

6.4.1 Hosting 261
6.4.2 Domain Name 262
6.4.3 SPA 262
6.4.4 Reverse Proxy 263
6.4.5 Load Balancer 263
6.4.6 CDN 264

6.5 Infrastructure Provisioning 264

6.5.1 Infrastructure as Code 264
6.5.2 Ansible 265
6.5.3 Terraform 266
6.5.4 Helm 269
6.5.5 Twelve-Factor App 272
6.5.6 Well-Architected Frameworks 273

6.6 Infrastructure Costs 274

6.6.1 Cloud Cost Estimates 274
6.6.2 FinOps 275
6.6.3 Best Practices 275

6.7 Dockerizing Angular Applications 277

6.7.1 Nginx 277
6.7.2 Docker Image 278
6.7.3 Best Practices 279

6.8 Observability and Monitoring 280

6.8.1 Fundamentals 280
6.8.2 Logging 281
6.8.3 Metrics 283
6.8.4 Tracing 285
6.8.5 Health Checks 288
6.8.6 Alerts 289

6.8.7 Dashboards 292
6.8.8 Best Practices 293

6.9 Web Security 296
6.9.1 Fundamentals 296
6.9.2 OWASP Top 10 297
6.9.3 Best Practices 298

6.10 Angular-Specific Security Strategies 302
6.10.1 Preventing Cross-Site Scripting 302
6.10.2 Preventing HTTP-level Vulnerabilities 305
6.10.3 Preventing Server-Side Request Forgery 308
6.10.4 Application Security Governance 309

6.11 Final Remarks 310

7 Web Application Performance 311

7.1 Overview 311

7.2 Fundamentals 312
7.2.1 Core Web Vitals 312
7.2.2 Performance Metrics 312
7.2.3 Performance Insights 315
7.2.4 Performance Patterns 315
7.2.5 Event Loop 318

7.3 Angular-Specific Performance Strategies 319
7.3.1 Zone.js vs Zoneless 319
7.3.2 Component Lifecycle 320
7.3.3 Change Detection Strategies 321
7.3.4 Web Workers 322
7.3.5 Service Workers 323
7.3.6 Lazy Loading 323
7.3.7 Bundle Optimization 324
7.3.8 Runtime Performance 324
7.3.9 Rendering Strategies 325
7.3.10 Signals with OnPush 325
7.3.11 Caching Strategies 326
7.3.12 API Layer Performance 326

7.4 Web Server Performance Strategies 327
7.4.1 Performance Concerns 327
7.4.2 Fallback Routes 328

7.4.3 Caching Headers 328
7.4.4 CDN and Edge Caching 330
7.4.5 Server Compression 331
7.4.6 HTTP/2, HTTP/3, and TLS Optimization 332
7.4.7 CORS Optimization 333
7.4.8 Server-Side Rendering 334
7.4.9 Server Scaling 335
7.4.10 API Latency Optimization 335

7.5 Performance Testing 336
7.5.1 Frontend vs Backend 336
7.5.2 Testing Approaches 337
7.5.3 Lighthouse 338
7.5.4 WebPageTest 340
7.5.5 k6 341
7.5.6 Best Practices 343

7.6 Performance Monitoring 344
7.6.1 Profiling Angular Applications 344
7.6.2 Chrome DevTools 345
7.6.3 Angular DevTools 346
7.6.4 Lighthouse CI 347
7.6.5 Profiling Node.js Applications 348

7.7 Best Practices 349

7.8 Final Remarks 354

Afterword 355

Index 357

Preface

I wrote this book for developers who want to take their Angular skills beyond the basics and apply them in complex enterprise environments. It assumes previous experience with Angular fundamentals and focuses on the concepts, practices, and decision-making required to build, test, deliver, operate, secure, and optimize large-scale applications with confidence.

Chapter 1 introduces the foundations of modern web development for Angular projects, including UX, TypeScript, web standards, dependency management, version control, code quality, and AI-assisted workflows.

Chapter 2 dives into Angular framework concepts and recommended practices, covering core building blocks, RxJS and Signals, HTTP and error handling, authentication, accessibility, SEO, and workspace organization.

Chapter 3 presents architectural thinking for Angular applications, from design principles and frontend architecture patterns to state management, API integration styles, rendering strategies, and architecture documentation.

Chapter 4 focuses on testing strategy and implementation, including unit, integration, functional, visual, and non-functional testing, along with tooling, BDD approaches, and practical test-suite guidance.

Chapter 5 explains how to automate and deliver Angular applications through CI/CD and GitOps-oriented workflows, including pipeline design, artifacts, deployment strategies, and release best practices.

Chapter 6 covers web infrastructure, operations and security topics such as hosting, Docker, cloud platforms, infrastructure as code, cost management, observability, and Angular security strategies.

Chapter 7 explores performance engineering for web applications, with emphasis on Angular-specific optimization techniques, server-side performance configurations, testing methodologies, and monitoring practices.

1 Web Development with Angular

1.1 Overview

Web development with Angular starts with strong foundations in HTML, CSS, and JavaScript, then evolves into building scalable single-page applications with a modern toolchain and disciplined engineering practices. When selecting a framework, teams typically evaluate functionality, scalability, performance, documentation quality, community strength, and long-term viability. Angular performs well across these dimensions, while still producing the same browser runtime artifacts: HTML, CSS, and JavaScript.

Since the Angular ecosystem is broad, it helps to prioritize fundamentals first. The most important building blocks are TypeScript and RxJS. TypeScript adds static typing, interfaces, decorators, and tooling that improve maintainability in large codebases. RxJS introduces functional reactive programming, providing a stream-based model for asynchronous data and events. Angular's class-oriented structure and RxJS's functional style are complementary and commonly used together.

Strong HTML and CSS knowledge remains essential. For styling at scale, a preprocessor such as Sass improves readability and reuse through variables, nesting, functions, and mixins. Beyond language fundamentals, Angular developers also need practical proficiency in testing, linting, formatting, and source control practices.

This chapter provides a practical roadmap to modern web development with Angular, covering frontend and backend context, core web technologies (HTML, TypeScript, and CSS), Browser APIs, web components, dependency and version control practices, code quality automation, user experience fundamentals, and responsible AI-assisted development.

1.2 Web Development

Frontend web development is the practice of converting data to a graphical interface, using HTML, CSS, and JavaScript, so that users can view and interact with that data. This traditional definition doesn't reflect the real complexity or dimension of the ecosystem of tools that we need to know. Web development today is completely different from what it was years ago. There are lots of different things that can easily baffle anyone entering web development.

To understand the frontend, we also must know the backend. The frontend, also called "client-side" programming, is what happens in the browser. It's everything the user sees and interacts with. The backend, also called "server-side" programming, happens on the server and the database. It's the machinery that works behind the scenes to power those fancy features users interact with on the client-side. Below are a couple of visuals to give us an idea of how frontend development flows.

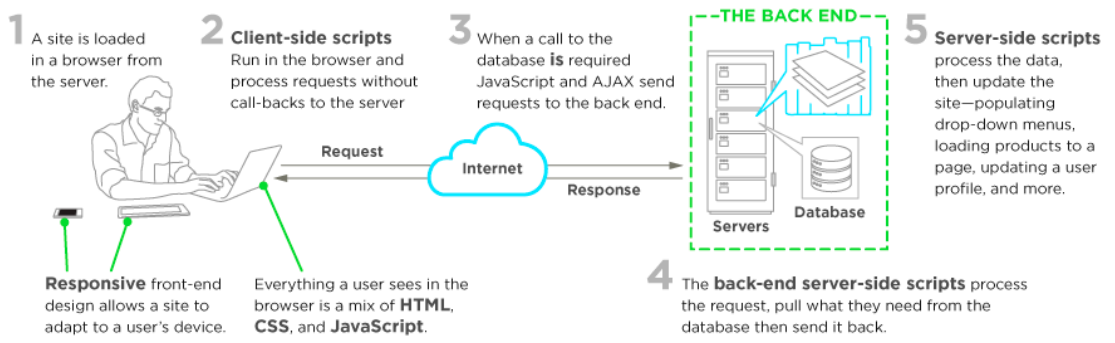


Fig. 1: Frontend development

We can see how the server-side manages all those client requests that make interactive websites possible. Frontend scripts process those user requests then volley them over to the server side. This process often happens in a constant loop of requests and responses. The roles of frontend developers are to deal with the user interface, forming an essential bridge between the designer and the backend programmer.

On the other side, we have the backend architecture. As we can see in the next visuals, the backend is a mix of the server, databases, APIs, and operating systems that power an app's frontend.

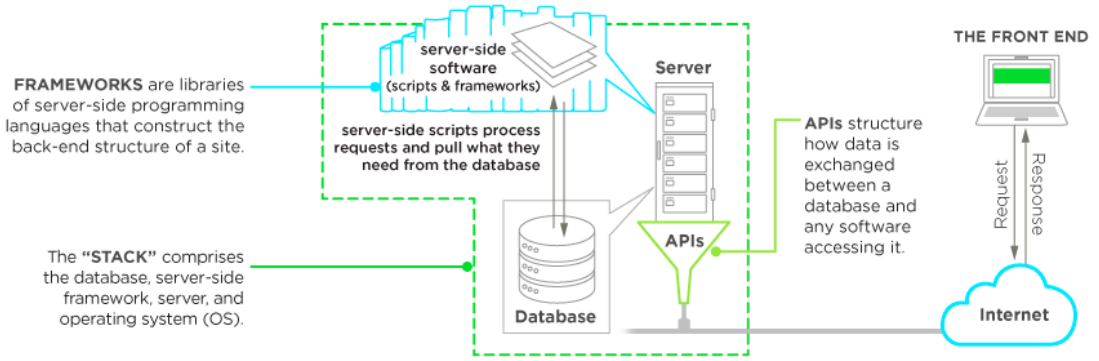


Fig. 2: Backend development

Frontend development has changed significantly over the last 15 to 20 years. Back in the day, websites were simple, static text sites with a bit of formatting and maybe even some animation. That was all thanks to HTML and CSS. JavaScript has evolved the most among the existing programming languages. It turned away from writing simple logic on websites by using ugly, unstructured code and plugins, and evolved into building completely functional Single Page Applications (SPA).

Web applications were using reloads every time a user interacted with UI. AJAX was still something new and was used only for updating small elements on the pages. The key player was jQuery, its simplified DOM data manipulations and added a nice wrapper for asynchronous code. Thousands of jQuery plugins were created which were responsible for specific UI interactions such as sliders, carousels, tabs and many more.

A few years later, between 2009-2010 a lot of things happened in the frontend world. Many new key players appeared on the scene. It's worth mentioning Backbone, RequireJS, AngularJS, Node, PhoneGap, Sass and Less. Evolution sped up after 2010. In the following years, thousands of new libraries appeared. Finally, JavaScript had package managers for dependencies. Most users started with Bower and slowly turned into npm packages when Node.js started to be more mature.

In the "good old days" any attempts to make some automation like concatenating files, minification or images optimization needed to be done outside of the JavaScript world. New task runners started to appear. Most developers began to use Grunt or Gulp for simple automation. Both tools started to lose popularity for the benefit of npm scripts.

The biggest changes appeared in JavaScript applications. Developers were inspired by solutions put forward by Backbone and Angular teams and started to recreate and improve on similar solutions. It's when Ember, Knockout, and React were gaining popularity. Another aspect of the development which changed slightly during this time was the modular approach to

development. In “the good old days”, modules in JavaScript did not exist. Several approaches were developed until webpack appeared, which became a standard for bundling and dealing with modules for client-side applications.

A big impact was also caused by UI Frameworks. Bootstrap, Foundation and, later, Angular Material. Each one included a complete set of components, fonts, icons and utils such as the frequently used grid system. Each one sped up the development process and helped to achieve consistent UI without any additional styling. In the case of styling, for a few years, Sass was unquestionably a leader. We have adopted nice architectural solutions for Sass such as BEM or SMACSS. Then there was an idea that styles could be created directly in JavaScript tied up to the components with the proper scopes. Currently, there are large numbers of similar CSS-in-JS solutions. In the case of SPA frameworks, the situation is clearer, there are 3 competitors, React, Angular and Vue.

The number of mobile application frameworks for JavaScript from which we can choose is also huge. There are React Native, Ionic, NativeScript and Cordova. Server-side rendering is a “hot” concept nowadays as we are in the SPA era, where all content is rendered client-side.

Security is a core component of modern web applications and should be built into design, implementation, and operations from the start. Frontend code must reduce browser attack surface, backend APIs must enforce authorization and input validation, and delivery pipelines must continuously verify dependencies and configurations. A practical baseline is the OWASP Top 10, complemented by implementation guidance from the OWASP Cheat Sheet Series.

Years ago, frontend developers could do everything their employer needed. It was not too difficult to be knowledgeable about CSS, vanilla JS, existing SPA frameworks. Nowadays it’s a bit more complicated, it’s hard to be a specialist in everything and it’s not possible to reach master level in everything related to frontend.

1.3 Web Technologies

Web sites are made with HTML, CSS and some JavaScript. However, we are developing web applications with complex UX/UI and business that require advanced technical solutions. This section will break down and broadly describes the practice of frontend engineering starting with a roadmap proposed by Frontend Developer Handbook 2024 with adaptations for web developers. Other roadmaps for Frontend, Backend and DevOps are also available at Developer Roadmaps.

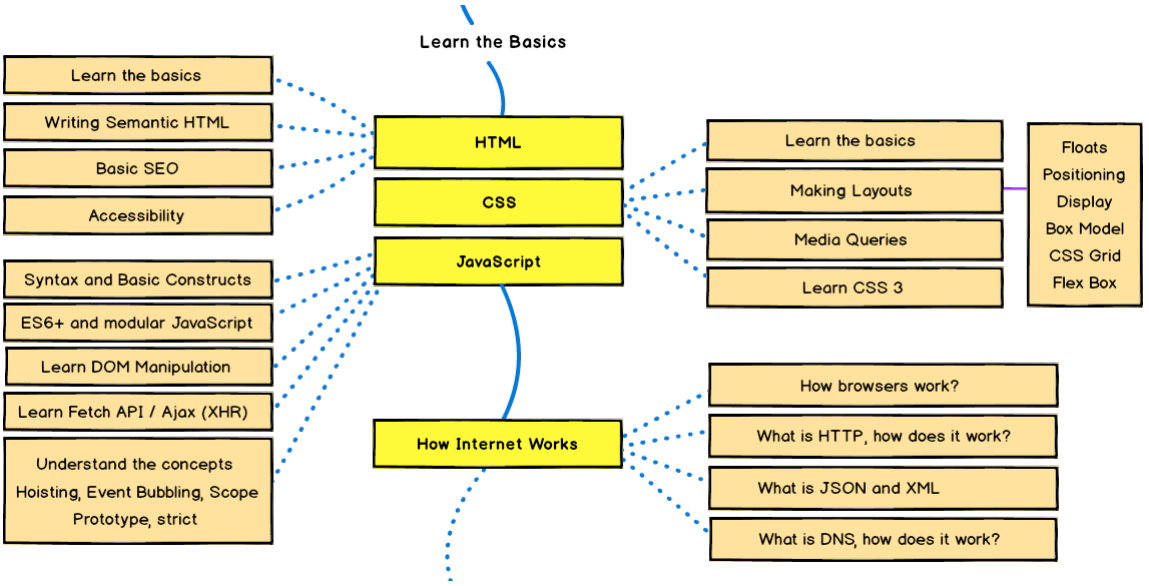


Fig. 3: Web development base

In yellow are defined what we need to learn and in orange are other options available. A person enters into the field of frontend development by learning to build a website or web application which relies on HTML, CSS, and JavaScript and commonly runs in a web browser but can also run in a headless browser, WebView, or as compilation input for a native runtime environment.

Frontend Developer Handbook 2024 recommends learning the actual underlying technologies before learning abstractions. “Don't learn jQuery, learn the DOM. Don't learn Sass, learn CSS. Don't learn JSX, learn HTML. Don't learn TypeScript, learn JavaScript. Don't learn Handlebars, learn JavaScript ES6 templates. Don't just use Bootstrap, learn UI patterns”. By learning the listed items, we should be able to work as web developers. We can achieve these goals creating a single page application. Once we are done with this, we need to learn about version control systems, the basic usage of Git and create a profile on GitHub or Bitbucket.

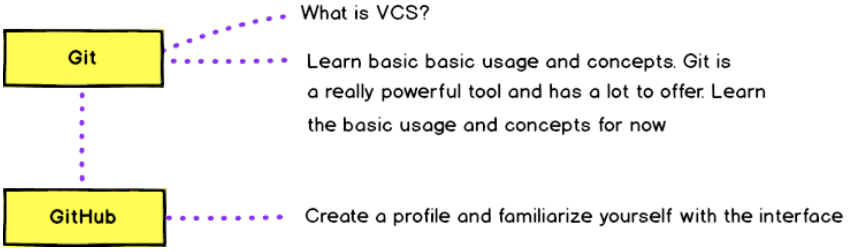


Fig. 4: Version control systems

Git is the most used version control system. Git tracks the changes we make to files, so we have a record of what has been done, and we can revert to specific versions should we ever need to. Git also makes collaboration easier, allowing changes by multiple people to all be merged into one source. Once we have covered the basics, the next step is about how to write maintainable CSS and use CSS frameworks. Here is how the roadmap should look:

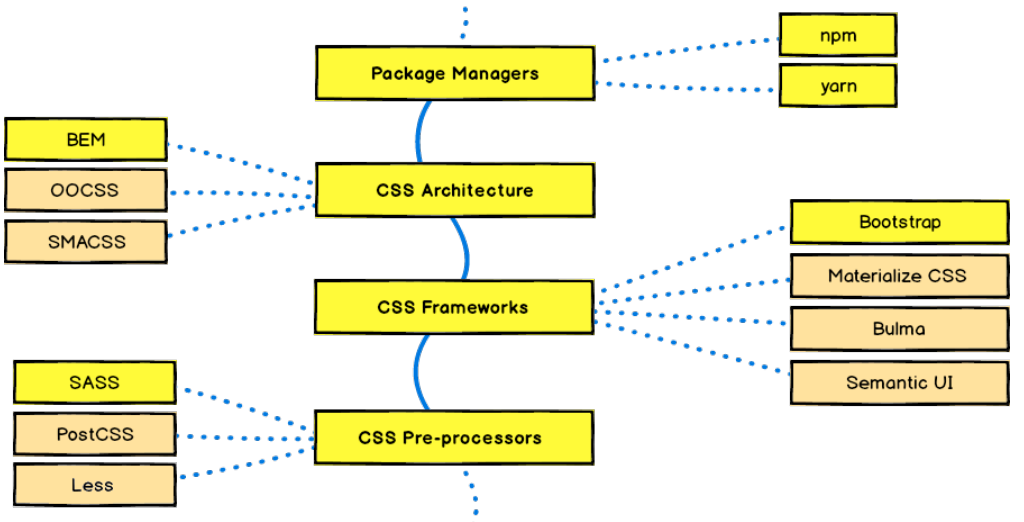


Fig. 5: CSS frameworks

To complete this step, we should be able to add Bootstrap in the project using npm or yarn. Write the CSS in Sass and convert the project to use BEM or other methodology. In general, we need to understand the importance of CSS in the code base. How the application looks is the first impression that users have. However, it can be hard to maintain a good appearance at all times with a poor code base. It's important to understand the good practices of CSS and the techniques available to write good CSS.

There are three pillars of maintainable CSS to consider when designing the CSS architecture of a software system. The first pillar defines the building blocks of the CSS architecture. These building blocks consist of a wide variety of solutions and tools, such as using Sass, writing efficient CSS selectors, the block-element-modifier (BEM) syntax, using classes instead of ID attributes, and using relative units where appropriate. Although this perspective gives us measurable improvements in CSS code quality, we need a higher level of organization to make our efforts systematic. Therefore, we need a second pillar that focuses on the orchestration of building blocks to establish solid, maintainable, hierarchical CSS. Unfortunately, neither the building blocks, nor a methodical usage of a framework or a CSS architecture give us the answer to writing

rock solid, maintainable CSS. Code becomes solid through the application of software engineering principles. This is the third pillar of writing maintainable CSS.

Next steps are about modern frontend development. To become a frontend developer, we need to learn more about JavaScript and its ecosystem. We need to learn what webpack is, understand the different concepts and why it was ever needed, understand what Babel is, why we use it and learn how to integrate it with webpack and lastly learn how to lint code using ESLint. All the items listed in this phase revolve around webpack.

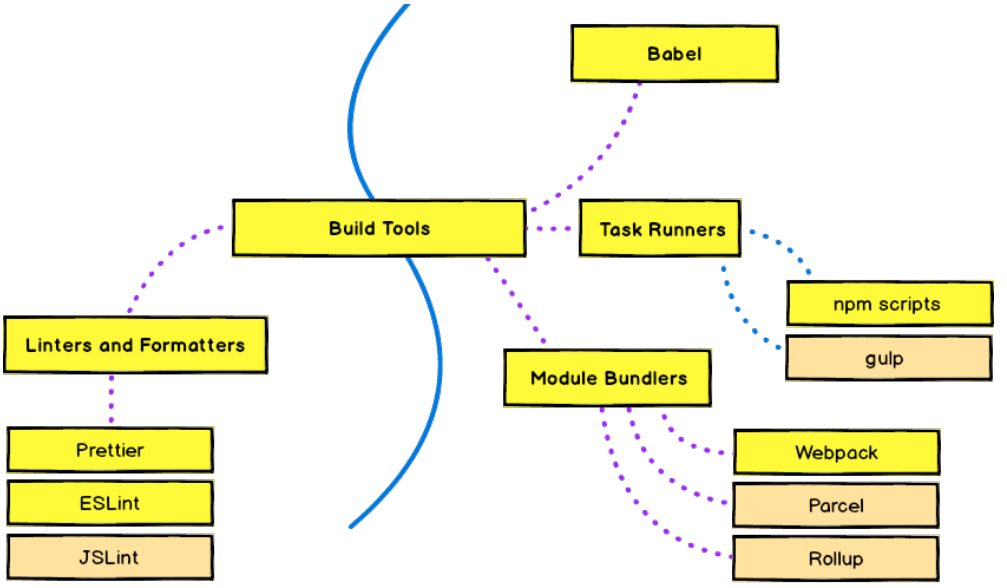


Fig. 6: Project tools

As a frontend developer, we need to master a set of tools such as transpilers, module bundlers, task runners, CLI commands, linters and formatters. We should be able to create a npm package with scripts for serve, test, lint and build. Create a small application that runs in the browser, bundle it with webpack, use babel to transpile it, use Biome or ESLint for linting and Prettier for formatting. It's important to apply Sass for the CSS, add Bootstrap for styles and apply BEM methodology. In the end, deploy the optimized production build in a web server. In general, we are working with TypeScript for Angular projects instead of Vanilla JavaScript. That means, we have small differences when we are configuring these types of projects.

Next step is to learn about some frontend framework. There are multiple options but the ones most commonly being used these days are Angular, React and Vue.

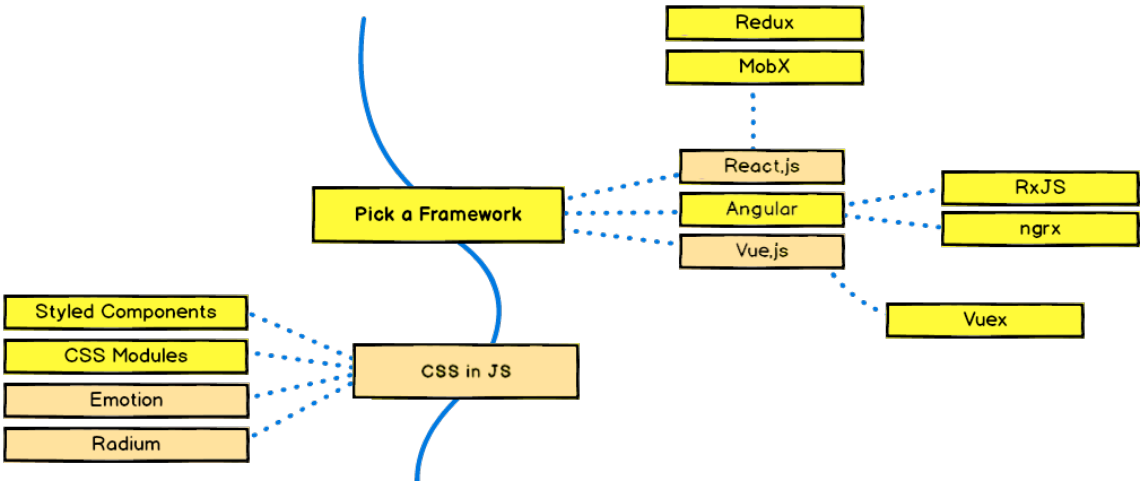


Fig. 7: JavaScript frameworks

Popular frameworks have a few core commonalities. Most frontend frameworks/libraries, from Vue to React, provide functionalities such as synchronization of state and view, routing, a template system, reusable components, browser extensions that help debugging things, they have a strong community and third-party components. However, these features aren't the main reason to use a JavaScript framework. The main reason why we need a framework is because these frameworks provide the ability to implement UIs that are guaranteed to be in sync with the internal state of the application. So, maintaining the UI in sync with the data without a framework involves writing a lot of tedious and fragile code.

Angular is the most popular framework to use with SOLID Principles for frontend projects. It has a large ecosystem, however, we don't need to know everything. It's fundamental to have knowledge in TypeScript and RxJS to work with Angular. Another important library to know is NgRx. NgRx Store provides reactive state management for Angular apps inspired by Redux. State management refers to the management of the state of one or more user interface controls such as text fields, buttons, radio buttons, etc., in a graphical user interface. For example, a state managed UI control such as a button will be in the enabled state when input fields have valid input values and the button will be in the disabled state when the input fields are empty or have invalid values.

CSS-in-JS is a styling technique where JavaScript is used to style components. When this JavaScript is parsed, CSS is generated and attached into the DOM. It allows us to abstract CSS to the component level itself, using JavaScript to describe styles in a declarative and maintainable

way. CSS-in-JS grew in popularity with React because it is a common approach in React projects. In Angular projects, it is more important to master in Shadow DOM.

In the end, we should be able to use a frontend framework, build a SPA, know about PWA, service workers, measuring performance, using Lighthouse and different Browser APIs. The most important APIs every frontend developer should be aware of are Fetch API, DOM API, Web Storage, Web Workers, Service Workers, Intersection Observer, WebSocket, Canvas, Notification API, and Payment Request API.

Automated testing will save us a lot of headaches in the future. Manual software testing is performed by a human sitting in front of a computer carefully going through application screens, trying various usage and input combinations, comparing the results to the expected behavior and recording their observations. Manual tests are repeated often during development cycles for source code changes and other situations like multiple operating environments and hardware configurations. An automated testing tool can playback pre-recorded and predefined actions, compare the results to the expected behavior and report the success or failure of these manual tests to a test engineer. Once automated tests are created, they can easily be repeated, and they can be extended to perform tasks impossible with manual testing. Automated software testing has long been considered critical for big software development organizations but is often thought to be too expensive or difficult for smaller companies to implement.

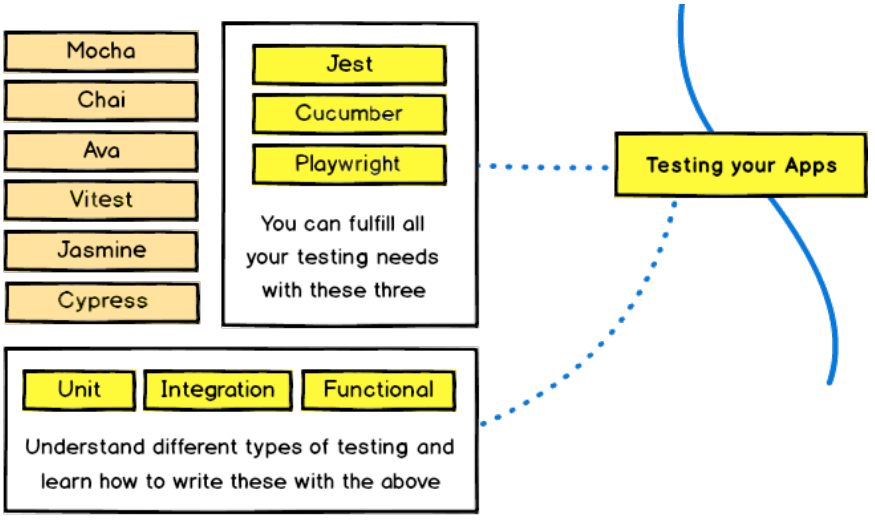


Fig. 8: Automated testing

These test types are the most recommended and are mandatory for frontend projects. Other types of tests, such as visual testing, are optional. So, as a frontend developer, we should be able to write unit, integration and functional tests.

Angular recommends Vitest testing framework for unit and integration tests. For functional tests is recommended Playwright. However, other tools with higher popularity are also available. Cypress and Jest are the most popular testing solutions. Cucumber is a framework that runs automated acceptance tests written in a BDD style. Each solution is good, but we will get a better experience by combining Angular recommendations with the strongest tools. This means that the better combination to test Angular apps is Jest + Cucumber + Playwright. However, it is only a proposal and other combinations are valid. Another important concept about testing is code coverage.

Server rendered applications allow achieving better performance and improved SEO as compared to the client rendered applications. Although not a requirement it would help us in crafting better frontend applications. There are different options available, the most popular is Next.js for React. In the next image, we have the most important JavaScript solutions for server-side rendering. For Angular projects, Angular Universal is recommended.

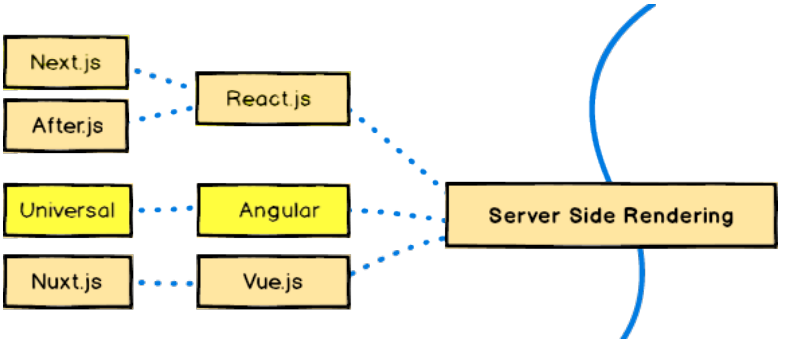


Fig. 9: Server-side rendering

As mentioned at the beginning, to become a frontend developer, we need to learn how the web platform works, make sure we know the "what" and "where" of HTML, CSS, DOM, JavaScript, Domains, DNS, URLs, HTTP, browsers, and servers/hosting. Learn about the fundamentals of user interface design (i.e. UI patterns, interaction design, user experience design, and usability), CLI/command line, the practice of software engineering (i.e., application design/architecture, templates, Git, testing, monitoring, automating, code quality, development methodologies), be opinionated and know how to customize the toolbox with whatever makes sense (e.g. Angular, NgRx, Vitest, and esbuild) and learn Node.js.

1.4 HTML

HTML (HyperText Markup Language) is the standard language used to describe the structure and meaning of content on the web. It is not a programming language, instead, it is a markup language based on elements and attributes. Browsers parse HTML documents and build a tree representation of the page, which is then styled with CSS and made interactive with JavaScript.

An HTML document is composed of nested elements that represent content such as headings, paragraphs, links, lists, forms, images, and sections. The browser reads this structure from top to bottom and turns it into the Document Object Model (DOM). Clean and predictable HTML structure improves maintainability, accessibility, and testability. Semantic HTML means choosing elements based on meaning, not only visual appearance. Instead of using generic `<div>` and `<span>` for everything, we should use semantic elements such as `<header>`, `<main>`, `<section>`, `<article>`, `<nav>`, `<aside>`, `<footer>`, `<button>`, and `<form>`. Using Semantic HTML improves accessibility for assistive technologies, SEO because search engines better understand document structure, code readability and reduces maintenance cost.

In modern frontend engineering, strong HTML knowledge is fundamental because every framework, including Angular, eventually renders HTML in the browser. Understanding the platform first helps us make better framework decisions, write accessible interfaces, and avoid implementation problems that appear only in production.

1.5 TypeScript

1.5.1 Fundamentals

TypeScript isn't a completely new language, it's a superset of ES6. If we write ES6 code, it's perfectly valid and compilable TypeScript code. Microsoft and Google are committed to moving the web forward and as developers we win because of it. TypeScript is heavily used in Angular, where types and decorators improve developer experience when working with Angular's dependency injection system.

There are five big improvements that TypeScript brings over ES5 such as types, classes, decorators, imports, and language utilities. TypeScript advantages that make people consider using it are:

- Code scalability with interface-oriented development and explicit contracts;
- Better collaboration in growing teams through predictable, typed module boundaries;

- Strong tooling and ecosystem support, especially in the editor and compiler pipeline;
- ES-next alignment, allowing modern JavaScript features with type safety;
- Better code quality and understandability through explicit types, interfaces, and generics;
- Safer and faster refactoring, because the compiler catches many issues before runtime;
- Types as living documentation for APIs and function signatures;
- Better testability through typed boundaries and easier mocking of dependencies;
- Better maintainability and predictability in large codebases;
- Easier application of architectural patterns such as SOLID principles;
- Reliable transpilation to JavaScript targets required by supported environments;
- IntelliSense and code navigation (completion, rename, go-to-definition, quick fixes) that improve productivity;
- Gradual adoption in existing JavaScript projects, allowing incremental migration.

TypeScript is unique in that it is a superset of JavaScript (ES2015+), but with optional types, interfaces, generics, and more. Unlike other compile-to-JavaScript languages, TypeScript does not try to change JavaScript into a new language. Instead, the TypeScript team takes extra care to align the language's extra features as closely as possible with what's available in JavaScript and its upcoming features. Because of this, TypeScript developers can take advantage of the latest features in the JavaScript language in addition to a power type system to write better-organized code while taking advantage of advanced tooling that using a statically typed language provides.

TypeScript supports object-oriented programming (OOP) with class-based inheritance. These features are great, but the truth is that they are also dangerous in modern JavaScript. We need a good object-oriented design (SOLID, Composite Reuse, etc.) to protect ourselves from these threats. As TypeScript developers, we need to know SOLID principles to write clean code. It's important to identify and apply Design Patterns to TypeScript projects. All patterns can be categorized by their intent, or purpose. They are creational patterns, structural patterns and behavioral patterns. It's important to understand the importance of dependency injection with InversifyJS to create testable, maintainable, and extensible code. We have more examples of implementations of design patterns in TypeScript in Márk Török repository. Also, Patterns.dev offers resources on design, rendering, and performance patterns for building powerful web apps with vanilla JavaScript or modern frameworks.

The real power in TypeScript comes from its tooling. Types are a means to bring world-class tooling to the JavaScript language, which allows for better structured projects that are easier to

maintain. This is especially important as JavaScript projects grow (both lines of code and developers on the project). Having fast, accurate completion, refactoring capabilities, and immediate feedback makes TypeScript the ideal language for large-scale JavaScript. TypeScript also makes it simple to get started. Since JavaScript is effectively TypeScript without type annotations, all or portions of an existing project can be converted immediately and then slowly take advantage of all that TypeScript has to offer over time.

To learn more about TypeScript, we have available the official documentation, books, articles and guides such as The Definitive TypeScript Guide.

1.5.2 Project Configuration

It's easy to work with TypeScript when using a starter project or a tool like the Angular CLI. First, we need TypeScript installed of course. We can decide to have it installed globally on the machine, or locally to the project.

```
npm i typescript -g
```

TypeScript comes with two binaries: *tsc* and *tsserver*. *tsc* is the TypeScript compiler and has a command line interface with plenty of available options. To initialize a TypeScript project, simply use the *--init* flag:

```
tsc --init
```

After running *tsc* with the *--init* flag, a *tsconfig.json* will be added to the project folder. The presence of a *tsconfig.json* file in a directory indicates that the directory is the root of a TypeScript project. The *tsconfig.json* file specifies the root files and the compiler options required to compile the project. A project is compiled in one of the following ways:

- By invoking *tsc* with no input files, in which case the compiler searches for the *tsconfig.json* file starting in the current directory and continuing up the parent directory chain.
- By invoking *tsc* with no input files and a *--project* (or just *-p*) command line option that specifies the path of a directory containing a *tsconfig.json* file, or a path to a valid *.json* file containing the configurations.

When input files are specified on the command line, *tsconfig.json* files are ignored. Below, we can see an example of *tsconfig.json* file.

```
{
  "compilerOptions": {
    "module": "commonjs",
    "noImplicitAny": true,
    "removeComments": true,
    "preserveConstEnums": true,
    "sourceMap": true
  },
  "include": [
    "src/**/*"
  ],
  "exclude": [
    "node_modules",
    "**/*.spec.ts"
  ]
}
```

Running *tsc* locally will compile the closest project defined by a *tsconfig.json*. We can compile a set of TypeScript files by passing in a glob of files we want. The following command transpiles any .ts files in the folder src, with the default settings:

```
tsc src/*.ts
```

Instead of running the TypeScript compiler every time we make a change, the `tsc -w` command compiles in watch mode, recompiling whenever there are changes to the TypeScript files.

1.5.3 Object-Oriented Programming

TypeScript objects are just syntax-sugar for JavaScript function-objects. There's a lot of repetitive code when it comes to using function-objects as classes in JavaScript, that's why class was implemented for ES6. TypeScript takes ES6 classes to a higher plane of reality by adding, not only types, but also object features such as public, private, abstract, etc.

Objects in TypeScript (like in all OO languages) are useful because they allow us to model a program based on real-world scenarios. Objects are instances (a variable with that object as its value type) of classes. Think of classes as just a group methods and variables. Classes are implemented using the class keyword, just like in ES6. To create an object, we call the class with the *new* keyword, which triggers the constructor and returns an instance-object just like in regular JavaScript.

TypeScript brings familiar OOP constructs to JavaScript. There are four main principles to Object Oriented Programming: Encapsulation, Inheritance, Abstraction, and Polymorphism. TypeScript can implement all four of them with its smaller and cleaner syntax. Using TypeScript doesn't

mean we're off the hook of knowing JavaScript well. The better we know JavaScript, the better we'll know TypeScript.

1.5.4 Functional Programming

TypeScript is not a purely functional language but offers a lot of concepts which are in line with functional languages. Functional programming is a programming paradigm, a style of building the structure and elements of computer programs, that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.

Using functional programming doesn't mean it's all or nothing, we can always use functional programming concepts to complement object-oriented concepts in TypeScript. The benefits of functional programming can be utilized whenever possible regardless of the paradigm or language we use.

Functional programming concepts, such as higher-order-functions, closures, currying, recursion, lazy evaluations or referential transparency, can be applied in TypeScript. TypeScript is a better option than JavaScript when it comes to functional programming. Adding type annotations can take a little bit of additional time, but the benefits will undoubtedly pay off. Our code will be less prone to errors, it will be self-documented, and the team members will be much more productive because they will spend less time trying to understand the pre-existing code.

1.5.5 Style Guide

A style guide is a set of standards that outline how code should be written and organized. Good projects have clear consistent coding conventions, with automated enforcement. We need a style guide because everyone writes code differently. Style guides are created so new developers can get up to speed on a code base quickly and then write code that other developers can understand quickly and easily.

We can build our custom style guide, but we have good public options available. The most popular are Airbnb JavaScript Style Guide, Google JavaScript Style Guide, Idiomatic JavaScript Style Guide and JavaScript Standard Style Guide. These rules are valid for TypeScript projects, but we can complete them with Do's and Don'ts from official documentation and contributions from some authors such as Google TypeScript Style Guide, TypeScript StyleGuide and Coding Conventions, TypeScript Style Guide and Clean Code TypeScript. Guidelines from Airbnb are very popular and are recommended for TypeScript projects. We can complete them with rules just for TypeScript.

These guidelines are helpful, but to apply them to TypeScript projects, we need a linter, formatter and strong code review. Linting code is very helpful for preventing minor issues that can escape the eye during development. Formatting code is very helpful for readability. When working in teams, it is nice to be able to look at code that is in the same format across the entire application.

1.5.6 Best Practices

The following best practices consolidate coding, design, and maintainability guidelines for writing clean, safe, and scalable TypeScript code:

- ☐ **Follow conventions**: Code conventions are base rules that allow the creation of a uniform code base across an organization. Following them increases uniformity and the quality of the code. To make it mandatory, we need a linter, formatter, and strong code review, and each team should define a dynamic convention for each project.
- ☐ **Cover the code with tests**: Testing is more important than shipping. If we have no tests or an inadequate amount, then every time we ship code, we will not be sure that we did not break anything. There is no excuse not to write tests, there are plenty of good JavaScript test frameworks with typing support for TypeScript, and we should always write tests for every new feature or module.
- ☐ **Use strict configuration**: The strict configuration should be mandatory and should be enabled by default because there is not much value in using Typescript without these few flags. Important options include `forceConsistentCasingInFileNames`, `noImplicitReturns`, `strict`, and `noUnusedLocals`, and we can progressively introduce `alwaysStrict`, `strictNullChecks`, `noImplicitAny`, `noImplicitThis` and `noPropertyAccessFromIndexSignature` in existing codebases.
- ☐ **Avoid any. Type everything**: Always declare variables or constants with a type other than `any`. Add return types to exported functions, public methods, and reusable helpers. Prefer `unknown` over `any` for external data. Without typing, TypeScript deduces a type from the assigned value, and this can cause unintended problems. Good typings make refactoring easier and safer.
- ☐ **Prefer unions over enums**: Literal unions are often simpler, easier to serialize, and more ergonomic than *enum* for closed sets of values.
- ☐ **Use interfaces and type aliases**: Prefer interfaces for object shapes and public contracts, and use type aliases for unions, intersections, and mapped types.
- ☐ **Use type guards and narrowing**: Prefer `typeof`, `instanceof`, `in`, and custom type guard functions (`value is T`) to narrow unions safely. This keeps runtime checks and compile-time guarantees aligned.

- ☐ **Avoid non-null assertions (!) as a default**: The non-null assertion operator bypasses null safety and can hide real runtime bugs. Prefer proper null checks, optional chaining, nullish coalescing, and narrowing before access.
- ☐ **Strings should be safe**: If we have a variable of type string that can have only a set of values, instead of declaring it as a string type we can declare a list of possible values as a union type. By declaring the type appropriately, we can avoid bugs during compile time rather than runtime.
- ☐ **Call things by their name**: Good variable and function names should be easy to understand and tell us what is going on, not more and not less. Avoid unreadable short names and avoid extremely long names. Also avoid marrying values and functionality in names. A function like `isLegalDrinkingAge()` communicates intent better than `isOverEighteen()`, and keeping to English is a good idea because programming languages are in English.
- ☐ **Use utility types**: TypeScript already has utility types such as `Partial<T>` and `Readonly<T>`, and they make code much easier to understand. Break interfaces or types into smaller nested interfaces only when it makes sense in the domain, because aggressively splitting types can hide structure and hurt readability, especially with code completion.
- ☐ **Use const and let**: JavaScript first searches local scope and then higher levels to global scope. `var` is function scope, while `let` and `const` are block scope. Using `let` and `const` makes declaration intention clearer and helps identify accidental reassignment to constants at compile time. Use a linter that checks and fixes this automatically.
- ☐ **Use satisfies and as const for safer literals**: `satisfies` verifies object shape without widening useful literal types, while *as const* preserves readonly literal values. Together they improve config objects and lookup tables without unsafe casting.
- ☐ **Use === instead of ==**: JavaScript has `===` | `!==` and `==` | `!=`. It is best practice to use strict equality operators. If two operands are the same type and value, `===` behaves predictably. With `==` and `!=`, type coercion can create bugs when comparing different types.
- ☐ **Use shortcut notation sparingly**: Shortcut notation keeps code small, but can hurt readability if overused. Prefer literal forms such as:
    - `{}` instead of `new Object()`
    - `""` instead of `new String()`
    - `0` instead of `new Number()`
    - `false` instead of `new Boolean()`
    - `[]` instead of `new Array()`
    - `/()/` instead of `new RegExp()`
    - `()=>{}` instead of `function (){}`

- ☐ **Avoid globals**: Global variables and function names are an incredibly bad idea because every JavaScript file on a page runs in the same scope. Scripts included later can overwrite variables and functions with the same names.
- ☐ **Avoid mixing with other technologies**: Although we can create everything in a document using JavaScript and DOM, it is not always the most effective way. Techniques such as CSS-in-JS are popular, but mixing JavaScript, HTML, and CSS in the same file can be dangerous. Good separation of concerns remains the best practice.
- ☐ **Avoid heavy nesting**: Nesting can explain logic and make code readable, but too much nesting makes code hard to follow. Readers should not have to scroll horizontally or fight wrapped lines. Deep nesting also creates messy iterator naming (`i, j, k, l`) and quickly becomes confusing.
- ☐ **Avoid long functions**: Long functions usually indicate they are doing too many things. Small functions are easier to read and faster to understand. If a function has more than 10 lines, ask if it should be split into smaller functions. Functions and methods around 5 to 10 lines are often easier to reason about and maintain.
- ☐ **Reduce function parameters**: Limiting function parameters makes testing easier. More than three parameters leads to a combinatorial explosion of test scenarios. One or two is ideal, three should be avoided when possible, and anything more should be consolidated, often into a higher-level object.
- ☐ **Do not use flags as function parameters**: Flags tell users a function does more than one thing. Functions should do one thing, so split functions when behavior changes based on a Boolean. Single-purpose functions are easier to compose, test, reason about, and refactor.
- ☐ **Comment as much as needed but not more**: Comments are messages to other developers, but if we are adding many comments, the code may not be self-explanatory. Good comments are informative and useful. Bad comments include commented-out code and noise comments that are not more informative than the code itself. Delete dead code instead of leaving it commented.
- ☐ **Use the fastest way to loop arrays**: There are many ways to iterate arrays (`for, for...of, forEach, map, filter, while`). The `for` loop is generally the fastest, and caching `length` can improve performance in some engines. `forEach` is slower for large arrays, but readability still matters unless performance is critical.
- ☐ **Prefer array methods**: A functional approach without intermediate variables is often shorter and more readable. While a base `for` loop can be more performant in some cases, benefits are noticeable mostly at very large scales. In most application code, favor clearer array methods and functional programming style over imperative style.
- ☐ **Do not use short-hand**: We can omit curly braces and semicolons and many browsers will still interpret code, but this approach is dangerous and not recommended. This is a

terrible practice that should be avoided. Omitting braces is debated even for one-liners, so favor explicit syntax.

- ☐ **Use parameter defaults**: When a function is called without a parameter, the missing argument is `undefined`. ES6 default parameters let us define safe defaults directly in the function signature. It is a good habit to assign default values to arguments. See next example:

```
function logNumber(num = 25): void {}
```

- ☐ **Use spread and destructuring**: Spread syntax lets us expand arrays, strings, objects, maps, sets, typed arrays, and other iterables into arguments, elements, or key-value pairs. Destructuring syntax lets us unpack object and array values into individual properties, improving clarity and reducing boilerplate.

- ☐ **Use template literals**: Template literals make working with strings much easier than long concatenation. We use backticks instead of single or double quotes and produce readable, flexible strings.

- ☐ **End the switches with defaults**: Do not leave switch statements without a default case, because something can go wrong and we want to detect it. In some cases *default* can be combined with another case to avoid duplicate conditions. See next example:

```
function getEmoji(key: string): string {
  switch (key) {
    case 'dog':
      return '🐶';
    case 'cat':
      return '🐱';
    // the rest of the emojis...
    default: case 'smile':
      return '🙂';
  }
}
```

- ☐ **Enforce exhaustive checks in switches**: When switching over unions, ensure all cases are handled and use a `never` fallback to catch missing branches at compile time during refactors.

- ☐ **Use the prefix is and has for booleans**: Prefixes like `is` and `has` make it clear a variable is Boolean. Code is read more often than written, and correctly used names reduce the need for comments that can become outdated.

- ☐ **Put declarations on top**: Put variable declarations at the top of each script or function for cleaner code and a single place to inspect local variables. This helps avoid implied globals and unwanted redeclarations.

- ☐ **Initialize variables**: Initialize variables when declaring them to keep code cleaner, avoid undefined values, and keep initialization in one place.

- ☐ **Use iterators and generators**: Iterators and generators are useful for stream-like collections. Benefits include decoupling calls from implementation, lazy execution on demand, built-in `for-of` iteration support, and optimized iterator patterns through iterables.
- ☐ **Prefer pure functions**: A pure function returns values determined only by inputs and has no observable side effects. Pure functions are easier to reason about, combine, test, debug, parallelize, memoize, and they support idempotence and referential transparency.
- ☐ **Prefer immutability**: Immutability helps differentiate objects and track changes. Mark configuration, DTOs, and class members as `readonly` when they should not change, and use immutable updates for state where practical. It increases predictability, change tracking, reactive UI implementation, testability, and single-source-of-truth design.
- ☐ **Avoid side effects**: Side effects are operations beyond input-output transformation, such as writing files or modifying global state. Programs need side effects sometimes, but they should be centralized to avoid shared mutable state and unstructured behavior.
- ☐ **Avoid magic numbers**: Magic numbers are unexplained values in source code that make code difficult to understand and maintain. Put special numbers into named constants to provide implicit documentation. Lint rules such as `no-magic-numbers` improve readability and refactoring.
- ☐ **Avoid many conditionals**: This sounds impossible, but in many cases polymorphism can replace `if` chains. The clean-code idea is that a function should do one thing. Functions with many conditionals often expose multiple responsibilities and become harder to maintain.
- ☐ **Handle JavaScript errors**: Thrown errors are useful because they identify runtime failures and provide stack traces. JavaScript and TypeScript allow throwing any object, and promises can reject with any reason, but it is advisable to throw `Error` types so catches in higher-level code are predictable.
- ☐ **Prefer promises over callbacks**: Promises are easy to use and callback APIs can be promisified. With promises and `async/await`, asynchronous code is cleaner and can improve throughput in JavaScript's single-threaded model.
- ☐ **Do not use weird JavaScript features**: Avoid unusual and risky features such as mutating array `length`, using `with`, using `void`, updating native object prototypes, calling `eval()`, or passing strings to `setTimeout` and `setInterval`. Language support does not mean these are good practices.
- ☐ **Place scripts at the bottom of the page**: While a script loads, browser rendering cannot continue, so users wait longer before seeing progress. Scripts that mainly add functionality after interactions can be placed near the closing body tag to improve perceived load speed.

- ☐ **Raw JavaScript is faster**: Libraries such as jQuery can save development time, especially for AJAX operations, but they cannot be as fast as raw JavaScript. For example, jQuery `each` is useful, yet a native `for` loop is generally quicker.
- ☐ **Keep imports clean and readable**: Group third-party and application imports with a blank line between groups, and keep imports alphabetized where practical. Import hygiene reduces merge noise and improves scannability.
- ☐ **Use aliases for imports**: Deep relative imports are hard to read and maintain. TypeScript path aliases provide a cleaner way to map imports to absolute paths through `compilerOptions` in `tsconfig.json`, making imports easier to read and refactor.
- ☐ **Prefer modularization**: Modularization is a general best practice. Functions that fulfill one job are easier to debug and change. This also applies to helper functions because if the same behavior appears across multiple places, extract it to a generic helper.
- ☐ **Use lazy-Loading**: Lazy or on-demand loading is a strong optimization practice. It keeps initial bundles smaller, decreases load times, and loads code only when users reach a related feature. Splitting code at logical breakpoints speeds initial load and can reduce overall application weight.
- ☐ **Remove unused code**: Unused or dead code is code that will never run, including files, conditions, loops, unused imports, variables, functions, and private members. Dead-code elimination reduces repository and bundle size, improves performance, and makes code easier to understand.
- ☐ **Compress source files**: Compress source code files and related static assets so payloads are smaller and delivery is faster. Compression improves loading performance and supports the same goal as lazy-loading.
- ☐ **Do not trust any data**: A key security point is not to trust input data. Make sure all data entering the system is clean and exactly what we need. This is especially important on the backend for URL parameters, but also on the client side for form inputs and DOM-derived values.
- ☐ **Validate runtime input at boundaries**: TypeScript types disappear at runtime, so validate external input at application boundaries. Schema validators such as Zod, Valibot, or io-ts can align runtime validation with static types.
- ☐ **Enforce CI quality and performance gates**: Automate `typecheck`, lint, unit/integration tests, and bundle budget checks in CI to prevent regressions from reaching production.

1.6 CSS

1.6.1 CSS Evolution

Since the early days of the web, teams have needed ways to style pages consistently. CSS has been the foundation for that, but both the language and the way we organize styles have changed significantly over time.

In the 1990s, the focus was often on visual impact, and inline styles were common. As websites became more dynamic, styling at scale became difficult. Different developers used different conventions, and large stylesheets quickly became hard to maintain.

In 2006, Sass was released. Sass introduced preprocessor capabilities such as variables, nesting, mixins, and reusable logic, helping teams structure styles more effectively. Other preprocessors like Less and Stylus followed similar goals. They introduced a compilation step that added features CSS did not yet provide natively. Today, CSS itself includes many of these ideas through custom properties, `calc()`, nesting, and other modern features, but preprocessors are still widely used.

Even with preprocessors and postprocessors, maintaining large CSS codebases remained challenging. This led to CSS methodologies: documented approaches for naming, structuring, and scaling styles. BEM (Block, Element, Modifier), introduced around 2010, was one of the most influential methodologies. It encouraged component-oriented naming and better reuse. Other relevant methodologies include OOCSS, SMACSS, SUITCSS, and Atomic CSS.

Still, Sass and naming methodologies do not provide true style encapsulation by themselves. They reduce risk through conventions, but collisions can still happen. CSS Modules addressed this by generating locally scoped class names at build time, reducing accidental overrides and visual regressions.

From 2014 onward, CSS-in-JS grew as another approach, especially in React ecosystems, by defining styles at the component level with JavaScript. This challenged traditional separation-of-concerns practices, but also improved co-location for some teams. At the same time, native features such as custom properties and cascade layers reduced the need for heavier abstractions in many projects. In Angular projects, understanding style encapsulation strategies, including Shadow DOM, is generally more relevant than adopting CSS-in-JS directly.

The practical takeaway is that no single approach is universally best. The right choice depends on team size, project complexity, framework conventions, and long-term maintainability goals.

1.6.2 Sass or SCSS

Sass is a CSS preprocessor that extends CSS with features such as variables, nesting, mixins, functions, and modules. In real projects, this helps keep styles more organized, reusable, and maintainable, especially as the codebase grows.

The terminology is often confusing, so it helps to separate two concepts:

- **Sass** is the preprocessor (the tool).
- **SCSS** is the most common Sass syntax, based on regular CSS syntax.
- **Indented Sass** (also called just "Sass syntax") is the older indentation-based syntax without braces and semicolons.

If a stylesheet is valid CSS, it is also valid SCSS. This makes SCSS easier to adopt incrementally and friendlier for teams already writing CSS.

For Angular projects, **SCSS is usually the best default** because it:

- Preserves familiar CSS syntax and reduces onboarding friction;
- Scales well with component-based styling and design tokens;
- Integrates naturally with modern tooling (for example, stylelint, PostCSS, and Autoprefixer).

When using Sass in modern projects, prefer the module system (`@use` and `@forward`) over legacy `@import`, and keep nesting shallow to avoid overly specific selectors.

1.6.3 Best Practices

The following practices present a practical baseline for writing scalable, consistent, and maintainable CSS:

- ☐ **Follow conventions**: Code conventions are base rules that allow the creation of a uniform code base across an organization. To make conventions mandatory, use a linter, formatter, and strong code review, and keep rules dynamic and adaptable for each team and project. Awesome Sass is also a useful collection of frameworks, libraries, style guides, articles, and resources.
- ☐ **Follow a CSS methodology**: CSS methodologies ensure consistency and future proof styles. They reduce CSS footprint, organize cooperation among programmers, and help maintain large codebases. BEM is a common methodology, and alternatives include ITCSS, OOCSS, SMACSS, SUITCSS and Atomic CSS.

- ☐ **Lint the CSS files**: Linting ensures we follow the rules we define for style and keep CSS consistent, well structured, and aligned with best practices. Use stylelint in CSS projects and integrate it with the IDE. Use lint-staged to run linting commands on staged files so committed code has no linting or formatting errors.
- ☐ **Alphabetize CSS properties**: Organization rules vary by developer, so define team rules and enforce them with a linter. stylelint can organize CSS and enforce strict property order, syntax checks, and style consistency for everyone.
- ☐ **Cross-browser compatibility**: Browser prefixes exist so browser makers can support new features before full standard support. With PostCSS and Autoprefixer, we can write normal CSS and let the postprocessor handle prefixes. Autoprefixer uses Browserslist to determine which browsers should be supported.
- ☐ **Prefer CSS over JavaScript**: If something can be done with CSS instead of JavaScript, prefer CSS. Even when JavaScript is needed, CSS should handle most style while JavaScript handles triggers and side effects. JavaScript is not fault tolerant and can be disastrous when misused. Applications are faster using CSS for styling behavior.
- ☐ **Comment the CSS**: Add comments for each group of CSS code when they provide value. Comments help explain blocks of code or temporary development changes. Keep comments clear and purposeful so maintenance is easier.
- ☐ **Avoid undoing styles**: CSS that unsets previous styles should ring alarm bells. Rulesets should inherit and add to previous ones, not undo them. See next bad example:

```
h2 {
  font-size: 2em;
  padding-bottom: 0.5em;
  border-bottom: 1px solid #ccc;
}

.no-border {
  padding-bottom: 0;
  border-bottom: none;
}
```

- ☐ **Avoid brute forcing**: Brute forcing CSS uses hard-coded magic numbers and hacks to force layouts, such as negative margins without structural reasoning. This usually indicates poor layout code or weak understanding of box model and computed styles. Well coded layouts should not need brute-force manipulation.
- ☐ **Avoid magic numbers**: A magic number is a value used because it just works. They become outdated, confuse developers, cannot be explained, and cannot be trusted. Never use numbers only because they seem to work right now. Lint's rules can detect this.
- ☐ **Avoid qualified selectors**: Qualified selectors like `ul.nav`, `a.button`, or `div.header` are needlessly prepended by an element. They inhibit reusability,

increase specificity, and increase browser workload. Prefer simple selectors like `.nav`, `.button`, and `.header` for better portability and performance.

- ☐ **Avoid dangerous selectors**: Dangerous selectors have too broad a reach, such as styling every `div` with specific appearance. These styles leak into places they should not, and then require undoing styles later. Keep selectors scoped and intentional.
- ☐ **Avoid extra selectors**: Extra selectors clutter stylesheets and make CSS less simple and clean. Deep chains like `body #container .someclass ul li` often can be reduced to `.someclass li` with the same result.
- ☐ **Avoid reactive !important**: `!important` is an important tool, but it should be used proactively, not reactively. Proactive usage is for intentional guarantees (for example, error text always red). Reactive usage to escape specificity problems only treats symptoms of ill-formed CSS.
- ☐ **Avoid IDs**: IDs cannot be reused in a page and are far more specific than classes. Their traits can usually be abstracted into reusable classes. For maintainable CSS, avoid IDs as styling hooks.
- ☐ **Avoid loose class names**: Loose names like `.board` or `.user` are vague and easy to accidentally reuse or override. Use stricter class names that communicate purpose clearly. BEM helps create names that are specific and maintainable.
- ☐ **Avoid string concatenation for classes**: Sass `&` concatenation is DRY, but it can hide full class names from source search. If `.foo-bar` is only generated indirectly, searching the CSS codebase becomes harder and tracing style origin is more difficult. See next example:

```
.foo {
  color: red;
  &-bar {
    font-weight: bold;
  }
}
```

- ☐ **Avoid duplicated key selectors**: The key selector is the selector being styled. Repeating the same key selector across multiple contexts creates mutation and removes a single source of truth. Keep shared classes defined consistently to avoid divergent outcomes.
- ☐ **Avoid using inline styles**: Inline styles mix content and presentation, hurt scalability, and reduce maintainability. They force repeated updates across many HTML files instead of one global stylesheet. As a rule of thumb, define styles in CSS files.
- ☐ **Avoid classes in wrong components**: Group styles by subject (the key selector) and place contextual styles where they logically belong. Keeping related component styles together reduces cognitive overhead, makes migration easier, and avoids chasing style fragments across many files.

- ☐ **Avoid shorthand syntax everywhere**: Shorthand syntax reduces keystrokes and bytes but can unset properties we did not intend to modify. Prefer longhand when accuracy matters. Write only as much as needed and avoid hidden side effects.
- ☐ **Avoid too many font files**: Too many fonts make websites chaotic and increase load/render delays. Font requests are often delayed until render tree construction, which can delay text rendering. Include only necessary fonts and optimize WebFont loading and rendering. It is recommended to optimize WebFont loading and rendering.
- ☐ **Prefer class selectors**: Class selectors (`.my-class`) are faster than attribute selectors (`[data-type="value"]`), pseudo-selectors (`:nth-child()`), or element selectors (`div`).
- ☐ **Use multiple classes**: It is often beneficial to apply multiple classes to an element, for example combining a base class and a contextual class. This improves reuse and keeps styling composable. See next example:

```
<div class="box warning"></div>
```

- ☐ **Use nested declarations**: CSS preprocessors improve organization and support nesting so we can see relationships between selectors. Nesting is powerful but easy to overuse. Avoid going deeper than about three levels to limit specificity and keep code readable.
- ☐ **Use "Margin: 0 auto" to center layouts**: Block-level elements cannot be centered with `float: center;`. Use `margin: 0 auto;` so left and right margins are equal and the element centers within its container.
- ☐ **Use Hex Code instead of Name Color**: Prefer hex colors over named colors for consistency and practical performance habits. Use naming conventions for color variables so colors have human-understandable names (for example with a naming reference like Veli's colorpedia).
- ☐ **Use a CSS reset**: Start with a reset stylesheet to standardize browser default differences and create a clean slate for app design. Include reset styles before other styles.
- ☐ **Use a CSS pre-processor**: CSS lacks some programming-language tools like variables and richer abstractions. Preprocessors such as Sass add variables, operators, interpolation, functions, and mixins, making style architecture more readable and easier to maintain.
- ☐ **Use a CSS post-processor**: If CSS load feels slow, compress and optimize CSS output. Minification and compression reduce file size by removing whitespace, line breaks, and redundant rules. Tools like PostCSS, cssnano and clean-css help automate this.
- ☐ **Use a CSS framework**: Frameworks such as Angular Material collect common styles and implement many best practices by default. They can save time and improve consistency, but they are not perfect for every job and can have side effects if used incorrectly.

- ☐ **Use a design system**: A design system is a set of reusable components guided by clear standards. It helps define global rules, modularize decisions, and apply best practices consistently. It is future proof because global changes and fixes become easier.
- ☐ **Use relative units**: Prefer relative units such as `em`, `rem`, `%`, `vw`, `vh`, and `fr`. Fixed units like `px` and `pt` are sometimes needed but should not dominate fluid web layout decisions.
- ☐ **Use CSS variables**: CSS variables remain available in the browser and support flexible, reusable UI design. Unlike preprocessor variables, they do not always require a compile step and are useful for design systems and dynamic features.
- ☐ **Prefer design tokens**: Centralize colors, spacing, typography, radii, shadows, and z-index scales behind tokens or CSS custom properties. Tokens reduce visual drift, make theming possible, and keep component styles aligned with the broader system.
- ☐ **Prefer modern layout primitives**: Use Flexbox, Grid, and `gap` for layout before reaching for floats, absolute positioning, or negative-margin hacks. Modern layout primitives express intent more clearly and usually reduce CSS complexity.
- ☐ **Prefer logical properties**: Prefer properties such as `margin-inline`, `padding-block`, `inset-inline`, and `border-block` over left/right-specific rules when layout should adapt to different writing modes. This improves internationalization support and reduces duplicated directional overrides.
- ☐ **Write descriptive media queries**: Media queries are central to responsive development, but they become complex quickly. Improve maintainability by naming query rules (for example with Sass variables/interpolation) and keeping responsive intent clear.
- ☐ **Use container queries**: When a component should adapt to the space provided by its parent rather than the viewport, prefer container queries over global breakpoints for component-driven responsiveness. This fits component-based architectures well and reduces coupling between page layout and component styling.
- ☐ **Separate global vs local style**: Distinguish global styles from component-specific styles. Keep global rules in dedicated files and keep local component styles near components. This aligns with modern frontend frameworks and improves maintainability.
- ☐ **Respect accessibility preferences**: Preserve visible focus indicators, meet contrast requirements, and support user preferences such as prefers-reduced-motion when adding animations. CSS should improve usability, not make interaction harder for keyboard users or motion-sensitive users.
- ☐ **Minimize expensive properties**: Complex painting properties (`box-shadow`, `filter`, frequent `width`/`height` changes, etc.) can trigger expensive recalculation and repainting. Use them carefully, and use `will-change` only as a last resort performance hint.

- ☐ **Animate compositor-friendly properties**: Prefer animating transform and opacity instead of layout-affecting properties such as top, left, width, or height. This usually produces smoother animations and avoids unnecessary layout and paint work.
- ☐ **Style to be responsive or at least adaptive**: People access apps on many devices and sizes. Responsive design improves user experience and is recommended by Google. If full responsiveness is not planned, keep layouts at least adaptive.
- ☐ **Let the content define the size**: Instead of fixed width and height, use padding and constraints like `max-width/max-height` unless strict size is required. This reduces layout bugs and improves component reusability.
- ☐ **Let the parent take care child position**: In content flow components, avoid embedding positioning and external spacing rules inside child styles. Let container context decide position and distance between elements for better reuse and layout control.
- ☐ **Create the HTML first**: Building the full HTML mockup first can save time because we know the complete structure before deciding CSS details. This supports more holistic, top-down styling decisions.
- ☐ **Combine elements with same styles**: When multiple elements share properties, group selectors rather than repeating declarations. Shared style blocks reduce duplication while allowing specific overrides later. See next example:

```
h1,
h2,
h3 {
  font-family: 'Times New Roman', Times, serif;
}
```

- ☐ **Modularize the styles**: Do not bundle all CSS in one file if it is not needed immediately. Serve only page-relevant styles and consider splitting essential vs non-essential styles (for example hidden dialogs/notifications) to reduce initial load.
- ☐ **Lazy load stylesheets**: Lazy loading defers non-critical resources at page load time. Since CSS is render blocking by default, defer non-critical styles with bundlers, dynamic import, loaders, or link strategies to improve initial rendering. There are many ways to lazy load the CSS files and it is often easier when using bundlers like webpack and playing around with dynamic import.
- ☐ **Remove unused CSS**: Unused CSS adds dead weight, slows performance, and makes maintenance harder. Keep styles clean and orderly by removing unused rules manually or with tools such as PurgeCSS, PurifyCSS, and UnCSS.
- ☐ **Minimize the CSS**: Minimize CSS before loading it in the browser. Smaller files load and process faster. Use post-processors or build steps with tools like webpack CSS minimizer plugin to optimize and minify output.

1.7 Browser APIs

Browser APIs (often called Web APIs) are built-in interfaces provided by web browsers that let JavaScript interact with the browser, the device, the network, and the page itself. They extend the core JavaScript language with capabilities that JavaScript alone does not have. Modern frontend applications leverage many Browser APIs to build rich and performant experiences. Below is a comprehensive reference of Browser APIs grouped by purpose.

Document & DOM APIs:

- DOM API: Manipulate HTML structure. e.g. update the page after user actions without reloading;
- History API: Control browser navigation (SPA routing). e.g. change routes with back/forward support in a single-page app;
- URL API: Parse and construct URLs easily. e.g. read and update query parameters for search and filters;
- MutationObserver: Watch DOM changes efficiently. e.g. react when third-party widgets insert or remove elements;
- Shadow DOM API: Encapsulate component markup and styles. e.g. build reusable UI components without global CSS conflicts.

Storage & Data APIs:

- LocalStorage / SessionStorage: Simple key-value storage. e.g. keep user theme preferences or short-lived form progress;
- IndexedDB: Powerful client-side database. e.g. store large offline datasets for a PWA;
- Cache API: Store HTTP responses (used in service workers). e.g. serve app shell assets offline for faster repeat visits;
- StorageManager API: Manage storage quotas. e.g. estimate remaining storage before caching large files;
- File System Access API: Read/write local files. e.g. implement browser-based editors that open and save local project files;
- Clipboard API: Read/write clipboard content. e.g. support one-click copy for code snippets or invite links.

Network & Communication APIs:

- Fetch API: Modern way to make HTTP requests. e.g. call a REST endpoint to load dashboard data;

- AbortController API: Cancel async operations such as fetches. e.g. cancel stale HTTP requests when users type quickly in an Angular search box;
- WebSocket API: Real-time bidirectional communication. e.g. power live chat, trading feeds, or multiplayer updates;
- Server-Sent Events: Server to client streaming. e.g. stream live notifications or status updates with simpler one-way messaging;
- Beacon API: Send analytics data reliably. e.g. flush usage metrics when users close or navigate away from a page;
- Broadcast Channel API: Communicate across browser tabs/windows of the same origin. e.g. sync logout, theme, or cache refresh events between multiple open app tabs;
- Channel Messaging API: Create structured message channels between browsing contexts. e.g. coordinate secure communication between an app shell, iframe, and worker;
- Network Information API: Detect connection type/speed. e.g. downgrade media quality on slow or metered networks.

Graphics & Media APIs:

- Canvas API: Draw 2D graphics dynamically. e.g. render charts, signatures, or custom image editors;
- WebGL API: Render 3D graphics in the browser. e.g. build interactive 3D product viewers;
- WebGPU API: Next-generation GPU access (faster than WebGL). e.g. run high-performance visualizations or compute-heavy simulations in-browser;
- SVG API: Manipulate vector graphics. e.g. animate scalable icons and diagrams;
- MediaDevices API: Access camera and microphone. e.g. capture webcam input for profile photos or meetings;
- WebRTC API: Real-time video/audio communication. e.g. enable peer-to-peer video calls and screen sharing.

Performance & Background Processing APIs:

- Web Workers API: Run scripts in background threads. e.g. process large JSON or image transformations without blocking the UI;
- Service Workers API: Offline support, caching, and push notifications. e.g. make a PWA installable with resilient offline behavior;
- Background Sync API: Defer network work until connectivity is available. e.g. retry queued form submissions after a mobile device comes back online;
- Periodic Background Sync API: Refresh content in the background on supported platforms. e.g. update cached news or inbox data for a PWA before the user reopens it;

- Performance API: Measure app performance. e.g. track page load milestones and custom timing metrics.

Device & Hardware APIs:

- Geolocation API: Get user location. e.g. show nearby stores or delivery options based on current position;
- Battery Status API: Check battery level (limited support). e.g. reduce background sync when battery is low;
- Device Orientation API: Detect device rotation. e.g. control game interactions by tilting the device;
- Vibration API: Trigger vibration (mobile). e.g. provide haptic feedback after important actions;
- Web Bluetooth API: Connect to Bluetooth devices. e.g. pair a web app with heart-rate sensors or IoT peripherals;
- Web USB API: Interact with USB devices. e.g. communicate with specialized hardware from a browser admin tool.

Security & Identity APIs:

- Web Crypto API: Encryption, hashing, and key generation. e.g. hash sensitive client data before sending it to the server;
- Credential Management API: Handle login credentials. e.g. simplify sign-in with saved credentials and reduced form friction;
- WebAuthn API: Passwordless authentication (biometrics, security keys). e.g. implement phishing-resistant MFA in enterprise apps;
- Federated Credential Management API: Support privacy-preserving federated sign-in flows. e.g. integrate identity providers without relying on third-party cookies;
- Permissions API: Query permission status (camera, location, etc.). e.g. check permission state before showing camera or location prompts.

Payments & Engagement APIs:

- Payment Request API: Streamline browser-native checkout flows. e.g. collect shipping, payer details, and supported payment methods with fewer form fields and less checkout friction;
- Payment Handler API: Integrate web-based payment apps with the browser payment flow. e.g. allow a wallet-style PWA to act as a payment method during checkout;
- Notification API: Display system notifications after permission is granted. e.g. notify users about order status changes or long-running job completion;

- Push API: Receive push messages through a service worker. e.g. send re-engagement or operational alerts even when the app tab is closed.

UI & Interaction APIs:

- Intersection Observer API: Detect element visibility. e.g. lazy load images when they enter the viewport;
- Resize Observer API: Watch element size changes. e.g. recalculate chart layouts when containers resize;
- Fullscreen API: Toggle fullscreen mode. e.g. switch dashboards or media players into distraction-free view;
- Pointer Events API: Unified mouse/touch input. e.g. implement drawing canvases that work consistently across devices;
- Drag and Drop API: Native drag-drop interactions. e.g. reorder Kanban cards or upload files by dropping them in a target area;
- Idle Detection API: Detect user inactivity. e.g. auto-set status to away and pause heavy background tasks;
- Screen Capture API: Capture screen content. e.g. share a browser tab during remote support or online meetings.

1.8 Web Components

1.8.1 Definition

Web Components create reusable HTML elements using Custom Elements, Shadow DOM, and HTML Templates. Web Components are especially useful when we need UI pieces that can be reused across technology boundaries. Web Components are framework-agnostic and work in any modern browser. This makes them ideal for building design systems and shared component libraries that need to be consumed across different frameworks (Angular, React, Vue) or in plain HTML pages.

Web Components are based on three browser primitives:

- Custom Elements: define new HTML tags such as `<customer-card>`;
- Shadow DOM: encapsulate markup and styles to avoid global CSS leaks;
- HTML Templates: define inert markup chunks that can be cloned and rendered.

This model gives us strong isolation and interoperability. A component can expose a stable API via attributes, properties, methods, and custom events, while hiding implementation details.

To share web components between projects, package them as versioned artifacts with clear contracts, following these recommendations:

- Publish to a private npm registry or CDN bundle;
- Version with semantic versioning and keep backward-compatible APIs stable;
- Document inputs, outputs, events, accessibility behavior and theming tokens;
- Provide a changelog and migration notes for breaking changes;
- Add visual and contract tests to detect regressions across hosts.

In enterprise environments, a common pattern is to maintain a "Web Components library" repository alongside the design system. Angular apps can consume those components directly, while non-Angular apps use the same compiled bundle.

1.8.2 Angular Elements

Angular Elements is an Angular package (`@angular/elements`) that allows Angular components to be packaged as custom elements (Web Components). Angular Elements wraps Angular components using the `createCustomElement()` API, which creates a bridge between Angular's component interface and the browser's Custom Elements API.

Use cases for Angular Elements:

- **Embedding Angular components in non-Angular applications**: Angular Elements can be used in CMS platforms, legacy applications, or applications built with other frameworks.
- **Micro-frontend integration**: Angular Elements can serve as the building blocks for micro-frontend architectures, where different parts of the page are built by different teams.
- **Dynamic content**: Content management systems can render Angular-powered interactive widgets alongside static content.
- **Shared component libraries**: Organizations can build a single component library in Angular and distribute it as Web Components for consumption across different technology stacks.

Considerations when using Angular Elements include bundle size (each element carries the Angular runtime unless shared), performance implications of multiple Angular zones, and the need for polyfills in older browsers.

1.8.3 Lit

Lit is a lightweight library for building Web Components using modern browser standards. Lit provides a small reactive layer on top of Custom Elements and Shadow DOM, helping teams

create fast, maintainable, framework-agnostic components with less boilerplate than vanilla Web Components.

Use cases for Lit:

- **Building reusable design system components**: Lit is well suited for creating shared UI primitives (buttons, inputs, dialogs, tables) that can be consumed by Angular, React, Vue, or plain HTML applications.
- **Incremental modernization**: Lit components can be introduced into legacy or multi-framework environments without requiring a full frontend rewrite.
- **Micro-frontend interoperability**: In distributed frontend architectures, Lit provides a neutral component format that different teams can adopt regardless of their framework choices.
- **High-performance UI widgets**: Lit's efficient reactive updates make it a strong option for interactive widgets and component-heavy interfaces where render performance matters.

Considerations when using Lit include defining clear conventions for state and event contracts, ensuring accessibility and theming standards across component libraries, and planning integration patterns in Angular apps (for example, input/output bindings, custom event handling, and form control interoperability).

1.9 Dependency Management

1.9.1 Management Tasks

Dependency management is the set of practices used to choose, install, update, audit and govern third-party packages during the full software lifecycle. In Angular applications, dependency management affects build stability, delivery speed, security posture and long-term maintainability.

Typical dependency management tasks in frontend and Angular projects include:

- Selecting libraries and tools based on maturity, maintenance activity, license and ecosystem fit;
- Adding runtime dependencies (`dependencies`) and development-only dependencies (`devDependencies`) with clear purpose;
- Defining safe version ranges and maintaining lockfiles for deterministic builds;
- Updating dependencies with a regular cadence (planned upgrades instead of ad hoc upgrades);

- Validating upgrades through linting, tests and production builds before merge;
- Auditing known vulnerabilities and handling security advisories with risk-based prioritization;
- Removing unused dependencies to reduce attack surface and bundle size;
- Monitoring package deprecations and migration guides (especially Angular major versions);
- Generating inventory artifacts such as SBOMs (Software Bill of Materials) for governance and compliance.

This discipline becomes increasingly important as projects grow. A medium Angular application can easily include hundreds or thousands of transitive packages. Without governance, teams may face brittle builds, security drift and difficult framework upgrades. For Angular projects, dependency updates should follow Angular update guidance and be validated through linting, tests and production build checks.

To apply these tasks consistently, teams need a package manager as the operational foundation. The package manager is the practical interface used every day to install dependencies, lock versions, run scripts and keep environments reproducible across local development and CI.

1.9.2 Package Managers

1.9.2.1 npm

npm is the default package manager distributed with Node.js. It is the most common baseline in enterprise projects and works out of the box with Angular CLI.

Key characteristics:

- Widely adopted and well documented;
- Uses `package-lock.json` for deterministic installs;
- Supports workspaces for monorepos;
- Integrates naturally with `npm run` scripts and CI pipelines.

Use `npm ci` in CI/CD environments because it installs strictly from the lockfile, improving reproducibility and speed.

1.9.2.2 Yarn

Yarn is an alternative package manager originally created to improve speed and reliability. Modern Yarn versions include advanced workspace and constraint features for larger repositories.

Key characteristics:

- Fast installs with efficient caching;
- Strong monorepo/workspace tooling;
- Supports `yarn.lock` for deterministic installs;
- Provides advanced controls (constraints, plugins, zero-install strategies).

For teams using Yarn in CI, `yarn install --immutable` helps fail fast when lockfile and manifest are out of sync.

1.9.2.3 pnpm

pnpm is a package manager focused on speed and disk efficiency. Instead of copying package contents for each project, pnpm uses a content-addressable store and links dependencies, which can significantly reduce disk usage in multi-project environments.

Key characteristics:

- Very fast installation in large codebases;
- Reduced disk usage via global content-addressable store;
- Strict dependency isolation that helps catch undeclared dependencies;
- Excellent workspace support for monorepos.

In CI, `pnpm install --frozen-lockfile` is the equivalent lockfile-strict workflow.

1.9.3 Dependency Security

Keep dependencies updated with a regular cadence and avoid maintaining old or private versions of Angular or any third-party dependency. Package managers also play a central role in security maintenance:

- **Audit regularly**: Use `npm audit` or `yarn audit` to identify known vulnerabilities in dependencies.
- **Lock files**: Always commit `package-lock.json` or `yarn.lock` to ensure consistent, reproducible builds.
- **Minimal dependencies**: Follow the principle of least privilege for dependencies. Only install what is needed, and prefer well-maintained libraries with active security policies. It will reduce significant the attack surface.
- **Secure scripts**: Review `post-install` scripts and package provenance when introducing new dependencies.

- **Automated scanning**: Integrate tools like Dependabot, Snyk or Socket into CI/CD pipelines for continuous dependency monitoring.

Lockfiles are mandatory for reliable builds and predictable dependency trees. Commit exactly one lockfile to version control and avoid mixing package managers in the same project unless there is a strong and explicit reason.

1.9.4 Dependency Governance

Dependency governance defines the organizational rules that keep package usage controlled, auditable and sustainable over time. While dependency management focuses on day-to-day package operations, governance defines who decides, which rules apply, and how compliance is verified.

An important governance artifact is the SBOM (Software Bill of Materials). SBOM provides a machine-readable inventory of project components and versions, which supports security response, compliance audits and incident analysis.

The script below shows a simple way to generate the SBOM file:

```
{
  "scripts": {
    "sbom": "cyclonedx-npm --output-file dist/sbom.json"
  }
}
```

CycloneDX is one of the most widely used SBOM standards for modern software supply chains. It defines a machine-readable format for describing application components, versions, dependencies, licenses and security-relevant metadata. In JavaScript and Angular projects, tools such as cyclonedx-npm can generate a CycloneDX SBOM directly from `package.json` and the lockfile, while platforms such as Dependency-Track can ingest that SBOM to support vulnerability management, compliance checks and ongoing governance.

1.10 Version Control

1.10.1 Version Control Systems

A version control system (VCS) is the software that manages project changes, allowing developers to store versions, compare updates, and revert when necessary. There are four main types of version control systems: distributed, centralized, lock-based, and optimistic. Each offers different approaches to managing changes across teams.

A VCS tracks every alteration to a file or set of files, enabling developers to journey back to earlier versions and collaborate seamlessly. Centralized version control systems (CVCS) streamline this process by housing all file versions on a single server. Developers borrow a file to tweak, then return it with updates, all neatly stored and cataloged by the server. This method shines in its simplicity, offering a straightforward path for managing changes.

Git is the dominant tool, but it is not the only version control system. Other alternatives exist and can still be found in some organizations or legacy environments:

- SVN, a centralized version control system that is simpler in some workflows but less flexible for distributed collaboration;
- Mercurial, another distributed version control system known for a simpler command model, though it has a much smaller ecosystem than Git today;
- Perforce Helix Core, often used in large enterprise, gaming or asset-heavy environments where centralized control and large binary handling are important;
- Fossil, a distributed system that bundles version control, wiki and issue tracking into a single tool.

1.10.2 Git

1.10.2.1 Fundamentals

Git is a distributed version control system used to track changes in source code, coordinate work between developers, and maintain a reliable history of how a project evolves over time. In a distributed model, every developer has a full copy of the repository and its history locally, which makes branching, inspecting history, and working offline fast and practical.

Git became the standard choice for modern software teams because it supports both individual productivity and team collaboration. It allows developers to experiment safely in branches, review changes before integration, and return to previous versions when a defect is introduced.

In day-to-day development, a team normally uses only a subset of Git commands regularly: `clone`, `status`, `add`, `commit`, `pull`, `push`, `branch`, `checkout or switch`, `merge`, `rebase`, `log` and `diff`. Mastering these commands is enough to work effectively in most projects. More advanced commands become relevant when resolving conflicts, cleaning up commit history, or recovering from mistakes.

Besides command-line tools, there are also graphical clients that sit on top of Git and can improve usability for some developers. Popular examples include GitHub Desktop, Sourcetree, GitKraken and built-in Git integrations in IDEs such as VS Code and IntelliJ.

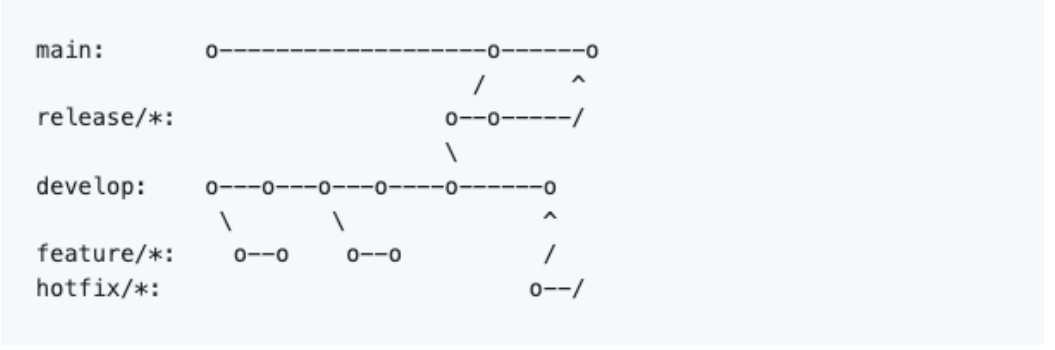
These tools do not replace Git itself, but they provide a visual interface over the same repository operations.

Git combined with a hosted platform such as GitHub, Bitbucket or GitLab is the most practical default. It supports feature branches, pull requests, automated checks, release tagging and team collaboration patterns that fit well with continuous integration and continuous delivery.

1.10.2.2 Git Branching Strategies

Different teams adopt different branching strategies depending on release cadence, team size, and governance requirements. The most common strategies are:

- **Git Flow:** Uses long-lived `main` and `develop` branches, with short-lived `feature/*`, `release/*`, and `hotfix/*` branches. Release and hotfix merges keep `main` and `develop` aligned. It is structured and predictable, but can feel heavy for fast delivery cycles. Good for enterprise applications.



- **Trunk-Based Development:** Developers integrate frequently into a single trunk branch (usually `main`), using very short-lived branches or direct commits with feature flags. It optimizes for continuous integration and rapid feedback. Recommended for mature engineering teams.



- **Release Branching:** A dedicated `release/x.y` branch is cut from `main` to stabilize a version while ongoing development continues elsewhere. Selected fixes can then be forward-merged into the next development line. It is useful when multiple versions must be maintained. Good for enterprise support.

```
main:          o---o---o-----------o
                    \             /
release/1.4:         o---o---o---/
                       \   \
fix branches:            o   o--o
                          \
next-dev:      o---o---o---o---o---o
```

- **Forking Workflow:** Common in open-source where each contributor works in a personal fork and opens pull requests to the upstream repository. It provides strong isolation and permission control for external contributions. Recommended for open-source projects.

```
upstream/main:   o---o---o---o---o
                     ^       ^
                     |       |
alice fork/main: o---o---o---o
                      \
alice feature:         o---o   -> PR to upstream/main
```

1.10.2.3 Git Rebase with Squash

Rebasing and merging are both designed to integrate changes from one branch into another branch but in different ways. The merge will result as a combination of commits, whereas rebase will add all the changes in feature branch starting from the last commit of the master branch. So:

- When we do rebase a feature branch onto master, we move the base of the feature branch to master branch's ending point.
- Merging takes the contents of the feature branch and integrates it with the master branch. As a result, only the master branch is changed. The feature branch history remains same.
- Merging adds a new commit to the branch history.

The major benefit of rebasing is that it gives us a much cleaner project history. First, it eliminates the unnecessary merge commits created by `git merge`. Second, rebasing results in a perfectly linear project history. Rebase presents conflicts one commit at a time, so it is recommended to squash commits before rebasing. This way, we get a linear and more comprehensive history. When we need to find a bug or revert a user story, reverting a single commit is much easier than reverting dozens. The worst-case scenario is having a long list of commits for a single user story, where we see every step of the developer's iterative process. Squashing removes these

unnecessary commits from the Git history, which is generally considered a best practice. Whether we use merge or rebase, the important thing is that, in the end, we squash the commits.

1.10.3 Conventional commits

Conventional Commits specification is a lightweight convention on top of commit messages. It provides an easy set of rules for creating an explicit commit history, which makes it easier to write automated tools on top of. The commit message should be structured as follows:

```
<type>([optional scope]): <description>
<BLANK LINE>
[optional body]
<BLANK LINE>
[optional footer]
```

Each commit message consists of a header, a body and a footer. The header has a special format that includes a type, a scope and a description. The header is mandatory, and the scope of the header is optional. Commit types allowed are: *build*, *ci*, *revert*, *fix*, *feat*, *chore*, *docs*, *style*, *refactor*, *perf* and *test*. By adding a git hook with husky, we can run custom scripts on the commit before letting it through. Husky can prevent bad *git commit*, *git push* and more. Next configuration allows us to add git hooks directly into the *package.json* via the *husky.hooks* field.

```
{
  "husky": {
    "hooks": {
      "commit-msg": "commitlint -E HUSKY_GIT_PARAMS"
    }
  }
}
```

It´s recommended to enforce commit conventions by linting on CI servers with commitlint. It's important to use conventional commits to:

- Automatically generating CHANGELOGs;
- Automatically determining a semantic version bump (based on the types of commits landed);
- Communicating the nature of changes to teammates, the public, and other stakeholders;
- Triggering build and publish processes;
- Making it easier for people to contribute to our projects, by allowing them to explore a more structured commit history.

1.10.4 Changelog

Following the Semantic Versioning specification helps other developers who depend on the code to understand the extent of changes in a given version and adjust their code if necessary. Conventional changelog is a tool for generating a *CHANGELOG.md* from git metadata. Using npm scripts effectively, we have:

```
{
  "scripts": {
    "version": "conventional-changelog -p angular -i CHANGELOG.md -s"
  }
}
```

This tool only works when we follow Conventional Commits rules.

1.11 Code Quality

1.11.1 Linters

Linters are automated tools that analyze source code and report problems based on predefined rules. They help detect bugs, unsafe patterns, inconsistent style, and maintainability issues before code reaches production. Linters are commonly used for TypeScript, HTML, CSS, JSON, and Markdown, and they are often integrated with editors, pre-commit hooks, and CI pipelines.

In the next table, we can see some of these tools and suggested npm scripts:

Language	Libraries	Recommended scripts
TypeScript	ESLint, typescript-eslint, Prettier	`"lint:ts": "eslint \"src/**/*.ts\"",` `"lint:ts:fix": "eslint \"src/**/*.ts\" --fix"`
HTML	HTMLHint, Prettier	`"lint:html": "htmlhint \"src/**/*.html\"",` `"lint:html:fix": "prettier --write \"src/**/*.html\""`
CSS	Stylelint, stylelint-config-standard, Prettier	`"lint:css": "stylelint \"src/**/*.css\"",` `"lint:css:fix": "stylelint \"src/**/*.css\" --fix"`
Gherkin	gherkin-lint, prettier-plugin-gherkin	`"lint:gherkin": "gherkin-lint \"**/*.feature\"",` `"lint:gherkin:fix": "prettier -write \"**/*.feature\""`
JSON	eslint-plugin-jsonc, Prettier	`"lint:json": "eslint \"**/*.json\"",` `"lint:json:fix": "eslint \"**/*.json\" --fix"`
Markdown	markdownlint-cli2, Prettier	`"lint:md": "markdownlint-cli2 \"**/*.md\"",` `"lint:md:fix": "prettier --write \"**/*.md\""`

These tools are recommended for use in frontend projects because:

- They keep everybody on the same page, following the same rules;
- They save time in code reviews, because we can safely ignore all style issues, and focus on things that matter, like the structure and semantics of the code;
- They catch errors. Prettier, not so much, but ESLint catches a lot of syntax errors and simple forms of type errors, such as undefined variables;
- Setting these things up is a one-time cost, but the time-saving benefits compound over time.

For Angular projects, we can supplement the previous list with angular-eslint and eslint-plugin-rxjs plugins. Additionally, for testing frameworks such as Jest and Playwright, we can use eslint-plugin-jest and eslint-plugin-playwright plugins.

1.11.2 ESLint + Prettier

ESLint and Prettier help us maintain code quality. All code in any codebase should look like a single person typed it, no matter how many people contributed. Besides checking style, linters are also excellent tools for finding certain classes of bugs, such as those related to variable scope. The code formatting is ensured by Prettier.

First, ensure the terminal is in the project root directory and then use npm to add Prettier and ESLint packages.

```
npm i -D prettier \
  eslint \
  @eslint/js \
  typescript-eslint \
  eslint-config-prettier \
  eslint-plugin-prettier
```

On the root of the project, create a file called *.prettierrc.json* with these prettier options:

```
{
  "endOfLine": "lf",
  "printWidth": 120,
  "singleQuote": true,
  "trailingComma": "all"
}
```

Then, create a file called *eslint.config.ts* with this structure:

```
export default defineConfig([
  {
    files: ['**/*.ts'],
    extends: [
      eslint.configs.recommended,
      tseslint.configs.recommendedTypeChecked,
      tseslint.configs.stylistic,
      prettier,
    ],
    plugins: {
      prettier: prettierPlugin,
    },
    rules: {
      'prettier/prettier': 'error',
    },
  },
]);
```

Finally, we just need to add a "lint" script to *package.json* as described in the table above. Prettier takes care of formatting, while ESLint handles code quality and style consistency.

1.11.3 IDE Integration

Depending on the code editor, we may want to add an extension to lint and format the code while we type or on-save. It's recommended to setup the editor to automatically run ESLint fix command (e.g., `eslint --fix`) and Prettier whenever a file is saved.

When adding an existent project, we will most likely see some errors. We can fix some of these errors by simply re-running the same command with the *--fix* option. It's recommended not to use the fix option until we have had a chance to look at the code that is causing the errors. That will give us a better understanding of why we are seeing the errors. Also, we may need to adjust some of the rules based on team preferences.

Alternatively, EditorConfig helps maintain consistent coding styles for multiple developers working on the same project across various editors and IDEs. The EditorConfig project consists of a file format for defining coding styles and a collection of text editor plugins that enable editors to read the file format and adhere to defined styles. EditorConfig files are easily readable and they work nicely with version control systems. However, with ESLint and Prettier, we no longer need to configure EditorConfig.

1.11.4 Git Hooks and CI

To ensure all files committed to git don't have any TypeScript, linting, or formatting errors, there is a tool called lint-staged that can be used. lint-staged allows to run linting commands on files

that are staged to be committed. When lint-staged is used in combination with husky, the linting commands specified with lint-staged can be executed to staged files on pre-commit.

To configure lint-staged and husky, add the following configuration to the *package.json* file:

```
{
  "husky": {
    "hooks": {
      "pre-commit": "lint-staged"
    }
  },
  "lint-staged": {
    "*.ts": ["eslint --fix", "prettier --write", "git add"],
    "*.html": ["prettier --write", "git add"],
    "*.css": ["stylelint --fix", "prettier --write", "git add"]
  }
}
```

The above configuration will run lint-staged when a user tries to commit. lint-staged will then run ESLint on any staged files with *.ts* extension. Any errors that can be fixed automatically will be fixed and added to the current commit. However, if there are any linting errors that cannot be fixed automatically, the commit will fail and the errors will need to be manually fixed before trying to commit the code again.

Unfortunately, it is not enough to only rely on lint-staged and husky to prevent linting errors because the git hooks can be by-passed if a user commits with the *--no-verify* flag. Therefore, it is also recommended to run a command on a Continuous Integration (CI) server that will verify that there are no linting errors.

1.11.5 SonarQube

SonarQube is a platform for continuous inspection of code quality. It analyzes source code and reports issues such as bugs, vulnerabilities, security hotspots, code smells, duplicated code and test coverage trends. In frontend and Angular projects, SonarQube complements tools like ESLint, Stylelint and unit tests by adding a broader quality view and enforcing quality gates in CI/CD. This helps teams detect maintainability and reliability problems early, prevent low-quality code from being merged and keep code quality visible over time. To integrate SonarQube into a JavaScript or Angular project, we can use the sonarqube-scanner package and expose it through an npm script.

The command below shows how to run the npm script in CI, pass the branch name to SonarQube server, authenticate with a repository secret and wait for the quality gate result.

```
npm run sonar -- -Dsonar.branch.name=<branch> -Dsonar.login=<sonar-token> \
  -Dsonar.qualitygate.wait=true
```

IDE plugins improve this workflow by surfacing quality issues before code reaches CI. SonarQube for IDE for VS Code, IntelliJ-based IDEs and other editors can highlight bugs and code smells directly in the editor. When connected to the SonarQube server, these plugins synchronize quality profiles and server-side rules, which helps developers fix problems locally before opening a pull request.

1.12 User Experience

1.12.1 Fundamentals

User Experience (UX) is the discipline of designing products that are useful, usable, accessible and pleasant to interact with. In web applications, UX is not limited to visual appearance. It includes how quickly users understand the interface, how efficiently they can complete tasks, how the application behaves in edge cases, how errors are communicated, how usable the experience is, and how confident users feel while using the product.

A user-centered design approach starts from real user goals, context, and constraints instead of internal technical preferences. Teams can align design and implementation by using personas, user journeys, and task flows to define navigation, content hierarchy, and interaction priorities before coding. These assumptions should then be validated continuously through usability sessions, analytics, A/B testing, support feedback, and behavioral data.

Modern UX must adapt to different screens, input modes, performance budgets, and user contexts. Responsive design is not just resizing layouts. It also includes adapting navigation patterns, information density, and interaction models for touch, keyboard, and hybrid usage, often with a mobile-first mindset. Inclusive experiences further require support for multiple languages, localization, right-to-left layouts, readable formatting, and tolerance for low-end devices or unstable connections.

UX quality should be measured continuously with both qualitative and quantitative evidence. Qualitative methods include usability testing, moderated interviews, heuristic evaluation and accessibility audits. Quantitative methods include funnel analysis, task success rate, error rate, bounce rate, retention, conversion, Core Web Vitals and support ticket trends.

Users perceive performance through responsiveness, stability, and smoothness, not only through technical benchmarks. Interfaces that block input, shift layout unexpectedly, or delay feedback feel unreliable even when requests eventually succeed, so UX teams should monitor quality with Core Web Vitals, Lighthouse, and the RAIL model. Practical priorities include fast initial rendering, quick interaction feedback, stable layout, progressive loading, and resilience on poor networks.

1.12.2 Design Systems

A design system is a shared language of reusable components, patterns, design tokens, principles and documentation that ensures consistency across products and teams. It usually includes typography scales, color palettes, spacing rules, iconography, accessibility rules, interaction patterns and coded UI components. In modern frontend development, design systems are a strategic asset because they reduce duplication, improve coherence and accelerate delivery.

A comprehensive design system typically includes:

- **Design tokens**: Abstract values for colors, spacing, typography, shadows, and other visual properties, stored in a platform-agnostic format that can be consumed by CSS, JavaScript, and design tools.
- **Component library**: A set of reusable UI components (buttons, inputs, modals, tables, etc.) implemented in code with well-defined APIs, accessibility support, and documentation.
- **Style guide**: Guidelines for visual design, branding, tone of voice, and iconography.
- **Pattern library**: Documented UI patterns for common scenarios such as forms, navigation, data display, error handling, and empty states.

In the Angular ecosystem, Angular Material is the most widely used component library, implementing Google's Material Design specification. The Angular CDK (Component Dev Kit) provides behavior primitives (overlay, drag-and-drop, virtual scrolling, accessibility utilities) that can be used to build custom design systems without adopting Material Design aesthetics.

1.12.3 Design Tools

Design tools are essential to transform product requirements into clear, testable and implementable user interfaces. In frontend projects, these tools are not only for visual design. They also support prototyping, collaboration, handoff to engineering, design system governance and user validation.

Core UI/UX design tools:

- Figma: The most used collaborative interface design tool. It supports wireframing, high-fidelity UI design, interactive prototyping, design systems, developer handoff and team collaboration in the browser.
- Sketch: A mature macOS-first UI design tool widely used in product teams. Strong for component-based design libraries, symbols and plugin-based workflows.
- Adobe XD: UI/UX design and prototyping tool from Adobe with support for flows, transitions and design specs.
- Penpot: Open-source collaborative design and prototyping tool, useful for teams that prefer open standards and self-hosting options.

Before high-fidelity screens, teams usually align on flows and concepts using whiteboarding tools:

- FigJam: Collaborative workshops, user flows, affinity maps and product discovery artifacts;
- Miro: Remote whiteboards for journey mapping, service blueprints and cross-team ideation.

These tools are useful to align product, UX and engineering before moving into implementation details. A good design workflow provides reusable components, interaction details, accessibility guidance and responsive behavior definitions that can be mapped directly to Angular components, routes and states.

1.12.4 Best Practices

The following practices summarize how to design web user interfaces that are consistent, accessible, and resilient across real usage conditions:

- ☐ **Start with user goals:** Before implementing a feature, define the primary user goal, critical tasks, expected context of use, and main risks so teams do not optimize for internal complexity instead of user value. Before building something like a vehicle configuration screen, clarify what the user must accomplish, expected task duration, and what happens if a session is lost midway, because those answers drive route structure, form persistence, guard logic, and error recovery decisions that are hard to retrofit later.
- ☐ **Build with consistency**: Use a design system, shared components, and documented interaction patterns so users learn interface rules once and apply them everywhere, reducing cognitive load. If the primary action is always the rightmost button in dialogs, users focus on the task instead of searching. Inconsistent validation timing, icon meanings, or destructive action placement increases errors and support requests.

- ☐ **Design all states**: Do not design only the happy path. Components that load data, accept input, or perform actions must deliberately handle loading, empty, error, offline, partial, disabled, and permission-denied states. A data table should show a skeleton while fetching, an empty state with call to action when no records exist, and a friendly retryable error on failure. In Angular, `@defer` with `@loading`, `@error`, and `@placeholder` provides structure for these states.
- ☐ **Prefer accessibility by default:** Start with semantic HTML, test keyboard navigation, ensure visible focus, verify contrast, label controls correctly, and treat WCAG compliance as a baseline requirement. In Angular, the CDK a11y module offers `FocusTrap`, `LiveAnnouncer`, and `FocusMonitor` for complex focus management in dialogs and drawers. Running Lighthouse and axe in CI helps detect accessibility regressions before production.
- ☐ **Reduce cognitive load**: Keep interfaces focused, group related information, use progressive disclosure, avoid unnecessary choices, and write clear microcopy. Cognitive load rises when users face too many options, vague labels, distant related controls, or unclear next steps. A complex vehicle order form should be split into steps so only controls relevant to the current step are visible.
- ☐ **Make feedback immediate**: Users should always understand whether the system is working, waiting, successful, or blocked because delayed or ambiguous feedback damages trust. If a button shows no visible response for two seconds, users may click again and create duplicate submissions. Show loading indicators within 100ms, confirm meaningful success outcomes, and provide errors that explain what went wrong and what to do next. Angular Material snack bars and the `HttpInterceptor` pattern are common tools for global feedback.
- ☐ **Optimize for real conditions**: Test on mobile devices, slower networks, large datasets, zoomed interfaces, and assistive technologies because UX quality depends on real usage conditions, not ideal environments. A feature usable on a fast desktop can fail on a mobile phone with 3G. Large tables need virtual scrolling like Angular CDK virtual scroll. Use Chrome DevTools network throttling, Angular SSR and hydration, and browser zoom at 200% to surface issues early.
- ☐ **Measure and iterate**: Observe user behavior, collect evidence, validate assumptions, and refine continuously since UX maturity comes from repeated improvement, not one-time design decisions. If analytics show users abandon a multi-step form at the same step, investigate complexity, unclear labels, or weak validation messaging. Combine Core Web Vitals, funnel drop-off, and error logs with usability sessions and interviews. Treat changes as hypotheses: implement, measure impact, then keep, refine, or revert.

1.13 Artificial Intelligence

AI can accelerate Angular development when used as an assistant for ideation, implementation, debugging, testing and documentation. Angular now provides dedicated guidance in Build with AI, which should be treated as the baseline reference for AI-assisted workflows. The key is to keep engineers in control where AI proposes and then developers validate. In practice, this means asking AI to generate a first draft and then reviewing architecture, security, accessibility, performance and coding standards before merging.

Where AI helps most in Angular projects:

- **Scaffolding and boilerplate**: Generate initial standalone components, services and route definitions aligned with Angular CLI, Angular Router and Standalone APIs;
- **Refactoring**: Convert imperative code into more declarative patterns using Signals, RxJS and modern template control flow;
- **Testing**: Create initial unit, integration and end-to-end test cases using Angular testing guidance and recommended tools like Jest and Playwright;
- **Debugging**: Analyze stack traces and runtime errors, then propose likely root causes in change detection, routing, dependency injection or reactive state;
- **Documentation**: Produce README sections, migration notes and code comments that reference Angular style guide conventions.

Angular AI workflows are most effective when connected to real project context through tools, not only free-form prompts:

- **MCP (Model Context Protocol)**: MCP servers can expose project-aware capabilities to AI agents, such as reading local Angular source code, running searches, invoking build/test commands and checking outputs. In an Angular project, this allows an agent to inspect route trees, component templates, form logic and test results before suggesting fixes;
- **Agent skills**: Skills are reusable instruction bundles for recurring tasks (for example: create a standalone feature, migrate from observable-heavy component state to Signals, or generate tests for reactive forms). They improve consistency and reduce prompt duplication across teams;
- **IDE agents**: In editors like VS Code, an agent can combine chat reasoning with concrete actions (file edits, CLI execution, test runs and diagnostics), which is especially useful for Angular migrations, refactors and bug-fix loops.

These capabilities should complement Angular official guidance, especially Build with AI and CLI documentation. To reduce risk, treat AI output as draft code and enforce normal engineering standards (code review, static analysis, tests and architectural review).

1.14 Final Remarks

In this chapter, we covered a practical overview of modern web development with Angular, grounded in Frontend Developer Handbook 2024 and Developer Roadmaps. The path starts with strong foundations in HTML, CSS, JavaScript, browser and server concepts, then evolves into Angular-focused development with TypeScript, RxJS, state management, testing, performance, and modern Browser APIs.

We also connected engineering fundamentals with delivery practices such as maintainable CSS and TypeScript patterns, web components, dependency management, linting and formatting, version control conventions, and automation through scripts, hooks, and CI checks. Finally, we emphasized that high-quality Angular applications are not built with code alone, but also with security, UX, accessibility, design systems, and responsible AI-assisted workflows. The objective is to build applications that are scalable, testable, maintainable, secure, and valuable to users.

2 Angular Framework, Style Guide and Best Practices

2.1 Overview

Angular is an open-source, TypeScript-based frontend web framework led by Google. It is used to build scalable, enterprise-grade single-page and dynamic web applications.

Angular is a mature ecosystem well suited for building scalable applications, offering strong tooling, robust testing capabilities, and long-term maintainability. TypeScript remains central to the Angular developer experience, enabling strict typing and safer refactoring at scale. In modern projects, teams commonly combine Angular with Sass for styling, RxJS, NgRx for state management depending on application complexity, Jest for unit and integration testing, Playwright for end-to-end testing, ESLint and Prettier for code quality, and Storybook for UI component development and documentation.

This chapter consolidates practical guidance for building modern Angular applications in a consistent, scalable, and maintainable way. The content covers Angular fundamentals, RxJS and Signals usage, style guide recommendations, and implementation practices that improve code quality, performance, testability, and long-term maintainability, based primarily on the official Angular documentation.

2.2 Angular Framework

2.2.1 Ecosystem

Angular is a development platform focused on productivity, performance, and testability. It is a TypeScript-first framework led by the Angular Team at Google and supported by an active open-source community. Applications built with Angular can target the web first and be adapted for mobile and desktop distribution when needed.

Angular may have a steeper initial learning curve than React and Vue, but it provides an integrated platform for routing, forms, HTTP, server-side rendering, hydration, and robust tooling. This integrated approach is valuable for teams building long-lived enterprise applications with clear conventions and predictable upgrades. Like other modern frameworks, Angular uses a Component-Based Architecture where the UI is modeled as reusable and composable building blocks. Angular includes first-class support for reactive programming with RxJS and complements it with Signals for fine-grained local reactivity. The same architecture can serve web, mobile, and desktop strategies depending on product requirements.

Angular does a lot of things, but maybe the next picture can give us a better idea:

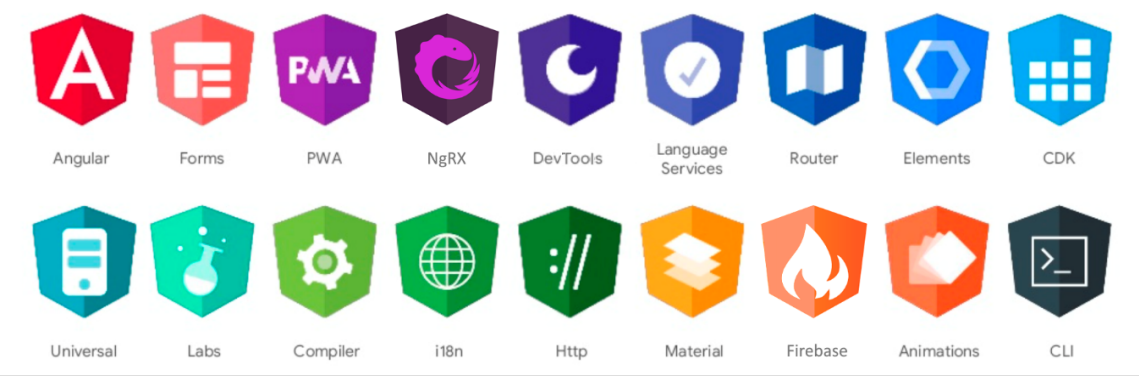


Fig. 10: Angular Ecosystem

As we can see, Angular provides a full-fledged platform with built-in features and conventions that help address non-functional requirements such as maintainability, testability, and cross-browser compatibility. With the Angular CLI, teams can scaffold projects, generate code, run tests, and produce optimized builds.

2.2.2 Features & Benefits

Cross Platform:

- **Web-first architecture**: Build responsive, installable web applications with PWA capabilities (offline support, caching strategies, background sync, and home-screen installation);
- **Mobile strategies**: Use Angular with Ionic and Capacitor to package web applications for iOS and Android while sharing most business and UI logic;
- **Desktop strategies**: Reuse Angular applications in desktop shells such as Electron or Tauri when native distribution is required.

Modern Performance:

- **Standalone-first architecture**: Build features with standalone components, directives, and pipes to reduce module boilerplate and improve lazy-loading boundaries;
- **Rendering flexibility**: Choose CSR, SSR, or prerendering per route and combine with hydration to improve Core Web Vitals and SEO for public pages;
- **Reactive UI updates**: Use Signals for fine-grained state updates and predictable synchronous derivations;
- **Smarter loading**: Route-level lazy loading, deferred views, and modern build tooling reduce initial bundle size and speed up first load.

Developer Productivity:

- **Strong TypeScript integration**: Strict typing in components, templates, forms, and API contracts improves refactoring safety and long-term maintainability;
- **Modern template syntax**: Cleaner control flow and template primitives improve readability and reduce template complexity;
- **Tooling and DX**: Angular CLI and ecosystem tooling provide fast local feedback, code generation, linting, testing, and production-ready builds;
- **IDE support**: Excellent language service support enables autocompletion, diagnostics, symbol navigation, and template-aware type checking.

Complete Engineering Story:

- **Testing**: Combine unit and integration testing using Jest with end-to-end testing using Playwright for reliable quality gates;
- **Accessibility and UX quality**: Build inclusive interfaces with semantic HTML, ARIA where needed, keyboard support, and automated accessibility checks;

- **Security and maintainability**: Leverage Angular defaults and patterns for sanitization, DI boundaries, route guards, HTTP interceptors, and consistent architecture at scale;
- **Long-term support**: Angular's regular release cadence, migration tooling, and official documentation make enterprise upgrades and maintenance more predictable.

2.2.3 Architecture

To build an Angular application, we define a set of components for each UI element, screen, and route. An application always has a root component that contains all other components. In other words, every Angular application has a component tree.

A component is a small unit that makes up part of the user interface:

- Components define views, which are sets of screen elements that Angular can render and update according to program logic and data;
- Components use services, which provide specific functionality not directly related to views. Service providers can be injected into components as dependencies, making code modular, reusable, and efficient.

Both components and services are classes with decorators that mark their type and provide metadata that tells Angular how to use them:

- The metadata for a component class associates it with a template that defines a view. A template combines standard HTML with Angular directives and binding markup that allow Angular to update the HTML before rendering it;
- The metadata for a service class provides the information Angular needs to make it available to components through dependency injection (DI).

The following diagram shows how these basic pieces are related.

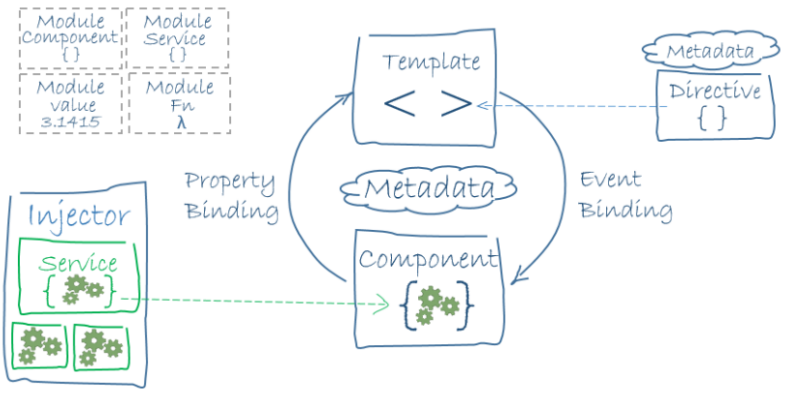


Fig. 2: Angular Architecture

A typical app defines many views arranged hierarchically. Angular provides the Router service to help us define navigation paths between views. The router provides sophisticated in-browser navigation capabilities. In modern Angular, Standalone APIs are the default recommendation and can be used without using NgModules. Both approaches are valid, and many enterprise applications use a hybrid model while migrating.

2.2.4 OOP vs FP style

Angular started with a very strong object-oriented style. Components, services, guards and modules were modeled as classes, with dependency injection as a central architectural pattern. This approach made Angular familiar to teams coming from Java or C#, and it helped structure large applications with clear responsibilities.

Over time, the Angular ecosystem moved to a more functional mindset for day-to-day implementation. While classes still exist and remain important, modern Angular encourages developers to write predictable logic using pure functions, immutable updates, and reactive composition.
In practice, this means:

- Keep components thin and declarative, delegating transformation logic to pure functions;
- Prefer composing streams with RxJS operators instead of imperative nested subscriptions;
- Model local state with Signals and computed values, avoiding mutable shared state;
- Derive UI state from inputs and streams, instead of manually synchronizing many class fields;
- Favor deterministic, testable functions over side-effect-heavy methods.

Angular today is not "OOP versus FP" as a strict choice. The framework still uses object-oriented building blocks (classes, DI, decorators), but the recommended coding style inside those blocks is increasingly functional and reactive. A practical rule is: use OOP for structure and boundaries, and use FP for behavior, state derivation and data flow. This hybrid approach improves readability, testability and performance, especially in large applications where predictable state transitions and explicit data flow are essential.

2.2.5 Dependency Injection

Dependency Injection (DI) is a design pattern we use to organize and share code across an application by supplying dependencies to a class instead of creating them inside it. In Angular, a class requests a dependency through the `constructor` or `inject()`, and Angular resolves it

from the nearest injector. This makes dependencies explicit, keeps classes loosely coupled, and lets us control scope and lifetime.

When we want consumers to depend on an abstraction instead of a concrete implementation, we can use an abstract class as the DI token. Unlike interfaces, abstract classes exist at runtime, so Angular can resolve them directly. This keeps consumers decoupled while allowing the provider configuration to choose the concrete implementation. See next example:

```
export abstract class Logger {
  abstract log(message: string): void;
}

@Injectable()
export class ConsoleLoggerService extends Logger {
  log(message: string): void {
    console.log(message);
  }
}

@NgModule({
  providers: [{
    provide: Logger, useClass: ConsoleLoggerService
  }]
})
export class AppModule { }
```

A consumer can then inject `Logger` through its constructor, and Angular will provide the class mapped to that token, wick is `ConsoleLoggerService`. The main advantages of this technique are cleaner constructor syntax, lower coupling between consumers and concrete services, easier test doubles in unit tests, and simpler implementation swaps per module or environment without changing component code. A provider registered at the root is usually shared across the whole application, while providers declared at route, feature, or component level create narrower instances for those boundaries.

2.3 Angular Core Concepts

2.3.1 Components

A Component is a reusable piece of a web application's UI that combines layout, behavior, and styling into one unit. Every Angular application has at least one component, the root component that connects a component hierarchy with the Document Object Model (DOM). Each component defines a class that contains application data and logic and is associated with an HTML template that defines a view to be displayed in a target environment. The *@Component()* decorator

identifies the class immediately below it as a component and provides the template and related component-specific metadata.

A well-structured component should focus on rendering and user interaction. Business rules, data access, and orchestration should usually be delegated to services. See next example:

```
@Component({
  selector: 'hello',
  template: `
    @if (showGreeting) {
      <div>Hello</div>
    }
  `,
})
export class HelloComponent {
  showGreeting = true;
}
```

Templates are where Angular binds component state to the DOM. Data binding keeps the UI declarative and predictable by defining how values flow between class and template. The most common binding modes are:

- **Interpolation**: `{{ value }}` for text output;
- **Property binding**: `[disabled]="isSaving"` for DOM/component properties;
- **Event binding**: `(click)="save()"` for user interactions;
- **Two-way binding**: `[(ngModel)]="model.name"` for editable fields (template-driven forms).

For performance-sensitive views, prefer OnPush change detection and immutable state updates. Also, prefer keeping complex presentation logic in computed values, signals, or pure pipes instead of embedding heavy expressions in templates.

2.3.2 Services

A Service is a reusable class that handles shared logic, data, or functionality across multiple components. A service class definition is immediately preceded by the `@Injectable()` decorator. The decorator provides the metadata that allows other providers to be injected as dependencies into the class. Services let us keep component classes lean and efficient.

Service example using `HttpClient` with explicit typing:

```
@Injectable({ providedIn: 'root' })
export class UsersService {
  constructor(private readonly http: HttpClient) {}

  getUsers(): Observable<User[]> {
    return this.http
      .get<UserDto[]>('/api/users')
      .pipe(map(users => users.map(u => ({ id: u.id, fullName:
`${u.first_name} ${u.last_name}` }))));
  }
}
```

Services can be scoped depending on lifecycle and sharing needs:

- **`providedIn: 'root'`**: Singleton for the whole application;
- **Component `providers`**: One instance per component subtree;
- **Route providers**: Scoped to a lazy route boundary.

We can also create a service by adding the `@Service()` decorator to a TypeScript class. This tells Angular that we can use the class as an injectable dependency. However, `@Service()` is not a complete replacement for `@Injectable()` because it is designed around the newer `inject()` API.

2.3.3 Directives

Directives are classes that attach behavior to existing DOM elements. They are ideal when we want to reuse UI behavior without creating a full component. Directives keep templates clean while centralizing repeatable behavior in a single testable unit.

The next example shows the signature of a directive for highlighting:

```
@Directive({
  selector: '[highlightOnFocus]',
})
export class HighlightOnFocusDirective { }
```

In practice, Angular directives are commonly used in two forms:

- **Attribute directives**: Change appearance or behavior of an existing element (for example, highlighting, permission-based disablement, input masking);
- **Structural directives**: Add, remove, or reshape DOM layout (for example, `@if`, `@for`, and related control-flow syntax).

As a rule of thumb, keep directives focused, side-effect free when possible, and centered on one behavior.

2.3.4 Pipes

Pipes transform values for display in templates while keeping components focused on state and interaction. They are ideal for formatting concerns such as dates, currency, labels, percentages, and small presentation-specific mappings.

Next example shows a pipe to transform the input into a full user name:

```
@Pipe({
  name: 'fullName',
})
export class FullNamePipe implements PipeTransform {
  transform(user: { firstName: string; lastName: string }): string {
    return `${user.firstName} ${user.lastName}`;
  }
}
```

When creating and using pipes, keep these practical guidelines in mind:

- Prefer pure pipes for deterministic transformations because they are memoized by Angular based on input references and are cheaper during change detection;
- Use impure pipes only when strictly necessary, since they execute on every change detection cycle;
- Keep pipes side-effect free and avoid putting business rules or API calls within them.

Angular provides built-in pipes (for example, *date*, *currency*, *percent*, *json*, and *async*) for common uses.

2.3.5 Forms

Forms are used to collect, validate, and process user input in web applications. They provide a structured way to build things like login forms, registration forms, search boxes, checkout pages, and data-entry screens.

See next example of a reactive typed form example:

```
@Component({
  selector: 'app-profile-form',
  template: `
    <form [formGroup]="form" (ngSubmit)="submit()">
      <input formControlName="name" placeholder="Name" />
      <input formControlName="email" placeholder="Email" />
      <button type="submit" [disabled]="form.invalid">Save</button>
    </form>
  `,
})
```

```
export class ProfileFormComponent {
  readonly form = new FormGroup({
    name: new FormControl('', [Validators.required]),
    email: new FormControl('', [Validators.required, Validators.email]),
  });

  submit(): void {}
}
```

In modern Angular, there are three types of forms:

- **Template-driven Forms**: Defined in the HTML template using directives such as `ngModel`. Suited for simple forms where validation logic is minimal and the form structure does not change dynamically;
- **Reactive Forms**: Defined programmatically in the component class using `FormGroup` and `FormControl`. Preferred for complex, dynamic, or highly validated forms because they offer explicit state management, stronger typing, and easier unit testing;
- **Signal Forms**: An emerging approach built on Angular Signals that models form state as reactive, synchronous signals. Reduces boilerplate and aligns form state management with the rest of the signal-based component model.

When native HTML controls are not enough, we can create custom form controls and integrate them with Angular forms through ControlValueAccessor. In practice, *ControlValueAccessor* is the contract that connects Angular form APIs (`formControl`, `formControlName`, and `ngModel`) to a custom component.

2.3.6 Guards & Interceptors

Guards control route access and navigation flow. Interceptors centralize HTTP cross-cutting concerns such as auth headers, retries, logging, and error normalization.

The next guard shows a simple approach for user control access:

```
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly authService: AuthService) {}

  canActivate(): Observable<boolean> {
    return this.authService.isAuthenticated();
  }
}
```

Guidelines for Guards and Interceptors:

- Guards should answer access/navigation questions (authentication, authorization, feature flags);

- Interceptors should enrich or transform HTTP requests and responses consistently across the app;
- Keep both focused and composable, avoiding heavy business logic.

In modern Angular, prefer functional guards and interceptors for simple, stateless policies because they reduce boilerplate and align well with Standalone APIs and `inject()`, while class-based implementations still make sense when richer encapsulation, shared behavior, or more complex dependencies improve clarity.

2.3.7 Routing

Routing defines navigation, feature boundaries, and lazy-loading strategy. A typical route configuration defines paths that map to components, protects sensitive routes with guards, and provides fallback navigation.

The following example shows a typical route configuration:

```
export const routes: Routes = [
  { path: 'admin', loadChildren: () => import('./admin/admin.module').then(m
=> m.AdminModule), canActivate: [AdminGuard] },
  { path: 'dashboard', component: HelloComponent, canActivate: [AuthGuard] },
  { path: '**', redirectTo: 'dashboard' },
];
```

This example demonstrates several key patterns:

- For feature modules and routes that are not needed on initial load, it is used `loadChildren` to lazy load route configurations.
- Each route specifies a `path` and the `component` to render at that path.
- The `canActivate` guard protects routes by preventing navigation if the guard rejects it (commonly used for authentication checks).
- The wildcard route (`**`) catches unmatched paths and redirects to a default route for graceful handling of invalid URLs.

To add routing to an Angular application, first define a route configuration array and then register it with the Angular router during application bootstrap or module setup. Angular will then match URLs to the appropriate views and display the correct screen as users navigate through the application.

2.3.8 Modules

NgModules declare a compilation context for a set of components that is dedicated to an application domain, a workflow, or a closely related set of capabilities. An NgModule can associate its components with related code, such as services, to form functional units.

Every Angular application has a root module, conventionally named `AppModule`, which provides the bootstrap mechanism that launches the application. An application typically contains many functional modules. Like JavaScript modules, NgModules can import functionality from other NgModules, and allow their own functionality to be exported and used by other NgModules.

Even in standalone-first applications, NgModules are still common in existing codebases and third-party integrations. For architecture decisions, the key point is to keep boundaries by feature (for example, *customers*, *billing*, *admin*) and lazy load those boundaries.

2.4 RxJS

2.4.1 Why RxJS?

Reactive programming is a declarative programming paradigm concerned with data streams and the propagation of change. RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using observables that makes it easier to compose asynchronous or callback-based code. The library also provides utility functions for creating and working with observables.

RxJS utility functions can be used for:

- Converting existing code for async operations into observables;
- Iterating through the values in a stream;
- Mapping values to different types;
- Filtering data streams;
- Composing multiple streams.

In Angular, observables provide support for passing messages between parts of the application. They are used frequently for event handling, asynchronous programming, and handling multiple values.

2.4.2 Fundamentals

Functional programming emphasizes pure functions, immutability, and composition. Reactive programming focuses on modeling values that change over time as streams and reacting to those changes declaratively.

Functional reactive programming combines these ideas. It represents time-varying values and events as streams, then transform and compose them with predictable and side-effect-minimized functions. RxJS follows this style in practice through observable pipelines and operators (`map`, `filter`, `switchMap`, etc.), even though not every use of RxJS is strictly FRP in the academic sense. This mindset helps keep asynchronous UI flows explicit, testable, and easier to reason about.

The observer pattern is a software design pattern in which an object, called the *subject*, maintains a list of its dependents, called *observers*, and notifies them automatically of state changes. In RxJS, a `Subject` is a practical implementation of this idea because it is both an observer (it can receive values) and an observable (it can broadcast values to subscribers).

The following example models a search input stream:

```
const inputValue = new Subject<string>();
```

This keeps asynchronous UI behavior declarative, because one stream represents user intent, then operators define transformation rules, and each subscriber handles a focused responsibility.

2.4.3 Promises vs Observables

A Promise represents the eventual result of one asynchronous operation. It resolves once (or rejects once) and then completes. An Observable represents a stream that can emit zero, one, or many values over time, then complete or error.

The following table summarizes the same capabilities for promises and observables:

Capability	**Promise**	**Observable**
Main goal	Represent the result of one asynchronous operation	Model asynchronous streams and event composition
Value model	Settles once with one value or error	May emit zero, one, or many values over time

Consumption	Read with `await` or handle with `then()`	Read with `subscribe()` or the `async` pipe
Operators	Chain with `then()`, `catch()`, and `finally()`	Compose with operators such as `map`, `filter`, and `switchMap`
Best use	Single asynchronous results, such as one request	HTTP streams, events, websockets, and multi-step async workflows

Both abstractions are useful for async code, but they model different kinds of workflows.

2.4.4 Best Practices

The following practices focus on building RxJS streams that are readable, predictable, and safe to maintain in Angular applications:

- ☐ **Use pipeable operators**: Use pipeable operators with RxJS. A pipeable operator returns a function with the signature `<T, R>(source: Observable<T>) => Observable<R>`. Pipeable operators are tree-shakable, so only the code we need is included, and it becomes easier to identify unused operators.
- ☐ **Use appropriate operators**: RxJS operators are the essential pieces that let operations be composed declaratively. There are operators for `creation`, `transformation`, `filtering`, `joining`, `multicasting`, `error handling`, `utility`, and more.
- ☐ **Use pure functions**: RxJS follows functional reactive programming concepts, so we should use pure functions to create reactive flow. Side effects may seem pragmatic at first, but they often mean we are not fully thinking reactively. Avoid side effects as much as possible.
- ☐ **Avoid manual subscribes**: If a component needs values from different streams, manually subscribing and mapping them to properties increases complexity. The `async` pipe unsubscribes automatically, triggers change detection, reduces the risk of forgetting to unsubscribe, and cleans up the code significantly.
- ☐ **Do not expose subjects**: A common pattern is to keep a private subject and expose only the observable via `asObservable()` method. This ensures that data can be changed only through the service interface and protects the write side of the stream. See next example:

```
export class DataService {
  private data = new BehaviorSubject<number>(0);

  readonly data$ = this.data.asObservable();

  increment(): void {
    this.data.next(this.data.getValue() + 1);
  }
}
```

- ☐ **Use subjects for state management**: A subject is both a hot observable and an observer, which lets us push values into the stream. Only use them when really needed because subjects tend to be overused by developers.
- ☐ **Share subscriptions**: Since most streams are cold by default, every subscription triggers the producer, and multiple subscriptions to `http.get()` can cause multiple HTTP calls. Angular template patterns like `async as else` can reduce the need to share streams broadly. See next example using `share` operator:

```
const users$ = this.http.get(...).pipe(share());
```

- ☐ **Handle RxJS errors**: Error handling is essential in RxJS. By default, when something goes wrong with an observable, it dies. RxJS Error Handling - Complete Practical Guide provides the most common error handling strategies.
- ☐ **Follow clean-code practices**: Consistent indentation and formatting improve readability of complex streams. Align operators below each other, extract into different streams when code becomes unreadable, move complex functionality into private methods, and avoid unnecessary brackets for readability.
- ☐ **Use marble diagrams**: Marble Diagrams provide a visual way to represent the behavior of an observable. They help us assert that an observable behaves as expected and create hot and cold observables as mocks. Marble testing is effectively a domain specific language for modeling observable interactions in tests.

2.5 Signals

2.5.1 Why Signals?

Signals are Angular's built-in reactive primitive for fine-grained state management. A signal stores a value and notifies Angular exactly where that value is read. This allows Angular to update only the affected parts of the UI instead of re-checking everything. Signals are synchronous and deterministic, which makes component state easier to model and reason about.

The most common use cases of Signals are:

- Local UI state in components (open/closed panels, selected tab, filters, counters);
- Derived state computed from other values (totals, flags, view models);
- Replacing simple Subjects/BehaviorSubjects used only for local state.

Angular Signals are strongly influenced by the observer pattern, reactive programming and fine-grained reactivity patterns. The code reading the signal is the consumer, and the signal being read is the producer.

2.5.2 Fundamentals

Signals have an easy-to-understand API for reporting data changes to the framework, allowing the framework to optimize change detection and re-rendering in a way that so far was just not possible. With Signals, Angular is able to determine exactly what parts of the page need to be updated and update only those and nothing more.

Angular provides three core APIs for signals:

- **signal()**: It creates writable state and acts as the single source of truth for a value;
- **computed()**: It derives read-only values from one or more signals, recalculating only when one of its tracked dependencies changes;
- **effect()**: It reacts to signal changes and is intended for side effects such as logging, analytics, persistence, or integration with non-signal APIs.

The following example shows these APIs in practice:

```
@Component({
  selector: 'cart-component',
  template: `
    <p>Items: {{ itemCount() }}</p>
    <p>Total: {{ total() | currency }}</p>
    <button (click)="addItem(25)">Add item</button>
  `,
})
export class CartComponent {
  readonly prices = signal<number[]>([]);
  readonly itemCount = computed(() => this.prices().length);
  readonly total = computed(() => this.prices().reduce((sum, p) => sum + p,
0));

  constructor() {
    effect(() => {
      console.log(`Items: ${this.itemCount()}, Total: ${this.total()}`);
    });
  }

  addItem(price: number): void {
    this.prices.update(items => [...items, price]);
  }
}
```

In templates, reading a signal is done by calling it like a function: *total().* This read tracking is what enables granular updates.

2.5.3 Signals vs Observables

Signals and Observables are complementary. Signals cannot fully replace Observables, but they can replace them in many UI state management scenarios.

The following table summarizes the same capabilities for signals and observables:

Capability	**Signal**	**Observable**
Main goal	Synchronous state and derived values	Model asynchronous streams and event composition
Value model	Always has a current value	May emit zero, one, or many values over time
Consumption	Read by calling `mySignal()`	Read with `subscribe()` or the `async` pipe
Operators	`computed(), effect()`	Rich operator ecosystem (`map`, `switchMap`, `mergeMap`, etc.)
Best use	Component/local state, view model state	HTTP, websockets, routing events, complex async workflows

As a rule of thumb, use signals for local state and synchronous derivations, use RxJS for asynchronous workflows and stream composition, and bridge both approaches when needed.

2.5.4 Best Practices

The following practices outline how to use Angular Signals effectively for local state, derived state, and controlled side effects:

- ☐ **Use signals for local state**: Use signals primarily for component and feature-local state. Keep long-lived asynchronous flows (HTTP polling, websocket streams, orchestration) in RxJS.
- ☐ **Prefer computed over imperative updates**: If a value can be derived from other state, use *computed()* instead of manually synchronizing variables. This avoids duplication and inconsistent state.
- ☐ **Keep effects side-effect oriented**: Use *effect()* for side effects (logging, persistence, analytics, integration points). Avoid using effects to propagate state that can be modeled with *computed()*.
- ☐ **Keep signal state immutable**: When storing arrays and objects in signals, prefer immutable updates using *set()* or *update()* with copied values instead of mutating in place. See the next example:

```
export class UsersComponent {
  readonly users = signal<Array<{ id: number; name: string }>>([]);

  addUser(user: { id: number; name: string }): void {
    this.users.update(current => [...current, user]);
  }
}
```

- ☐ **Keep computed values pure and cheap**: Use *computed()* for deterministic derivations only. Avoid side effects and avoid heavy processing inside computed signals. If a computation is expensive, break it into smaller computed values or move the expensive part to a dedicated service.
- ☐ **Minimize and scope effects carefully**: Prefer a small number of focused effects close to the feature that needs them. This keeps dependencies explicit, reduces accidental reactivity chains, and improves maintainability.
- ☐ **Bridge signals and observables intentionally**: Use *toSignal()* when an observable result needs to be consumed as synchronous UI state, and *toObservable()* when signal state needs to participate in RxJS pipelines.
- ☐ **Prefer signal inputs**: Signal-based component APIs can simplify change tracking and reduce boilerplate around input propagation. Keep public component APIs explicit and strongly typed to improve readability and refactoring safety.
- ☐ **Prefer signal forms**: An emerging approach built on Angular Signals that models form state as reactive, synchronous signals. It reduces boilerplate and aligns form state management with the rest of the signal-based component model.
- ☐ **Test signals as state transitions**: Unit tests for signal-based components should verify state transitions and derived values, not implementation details. Assert that changing writable signals updates computed state and expected side effects in a predictable way.

```
it('updates total when quantity changes', () => {
  const cmp = new CartComponent();

  cmp.addItem(25);

  expect(cmp.total()).toBe(25);
});
```

2.6 Angular Advanced Concepts

2.6.1 CDK

The Angular Component Dev Kit (CDK) is a set of behavior-focused primitives for building robust UI components without imposing a visual style. While Angular Material provides fully styled components, the CDK provides low-level building blocks.

Some popular CDK categories are:

- **Accessibility**: Accessibility primitives for focus management, keyboard interaction, ARIA helpers, and screen-reader announcements;
- **Overlay**: Infrastructure for floating layers (dialogs, popovers, tooltips, context menus) with positioning, backdrop, and scroll strategies;
- **Portal**: APIs to dynamically render a component or template into a different host container;
- **Scrolling**: Scroll utilities and virtual scrolling to efficiently render very large lists;
- **Drag and Drop**: Declarative drag-and-drop behavior for sortable lists, transfer between containers, and constrained drag areas;
- **Table**: Unstyled table foundation (data source, row/column definitions, rendering pipeline) for custom data grids;
- **Tree**: Primitives for rendering hierarchical data with expandable/collapsible nodes and nested or flat tree structures;
- **Menu**: Headless menu behavior with keyboard navigation, focus handling, nested submenus, and accessibility semantics;
- **Bidirectionality**: The `bidi` package provides support for RTL/LTR layouts and direction-aware component behavior;
- **Observers**: Wrappers for browser observer APIs (for example resize and mutation) to react to DOM changes safely in Angular;
- **Layout**: Responsive layout helpers and media-query observation to adapt UI behavior across breakpoints;
- **Testing**: Component harness utilities for stable implementation-agnostic UI tests.

In short, Angular CDK is a toolkit of low-level building blocks that help us create sophisticated interfaces while keeping full control over the interface and styling.

2.6.2 Animations

Animations help communicate state changes, provide continuity between views, and improve perceived responsiveness. In Angular applications, animations should support UX goals, not decorate every interaction. Good motion clarifies what changed (for example entering, leaving, expanding, collapsing, and reordering) and guides user attention without slowing task completion.
In practice, teams usually combine two approaches:

- **Native CSS/Web Animations for simple transitions**: Great for hover/focus effects, opacity fades, and small transforms with minimal code;

- **Angular Animation API for stateful UI transitions**: Useful when animation behavior depends on component state, structural changes, route transitions, or coordinated enter/leave sequences.

For larger applications, route-level animations can be introduced gradually by defining transition data on routes and applying animations at the layout shell level. This keeps motion centralized, consistent, and easier to maintain.

2.6.3 User Access Control

User access control is the set of rules that determines who can access the application (authentication) and what each authenticated user can do (authorization). In Angular, this is typically implemented with route guards to block navigation for users without the required role or permission, and with UI-level checks to hide or disable actions that the user is not allowed to perform.
Authentication in Angular SPAs typically follows one of these patterns:

- **Token-based (JWT)**: The most common approach. After login, the server issues a JSON Web Token that the client stores and sends with each request. The token is typically stored in `localStorage` and attached to requests via an HTTP interceptor.
- **Session-based with cookies**: The server manages sessions and sets `HttpOnly` cookies. This approach benefits from browser-managed cookie handling and CSRF protection but requires the API and SPA to share the same domain or use proper CORS configuration.
- **OAuth 2.0 / OpenID Connect (OIDC)**: For enterprise or third-party authentication, Angular applications integrate with identity providers (Auth0, Keycloak, and others) using OIDC flows. Libraries like angular-oauth2-oidc simplify the integration.

Authorization determines what authenticated users can access. In Angular, authorization is enforced at multiple levels:

- **Route guards**: Protect entire routes based on authentication status, user roles, or permissions via `canActivate`, `canMatch`, or `canActivateChild` guards.
- **UI-level**: Show or hide UI elements (buttons, menu items, form fields) based on the user's permissions using structural directives or pipes. Libraries like ngx-permissions can help centralize role/permission checks in templates and routes.
- **API-level**: The backend enforces authorization independently because the frontend authorization is a UX convenience, not a security boundary.

Authentication and authorization in Angular require a combination of services, route guards, and interceptors to secure an application.

2.6.4 Error Handling

Angular provides a built-in ErrorHandler class that catches all unhandled exceptions in the application. Replacing the default handler with a custom implementation allows centralized error logging, user notification, and integration with monitoring services.
HTTP errors should be handled at multiple levels:

- **Interceptor level**: Catches common HTTP errors (401, 403, 500, network errors) and applies global handling such as token refresh, redirect to login, or retry.
- **Service level**: Catches domain-specific errors and transforms them into meaningful error types using RxJS `catchError`.
- **Component level**: Handles presentation-specific errors, displaying inline error messages or fallback UI.

A well-designed error handling architecture distinguishes between:

- **Expected errors**: Validation failures, business rule violations, and not-found responses. These should be handled gracefully with user-friendly messages.
- **Unexpected errors**: Server errors, network failures, and unhandled exceptions. These should be logged to an error tracking service and display a generic error message to the user.

Error tracking services like Sentry, Datadog, or Application Insights provide real-time visibility into production errors. These services capture stack traces, source maps, user context, and breadcrumbs, enabling teams to diagnose and fix issues quickly. Angular's error handler and HTTP interceptors serve as the integration points for these services.

2.6.5 Internationalization

Internationalization (i18n) is the process of designing an application so it can be adapted to different languages and regions without engineering changes. Localization (l10n) is the process of adapting the internationalized application to a specific locale.
We have two primary approaches to internationalization:

- **Built-in i18n**: Angular's compile-time i18n uses `i18n` attributes in templates, the @angular/localize package, and the Angular CLI extraction tool (`ng extract-i18n`) to produce translation files. Each locale generates a separate application build, optimized for that language with no runtime overhead. This approach is best for applications where the locale is determined at deployment time.

- **ngx-translate**: A runtime i18n library that loads translation files dynamically and allows switching languages without reloading the application. This approach is more flexible but adds bundle size and a small runtime cost.

Choose the approach based on product requirements. Prefer built-in Angular i18n for static, deployment-time locale builds and use `ngx-translate` when runtime language switching is a core user requirement. Use ICU message syntax for pluralization, gender, and complex conditional messages.

2.6.6 Accessibility

Accessibility (a11y) is a core part of UX, not an extra feature. A web application is accessible when people with different abilities can use it well. They should be able to perceive, understand, navigate, and interact with it. This includes people who use screen readers, keyboard navigation, voice input, captions, zoom, high contrast settings, or reduced motion. It also includes people with temporary or situational limits, such as a broken mouse, poor lighting, low bandwidth, or a noisy environment.

The Web Content Accessibility Guidelines (WCAG) are the main reference for accessible web experiences. In practice, frontend teams should focus on semantic HTML, proper labels, enough color contrast, visible focus states, keyboard support, accessible forms, meaningful link text, alternative text for images, correct heading structure, and proper WAI-ARIA usage when native semantics are not enough. Angular Aria is a set of headless, accessible directives that implement common WAI-ARIA patterns. These directives handle keyboard interactions, ARIA attributes, focus management, and screen reader support.

Accessibility also improves quality for all users. Captions help in noisy places, larger click targets help on mobile devices, better contrast helps in bright environments, and clearer labels reduce mistakes for everyone. Accessible design often leads to clearer, more robust, and more resilient interfaces. In this sense, accessibility is both an ethical responsibility and a strong engineering practice. Angular applications should also treat prefers-reduced-motion, keyboard-only navigation, and screen-reader announcements as normal parts of implementation work.

2.6.7 Analytics

Web analytics is the practice of collecting, measuring, and interpreting user behavior data. It helps teams understand how people use an application and make better product decisions. Analytics often tracks page views, feature usage, conversion funnels, performance timings, and errors.

Popular tools include Google Analytics, Google Tag Manager, Adobe Analytics, Mixpanel, and Amplitude. Product observability platforms such as Datadog RUM and Sentry also provide error and session insights.

In Angular, a Redux-like approach or NgRx Effects works well because we can observe actions, combine them with the latest state when needed, map them to an analytics payload, and send that payload to the analytics API. Only selected actions should be tracked to avoid noisy or duplicated telemetry. This keeps components focused on UI concerns and makes analytics behavior easier to change over time.

2.6.8 SEO

Search Engine Optimization (SEO) is the practice of making a site easier to discover, understand, and rank in organic search. Angular SPAs can create SEO risk when important content, metadata, or navigation depends on late client-side rendering. Modern search engines can run some JavaScript, but relying only on that is less predictable than serving meaningful HTML up front. Angular provides several practical strategies to improve SEO:

- **Server-side and hybrid rendering**: Angular supports route-level rendering strategies, so teams can choose CSR, SSR, or prerendering for each route. Use SSR for public, request-specific pages where crawlers and users should receive fully rendered HTML in the first response.
- **Hydration**: Hydration reuses the server-rendered DOM on the client instead of tearing it down and rendering it again. This reduces flicker and helps preserve strong Core Web Vitals such as Largest Contentful Paint (LCP) and Cumulative Layout Shift (CLS), which indirectly support SEO.
- **Build-time prerendering)**: Static Site Generation (SSG) creates HTML at build time for routes whose content is known in advance. This is a strong fit for landing pages, marketing sites, blogs, and documentation because the pages are immediately cacheable and indexable.
- **Meta tags and titles**: Angular's Meta and Title services let teams set route-specific titles, descriptions, robots directives, and social preview tags.
- **Structured data**: Adding JSON-LD structured data helps search engines understand entities such as articles, products, FAQs, and breadcrumbs, which improves eligibility for rich results.
- **Canonical URLs**: Adding `<link rel="canonical">` tags helps prevent duplicate-content issues when the same page can be reached through multiple URLs, query strings, or campaign parameters.

- **Sitemaps and crawl controls**: A `sitemap.xml` file helps search engines discover public routes, while a correctly configured `robots.txt` file tells them which routes should or should not be crawled.

In short, good SEO in Angular depends on serving meaningful HTML early and keeping each public route clear, stable, and easy for search engines to understand.

2.6.9 PWA

A Progressive Web Application (PWA) is a web application that uses modern browser capabilities to deliver an installable, app-like experience. PWAs are reliable (load quickly and can work offline), fast (respond quickly to user interactions), and engaging (feel close to native on supported devices). Key technologies behind PWAs include the Service Worker API, Web Application Manifest, and Cache API.

Angular provides first-class support for building PWAs through the @angular/pwa package. Angular CLI configures the Angular Service Worker, generates a `manifest.webmanifest`, adds icons, and sets up the `ngsw-config.json` file for caching and update strategies.

PWAs are particularly valuable for applications that must tolerate unreliable network connectivity or provide an app-store-like installation experience without the overhead of native application development and distribution.

2.6.10 WebMCP

WebMCP is an emerging web standard that enables web applications to expose structured tools directly to AI agents running natively in the browser. Rather than requiring an agent to navigate complex UI flows through DOM interactions, WebMCP tools let agents call application logic directly, for example, submitting a user-registration form by invoking a tool instead of filling in individual fields.

Angular WebMCP provides a framework-supported way to connect those agent interactions to application features. At a high level, this makes it easier to enable AI-assisted workflows in Angular applications.

Some best practices for WebMCP integrations include keeping tool names globally unique (duplicate registrations throw at runtime), preferring application-level or root-service providers over component constructors to avoid accidental multi-registration, and always validating tool inputs explicitly inside `execute` callback.

2.7 Development Workspace

2.7.1 Project Structure

A well-organized file structure is essential for maintaining a scalable Angular application. The Angular Style Guide recommends organizing files by feature rather than by type. The LIFT principle (Locate, Identify, Flattest structure, Try to be DRY) should guide the folder organization.

A recommended folder structure for a large Angular application:

```
src/
├── app/
│   ├── core/              # Services, guards, interceptors, models
│   │   ├── services/
│   │   ├── guards/
│   │   ├── interceptors/
│   │   └── core.module.ts
│   ├── shared/            # Shared components, directives, pipes
│   │   ├── components/
│   │   ├── directives/
│   │   ├── pipes/
│   │   └── shared.module.ts
│   ├── features/          # Feature modules (lazy-loaded)
│   │   ├── feature-a/
│   │   │   ├── components/
│   │   │   ├── models/
│   │   │   ├── services/
│   │   │   └── feature-a.module.ts
│   │   └── feature-b/
│   ├── app.component.html
│   ├── app.component.spec.ts
│   ├── app.component.ts
│   ├── app.module.ts
│   └── app-routing.module.ts
├── assets/
├── environments/
├── index.html
├── main.ts
└── styles.scss
```

Key guidelines for structure the project:

- **Core module**: Contains application-wide singleton services, HTTP interceptors, authentication guards, and global error handling. This module is imported only once in the `AppModule`.
- **Shared module**: Contains reusable components, directives, and pipes that are shared across multiple feature modules. Exports all declarations so consuming modules can use them.
- **Feature modules**: Each feature or domain area of the application is encapsulated in its own module. Feature modules should be lazy-loaded whenever possible to reduce the initial bundle size.

With standalone components, `NgModules` become optional. Standalone components can import their dependencies directly, simplifying the module structure and enabling more granular lazy loading.

2.7.2 Project Documentation

Project documentation is critical for onboarding, maintenance, and long-term sustainability of an Angular project. It shortens onboarding, preserves architectural decisions, reduces maintenance and refactoring risk, improves debugging and incident response, and aligns developers, QA, and DevOps teams around shared standards. In practice, clear documentation prevents knowledge silos, reduces rework, and protects long-term project sustainability as the codebase and team evolve.

Essential project documentation artifacts include:

- **README.md**: The entry point for any project. It should contain a project description, prerequisites, setup instructions, available scripts (`ng serve`, `ng build`, `ng test`), and links to further documentation.
- **CHANGELOG.md**: Maintained automatically using tools like semantic-release following Conventional Commits to generate meaningful changelogs.
- **CONTRIBUTING.md**: Guidelines for contributing to the project, including branch naming conventions, commit message format, code review process, and testing requirements.
- **API documentation**: Documentation for source code and essential for shared libraries. Use Compodoc to generate Angular-specific documentation from code comments, decorators, and module declarations.
- **Component catalog**: Documentation for UI components. Tools like Storybook serve as living documentation for UI components, allowing developers and designers to browse and interact with components in isolation.

In delivery pipelines, teams often publish the generated documentation as a build artifact or a static site using GitHub Pages or a similar platform. This creates a single, searchable reference for developers, QA, and architects without requiring manual documentation assembly.

2.7.3 Build Environments

Angular CLI supports multiple named build configurations such as `development`, `staging`, and `production`. Each configuration can override any builder option (source maps, optimization, API URLs, feature flags) allowing the same codebase to target different environments without code changes.

The `src/environments/` folder holds per-environment TypeScript files. The environments structure folder looks like this:

```
src/environments/
├── environment.ts
└── environment.prod.ts
```

Angular CLI replaces `environment.ts` with the matching file at build time. This is configured automatically in `angular.json` under `fileReplacements`:

```
{
  "configurations": {
    "production": {
      "fileReplacements": [
        {
          "replace": "src/environments/environment.ts",
          "with": "src/environments/environment.prod.ts"
        }
      ]
    }
  }
}
```

The `environment.ts` file is bundled into the client-side application and visible to anyone who loads the page. Never store secrets such as API keys here. Use a server-side proxy or a secrets manager instead.

2.7.4 Build Configuration

The Angular CLI `ng build` command compiles Angular templates and TypeScript, then bundles and optimizes the application for the target environment. The builder used by default for new projects is `@angular/build:application`, which is powered by esbuild and produces a `dist/` output directory.

Angular provides two compilation strategies for turning component templates and TypeScript code into JavaScript:

Compiler	Description
Just-in-Time (JIT)	Compiles the application in the browser at runtime. Was the default before Angular 9. Slower initial load, easier to debug in some cases.
Ahead-of-Time (AOT)	Compiles templates and TypeScript at build time, before the browser downloads any code. Default since Angular 9.

The CLI supports named configurations defined in `angular.json`. The two standard configurations are `production` (the default for `ng build`) and development (the default for `ng serve`).
A typical production build enables several performance-focused optimizations, namely, it precompiles templates with AOT, switches Angular to production mode to remove development-only checks, bundles files into fewer assets, minifies and mangles code to reduce size, eliminates unused code, and add inlines critical CSS to improve initial render speed.

During local development, `ng serve` skips these optimizations intentionally to enable faster rebuilds, detailed error messages, and Angular DevTools support.

2.7.5 Proxy Configuration

During local development it is common to call a backend API running on a different port or host. Browsers block cross-origin requests unless the server explicitly allows them. The Angular CLI Dev Server supports a proxy configuration that forwards matching requests to a backend server, avoiding CORS issues without changing production code.

To set up a proxy, create `config/proxy.conf.json` file with:

```
{
  "/api/**": {
    "target": "http://localhost:3000",
    "secure": false,
    "logLevel": "info"
  }
}
```

Then, register the proxy file in `angular.json` under the `serve` target. Restart `ng serve` after editing the files for changes to take effect. Then, all requests to `/api/**` are forwarded to `http://localhost:3000`. For advanced use cases such as attaching a bearer token to every proxied request, a JavaScript proxy configuration file provides more flexibility.

2.7.6 Template Type Checking

Angular performs static type checking of template expressions and bindings at compile time. Just as TypeScript catches type errors in source code, Angular checks expressions and bindings within application templates and reports any type errors it finds. The depth of this checking is controlled by compiler options in `tsconfig.json`.
Angular provides three progressive levels of [template type checking]:

- **Basic mode**: Validates only top-level expressions. Embedded views such as `@if` and `@for` are not checked, and many types default to `any`;
- **Full mode**: Checks embedded views, pipe return types, and directive references;
- **Strict mode**: Verifies binding assignability, respects `strictNullChecks`, infers generic types, and correctly types `$event` in event bindings and local template references.

Strict mode is the recommended setting and can be fine-tuned with individual flags such as `strictNullInputTypes`, `strictInputTypes`, `strictDomLocalRefTypes`, `strictOutputEventTypes`, and others. When a template expression cannot be typed correctly (for example, due to incomplete library typings) use the `$any()` pseudo-function to opt out of type checking for that expression. `$any()` is equivalent to a TypeScript `as any` cast.

2.7.7 Source Maps

Source maps map compiled JavaScript back to original TypeScript and templates. They are essential for local debugging because breakpoints, stack traces, and browser DevTools can point to the real source instead of transformed output. In Angular projects, source maps are typically enabled in development and disabled in production to avoid exposing source code and to reduce bundle size.

Source maps are also useful for bundle analysis. [esbuild-visualizer] visualizes which packages occupy the most space and helps identify oversized dependencies, duplicated libraries, or missed lazy-loading boundaries. For more details, see the script bellow:

```
{
  "scripts": {
    "analyze": "ng build --stats-json && esbuild-visualizer --metadata
dist/my-app/stats.json --filename dist/stats.html --open"
  }
}
```

This opens an interactive treemap in the browser showing the size contribution of each module. Run the command regularly in performance reviews to catch regressions early and keep

JavaScript payloads under control. Also, use Bundlephobia to identify large dependencies and optimization opportunities.

2.7.8 Schematics

Schematics are template-based code generators that the Angular CLI uses to scaffold and transform projects. Every `ng generate` sub-command (`component`, `service`, `pipe`, `directive`, `guard`, and so on) is backed by a schematic from the @schematics/angular package.
Developers can create custom schematics to:

- **Enforce project conventions**: Generate components, services, and modules with a predefined structure, naming conventions, and boilerplate that matches team standards.
- **Automate repetitive tasks**: Create schematics for common patterns such as CRUD features, state management boilerplate, or page scaffolding.
- **Library setup**: When building Angular libraries, schematics can be packaged alongside the library to help consumers configure and integrate it (via `ng add`).
- **Migration scripts**: Schematics can automate breaking changes during library or framework upgrades, similar to how Angular itself uses schematics for major version migrations.

Popular third-party libraries such as NgRx, Angular Material, and Nx provide their own schematics to streamline development workflows.

2.7.9 Debugging

Debugging is a core skill for any programmer. The first step is always to understand what is happening and determine where the problem originates. The simplest approach, in terms of tooling, is to read the code carefully. If that reveals nothing, the next step is to add a few diagnostic lines. In JavaScript, developers commonly use *alert()* and *console.log()* for this purpose. The output of *console.log()* appears in the browser's JavaScript console, which is available in all major browsers.
Modern browsers ship with built-in developer tools, commonly known as DevTools. These tools help inspect and analyze HTML, CSS, and JavaScript, examine HTTP headers, and evaluate page accessibility. In the debugger panel, breakpoints can be set in JavaScript code. When a breakpoint is reached, execution pauses and the program state becomes inspectable. Variable values can be checked and execution can be resumed one line at a time. Another option is to open the file in the Sources panel and click a line number to add a breakpoint there. The *debugger* keyword in code has the same effect because it pauses execution when developer tools are open.

For IDE-based debugging, Visual Studio Code, WebStorm, and IntelliJ IDEA are the recommended options. Each supports debugging client-side Angular code with breakpoints, variable inspection, and call stack navigation, either natively or through plugins. For further details, see Using Angular in Visual Studio Code, Running and debugging an Angular application with WebStorm, and Running and debugging an Angular application with IntelliJ IDEA.

Finally, Angular DevTools is the official browser extension for Angular, developed by the Angular team and available for Google Chrome and Mozilla Firefox. It provides debugging and profiling capabilities including component tree visualization, component state inspection, change detection tracking, and performance analysis.

2.8 Style Guide

This section is based on the official Angular Style Guide, with some adaptations. Keep these rules as defaults, but when in doubt, prefer consistency within each file and feature.

2.8.1 Naming

Use the naming rules below to standardize files, symbols, selectors, and tests:

- **General Naming Guidelines**: Do use consistent names for all symbols. Do follow a pattern that describes the symbol's feature then its type. The recommended pattern is *feature.type.ts*. The naming conventions should simply help find desired code faster and make it easier to understand. Names of folders and files should clearly convey their intent. For example, *app/heroes/hero-list.component.ts* may contain a component that manages a list of heroes.
- **Separate filenames with dots and dashes**: Do use dashes to separate words in the descriptive name and dots to separate the descriptive name from the type. Do use conventional type names including *.service*, *.component*, *.pipe*, *.module*, and *.directive*. Type names provide a consistent way to quickly identify what is in the file, make it easy to find and provide pattern matching for any automated tasks.
- **Symbols and file names**: Do use upper camel case for class names. Do match the name of the symbol to the name of the file. Do give the filename the conventional suffix (such as *.component.ts*, *.directive.ts*, *.module.ts*, *.pipe.ts*, or *.service.ts*) for a file of that type.
- **Bootstrapping**: Do put bootstrapping and platform logic for the app in a file named *main.ts*. Do include error handling in the bootstrapping logic.
- **Pipe names**: The pipe class name should use *UpperCamelCase* (the general convention for class names), and the corresponding name string should use *lowerCamelCase*. The name string cannot use hyphens ("dash-case" or "kebab-case").
- **Unit test file names**: Do name test specification files the same as the component they test. Do name test specification files with a suffix of *.spec*.

- **E2E test file names**: Do name end-to-end test specification files after the feature they test with a suffix of *.spec*.
- **Module names**: When using NgModules, do append the symbol name with the suffix *Module* and use *.module.ts* files. Do suffix a *RoutingModule* class name with *RoutingModule* and end the filename with *-routing.module.ts*.

2.8.2 Coding Conventions

Apply the following coding conventions to keep Angular TypeScript code consistent and reviewable:

- **Classes**: Do use upper camel case, also known as *PascalCase*, when naming classes.
- **Constants**: Do declare variables with *const* if their values should not change during the application lifetime. Lower camel case variable names (*heroRoutes*) are easier to read and understand than the traditional UPPER_SNAKE_CASE names (*HERO_ROUTES*).
- **Interfaces**: Do name an interface using upper camel case. Consider naming an interface without an *I* prefix. Consider using a class instead of an interface for services and declarables (components, directives, and pipes). Consider using an interface for data models.
- **Properties and methods**: Do use lower camel case to name properties and methods. Avoid prefixing private properties and methods with an underscore.
- **Imports**: Consider leaving one empty line between third-party imports and application imports. Consider listing import lines alphabetized by the module.
- **Comments**: Prefer self-explanatory code. Use comments only when they add context that code cannot express (for example, non-obvious constraints, trade-offs, or links to specifications).

2.8.3 Application Structure

Use the following structure rules to organize source files by feature and clear boundaries:

- **LIFT**: Do structure the app such that we can Locate code quickly, Identify the code immediately, keep the Flattest structure we can, and Try to be DRY. Do define the structure following these four basic guidelines, listed in order of importance:
    - **Locate**: Do make locating code intuitive, simple, and fast.
    - **Identify**: Keep each file focused on a single primary concept (typically one component, directive, pipe, service, or class). This improves discoverability, reduces merge conflicts, and makes refactoring and testing easier.
    - **Flat**: Do keep a flat folder structure if possible. Consider creating sub-folders when a folder reaches seven or more files.

    - **T-DRY**: Do be DRY (Don't Repeat Yourself). Ensure the same code is not copied into different places in the codebase. Extract repeating code and use it in place of repeated code.
- **Overall structural guidelines**: Do start small but keep in mind where the app is heading down the road. Do have a near-term view of implementation and a long-term vision. Consider creating a folder for a component when it has multiple accompanying files (*.ts*, *.html*, *.css* and *.spec*).
- **Application entry**: Do bootstrap the application in *src/main.ts*. In standalone-first applications, central app setup is typically done with `bootstrapApplication` and app-level providers/config.
- **Folders-by-feature structure**: Do create folders named for the feature area they represent. Avoid top-level folders such as *components*, *directives*, and *services* for unrelated code.
- **Feature boundaries**: Keep each feature independently understandable and route-oriented. Use lazy loading (or deferred loading) whenever it improves startup performance.
- **Shared code**: Keep reusable UI pieces and utilities in a shared area by feature/theme. In NgModule-based projects, a *SharedModule* is still valid. In standalone projects, prefer direct imports and shared entry points.
- **Lazy loaded folders**: A distinct application feature or workflow may be lazy loaded or loaded on demand rather than when the application starts. A typical lazy loaded folder contains a routing component, its child components, and their related assets and modules.

2.8.4 Components

Follow the component rules below for selectors, public APIs, member organization, and template logic:

- **Component names**: Do suffix a service class name with Component. For example, something that gets *product* or *heroes* should be called a *ProductComponent* or a *HeroComponent*.
- **Selector conventions**: Do use dashed-case or kebab-case for naming the element selectors of components. Do use a hyphenated, lowercase element selector value, for example, *admin-users.* Do use a prefix that identifies the feature area or the app itself.
- **Templates and styles to their own files**: Prefer to extract templates and styles into a separate file. Do name the template file *[component-name].component.html* and style file *[component-name].component.css*, where *[component-name]* is the component name. However, inline templates and styles are very popular.
- **Inputs and outputs**: Do use the *@Input()* and *@Output()* class decorators. Alternatively, use function-based APIs such as `input()` and `output()` when appropriate.

- **Member sequence**: Do place Angular-specific members near the top (injected dependencies, inputs, outputs, queries), then other properties, and then methods. Placing members in a consistent sequence makes classes easier to scan.
- **Delegate complex component logic to services**: Do limit logic in a component to only that required for rendering. All other logic should be delegated to services.
- **Don't prefix output properties**: Do name events without the prefix *on*. Prefer handler names that describe intent (for example, *saveProfile*), not generic trigger names.
- **Name event handlers for what they do**: Prefer naming event handlers for the action they perform rather than for the triggering event. For example, use `(click)="saveUserData()"` for event handler.
- **Avoid overly complex logic in templates**: Keep template expressions simple and readable. Complex derivations should move to the class (for example through computed values, signals, or pure helper methods).
- **Use protected for template-only members**: Members used only by a component template should generally be `protected` instead of `public`, keeping the component public API minimal.
- **Use readonly for Angular-managed properties**: Prefer `readonly` for outputs, queries, and function-based input/model/output declarations that should not be reassigned.
- **Prefer class/style bindings**: Prefer `[class]`, `[class.foo]`, `[style]`, and `[style.prop]` over `ngClass` and `ngStyle` for clearer syntax and lower overhead.
- **Keep lifecycle hooks simple**: Keep lifecycle hooks short and delegate behavior to well-named methods.

2.8.5 Directives

Use the following directive rules for selector naming and host interaction patterns:

- **Selector conventions**: Keep directive selectors camelCase and prefixed consistently with your application namespace.
- **Use directives to enhance an element**: Use attribute directives when we have presentation logic without a template.
- **HostListener/HostBinding decorators versus host metadata**: Consider preferring the *@HostListener* and *@HostBinding* to the host property of the *@Directive* and *@Component* decorators.

2.8.6 Services

Follow the service rules below for scope, provider configuration, and separation of concerns:

- **Service names**: Do suffix a service class name with Service. For example, something that gets *data* or *heroes* should be called a *DataService* or a *HeroService*.

- **Services are scoped by injector**: Use root-scoped services for cross-application shared behavior and feature or component-scoped services when isolation is required.
- **Single responsibility**: Do create services with a single responsibility that is encapsulated by its context. Do create a new service once the service begins to exceed that singular purpose.
- **Talk to the server through a service**: Do make data services responsible for XHR calls, local storage, stashing in memory, or any other data operations. The component's responsibility is for the presentation and gathering of information for the view.
- **Use abstraction tokens when needed**: For swappable implementations, use abstract classes or `InjectionTokens` as provider tokens.

2.9 Best Practices

The following recommendations summarize the Angular engineering practices that most improve consistency, maintainability, and delivery quality:

- ☐ **Follow conventions**: Following conventions increases uniformity and code quality, improves cross-team collaboration, and enables new developers to integrate quickly. We should follow the Angular Style Guide to get the most out of the Angular Framework.
- ☐ **Follow separation of concerns**: Angular is built around separation of concerns, which makes code easier to maintain, extend, reuse, and test. It helps us limit component logic to what the template needs and nothing more. This is central to writing clean Angular code and is reinforced by the Angular Style Guide.
- ☐ **Follow LIFT Principle**: The LIFT Principle helps us locate code quickly, identify code immediately, keep the structure flat, and try to be DRY. Apply the 5-seconds rule: if we cannot understand it in 5 seconds, it probably needs refactoring.
- ☐ **Type everything**: Always declare variables or constants with a type other than any. Without typing, TypeScript deduces types from assigned values, which can cause unintended problems. Good typings make refactoring easier and safer, and in most cases there is a better alternative than `any`. Set `strict: true` in `tsconfig.json` file.
- ☐ **Make use of lint rules**: Lint rules provide immediate feedback when we do something we shouldn't be doing. The combination of typescript-eslint and angular-eslint rules reinforces consistency, improves readability, and helps find errors, such as variable-scope issues.
- ☐ **Keep imports clean and readable**: Group third-party and application imports with a blank line between groups, and keep imports alphabetized where practical. Import hygiene reduces merge noise and improves scannability.

- ☐ **Use aliases for imports**: Deep relative imports are hard to read and maintain. TypeScript path aliases provide a cleaner way to map imports to absolute paths through `compilerOptions` in `tsconfig.json`, making imports easier to read and refactor.
- ☐ **Avoid long functions**: Long functions usually indicate they are doing too many things. Small functions are easier to read and faster to understand. If a function is more than 10 lines, ask whether it should be split into smaller functions. Use the Single Responsibility Principle and watch for cyclomatic or cognitive complexity.
- ☐ **Avoid useless code comments**: If we need a comment, the code may not be self-explanatory enough. Good comments provide useful information such as legal notes, warnings, or realistic TODOs. Bad comments include commented-out code, noise comments, and redundant comments that merely restate the obvious.
- ☐ **Remove unused code**: Unused or dead code is code that will never run, including files, conditions, loops, unused imports, variables, functions, and private members. Dead-code elimination reduces repository and bundle size, improves performance, and makes code easier to understand.
- ☐ **Prefer signal-first reactivity**: Use Signals (`signal`, `computed`, `effect`) as the preferred foundation for application state and reactivity, and document recommended patterns for interoperating with RxJS through `toSignal` and `toObservable`.
- ☐ **Prefer Observables over Promises**: Use Observables for streams, cancellation, composition, and multi-value sources (for example `HttpClient` pipelines, router data, and UI events). Use Promises for one-off async operations where stream semantics are unnecessary. Keep boundaries explicit and avoid mixing patterns without clear reason.
- ☐ **Subscribe in component template**: Avoid subscribing to observables in component classes and instead subscribe from the template using the async pipe. The `async` pipe subscribes and unsubscribes automatically. Manual `subscribe()` also creates the need to unsubscribe at the end of the component's lifecycle. See next example with a partial class:

```
@Component({
  selector: 'my-component',
  template: `<child [data]="data$ | async"></child>`
})
export class MyComponent {
  data$: Observable<object>;
}
```

- ☐ **Avoid memory leaks**: When subscribing to observables, always unsubscribe appropriately with operators like `take`, `takeUntil`, or `unsubscribe()`. A subscription stays alive until the stream completes or we unsubscribe manually. Failing to unsubscribe can cause memory leaks after a component is destroyed or a user navigates away. This risk can also be reduced with eslint-plugin-rxjs that detect unsubscribed observables.

- ☐ **Use framework-assisted teardown**: Prefer `async` pipe in templates, `OnDestroy` and `takeUntilDestroyed()/DestroyRef` in classes for deterministic cleanup. This reduces boilerplate and prevents lifecycle-related leaks.
- ☐ **Avoid nested subscriptions**: Nested subscriptions make code unreadable and complex, introduce side effects, and push us away from reactive thinking. Prefer flattening operators and reactive composition over `subscribe`. See next example of a bad implementation:

```
ngOnInit(): void {
  this.route.params
    .pipe(map(param => param.id))
    .subscribe(id => this.userService.fetchById(id)
      .subscribe(user => (this.user = user)));
}
```

- ☐ **Keep child component contracts simple**: Prefer passing plain values or signal inputs to child components. Do not pass streams to components directly, instead, handle the subscription in the parent component, ideally with the `async` pipe.
- ☐ **Use minimal component API surface**: Prefer `protected` for members used only by templates and `readonly` for Angular-managed properties that should not be reassigned. This keeps public APIs intentional and reduces accidental mutations.
- ☐ **Keep lifecycle hooks thin**: Keep lifecycle hooks short and delegate non-trivial behavior to clearly named private methods or services. This improves readability and testability.
- ☐ **Avoid changing the DOM directly**: Angular uses Lifecycle Hooks that determine how and when components will be rendered and updated. Direct DOM access or manipulation can corrupt these lifecycle hooks, leading to unexpected behavior of the whole app. Use Angular templating and data binding instead.
- ☐ **Avoid computing values in the template**: Binding methods in templates is convenient but expensive because Angular may call those methods on every change detection cycle. This can create serious performance issues. Prefer pure pipes or precomputed values set through lifecycle hooks.
- ☐ **Avoid complex logic in templates**: Template logic, even a simple `&&`, should usually be extracted into the component. Logic in templates is harder to test in isolation and spreads presentation logic across two places. Keeping logic in the component class improves testability, maintainability, and reusability.
- ☐ **Prefer direct class/style bindings**: Prefer `[class]`, `[class.foo]`, `[style]`, and `[style.prop]` over `ngClass and ngStyle` in most cases for clearer templates and lower runtime overhead.
- ☐ **Components should only deal with display logic**: Components should be as simple as possible and focus on presentation. Business logic should be extracted into services when appropriate. This separation makes components easier to reason about, easier to test, and easier to reuse.

- ☐ **Create small reusable components**: A single component should ideally render one specific part of the page or handle one behavior. Keep components small and as dumb as possible, driven by inputs and outputs.
- ☐ **Use smart and dumb components**: A common Angular pattern is separating smart and dummy components. Dummy components are presentation-only and do not know where data came from. Smart components provide or orchestrate data and behavior. This separation makes components simpler and less likely to contain bugs.
- ☐ **Use base component classes**: A base component class can help when many components share the same behavior, such as forms or CRUD pages. Instead of duplicating common logic everywhere, extract shared data and methods into a base class so changes happen in one place.
- ☐ **Do not remove view encapsulation**: Angular component styles are encapsulated by default so they do not affect the rest of the application. Disabling encapsulation with `ViewEncapsulation.None` makes styles global and can create unintended side effects. If we use it, we need extra discipline such as nested CSS or naming conventions like BEM.
- ☐ **Avoid poorly structured CSS**: Common mistakes include deep selectors and inline styles. Inline styles are bad practice because of poor scalability and maintainability, so styles should live in CSS files. Overusing `::ng-deep` is also problematic because it pierces component style isolation and encourages poor styling practices, even though it still exists as a temporary solution.
- ☐ **Use mandatory inputs**: We can make required inputs explicit by using the component selector to require a specific attribute or using signal inputs. This produces an error at startup or compile time if the attribute is missing and improves readability and integration for other developers without needing extra validation code.
- ☐ **Provide private services**: Many providers are designed for global scope, but not all services need to be provided globally. If a service is used only by one component and directly tied to it, provide it inside that component. This improves tree-shaking and reduces dead code risk.
- ☐ **Use OnPush by default**: `ChangeDetectionStrategy.OnPush` limits checks to input reference changes, component-originated events, observable emissions consumed with `async`, or manual change detection, which substantially improves performance.
- ☐ **Defer non-critical code**: Deferrable views, also known as `@defer` blocks, reduce the initial bundle size of the application by deferring the loading of code that is not strictly necessary for the initial rendering of a page.
- ☐ **Use pure pipes**: Pure pipes run only when Angular detects a pure input change (primitive value change or object/array reference change), instead of on every change detection cycle. This reduces repeated template computation, lowers CPU usage, and keeps rendering predictable in large component trees.

- ☐ **Use virtual scrolling**: Virtual scrolling reduces DOM rendering and calculation costs by rendering only the visible window plus a small buffer. This improves performance for large lists. Angular CDK provides support for virtual scrolling.
- ☐ **Use lazy loading**: Lazy loading, combined with code splitting, decreases the initial JavaScript payload by partitioning the application into logical chunks that are fetched only when the corresponding features are accessed.
- ☐ **Use SSR/prerender/hydration when beneficial**: For content-heavy, SEO-sensitive, or slow-network scenarios, prefer Angular rendering strategies (SSR, prerendering, hydration) to improve perceived performance and discoverability.
- ☐ **Develop in a modular, standalone-first way**: Organize by feature boundaries and route ownership. In modern Angular, prefer standalone APIs and feature-level provider scoping, while using NgModules only when they add clear value.
- ☐ **Use reactive typed forms**: Reactive forms provide a model-driven approach to inputs whose values change over time. They move validation logic out of templates, keep templates cleaner, and use an explicit, immutable approach to form state so every change returns a new consistent state. Also, they are strictly typed by default.
- ☐ **Use functional features**: Using standalone components, functional guards, and signals make code more concise, easier to reason about, and less tied to heavy patterns. They reduce boilerplate and improve performance by aligning better with modern reactive programming practices.
- ☐ **Use a state management library**: In large and complex apps, or where state is shared across many components, we should consider a state management library. A Store keeps state logic in one place, reduces communication between components, improves predictability, and can improve performance through memoization and Angular change detection integration.
- ☐ **Add caching mechanisms**: Some API responses do not change often, so caching them can avoid repeated requests. This avoids unnecessary network calls, improves speed, and prevents downloading the same information repeatedly.
- ☐ **Provide a global error handling**: Centralize cross-cutting error handling (global error handler, HTTP interceptors, route-level guards/resolvers), enforce sensible timeout/retry policies only where idempotent, and expose user-friendly fallback states.
- ☐ **Instrument production behavior**: Add logging, tracing, and UX/performance telemetry (for example Web Vitals and API latency) so regressions are visible before users report them.
- ☐ **Do not yield to web browser whims**: Browser-specific code quickly becomes hard to maintain and outdated. Scripts that depend on a single browser can fail with new versions or different engines. Build on agreed standards so code works for everyone and requires fewer maintenance fixes.

- ☐ **Add meaningful unit tests**: Angular CLI generates `*.spec.ts` files, but they should not be left empty or reduced to component initialization without real tests. Empty or trivial specs give a false sense of security. We should test the most fragile parts, not just what is easiest to test.
- ☐ **Treat security as a default concern**: Avoid bypassing Angular sanitization (`bypassSecurityTrust*`) unless strictly required and reviewed, never trust client input, and validate/encode data at every boundary.
- ☐ **Use Angular CLI**: Use Angular CLI because it standardizes the workflow, speeds up development, and simplifies testing, builds, and maintenance.

2.10 Final Remarks

This chapter brought together the foundations and day-to-day practices required to build modern Angular applications with consistency, scalability, and long-term maintainability. We reviewed Angular's ecosystem, architecture, and core concepts, then connected those fundamentals to modern reactive patterns with RxJS and Signals, showing how Angular combines class-based structure with a more functional and declarative implementation style.

We also covered the broader engineering concerns such as authentication and authorization, error handling, accessibility, internationalization, SEO, analytics, debugging, and development workspace organization. Finally, the style guide and best-practice sections emphasized the importance of keeping the code predictable, small, testable, and easy to evolve.

3 Architecting Angular Applications

3.1 Overview

Software Architecture defines the fundamental organization of a system, namely, its major components, the relationships between them, and the principles guiding its evolution.

People often mix concepts related to software architecture. Software architecture describes the high-level structure of the system, while software design focuses on implementation-level decisions in modules, classes, and functions. Likewise, design principles provide foundational guidelines for making design decisions, design patterns solve recurring problems inside components, and architecture patterns organize how larger subsystems collaborate.

Modern web systems must satisfy scalability, availability, security, reliability, and resiliency requirements. To meet these demands, applications are commonly organized into layers (for example three-tier architecture) and delivered either as multi-page systems, single-page applications, or hybrids. In a traditional setup, frontend and backend are tightly coupled and released together, making it simple but less flexible. A decoupled architecture separates them through APIs, allowing independent development and deployment, though it increases integration complexity. A headless approach goes further, with backend services exposing only APIs while the frontend fully handles presentation, enabling multi-channel delivery but requiring more advanced coordination.

This chapter focuses on practical ways to design Angular applications, avoid misapplying backend patterns to frontend concerns, and adopt principles that support sustainable delivery speed and long-term maintainability. Angular is an opinionated framework that helps enforce structure, but architectural quality still depends on decisions made by the developer team or software architects.

3.2 Fundamentals

3.2.1 Design Principles

3.2.1.1 Keep It Simple, Stupid

KISS, an acronym for keep it simple, stupid, is a design principle that states that most systems work best if they are kept simple rather than made complicated. Therefore, simplicity should be a key goal in design, and unnecessary complexity should be avoided. This is because more complex code is written, more difficult it becomes to modify at any later point of time. Messy code means writing solutions of multiple problems in one block, method, or class. This might lead to add some of unnecessary lines in code. There may also be a situation where there could be multiple solutions to one problem. For instance, in some cases, both switch statement and if-else statements provide solution to a problem.

3.2.1.2 Don't Repeat Yourself

DRY, don't repeat yourself, is a principle of software development aimed at reducing repetition of software patterns, replacing it with abstractions or using data normalization to avoid redundancy. The programmers usually write enormous duplicate codes accidentally or un-accidentally. This principle forces us to avoid this habit. The application should avoid specifying behavior related to a particular concept in multiple places as this practice is a frequent source of errors. Some of examples could be declaring excessive local variables for each assignment and initialization. Writing multiple methods and classes for the same task again and again.

3.2.1.3 LIFT

LIFT principle helps us to find code quickly. Structuring the app such that we can *locate* code quickly, *identify* the code immediately, keep the *flattest* structure we can, and *try* to be DRY. keep a flat folder structure as long as possible. It's very important to give good names for methods, variables, and parameters. 5 seconds rule says that if we can't understand in 5 seconds, we probably need a refactor. We must organize the class code by putting the most important things first, properties followed by methods, grouped and sorted with consistent naming and spelling matter.

3.2.1.4 Single Responsibility Principle

The single responsibility principle applies to object-oriented design but can also be considered as an architectural principle like separation of concerns. It states that objects should have only one responsibility and that they should have only one reason to change. Following this principle

helps to produce more loosely coupled and modular systems, since many kinds of new behavior can be implemented as new classes, rather than by adding additional responsibility to existing classes. It is one of SOLID Principles.

3.2.1.5 Open–Closed Principle

The open–closed principle states "software entities should be open for extension, but closed for modification" that is, such an entity can allow its behavior to be extended without modifying its source code. Which means a class, methods, or functions should be written in a way that it is ready for adopting/adding new features but not interested in any modification. In the context of object-oriented design, this could be achieved with the concept of Abstraction and Polymorphism. Open–closed principle is one of the five SOLID principles.

3.2.1.6 Liskov Substitution Principle

One of the basic principles of object-oriented design where any base class can appear, subclasses must appear. LSP is the cornerstone of inheritance and reuse. Only when the derived class can replace the base class, the function of the software unit is not affected, the base class can be truly reused, and the derived class can also add new behavior based on the base class. The user of a base class should be able to use an instance of a derived class without knowing difference. It is one of SOLID Principles.

3.2.1.7 Interface Segregation Principle

The interface-segregation principle (ISP) states that no client should be forced to depend on methods it does not use. This usually happens when an interface contains more than one function and client needs only one but not all. In this case client is forced to implement all functionality of that interface. which is insane and should be avoided. Hence, one must always keep interface cohesive and narrow and focused on one and only one thing. Interface-segregation principle is one of the five SOLID principles.

3.2.1.8 Dependency Inversion Principle

Dependency inversion principle is one of SOLID Principles and a key part of building loosely coupled applications, since implementation details can be written to depend on and implement higher-level abstractions, rather than the other way around. High-level modules should not rely on low-level modules, both of which should rely on their abstractions. Abstractions should not rely on details. Details should rely on abstractions. The resulting applications are more testable,

modular, and maintainable as a result. The practice of dependency injection is made possible by following the dependency inversion principle.

3.2.1.9 Separation of Concerns

Separation of concerns asserts that software should be separated based on the kinds of work it performs. Architecturally, applications can be logically built to follow this principle by separating core business behavior from the user-interface logic. Ideally, business rules and logic should reside in a separate layer, which should not depend on other layers in the application. This separation helps ensure that the business model is easy to test and can evolve without being tightly coupled to low-level implementation details. Separation of concerns results in more degrees of freedom for some aspect of the program's design, deployment, or usage.

3.2.1.10 YAGNI

You aren't gonna need it (YAGNI) is a principle of extreme programming that states a programmer should not add functionality until deemed necessary. The principle helps developers avoid wasted effort on features that are assumed to be needed at some point. Even if a feature ends up being desired, it still may turn out that the implementation is not necessary. The argument is for developers to not waste time on creating extraneous elements that may not be necessary and can hinder or slow the development process.

3.2.1.11 Encapsulation

Different parts of an application should use encapsulation to insulate them from other parts of the application. Application components and layers should be able to adjust their internal implementation without breaking their collaborators as long as external contracts are not violated. Proper use of encapsulation helps achieve loose coupling and modularity in application designs, since objects and packages can be replaced with alternative implementations so long as the same interface is maintained.

3.2.1.12 Bounded Contexts

Bounded contexts are a central pattern in Domain-Driven Design. They provide a way of tackling complexity in large applications or organizations by breaking it up into separate conceptual modules. Each conceptual module then represents a context that is separated from other contexts (hence, bounded), and can evolve independently. Each bounded context should ideally be free to choose its own names for concepts within it and should have exclusive access to its own persistence store.

3.2.2 Design Patterns

3.2.2.1 Creational Patterns

Creational patterns focus on how objects are created so the code remains flexible, testable, and less tightly coupled to concrete implementations. They centralize or abstract instantiation logic to improve reuse and make future changes easier. Examples: Abstract Factory, Builder, Factory Method, Prototype, Singleton.

3.2.2.2 Structural Patterns

Structural patterns describe how classes and objects can be composed into larger systems while preserving clarity, flexibility, and maintainability. They help teams manage complexity by defining stable relationships and delegation boundaries between parts of the codebase. Examples: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.

3.2.2.3 Behavioral Patterns

Behavioral patterns define how objects collaborate, communicate, and distribute responsibilities to keep workflows explicit and adaptable. They are especially useful when control flow, event handling, or algorithm selection must vary without rewriting existing code. Examples: Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

3.2.3 Architectural Patterns

3.2.3.1 Layered

The most common architecture pattern is the layered pattern, otherwise known as the n-tier architecture pattern. Components within the layered architecture pattern are organized into horizontal layers, each layer performing a specific role within the application. Although the layered architecture pattern does not specify the number and types of layers that must exist in the pattern, most layered architectures consist of four standard layers: presentation, business, persistence, and database. Thus, smaller applications may have only three layers, whereas larger and more complex business applications may contain five or more layers. One of the powerful features of the layered architecture pattern is the *separation of concerns* among components. Components within a specific layer deal only with logic that pertains to that layer. For example, components in the presentation layer deal only with presentation logic, whereas components residing in the business layer deal only with business logic.

3.2.3.2 Client-Server

The client-server pattern is a distributed application structure that partitions tasks between the providers of a resource or service, called servers, and service requesters, called clients. A client sends a request to the server and waits for a response. The server processes the request and returns the appropriate response. This is the foundational model for most Angular applications where the Angular application runs as the client in the browser and communicates with a backend server via HTTP or WebSocket protocols. Angular's built-in HttpClient module provides a powerful and ergonomic way to communicate with server-side APIs using RESTful or GraphQL endpoints.

3.2.3.3 Master-Slave

The master-slave pattern consists of two parties, namely, the master distributes the work among identical slave instances and computes a final result from the results which the slaves return. This pattern is commonly used in database replication, where the master database is regarded as the authoritative source and the slave databases are synchronized to it. In the context of Angular, a similar concept can be observed in Web Workers. Angular supports Web Workers through the CLI (`ng generate web-worker`), allowing computationally heavy tasks to be offloaded from the main thread (master) to worker threads (slaves), keeping the UI responsive.

3.2.3.4 Pipe-Filter

The pipe-filter pattern provides a structure for systems that process a stream of data. Each processing step is enclosed within a filter component, and data is passed through pipes between adjacent filter components. The filters can be recombined and reused to build related pipelines. In Angular, RxJS operators follow this pattern closely. The `pipe()` method in RxJS allows developers to chain operators such as `map`, `filter`, `switchMap`, `debounceTime`, and others to transform, filter, and compose asynchronous data streams in a declarative and reusable manner. Angular Pipes in templates also follow this concept, transforming displayed values within a view.

3.2.3.5 Broker

The broker pattern is used to structure distributed systems with decoupled components that interact by remote service invocations. A broker component is responsible for coordinating communication, such as forwarding requests, as well as for transmitting results and exceptions. Message brokers like RabbitMQ or Apache Kafka are typically used on the server side. On the client side, Angular applications can leverage WebSockets or libraries like Socket.IO to establish

real-time communication channels, effectively using the backend broker to decouple producers and consumers of messages.

3.2.3.6 Peer-to-Peer

The peer-to-peer pattern is a distributed architecture in which individual components, known as peers, are equally privileged and can act both as clients and servers. Each peer can initiate or complete transactions. In the web context, WebRTC enables peer-to-peer communication directly between browsers for audio, video, and data sharing. Angular applications can integrate WebRTC using libraries like PeerJS to build real-time collaborative features, video conferencing, or file-sharing applications without routing all data through a centralized server.

3.2.3.7 Event-Bus

The event-bus pattern uses an event-driven architecture, where components communicate by publishing and subscribing to events through a central bus. This pattern promotes loose coupling between components. In Angular, a simple event bus can be implemented using an RxJS Subject or BehaviorSubject within a shared service. Components inject this service and either emit events or subscribe to them. For more complex scenarios, state management libraries like NgRx implement a global event bus through actions and effects, where dispatched actions serve as events that multiple reducers and effects can respond to.

3.2.3.8 Model-View-Controller

Model-View-Controller (MVC) is one of the most widely used architectural patterns. It divides an application into three interconnected components: the Model (data and business logic), the View (user interface), and the Controller (handles input and updates the model). The predecessor of Angular, AngularJS, was heavily influenced by the MVC pattern. Modern Angular has evolved beyond traditional MVC into a component-based architecture, but the conceptual separation remains, where templates serve as the View, component classes act as the Controller, and services with models represent the Model layer.

3.2.3.9 Model-View-Presenter

Model-View-Presenter (MVP) is a derivation of MVC where the Presenter assumes the role of the middle-man. In MVP, the View is passive and delegates all user interactions to the Presenter, which then manipulates the Model and updates the View. Angular's smart and dumb component pattern closely resembles MVP. The container (smart) component acts as the Presenter, handling state and logic, while the presentational (dumb) component acts as the View, only

receiving data via `@Input()` and emitting events via `@Output()`. This approach makes testing easier and promotes reusability of presentational components.

3.2.3.10 Model-View-ViewModel

Model-View-ViewModel, MVVM, is an architectural pattern that facilitates the separation of the graphical user interface (View) from the business logic (Model) through an intermediary (ViewModel). The ViewModel exposes data and command objects that the View can bind to, enabling a clean separation of concerns while supporting two-way data binding. Unlike MVC, the ViewModel has no direct reference to the View. It simply exposes an observable state that the View consumes. Angular's architecture aligns closely with MVVM. Angular's template binding syntax (`[property]`, `(event)`, `[(ngModel)]`) provides the declarative data-binding mechanism that is central to MVVM. The component class manages the presentation logic, transforms data from services into view-friendly formats, and handles user interactions without the View needing to know about the underlying services or data sources. This makes Angular components highly testable, as the ViewModel logic can be unit tested independently of the DOM.

3.2.3.11 Blackboard

The blackboard pattern is useful for problems for which no deterministic solution strategy is known. The central idea is a shared data store (the blackboard) that multiple specialized subsystems (knowledge sources) can read from and write to. A control component monitors the blackboard and coordinates the knowledge sources. While not commonly used directly in frontend applications, the concept is analogous to a centralized state store like NgRx Store. The store acts as the blackboard, reducers and effects act as knowledge sources, and selectors enable components to reactively observe specific slices of state.

3.2.3.12 Interpreter

The interpreter pattern defines a grammatical representation for a language and provides an interpreter to deal with this grammar. It is used to evaluate sentences in a language by representing each grammar rule as a class. In Angular, the template compiler itself is an interpreter that parses Angular template syntax (control-flow blocks like `@if`, `@for`, binding expressions, and pipes) and translates them into executable instructions. Angular's expression language within templates is a domain-specific language interpreted at runtime to resolve bindings, evaluate conditions, and render dynamic content.

3.2.3.13 Microservices

Microservices architecture structures an application as a collection of loosely coupled, independently deployable services. Each service is built around a specific business capability and communicates via well-defined APIs. While microservices are primarily a backend concern, the frontend equivalent is the micro-frontends architecture, which applies the same principles to the UI layer. In Angular, micro-frontends are commonly implemented with Module Federation in Webpack 5, Native Federation for modern Angular builds, or with single-spa when orchestration across heterogeneous frameworks is needed. This enables large organizations to scale their frontend teams and codebases effectively.

3.3 Angular-Based Architectures

3.3.1 Clean Architecture

The Clean Architecture keeps business rules and application workflows independent from frameworks and delivery mechanisms. On the frontend, that means Angular, *HttpClient*, router configuration, browser storage, and UI widgets should remain at the edges of the system rather than shaping the core of the application. Dependencies point inward, namely, UI and infrastructure depend on application logic, and application logic depends on domain concepts, but the domain does not depend on Angular.

For Angular applications, Clean Architecture is usually easier to implement with the following four practical layers:

- Domain: Pure business concepts such as entities, value objects, and domain rules. This layer should be framework-agnostic using TypeScript.
- Application: Use cases, orchestration logic, facades, and ports. This layer coordinates workflows such as “load customer profile”, “submit checkout”, or “save draft”.
- Infrastructure: Adapters for HTTP APIs, browser storage, analytics, feature flags, and other external systems. This is where `HttpClient`, `localStorage`, IndexedDB, and third-party SDKs belong.
- Presentation: Angular components, routes, templates, guards, and view state composition. This layer translates user interaction into use-case execution and renders the resulting state.

A practical folder structure for Angular can look like this:

```
src/
├── app/
│   ├── core/                    # Cross-cutting setup, shared policies
│   ├── domain/                  # Business rules, framework-agnostic
│   │   └── entities/
│   ├── application/             # Use cases and workflow orchestration
│   │   └── use-cases/
│   ├── infrastructure/          # Framework-dependent adapters
│   │   ├── api/                 # HttpClient implementations
│   │   ├── persistence/         # localStorage/indexedDB adapters
│   │   └── mappers/
│   └── presentation/            # Angular UI layer
│       ├── pages/
│       ├── components/
│       └── routing/
```

Notes for applying Clean Architecture in Angular:

- Keep `domain` and most of `application` as pure TypeScript with no Angular imports. This keeps business logic highly testable and portable.
- Make dependencies point inward only: presentation and infrastructure may depend on application/domain, but domain must not depend on Angular, `HttpClient`, Router, or browser APIs.
- Define repository and gateway interfaces in inner layers and implement them in infrastructure adapters.
- Use Angular dependency injection to bind interfaces (ports) to concrete implementations through provider tokens.
- Keep components thin. Let facades or use cases coordinate workflows and expose view-friendly state.
- Map external DTOs to domain models in infrastructure mappers to avoid leaking backend contracts into core business logic.
- Test by layer. Fast unit tests for domain and use cases, adapter tests for infrastructure, and component integration tests for presentation.

3.3.2 Hexagonal Architecture

Hexagonal Architecture, also known as the Ports and Adapters pattern, was introduced by Alistair Cockburn. The core idea is to place the application's business logic at the center and

define explicit boundaries (ports) through which it communicates with the outside world. Adapters implement these ports, translating between the application's internal API and external systems such as HTTP APIs, databases, UI frameworks, or message queues.

In the context of Angular applications:

- **Core (Domain)**: Contains business logic, domain models, and use cases. This layer has no dependencies on Angular, RxJS, or any framework, therefore, it is pure TypeScript. Entities, value objects, and business rules live here.
- **Ports**: Interfaces that define the contracts for communication between the core and the outside world. Inbound ports (use cases) define what the application can do. Outbound ports (repository interfaces, gateway interfaces) define what the application needs from external systems.
- **Adapters**: Concrete implementations of ports. Inbound adapters are Angular components and route handlers that invoke use cases. Outbound adapters are Angular services that implement repository interfaces using `HttpClient`, `localStorage`, or other external APIs.

The folder structure looks like:

```
src/
├── app/
│   ├── domain/                  # Pure TypeScript - no Angular imports
│   │   ├── models/
│   │   ├── ports/               # Interfaces (inbound and outbound)
│   │   └── use-cases/
│   ├── infrastructure/          # Outbound adapters
│   │   ├── api/                 # HttpClient-based implementations
│   │   └── storage/             # LocalStorage/IndexedDB implementations
│   └── presentation/            # Inbound adapters
│       ├── components/
│       └── pages/
```

The key benefit of hexagonal architecture for Angular applications is that the domain layer can be tested without Angular's dependencies, making tests faster and more focused. It also allows swapping external dependencies (e.g., replacing a REST API with GraphQL or a mock implementation) without changing business logic.

3.3.3 Layered Frontend Architecture

Layered Frontend Architecture is a practical way to organize Angular code by responsibility. It applies classic Layered Architecture concepts to the client side so that UI concerns, business logic, and data access stay separated and can evolve independently.

In the context of Angular applications:

- **Presentation**: Components, templates, routing adapters, form handling, and view state. It should avoid direct `HttpClient` calls and complex business rules.
- **Domain**: Use cases, orchestration, validation rules, derived state, and decision logic. It should be framework-light and easy to unit test.
- **Data**: HTTP clients, persistence adapters (`HttpClient`, `localStorage`, IndexedDB), DTO interfaces, and mapping functions.

Data flow is usually top-down and event-up:

- User interacts with a component in the Presentation layer.
- The component calls a facade or application service in the Domain layer.
- The Domain layer orchestrates use cases and requests data from the Data layer.
- The Data layer performs I/O and maps DTOs to domain models.
- The result propagates back to the Domain layer, then to the Presentation layer for rendering.

The folder structure looks like:

```
src/
├── app/
│   ├── presentation/
│   │   ├── pages/
│   │   └── components/
│   ├── domain/
│   │   ├── models/
│   │   └── services/
│   └── data/
│       ├── api/
│       └── mappers/
```

For small projects, a lightweight version of layered architecture is usually enough. Keep components in presentation folder, move business rules to domain services, and centralize all backend communication in dedicated API services. As complexity grows, teams can progressively introduce stricter boundaries (ports, repositories, and dedicated mappers) without rewriting the whole application.

3.3.4 Feature-Based Architecture

Feature-Based Architecture organizes an Angular application around business capabilities (features) instead of technical types. Rather than keeping all components in one folder and all services in another, each feature owns its UI, state, services, models, and routes. This makes the codebase easier to scale as teams and product scope grow. Typical examples of features are `auth`, `users`, `orders`, `billing`, and `admin`.

Core principles:

- **Slice by business domain**: Group files by feature, not by artifact type.
- **High cohesion**: Keep code that changes together in the same feature folder.
- **Low coupling**: Reduce dependencies between features; communicate through explicit contracts.
- **Encapsulation**: Expose a small public API from each feature and keep implementation details private.

A common structure looks like this:

```
src/
├── app/
│   ├── core/                    # Singletons (auth, interceptors, guards)
│   ├── shared/                  # Reusable UI blocks (no business logic)
│   ├── features/                # Business capabilities
│   │   └── users/
│   │       ├── pages/
│   │       ├── components/
│   │       ├── services/
│   │       ├── state/
│   │       ├── models/
│   │       └── users.routes.ts
│   └── app.routes.ts
```

Routing aligns naturally with feature boundaries. Each feature should define its own route configuration and be lazy-loaded when possible. This improves initial load performance and keeps boundaries explicit. In practice, Feature-Based Architecture and Layered Frontend Architecture are complementary. A common approach is to structure the app by features first, and then apply internal layering (presentation, domain, data) inside each feature when complexity justifies it.

3.3.5 Modular Architecture

Modular Architecture refers to software architecture that can be built from self-contained modules which are deployed and developed independently. This allows for greater flexibility and speed when adapting to fast market changes. Modular architecture in Angular refers to dividing an application into smaller, reusable, and maintainable chunks called “modules.” Each module encapsulates a set of related components, services, directives, pipes, and other dependencies. Angular itself revolves around the concept of NgModules, making modular design a natural fit for any Angular project.

Core principles:

- **Encapsulation**: Each module hides its internal implementation and exposes only a well-defined public API through declarative exports.
- **Single responsibility**: A module should have one clear purpose (e.g., authentication, user management, billing).
- **Loose coupling**: Modules communicate through contracts (exported services, models, interfaces) rather than direct internal dependencies.
- **High cohesion**: Related functionality stays together, but unrelated code is separated into different modules.
- **Lazy loading**: Feature modules can be loaded on demand, reducing initial bundle size and improving startup performance.

A common structure looks like this:

```
src/
├── app/
│   ├── core/                    # Singleton services, guards, interceptors
│   ├── shared/                  # Reusable, non-domain components
│   ├── features/                # Feature modules (lazy-loaded)
│   │   ├── users/
```

```
│   │   │   ├── components/
│   │   │   ├── services/
│   │   │   ├── models/
│   │   │   ├── users.module.ts
│   │   │   └── users-routing.module.ts
│   │   └── orders/
│   ├── app.module.ts
│   └── app-routing.module.ts
```

With this modular structure, we get the benefit of lazy loading, which optimizes performance by loading feature modules only when they are required. This is especially useful in large applications like e-commerce platforms, where some features may not be needed until specific user interactions occur. This approach also makes testing simpler and faster, as each module can be tested independently. It enables parallel development, where multiple teams or developers can work on different modules without interfering with each other, leading to faster development cycles.

3.3.6 Micro-Frontends Architecture

Micro-frontends architecture extends microservice principles to the frontend by splitting a large UI into smaller, independently developed and deployed frontend applications. Instead of one Angular application owning every route and feature, a host (shell) application composes multiple feature-focused remotes at runtime. This architecture is particularly useful when multiple teams need to deliver frontend features independently, different business domains require separate release cycles, a large SPA has become difficult to scale and maintain, or an incremental migration from legacy frontend systems is needed.

Core implementation considerations:

- **Clear ownership boundaries**: Each remote should map to a business capability, not a technical layer.
- **Shared dependency strategy**: Share Angular, RxJS, and UI libraries carefully to avoid version conflicts and duplicated bundles.
- **Contract-first integration**: Define route contracts, events, and cross-app APIs explicitly (and version them).
- **Runtime resilience**: The shell should handle remote loading failures gracefully with fallbacks and error boundaries.

- **Consistent UX governance**: Use a shared design system so independently deployed remotes still feel like one product.

A typical structure looks like this:

```
apps/
├── shell/                    # Host app, top-level routing and layout
├── mfe-auth/                 # Remote app owned by auth team
├── mfe-orders/               # Remote app owned by orders team
├── mfe-billing/              # Remote app owned by billing team
└── shared-lib/               # Shared design system and others
```

Micro-frontends improve team autonomy and deployment flexibility, but they also introduce operational and architectural complexity (orchestration, observability, testing across app boundaries, and shared-state coordination). A practical strategy is to start with a modular monolith and introduce micro-frontends only for domains that clearly benefit from independent delivery.

3.4 Data Flow Architecture

3.4.1 Reactive Architecture

Reactive Architecture is an approach to building systems that are responsive, resilient, elastic, and message-driven. In Angular, reactivity is a first-class citizen through the deep integration of RxJS. Reactive programming allows developers to work with asynchronous data streams, compose complex event-driven logic, and manage state changes predictably. Angular's `HttpClient`, `Router`, reactive forms, and many other built-in services are all built on top of Observables.

The reactive approach in Angular encourages:

- **Push-based data flow**: Components subscribe to observable streams and react to changes as they are emitted, rather than polling or manually checking for updates.
- **Declarative composition**: Using RxJS operators (`combineLatest`, `merge`, `switchMap`, `withLatestFrom`, etc.) enables developers to express complex asynchronous logic declaratively.
- **Async Pipe**: The AsyncPipe in templates automatically subscribes and unsubscribes from observables, reducing boilerplate and preventing memory leaks.

- **Reactive Forms**: Angular's Reactive Forms Module provides an observable-based API for form inputs, enabling real-time validation, conditional logic, and dynamic form construction.

Angular Signals, introduced in Angular 16+, complement RxJS by providing a synchronous, fine-grained reactivity primitive. Signals allow developers to define reactive state that automatically tracks dependencies and triggers efficient re-renders. The combination of Signals for synchronous UI state and RxJS for asynchronous event streams provides a powerful and flexible reactive architecture.

3.4.2 Component-Based Architecture

Component-based architecture has gained a lot of popularity in frontend development over the past years. It's a pattern that fits nicely in the context of developing frontend applications and enables developers to write maintainable and extensible code. A component is a piece of HTML and JavaScript combined. The central idea is to decompose the user interface into a tree of reusable, self-contained components. Each component encapsulates its own template, styles, and behavior, communicating with other components through well-defined inputs and outputs. This encapsulation enables independent development, testing, and reuse.

A key distinction in component-based architecture is between **Container Components** (Smart) and **Presentational Components** (Dumb):

- **Container (Smart) Components**: These components are aware of the application state and services. They fetch data, dispatch actions, and coordinate logic. Container components inject services, interact with the store, and pass data down to presentational components. They typically have minimal template logic and delegate rendering to child components.
- **Presentational (Dumb) Components**: These components are purely concerned with how things look. They receive data via `@Input()` properties and communicate events back to their parent via `@Output()` event emitters. Presentational components have no knowledge of services, stores, or routing because they are fully controlled by their parent. This makes them highly reusable, easy to test, and simple to reason about.

Container component

```
@Component({
  selector: 'app-users-container',
  template: `<app-user-list [users]="users$ | async"
(selectUser)="onSelect($event)" />`
})
export class UsersContainerComponent {
  readonly users$ = this.userService.getUsers();

  constructor(private readonly userService: UserService) {}

  onSelect(user: User) {
    this.router.navigate(['/users', user.id]);
  }
}
```

Presentational component

```
@Component({
  selector: 'app-user-list',
  template: `
    <ul>
      @for (user of users; track user.id) {
        <li (click)="selectUser.emit(user)">{{ user.name }}</li>
      }
    </ul>`
})
export class UserListComponent {
  @Input() users: User[] = [];

  @Output() selectUser = new EventEmitter<User>();
}
```

Following this separation leads to a cleaner component tree where data flows downward through inputs and events bubble upward through outputs, establishing a unidirectional data flow that makes the application predictable and easier to debug.

3.4.3 Event-Driven Architecture

Event-Driven Architecture (EDA) is a software design paradigm where the flow of an application is determined by events (signals that represent something significant has occurred in the system). Instead of components directly calling one another, they communicate asynchronously through events, enabling loose coupling and scalable systems. In Angular applications, event-driven design is commonly implemented with RxJS streams and framework primitives such as `@Output()/EventEmitter`, `Subject/BehaviorSubject`, Router events, and store actions.

In an event-driven model:

- **Event Producers** (emitters) generate and broadcast events when something happens (e.g., user action, data change, API response).
- **Event Consumers** (listeners/subscribers) react to those events independently without knowing who triggered them.
- **Event Bus** (or broker) acts as a mediator, managing event routing and delivery between producers and consumers.

Core principles of event-driven architecture:

- **Loose coupling**: Producers and consumers don't depend on each other directly. They only know about events.
- **Asynchronous communication**: Events are processed asynchronously, allowing components to work concurrently without blocking.
- **Scalability**: Multiple services can consume the same event simultaneously, enabling horizontal scaling.
- **Real-time responsiveness**: Applications react to state changes immediately as events are emitted.
- **Extensibility**: New consumers can be added without modifying producers or existing code.

Event-driven patterns are most valuable for real-time applications (chat, notifications, live dashboards), decoupled feature interactions within large applications, asynchronous workflows and side effect orchestration, systems where multiple services need to react to the same occurrence, or applications built around domain events (Domain-Driven Design). For simple CRUD applications with linear workflows, event-driven architecture may introduce unnecessary complexity. NgRx is a practical way to formalize event-driven architecture at scale for Angular applications.

3.4.4 Signal-Based Architecture

Angular Signals, introduced in Angular 16 and progressively enhanced in subsequent versions, represent a fundamental shift in how Angular applications manage reactivity and state. Unlike RxJS Observables, which are push-based and require explicit subscription management, Signals provide a synchronous, pull-based reactivity model with automatic dependency tracking.

Core Signal primitives:

- **signal()**: Creates a writable signal that holds a value and notifies consumers when it changes.
- **computed()**: Creates a derived signal that automatically recalculates when its dependencies change. Computed signals are lazily evaluated and memoized.
- **effect()**: Runs a side-effect function whenever any signal it reads changes. Effects are primarily used for logging, persistence, or synchronization with external systems.

Signal-based architecture brings several architectural advantages:

- **Simplified change detection**: Signals enable zoneless change detection, where Angular can precisely identify which components need to be re-rendered without relying on Zone.js or running change detection across the entire component tree.
- **Reduced boilerplate**: Signals eliminate the need for `async` pipe, manual subscription management, and `OnDestroy` cleanup for component-local state.
- **Interoperability with RxJS**: The `toSignal()` and `toObservable()` functions from `@angular/core/rxjs-interop` provide seamless conversion between Signals and Observables, allowing gradual adoption.
- **Signal-based inputs and outputs**: Angular 17+ introduces signal-based `input()` and `output()` functions that replace the `@Input()` and `@Output()` decorators, providing a more consistent reactive API.
- **Signal store**: The NgRx SignalStore offers a lightweight, signal-based state management solution that leverages Angular's native reactivity without the overhead of actions and reducers.

The emerging best practice is to use Signals for synchronous component and application state, and reserve RxJS for asynchronous operations (HTTP requests, WebSocket streams, complex event composition).

3.5 State Management Architecture

3.5.1 Application State Management

The state of an application is a representation of a system at a given time. The state keeps the data of different components in sync because each update will re-render all relevant

components. Ensuring that the UI components have access to the correct and current state is essential for the application to render correctly. In Angular, state changes are typically triggered by events or async updates.

Types of state:

- **Data + loading state**: The list of todo items the frontend renders and whether the list is loading. Put into NgRx/NGXS/etc.
- **Global UI state**: Whether the user is logged in or a value of a global search bar. The server doesn't store this data at all. Put into NgRx/NGXS/etc.
- **Local UI state**: Whether a dropdown is expanded. The rest of the frontend doesn't care about this. Use component state.
- **Form state**: The values of fields in a form. This is a subset of local UI state. Use Angular's Reactive Forms to manage form state as a controlled component.
- **URL state**: The route the user is on now. Manage via the Angular Router.
- **Page state**: A page may have components that interact with each other in a complex way, but not with components on other pages. Store data just for the page (or pass down a plain object with Inputs/Services/etc.)

When to use a state management solution:

- Use only component/local state when data is needed by a single component subtree and does not need persistence, cross-page sharing, or complex synchronization.
- Use feature-level state (service + `BehaviorSubject`, signals, or a small store) when multiple components in the same feature need the same state and coordinated updates.
- Use global state management (NgRx/NGXS/SignalStore) when state is shared across distant features, requires predictable transitions, or needs advanced tooling.

Effective state management is crucial in larger frontend applications. Redux is probably the most popular way of managing state. Start with component/local state and RxJS Subjects for small features. As complexity grows and state becomes shared across distant features, migrate to NgRx SignalStore for lighter lift, to full NgRx for strict Redux-style structure and advanced tooling, or to NGXS for a class-oriented approach with reduced boilerplate and strong plugin support.

3.5.2 Solutions for State Management

3.5.2.1 RxJS Subjects

Observable data services are a state management solution implemented using RxJS to mimic Redux Architecture. Using RxJS Subjects for state management is a lightweight pattern that works well when NgRx would be too heavy. The idea is simple: keep state in a service, expose it as read-only observables, and update it through explicit methods.

The most commonly used Subject types are:

- **Subject<T>**: Emits only new values to current subscribers. Useful for events.
- **BehaviorSubject<T>**: Stores the latest value and emits it immediately to new subscribers. Most common for feature state.
- **ReplaySubject<T>**: Replays a configurable number of past values. Useful for cached streams or history-like behavior.

This approach gives many Redux-like benefits (single source of truth, predictable updates, testability) with less boilerplate code. The main tradeoff is that as applications grow, conventions may drift without strict tooling. A practical strategy is to start with RxJS Subject-based stores for feature-local state and adopt NgRx when global state complexity, team size, and side-effect orchestration require stronger structure.

3.5.2.2 NgRx

NgRx is a framework for building reactive applications in Angular. It provides libraries for managing global and local state, isolation of side effects, entity collection management, router integration, and developer tooling. NgRx is the Angular-specific implementation of the Redux pattern, powered entirely by RxJS.

NgRx consists of several packages, the most important of which are:

- **@ngrx/store**: A global state management solution based on RxJS, powered by actions and reducers. The store is a single, immutable data structure that represents the entire application state.
- **@ngrx/effects**: Handles side effects in NgRx. Effects listen for actions dispatched from the store, perform asynchronous work (API calls, navigation, etc.), and dispatch new actions based on the results.

- **@ngrx/router-store**: Connects the Angular Router to the NgRx Store, serializing router state and enabling route-based actions and selectors.

NgRx is recommended for applications with a significant amount of shared state, complex state transitions, or when team scalability and predictability of data flow are priorities. The strong tooling support through Redux DevTools and ngrx-store-devtools provides powerful debugging capabilities.

3.5.2.3 NgRx SignalStore

NgRx SignalStore is a lightweight, modern state management solution built on Angular Signals. It was introduced as a simpler alternative to the traditional NgRx Store + Effects pattern while maintaining the benefits of NgRx's ecosystem and developer tooling.

NgRx SignalStore key features include:

- **Signal-based state**: State is defined using Angular Signals, providing automatic dependency tracking and fine-grained reactivity without RxJS boilerplate.
- **Simplified data flow**: No need for actions, reducers, or effects for basic state management. Direct mutations and computed selectors are simpler to reason about.
- **Composable store features**: Plugins and features (such as `withState`, `withComputed`, `withMethods`) compose together for clean architecture.
- **Interop with RxJS**: Signals and observables can be seamlessly converted, allowing gradual migration from RxJS-heavy patterns.
- **Type safety**: Full TypeScript support with strict typing for store shape, methods, and computed values.

NgRx SignalStore is recommended for teams that want a modern, lightweight state management approach with Angular's native reactivity model. It reduces boilerplate compared to traditional NgRx while maintaining consistency with the NgRx ecosystem. Use it for feature-level or global state when complexity doesn't yet justify the Redux pattern.

3.5.2.4 NGXS

NGXS is a state management pattern and library for Angular that takes a different approach from NgRx. NGXS aims to reduce boilerplate by using TypeScript classes and decorators to define state, actions, and selectors. NGXS is modeled after the CQRS pattern commonly used in backend architectures.

Key features of NGXS include:

- **Class-based state**: States are defined as decorated TypeScript classes, making them more intuitive for object-oriented developers.
- **Action handlers**: Actions are handled within the state class itself using `@Action()` decorators, co-locating state mutations with state definitions.
- **Selectors**: Defined as static methods on the state class using `@Selector()` decorators, enabling memoized data selection.
- **Plugins**: NGXS provides a rich plugin ecosystem including logger, devtools, storage, router, form, HMR, and WebSocket plugins.

NGXS is a good choice for teams that prefer a more class-oriented and less boilerplate-heavy approach to state management while still maintaining the benefits of a predictable, centralized state container.

3.5.2.5 Elf

Elf is a lightweight, RxJS-first state management library focused on composable stores and minimal boilerplate. Unlike Redux-style solutions, Elf does not require actions, reducers, or effects by default. Instead, state is managed through small store instances, reusable operators, and focused feature plugins.

Key features of Elf include:

- **Store-first API**: Create focused stores with `createStore()` and update state through explicit immutable operations.
- **Composable plugins**: Add capabilities such as entities, UI entities, requests status, pagination, and history through optional plugins.
- **Entity management**: The `@ngneat/elf-entities` package provides normalized entity collections with helpers for CRUD operations and selectors.
- **RxJS-native design**: Selectors and updates are observable-based, making integration with Angular services and existing RxJS flows straightforward.
- **Small footprint and incremental adoption**: Elf can be introduced per feature without forcing a whole-application migration strategy.

Elf is a strong option for teams that want more structure than ad hoc `BehaviorSubject` services but less ceremony than full NgRx. It works particularly well for feature-level stores,

medium-complexity applications, and projects that prefer a pragmatic, plugin-based approach to state management.

3.5.2.6 Mobx

MobX is a state management library that makes state management simple and scalable by transparently applying functional reactive programming. mobx-angular provides Angular bindings for MobX.

MobX follows a different philosophy from Redux-inspired libraries:

- **Observable state**: Any value can be made observable using decorators or utility functions. Changes to observable state are tracked automatically.
- **Computed values**: Derived values are computed automatically and cached until their dependencies change.
- **Reactions**: Side effects that run automatically when observed state changes, similar to Angular's effect system.
- **Actions**: Methods that modify state, ensuring all mutations are tracked and batched.

MobX allows for a more mutable programming model, which can feel more natural to developers coming from object-oriented backgrounds. The tradeoff is less explicit data flow compared to Redux-style patterns, which may make debugging and tracing state changes more difficult in very large applications.

3.5.2.7 State Machines

State machines provide a mathematical model for representing a finite number of states and the transitions between them. In frontend development, state machines help model complex UI flows such as multi-step forms, authentication flows, media players, and data loading states. Instead of managing boolean flags and conditional logic, state machines make valid states and transitions explicit. XState is the most popular JavaScript/TypeScript library for creating and interpreting finite state machines and statecharts.

Benefits of using state machines in Angular applications:

- **Explicit state modeling**: All possible states and transitions are defined upfront, eliminating impossible states by design.

- **Visual tooling**: Stately Studio allows developers to model, visualize, and simulate state machines, improving communication between developers and stakeholders.
- **Testability**: State machines can be tested in isolation without rendering components, verifying that correct transitions occur for given events.
- **Complex flow management**: Multi-step wizards, authentication flows, and real-time features become easier to reason about and maintain.

State machines are particularly valuable when the UI has well-defined states with strict transition rules, reducing bugs caused by impossible state combinations. In Angular, XState is typically integrated through a thin facade/service that owns the machine or actor lifecycle and exposes state to components via Signals or Observables. For store-based usage, official Angular bindings are available via @xstate/store-angular.

3.5.3 Best Practices

The following practices provide a practical checklist for designing state management that remains explicit, scalable, and testable as the application grows:

- ☐ **Share state only when needed**: Keep state local by default and promote it only when two or more parts of the application need the same source of truth. Over-sharing state makes features tightly coupled and harder to change.
- ☐ **Keep state close to usage**: Place state in the narrowest scope that satisfies the requirement (component -> feature service/store -> global store). This minimizes the range of changes when requirements change.
- ☐ **Avoid global form state**: Form values, touched/dirty flags, and validation errors usually belong in Angular Reactive Forms. Promote to shared/global state only when another feature truly depends on in-progress form values.
- ☐ **Maintain a single source of truth**: Store each business fact in one canonical place (for example, `users.entities`), then read it everywhere else via selectors/computed values. Duplicate copies drift and create synchronization bugs.
- ☐ **Model canonical state explicitly**: Define state shape intentionally (entities, ids, selectedId, filters, pagination, request status) instead of letting it evolve ad hoc. A clear schema makes state predictable and easier to debug.

- ☐ **Compute derived state**: Do not persist values that can be calculated from canonical state (totals, filtered lists, badges, permissions). Use selectors, `computed()`, or pure functions to derive them consistently.
- ☐ **Use selectors as the read API**: Read shared state through selector functions or `computed()` signals instead of scattering direct state-tree access across components. Memoize expensive derivations to avoid unnecessary recalculation.
- ☐ **Define dependency direction**: Keep dependency flow unidirectional (UI -> actions/methods -> state update -> selectors -> UI). Avoid circular dependencies between stores/services/features to preserve testability and readability.
- ☐ **Prefer flatter and normalized structures**: For collections, use normalized `entity-by-id + ids` structures instead of deeply nested trees. Flat state reduces update complexity and prevents accidental deep mutation.
- ☐ **Use immutable update patterns**: Replace objects/arrays with new references for updates instead of mutating in place. Immutability improves change detection behavior, memoization correctness, and time-travel debugging.
- ☐ **Prefer serializable shared state**: Keep shared/global store state serializable (plain objects, arrays, primitives) to improve debugging, persistence, replayability, and tooling interoperability. Keep non-serializable runtime objects (`Date`, `Map`, `Set`, class instances, DOM refs, functions, observables) out of canonical shared state.
- ☐ **Let the store layer own state shape**: Reducers/store updaters should validate and control the resulting structure. Avoid blind replacements/merges such as assigning arbitrary payload objects directly into state without schema awareness.
- ☐ **Control complexity deliberately**: Start with the simplest viable approach (component state or small feature store), then introduce heavier patterns (NgRx/NGXS/SignalStore) only when cross-feature coordination or side effects justify it.
- ☐ **Track request lifecycle explicitly**: Model async status as first-class state (`idle`, `loading`, `success`, `error`) and keep error details available for UI and telemetry. This prevents spinner and retry logic from becoming scattered.
- ☐ **Model transitions as state machines**: Gate updates by both event and current state (for example, accept `loadSuccess` only from `loading`). This prevents impossible transitions and race-condition bugs in async flows.

- ☐ **Keep side effects outside state calculation**: Reducers/computed functions should stay pure. Put HTTP calls, storage access, timers, analytics, and navigation in effects, facades, or dedicated services.
- ☐ **Model updates as domain events, not UI setters**: Prefer event-based updates (`orderSubmitted`, `profilePatched`) over generic setters (`setData`, `updateStore`). Event-style naming improves traceability and scales better across features.
- ☐ **Avoid multiple dispatches for one user intent**: Do not emit many sequential updates for a single conceptual transaction when one event (or an explicit batch/transaction) can represent it. This reduces intermediate invalid states and unnecessary re-renders.
- ☐ **Persist only what is necessary**: Persist durable state (auth session metadata, user preferences, draft identifiers) and avoid persisting transient UI state (open panels, hover flags). Never persist sensitive data without a clear security model.
- ☐ **Test state behavior directly**: Unit test update logic and selectors/computed rules as pure behavior. Add focused tests for effects/async orchestration so state transitions remain deterministic and refactoring-safe.

3.6 API Layer Architecture

3.6.1 Service Organization

The API layer is responsible for all communication between the Angular application and backend services. A well-structured API layer separates concerns, centralizes configuration, and provides a consistent developer experience.

The following example shows a recommended approach for organizing API services by domain:

```
src/app/
├── core/
│   ├── api/
│   │   ├── interceptors/
│   │   │   ├── auth.interceptor.ts
│   │   │   ├── error.interceptor.ts
│   │   │   └── loading.interceptor.ts
│   │   ├── models/
│   │   │   ├── api-response.model.ts
│   │   │   └── pagination.model.ts
```

```
│   │   └── services/
│   │       ├── user-api.service.ts
│   │       ├── product-api.service.ts
│   │       └── order-api.service.ts
```

Each API service encapsulates HTTP calls for a specific domain entity and maps server responses to domain models. This mapping layer (often called DTO mapping or adapter pattern) insulates the application from backend contract changes. Centralize cross-cutting concerns such as authentication, logging, and generic error handling in interceptors. To improve efficiency, avoid duplicate HTTP requests by applying sharing or caching strategies. Finally, keep all API communication within dedicated services, allowing components to focus on UI orchestration rather than HTTP implementation details.

Maintaining all this code manually is not a good practice, since much of it can be generated from the API contract.

3.6.2 Client Code Generation

Writing client services by hand is error-prone and creates drift between the backend contract and the frontend implementation. When the backend exposes an OpenAPI specification, code generators can automatically produce type-safe Angular services, models, and enums. This approach, when automated, it keeps the API layer in sync with the server at all times.

The ng-openapi-gen is one of the most popular OpenAPI code generators specifically designed for Angular. It reads an OpenAPI 3.0 specification (JSON or YAML) and generates Angular services and models that use `HttpClient` natively:

```
npx ng-openapi-gen -i https://api.example.com/swagger.json -o dist/@api
```

To maximize the value of API code generation, integrate it into the development workflow:

- **npm scripts**: Add a generation script (e.g., `"api:generate": "ng-openapi-gen"`) to `package.json` so developers can regenerate the API layer with a single command.
- **CI/CD validation**: Run the generator in CI and fail the build if the generated output differs from the committed code. This catches spec-vs-code drift early.

Several alternative generators support Angular or produce framework-agnostic TypeScript clients:

- **openapi-generator**: A comprehensive, language-agnostic generator with a dedicated `typescript-angular` template. It supports OpenAPI 2.0 and 3.x and provides fine-

grained control over output through templates and configuration options. Available as a CLI, Maven/Gradle plugin, and Docker image.

- **swagger-codegen**: The original Swagger code generator, maintained by SmartBear. It supports Angular via the `typescript-angular` language option but has been largely superseded by openapi-generator.
- **Orval**: A modern TypeScript-first code generator that supports OpenAPI 3.x and produces Angular services using `HttpClient`, as well as mock data generation using MSW or faker.js. Orval supports custom transformers and output overrides.
- **openapi-typescript**: Generates TypeScript types from OpenAPI schemas without opinionated service code. Paired with its companion `openapi-fetch`, it provides a lightweight, type-safe HTTP client that can be wrapped in Angular services.
- **GraphQL Code Generator**: For GraphQL APIs, this tool generates TypeScript types, Angular services, and typed query/mutation hooks from GraphQL schemas and operations. The `@graphql-codegen/typescript-apollo-angular` plugin generates ready-to-use Angular services for Apollo Angular.
- **gRPC-Web**: For backends using gRPC, the `protoc` compiler with the `protoc-gen-grpc-web` plugin generates TypeScript client stubs from `.proto` files that can be consumed in Angular applications via the `grpc-web` runtime.

3.6.3 REST API Integration

REST (Representational State Transfer) remains the most widely used architectural style for web APIs, where resources are addressed by URLs and manipulated through standard HTTP methods such as `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.

In Angular applications, communication with REST endpoints is handled through HttpClient from `@angular/common/http`, typically configured in modern projects with `provideHttpClient()` (or, in module-based setups, `HttpClientModule`). The standalone configuration is generally preferred because it composes cleanly with feature functions like `withInterceptors()` for cross-cutting concerns (for example authentication headers, logging, and centralized error handling) and `withFetch()` for Fetch API support, which is important for Angular SSR compatibility.

Since transient failures are unavoidable in distributed systems, REST calls should also include resilience strategies, and RxJS operators such as retry with a delay function are commonly used to implement controlled exponential backoff.

3.6.4 GraphQL Integration

GraphQL is a query language and runtime for APIs, originally developed at Facebook and now maintained by the GraphQL Foundation. Unlike REST, where the server defines the shape of each response, GraphQL allows clients to request exactly the data they need in a single request. This eliminates over-fetching (receiving more data than needed) and under-fetching (needing multiple requests to assemble related data). Core concepts include **queries** (read operations), **mutations** (write operations), **subscriptions** (real-time push, commonly over WebSocket), and a strongly typed **schema** that serves as the API contract.

Apollo Angular is the most widely adopted GraphQL client for Angular. It can be installed with `ng add apollo-angular` (or standard package installation) and configured through the `provideApollo()` function. Apollo integrates with Angular's dependency injection and exposes RxJS observables for application data flows. The `watchQuery` method provides reactive queries that automatically update when the cache changes, while `query` performs one-time fetches. Mutations support an `update` callback for manual cache updates to keep the UI in sync without re-fetching. Subscriptions commonly use the `graphql-ws` library with link splitting to route queries and mutations over HTTP while subscriptions flow over WebSocket.

The GraphQL Code Generator automates TypeScript type and Angular service generation from GraphQL schemas and `.graphql` operation files. Using the `typescript`, `typescript-operations`, and `typescript-apollo-angular` plugins, it produces fully typed, ready-to-use Angular services. This eliminates manual type definitions and provides compile-time safety when the schema changes. The generator should be integrated into the development workflow via npm scripts and validated in CI.

3.6.5 REST vs GraphQL

The following table summarizes the main considerations for deciding which solution we should use:

Concern	**REST**	**GraphQL**
Data fetching	Fixed response shapes, potential over/under-fetching	Client specifies exact data needs
Caching	HTTP caching (ETags, Cache-Control) works natively	Requires client-side cache (Apollo Cache)
File uploads	Native multipart support	Requires workaround (e.g., graphql-upload)

Versioning	URL or header-based versioning	Schema evolution with deprecation
Real-time data	Requires separate WebSocket implementation	Built-in subscriptions
Tooling maturity	Mature (OpenAPI, Postman, Swagger)	Growing (GraphiQL, Apollo Studio)
Error handling	HTTP status codes	Always 200 with `errors` array
Learning curve	Low	Moderate

Use REST when the API is simple, CRUD-oriented, and consumed by multiple client types that benefit from HTTP caching. Use GraphQL when the application has complex data requirements, multiple nested resources, or needs real-time subscriptions.

3.6.6 WebSocket Integration

WebSockets provide full-duplex communication channels over a single TCP connection, enabling real-time, bidirectional data flow between client and server. Unlike HTTP request-response cycles, WebSockets maintain a persistent connection where both sides can send messages at any time. Common use cases include chat and messaging, live notifications, collaborative editing, live dashboards, gaming, and IoT data streams.

RxJS provides a built-in WebSocketSubject that wraps the native WebSocket API with an observable-based interface. It supports configurable serialization/deserialization, open and close observers, and allowing the same connection to handle multiple logical channels via the `multiplex()` method. An Angular service wrapping `WebSocketSubject` typically exposes `connect()`, `send()`, and `disconnect()` methods alongside a `messages$` observable stream. WebSocket connections can drop due to network changes, server restarts, or idle timeouts. A robust implementation should use the RxJS `retry` operator with exponential backoff and jitter to prevent thundering herd problems when many clients reconnect simultaneously. WebSocket messages can be mapped to NgRx actions within effects or exposing them through Angular signals.

Socket.IO provides a higher-level abstraction over WebSockets with automatic reconnection, room-based broadcasting, acknowledgments for message delivery confirmation, and fallback to HTTP long-polling when WebSockets are blocked. The `socket.io-client` library is framework-agnostic and integrates with Angular by wrapping event listeners as RxJS observables within an injectable service. Socket.IO is preferred over raw WebSockets when the application needs automatic transport fallback, room-based messaging, or delivery acknowledgments.

3.6.7 Server-Sent Events Integration

Server-Sent Events (SSE) are a lightweight option for server-to-client real-time streaming in Angular applications. SSE uses a single long-lived HTTP connection with the `text/event-stream` content type, allowing the server to continuously push updates to the browser without repeated polling. SSE is ideal for use cases where communication is one-way and clients primarily listen for updates, such as notifications, activity feeds, job progress updates, monitoring dashboards, and live status indicators.

Angular does not provide a dedicated SSE abstraction in `HttpClient`, so the usual approach is to wrap the browser's EventSource in an RxJS observable or an Angular service. This keeps connection lifecycle management, parsing, error handling, and teardown logic in one place.

3.6.8 WebSocket vs SSE

The following table summarizes the main considerations for deciding which solution we should use:

Concern	**WebSocket**	**SSE**
Browser API	WebSocket	EventSource
Communication model	Full-duplex (client and server can both send messages)	Server-to-client only (client uses HTTP for writes)
Transport	Dedicated persistent connection (HTTP upgrade to `ws`/`wss`)	Standard HTTP streaming with `text/event-stream`
Reconnection	Typically implemented manually or via libraries (Socket.IO)	Built-in automatic reconnect behavior in `EventSource`
Message direction for interactive apps	Ideal for two-way, low-latency interactions (chat, multiplayer, collaborative editing)	Better for one-way live updates (notifications, progress, dashboards)
Complexity	Higher (stateful connection management, scaling strategy)	Lower (works well with standard HTTP infrastructure)

Use SSE when the client mainly consumes real-time updates and simplicity over HTTP is preferred. Use WebSockets when the client must also send frequent low-latency messages to the server. A common architecture is REST for CRUD, SSE for live read updates, and standard HTTP write requests (`POST`, `PUT`, `PATCH`, `DELETE`) for data changes.

3.7 CSS Architecture

3.7.1 Fundamentals

CSS architecture defines how styles are organized, structured, and maintained in a project. As applications grow, unstructured CSS becomes a major source of technical debt, leading to specificity wars, duplicated rules, and unexpected side effects. It covers naming conventions, selector strategy, component boundaries, design tokens, and responsive behavior, not just syntax choices. Without an explicit architecture, teams usually drift into duplicated rules, specificity conflicts, and fragile overrides.

Historically, Sass and preprocessors such as Less and Stylus improved authoring with variables, mixins, and modular partials. Later, methodologies such as BEM, SMACSS, OOCSS, ITCSS, and SUITCSS introduced stronger structure for large teams. Today, modern projects combine these ideas with native CSS capabilities such as custom properties, cascade layers, and container queries.

In practice, CSS architecture is also enabled by tooling. Build-time tools such as PostCSS and Autoprefixer, quality gates such as Stylelint, and browser diagnostics via Chrome DevTools help enforce consistency, catch regressions early, and validate layout behavior across breakpoints. In Angular specifically, View Encapsulation, project-level style configuration in Angular CLI, and optional Shadow DOM strategies shape how local and global styles interact.

The following sections apply this foundation progressively: first through concrete **CSS methodologies** and **styling patterns**, then through **project structure** and **responsive strategies** (fluid, adaptive, responsive, mobile-first, and responsive media), and finally through **perceived responsiveness** and **resilient CSS** practices. Together, these topics form a practical CSS architecture for long-lived Angular applications.

3.7.2 CSS Methodologies

CSS methodologies are structured rule systems for organizing selectors, naming classes, layering rules, and defining how styles should evolve across a codebase. In other words, a methodology gives a team a shared way to write CSS consistently, not just individual syntax features or framework-specific tooling.

The following established CSS methodologies help teams write scalable and maintainable stylesheets:

- **BEM** (Block, Element, Modifier): A naming convention that creates clear, self-documenting class names (e.g., `.card__title--highlighted`). BEM prevents naming collisions and makes the relationship between HTML and CSS explicit.
- **SMACSS** (Scalable and Modular Architecture for CSS): Categorizes CSS rules into base, layout, module, state, and theme categories, promoting modularity and reuse.
- **OOCSS** (Object-Oriented CSS): Encourages separating structure from skin and container from content so visual patterns are reusable across components.
- **ITCSS** (Inverted Triangle CSS): Organizes CSS layers from generic to explicit (settings, tools, generic, elements, objects, components, utilities), ensuring that specificity increases progressively.
- **SUITCSS**: A component-oriented naming methodology (for example, `ComponentName`, `ComponentName--modifier`, `ComponentName-descendant`) that promotes clear boundaries and predictable selector intent.
- **CUBE CSS** (Composition, Utility, Block, Exception): A methodology that combines compositional layout rules with utility classes and component blocks, emphasizing low specificity, predictable styling, and maintainable, context-aware exceptions instead of deeply nested selectors.

Angular provides built-in view encapsulation that scopes styles to individual components by default (using emulated Shadow DOM). This significantly reduces global style conflicts. If view encapsulation is disabled (for example, `ViewEncapsulation.None`), global style collisions become much more likely, so following a strict CSS methodology is essential rather than optional.

3.7.3 CSS Styling Patterns

CSS styling patterns describe how styles are authored, scoped, and applied in practice. Unlike methodologies such as BEM or ITCSS, these patterns are more concerned with the implementation model. In Angular projects, the most natural patterns are usually component-scoped styles, utility classes, and token-driven global styles, while some patterns such as CSS Modules or styled-components are more common in other ecosystems.

The following styling patterns are common in frontend development:

- **Atomic CSS**: Builds interfaces by composing small, single-purpose utility classes, reducing custom CSS surface area and improving consistency when paired with design tokens and conventions.

- **Utility-First CSS**: A practical evolution of atomic ideas where teams compose interfaces from predefined utility classes for spacing, typography, color, layout, and state. Frameworks such as Tailwind CSS popularized this pattern.
- **Component-Scoped CSS**: Organizes styles around UI components rather than global pages or broad selectors. Each component owns its template, styles, and behavior, which aligns naturally with Angular's component model. This pattern reduces accidental coupling between unrelated screens and works especially well when shared design tokens and reusable primitives are defined globally.
- **CSS Modules**: Scope class names locally by default and generate unique identifiers at build time so styles do not leak globally. CSS Modules are popular in React and other bundler-driven ecosystems, but they are not a standard Angular styling mechanism because Angular already provides style scoping through view encapsulation.
- **CSS-in-JS**: Generates or attaches styles from JavaScript rather than relying only on standalone stylesheet files. This pattern often enables dynamic theming, co-located styles, and runtime style composition, but it also adds abstraction and can increase runtime cost or tooling complexity.
- **styled-components**: A specific CSS-in-JS library that lets developers create styled UI primitives directly in JavaScript. It is strongly associated with React and is generally not used in Angular applications.

3.7.4 CSS Project Structure

For Angular projects, keep application code under `src/app/` and shared style assets under `src/assets/` so the Angular structure and the CSS structure stay aligned with the default Angular workspace layout. Keep shared/global styles in `src/assets/styles/` and keep feature-specific styles close to each component (`*.component.scss`).

A project structure organized by file type can look like this:

```
src/
├── app/
├── assets/
│   ├── fonts/
│   ├── images/
│   └── styles/
│       ├── base/
│       │   ├── _reset.scss
```

```
│       │   ├── _typography.scss
│       │   └── _globals.scss
│       ├── tokens/
│       │   ├── _colors.scss
│       │   ├── _spacing.scss
│       │   ├── _typography.scss
│       │   └── _z-index.scss
│       ├── utilities/
│       │   ├── _mixins.scss
│       │   ├── _functions.scss
│       │   └── _helpers.scss
│       ├── layouts/
│       │   ├── _grid.scss
│       │   ├── _containers.scss
│       │   └── _shell.scss
│       ├── themes/
│       │   ├── _light.scss
│       │   └── _dark.scss
│       ├── vendor/
│       │   └── _overrides.scss
│       └── styles.scss
└── index.html
```

The recommended import order in `src/assets/styles/styles.scss` is: tokens -> utilities -> base -> layouts -> themes -> vendor. This ordering keeps variables and mixins available before rule declarations and reduces accidental override conflicts. Files prefixed with _ (for example, `_colors.scss` or `_mixins.scss`) follow the Sass partial convention. They are shared SCSS files meant to be imported into another stylesheet rather than treated as standalone entry-point files.

For most Angular projects, we don't need all of these folders. Typically, we only need to place the relevant files described in the project structure into the `styles` folder.

3.7.5 Responsive Strategies

3.7.5.1 Fluid vs Adaptive vs Responsive Design

Responsive strategies answer questions like **"How does the layout adapt to different screens and input contexts?"**. They are primarily about viewport, container, and media behavior in CSS. Common strategies include:

- **Responsive design**: It is a web design approach where a site's layout and content adapt to different screen sizes and resolutions using flexible grids, flexible images, and CSS media queries. The layout fluidly adjusts between breakpoints.
- **Adaptive design**: It serves different fixed layouts for specific screen sizes (breakpoints). Instead of fluidly resizing, the layout snaps between distinct versions designed for specific devices.
- **Fluid design**: It uses relative units (`%`, `vw`, `vh`, `rem`, `em`) and CSS functions like `clamp()` and `calc()` to create layouts and typography that scale proportionally to the viewport, without relying on media query breakpoints.

In Angular applications, responsive design is the most common approach. Angular Material and other libraries provide responsive utilities. The BreakpointObserver from Angular CDK allows components to reactively respond to viewport changes. Modern CSS features like CSS Grid, Flexbox, and Container Queries are the preferred approach for building responsive layouts without additional libraries.

3.7.5.2 Mobile First

Mobile First is a design and development strategy that prioritizes the mobile experience. The application is designed and built for mobile devices first, then progressively enhanced for larger screens (tablets, desktops). This approach ensures that the core experience works well on constrained devices and avoids the common pitfall of trying to squeeze a desktop layout into a small screen.

Practical Mobile First principles to guide web design:

- **Keep it simple**: Use whitespace intentionally, reduce clutter, and limit navigation to essential items.
- **Prioritize visual hierarchy**: Surface the most important content first with clear headings and text structure.
- **Optimize for scanning:** Place key information above the fold and keep paragraphs short.
- **Design for touch**: Avoid hover-only interactions and rely on familiar gestures.
- **Adapt media for mobile**: Simplify complex visuals and optimize images for small screens.
- **Ensure tap accessibility**: Design large touch targets, meaning targets that are not too small.
- **Respect usage context**: Account for device constraints, environment, and user intent.

Angular supports mobile-first development through responsive component design, lazy loading to minimize initial payload on resource-constrained devices, and PWA capabilities for offline support. The Angular CDK provides layout utilities that help build mobile-first responsive interfaces.

3.7.5.3 Responsive Media

Responsive architecture should also include media delivery strategy, not only layout adaptation. Large images and videos are often the main source of payload and rendering cost on mobile devices.

Core practices for responsive media include:

- **Use responsive images**: Combine `srcset` and `sizes` so browsers download the smallest acceptable asset per viewport and DPR.
- **Use art direction when needed**: Use the `<picture>` element when the crop itself should change by breakpoint, not only the resolution.
- **Prefer modern formats**: Serve AVIF/WebP with fallback formats where required.
- **Delay non-critical media**: Use `loading="lazy"` and `decoding="async"` for offscreen media.
- **Stabilize layout**: Set intrinsic dimensions (`width` and `height`) or `aspect-ratio` to reduce CLS.

In Angular, image optimization should be treated as an architecture concern. Use framework-supported image tooling, define media size conventions in design tokens, and validate real device behavior in performance testing.

3.7.6 Perceived Responsiveness

Perceived responsiveness answers questions like **"How fast does the product *feel* while work is in progress?"**. It is primarily about interaction feedback, loading behavior, and reducing user uncertainty. An application can still feel fast even with network delay if it gives immediate feedback through techniques like skeleton screens, optimistic updates, virtualization, and section-level loading states.

The following patterns are among the most widely used techniques to improve perceived responsiveness in Angular applications:

- **Skeleton screens**: Show layout-shaped placeholders while content loads. This reduces perceived waiting time compared to generic spinners.
- **Optimistic updates**: Apply expected UI changes immediately after user actions, then confirm or roll back when the server responds.
- **Virtualization**: Render only visible rows/items in large lists and tables (for example, with Angular CDK Virtual Scroll) to keep scrolling and interaction smooth.
- **Stale-while-revalidate**: Show cached data instantly and refresh in the background, updating the UI when fresh data arrives.
- **Prefetching likely next views**: Preload probable next routes/data to reduce navigation latency.
- **Granular loading states**: Use section-level loading indicators instead of blocking the entire page, so users can continue interacting with ready parts of the UI.
- **Layout stability**: Reserve space for dynamic content before it loads (images, ads, async sections, and fonts) by setting explicit dimensions, `aspect-ratio`, and predictable placeholders.

3.7.7 Resilient CSS

3.7.7.1 Progressive Enhancement

Progressive Enhancement is a strategy where we start with a robust baseline experience and then add advanced styling and interactions only when the browser supports them. The baseline should always preserve content readability, information hierarchy, and core user tasks. Enhancements then improve the experience (for example richer layouts, motion, or advanced CSS features) without making the baseline unusable.

Progressive enhancement principles can be applied through:

- **Angular Universal (SSR)**: Server-side rendering generates static HTML for the initial page load, ensuring content is visible even before JavaScript bootstraps.
- **Semantic HTML**: Using proper HTML elements (`<nav>`, `<main>`, `<article>`, `<button>`) ensures basic structure and accessibility regardless of JavaScript execution.
- **Feature detection**: Using feature detection (e.g., checking for browser API support) rather than browser sniffing to provide appropriate functionality.

3.7.7.2 Graceful Degradation

Graceful Degradation takes the opposite direction: build the full modern experience first, then define acceptable fallbacks when parts of that experience are unavailable. The goal is not identical behavior across every environment, but controlled degradation where the application remains understandable and usable even when some enhancements are missing.

In Angular applications, graceful degradation is common because most apps are SPA-first and rely on JavaScript for interaction. In practice, teams can still improve resilience by combining this approach with progressive enhancement techniques such as semantic HTML, SSR/prerendered content, and feature-based fallbacks for non-critical visual effects.

3.7.7.3 Defensive CSS

Defensive CSS is a practical approach to writing styles that remain stable under real-world stress, for example, long content, missing media, localization expansion, user zoom, and unexpected container sizes. Instead of assuming ideal markup and perfect content, defensive rules anticipate failure modes and prevent layout breakage.

Defensive CSS usually means defining robust defaults in shared components and feature styles:

- **Overflow safety**: Prevent layout breakage from long words, URLs, and dynamic text using wrapping and clipping strategies.
- **Flexible constraints**: Use sensible `min-width`, `max-width`, and container constraints so components adapt to narrow and wide contexts.
- **Resilient media behavior**: Ensure images, avatars, and cards degrade gracefully when assets fail or dimensions vary.
- **Content-agnostic components**: Design reusable components to tolerate optional fields, empty states, and translated content length changes.

3.7.7.4 Feature Queries and Fallbacks

Feature queries allow teams to safely adopt modern CSS while preserving a reliable baseline. The idea is to define a default style that works broadly, then apply enhanced rules only when the browser explicitly supports a feature.

This pattern is especially useful in Angular design systems where teams progressively introduce newer CSS capabilities such as container queries, advanced color spaces, or modern layout

primitives. Combined with progressive enhancement, feature queries reduce regression risk and make browser support decisions explicit in the stylesheet.

3.7.7.5 Accessible Motion Preferences

prefers-reduced-motion should be treated as a resilience concern, not only an accessibility add-on. A resilient UI should remain understandable and usable even when animation is reduced or disabled. This means avoiding motion-dependent meaning and providing equivalent non-animated feedback for state changes.

In Angular projects, the most robust baseline is to keep motion decisions in CSS first, then add runtime control only when needed. This aligns with Angular's current direction to favor native CSS animations and migrate away from framework-managed animations where possible. When disabling animation globally (for accessibility mode, tests, or emergency fallback), ensure the UI still communicates progress and completion.

3.8 Workspace Architecture

3.8.1 Standalone

A standalone repository is the default and most pragmatic starting point for the majority of Angular projects. In practice, this usually means one Angular application, possibly with a few internal libraries, deployed as a single frontend. Even at larger scale, this remains effective when teams enforce boundaries between workspace areas (for example `apps/`, `libs/`, `core/`, and `shared/`), keep reusable code in libraries, and standardize build, test, lint, and CI policies at the workspace level.

This workspace model is often the best choice when a product is managed by a single team (or closely coordinated teams), features are released together and share common infrastructure, there is significant code reuse between features, and maintaining a simple development and deployment process is more important than enabling independent deployments.

Move beyond a standalone repository workspace only when there is a strong organizational or delivery driver, such as independently versioned platforms, strict team autonomy with separate release trains, or cross-organization governance that cannot be handled well in a single repository. In practice, frontend teams should start with a standalone workspace repository and adopt monorepo-at-scale patterns, multi-repo, or micro-frontends only after clear constraints justify the added complexity.

3.8.2 Monorepo

A monorepo is a software development strategy where code for multiple projects, libraries, and applications is stored in a single version-controlled repository. Instead of having separate repositories for each package or application, all code lives together, shared build tools, dependencies, and configurations.

Monorepos help address some of these challenges by breaking the monolith into multiple libraries and applications within the same repository while maintaining the benefits of shared code and unified tooling. They enable atomic commits across related projects, provide unified development tooling and standards, simplify large-scale refactoring, and ensure all projects use the same dependency versions, reducing compatibility issues. Common trade-offs include larger repository size, stricter CI discipline to avoid long pipelines, and the need for clear ownership boundaries to prevent cross-team coupling.

The JavaScript ecosystem offers several mature options for monorepo management. **Package-manager workspace tooling** provides the foundation for linking local packages, sharing dependencies, and standardizing installs:

- **npm workspaces**: Native workspace support in npm that links local packages inside a single repository, making dependency sharing and multi-package development straightforward without introducing an additional monorepo framework.
- **Yarn workspaces**: Similar to npm workspaces, Yarn provides built-in monorepo support.
- **pnpm workspaces**: The pnpm package manager provides workspace support with efficient disk usage through content-addressable storage and strict dependency isolation.

Orchestration and release tooling builds on that foundation with higher-level capabilities such as task pipelines, affected-project execution, caching, and coordinated versioning:

- **Nx**: A full-featured monorepo platform with project graph analysis, affected-task execution, generators, and local/remote caching to speed up builds and tests in large Angular codebases.
- **Lerna**: A multi-package management tool focused on versioning and publishing workflows. Lerna optimizes the workflow around managing multi-package repositories with Git and npm.

- **Turborepo**: A high-performance build system for JavaScript and TypeScript monorepos, focused on task caching and parallelization.
- **Rush**: A scalable monorepo manager developed by Microsoft, designed for large teams with enterprise-grade features like phased builds and custom policies.

For Angular projects, Nx is a common choice because of its Angular-focused generators, dependency graph tooling, and caching capabilities, but the best fit depends on team size, release model, and existing package-manager strategy.

3.8.3 Multi-Repo

A multi-repo strategy stores each application or library in its own repository. This model provides stronger repository-level isolation, independent permissions, and simpler per-repo CI/CD pipelines, which can fit organizations with independent product teams and separate release governance.

The trade-off is higher coordination overhead for cross-repository changes, more complex shared-library version management, and reduced visibility across dependent projects. In Angular ecosystems, multi-repo can work well when teams are operationally independent, while monorepo is often preferred when teams need frequent shared refactoring and unified standards.

3.9 Angular Rendering Strategies

3.9.1 Client-Side Rendering

Client-Side Rendering (CSR) is the default rendering model for Angular single-page applications. The browser initially loads a minimal HTML shell plus JavaScript bundles, and Angular then bootstraps the application and renders views on the client.

In CSR, route transitions happen without full page reloads via the Angular Router. Data is typically fetched from APIs after navigation and rendered once it arrives. This creates highly interactive user experiences and enables rich client-side state management. However, CSR has trade-offs, including slower initial rendering on slower devices because the browser must download and execute JavaScript before content becomes visible, potential SEO limitations when content is not server-rendered or statically generated, and a dependency on JavaScript, meaning that if scripts fail, users may see little or no usable content.

Practical CSR optimization focuses on reducing initial load cost and improving perceived performance. A key approach is route-level code splitting through lazy loading, which loads only the necessary feature modules on demand and reduces the initial bundle size. Performance can be further improved by deferring non-critical parts of the UI using deferrable views, allowing Angular to prioritize essential rendering work before loading secondary template sections. Another important strategy is optimizing delivery through fast transfer and caching, typically achieved with service workers and properly configured HTTP caching headers to minimize repeat load costs.

Alternatively, a hybrid approach can be used, where CSR is reserved for highly interactive or private routes, while SSR or SSG is applied to public entry points to improve first-load performance and SEO.

3.9.2 Server-Side Rendering

Server-Side Rendering (SSR) is a rendering strategy where the initial HTML for a route is generated on the server and sent to the browser, instead of shipping an almost empty HTML shell and waiting for client-side JavaScript to render the first view. After the server-rendered HTML is delivered, Angular hydrates the page on the client so it becomes fully interactive. In modern Angular versions, SSR is integrated directly into the framework and CLI via @angular/ssr, but many teams still use the term "Angular Universal" to describe the same concept.

In Angular, server-side rendering (SSR) is particularly valuable for improving the initial user experience, as it enables faster first render so users can see meaningful content sooner, especially on slower devices or networks. It also enhances SEO and social sharing, since crawlers and link preview systems can directly read server-rendered HTML and metadata, and it can improve Core Web Vitals metrics such as Largest Contentful Paint (LCP).

When adopting SSR, applications need to be designed to run correctly in both server and browser environments. This includes being aware of platform differences, since browser-only APIs such as `window`, `document`, `localStorage`, and `navigator` do not exist on the server, so access to them must be guarded using platform checks like isPlatformBrowser or abstracted behind environment-safe services. It is important to use HTTP and caching strategies such as CDNs or reverse proxies for cacheable routes because SSR increases load on the server. Use SSR where SEO and initial paint are important, while keeping purely internal or highly interactive pages client-rendered when that approach is simpler and more efficient.

SSR without hydration would force Angular to re-render the full page on the client, wasting work and causing visual instability. With hydration, Angular reuses server-rendered DOM and attaches

event handlers, preserving the initial HTML. To avoid duplicate data fetching (server fetch + client fetch), Angular can transfer fetched data from server to browser via TransferState. This reduces extra API calls and improves startup performance.

Alternatively, AnalogJS is a meta-framework for Angular inspired by the developer experience of frameworks like Next.js/Nuxt. It provides SSR, file-based routing, API routes, static site generation, and Vite-powered tooling on top of Angular.

3.9.3 Static Site Generation

Static Site Generation (SSG), also called prerendering, generates HTML files for selected routes at build time. Instead of rendering pages on each request, Angular produces static assets that can be served from a CDN or static hosting platform. SSG is part of the Angular SSR toolchain and can be configured so that known routes (for example, `/`, `/about`, `/docs/*`) are emitted as static HTML during build. At runtime, the browser loads prebuilt HTML and then Angular hydrates/bootstraps the page for interactivity.

SSG is well suited for marketing pages and documentation where content is mostly stable, as well as SEO-sensitive landing pages that benefit from immediately available, crawlable HTML. It is also effective for high-traffic routes because serving pre-rendered static files significantly reduces server load and operational cost. However, it comes with trade-offs. Per-request personalization is limited in the initial HTML, making it less suitable for highly user-specific experiences. Large sets of dynamic routes can increase build time and output size, and content freshness is tied to rebuild or redeployment cycles unless augmented with runtime data hydration from APIs.

In practice, many frontend teams adopt a hybrid model: pre-render public/static routes, use SSR for request-specific pages, and keep fully CSR behavior for highly interactive or private routes.

3.9.4 Deferrable Views

Deferrable Views are an Angular template-level performance feature that allows postponing rendering and loading of non-critical content. Using the `@defer` block, Angular can split code and load a section only when a trigger condition is met. This approach improves startup performance by prioritizing above-the-fold content while deferring expensive UI elements such as charts, editors, maps, or secondary panels until they are actually needed. Loading can be triggered in several ways, including when the browser is idle, when content enters the viewport, after user interaction, or based on a timer or custom condition.

From an architectural perspective, only non-critical content that is not required for initial interaction should be deferred. Placeholders should remain lightweight and visually stable to prevent layout shifts. These deferrable views work best when combined with route-level lazy loading for cumulative performance benefits. Their impact should be validated using tools like Angular DevTools and standard Core Web Vitals metrics.

In Angular, deferrable views also support companion UI states to improve user experience during loading transitions. A `@placeholder` block can display lightweight fallback content before the main component loads, `@loading` provides an explicit loading state while resources are being fetched, and `@error` enables graceful handling when deferred content fails to load.

Deferrable views complement SSR and hydration by rendering critical content on the server first, then progressively activating lower-priority islands as needed.

3.9.5 Incremental Hydration

Incremental Hydration extends Angular hydration by enabling parts of a server-rendered page to become interactive in stages, instead of hydrating the entire application at once. This reduces main-thread contention during startup and improves responsiveness on slower devices. With classic full hydration, Angular attaches listeners and initializes component trees broadly after SSR HTML arrives.

Incremental Hydration is a performance strategy designed for pages with large component trees where full hydration would be expensive, as well as layouts with mixed criticality zones where primary UI elements should become interactive first while secondary sections are deferred. It is also particularly useful for mobile-heavy traffic, where CPU and network constraints make full-page hydration costly.

Incremental Hydration is best introduced as an optimization layer once baseline SSR + Hydration are stable and the route and data architecture is already well established.

3.10 Software Architecture Documentation

3.10.1 C4 Model

The C4 Model is a hierarchical approach to visualizing software architecture at four levels of abstraction: **System Context**, **Container**, **Component**, and **Code**. Each level provides increasing

detail, allowing different stakeholders (business, technical leads, developers) to understand the architecture appropriate to their concerns.

Level 1 - System Context: Shows the software system as a single box with external systems and users around it. Answers the question: "What is the system?" and how does it interact with external systems.

Level 2 - Container: Decomposes the system into containers (applications, services, databases, message queues). Each container is a separately deployable/executable unit. For an Angular application, containers might include the SPA, API, database, and external services.

Level 3 - Component: Shows the major structural building blocks within a container. In an Angular application, components correspond to modules, services, and architectural layers. This level shows how the container is composed and internal dependencies.

Level 4 - Code: The most detailed view, showing classes, interfaces, and functions. This level is often automated through IDE navigation and code documentation rather than manually maintained diagrams.

The C4 Model is particularly useful for communicating architecture to non-technical stakeholders (System Context), aligning teams on module boundaries (Component level), and planning micro-frontend or monorepo layouts (Container level). Diagrams can be created using Structurizr, Miro, or Draw.io with DSL-based options such as C4-PlantUML.

3.10.2 arc42

arc42 is a template for documenting and communicating software architecture. It provides a standardized structure with 12 sections that balance brevity with comprehensive coverage, suitable for systems of varying complexity. Unlike narrative documentation, arc42's structured sections ensure architectural decisions are captured consistently across teams and projects.

The 12 sections are:

1. **Introduction and Goals**: Business context, stakeholders, and key requirements.
2. **Constraints**: Technical, organizational, and regulatory constraints shaping decisions.
3. **Context and Scope**: System boundaries and external dependencies.
4. **Solution Strategy**: Core architectural approaches and technology choices.
5. **Building Block View**: Static decomposition of the system into components and their relationships (equivalent to C4 levels 2-3).

6. **Runtime View**: Dynamic behavior showing how components interact during execution (sequence diagrams, use case scenarios).
7. **Deployment View**: Infrastructure, deployment units, and runtime environments.
8. **Crosscutting Concepts**: Overall, principal regulations and solution approaches relevant in multiple parts (cross-cutting) of the system.
9. **Architectural Decisions**: Important, expensive, critical, large scale or risky architecture decisions including rationales (often cross-referenced with ADRs).
10. **Quality Requirements**: How the system achieves quality attributes like performance, security, and scalability.
11. **Risks and Technical Debt**: Known limitations and future improvement areas.
12. **Glossary**: Domain terminology and abbreviations.

arc42 is valuable for communicating architecture to distributed teams and ensuring comprehensive documentation of design rationale. Solutions like AsciiDoc or Markdown combined with build systems can generate HTML/PDF documentation from arc42 templates. The arc42 provides free templates in multiple formats and languages.

3.10.3 Architecture Decision Records

Architecture Decision Records (ADRs) are lightweight, time-stamped documents that capture significant architectural decisions, their context, rationale, and consequences. Each ADR is typically a single Markdown file following a standard template, stored in version control alongside code or in documentation platforms. ADRs create a searchable, immutable history of why architectural choices were made, enabling new team members to understand the decision context without extensive oral knowledge transfer.

A typical ADR contains:

- **Title**: A concise name describing the decision (e.g., "Use NgRx for global state management").
- **Status**: Proposed, Accepted, Deprecated, Superseded (with a reference to the superseding ADR).
- **Context**: The situation that prompted the decision, including constraints and alternatives considered.
- **Decision**: The chosen solution and why it was selected.
- **Consequences**: Benefits and trade-offs resulting from the decision.

- **Related ADRs**: References to related decisions or dependencies.

Tools like adr-log can generate an index of all ADRs and auto-linkage between related decisions. ADRs should be reviewed to ensure decisions are properly documented before implementation and should be consulted during architecture reviews to understand the reasoning behind existing patterns.

3.10.4 Documentation as Code

Documentation as Code (DaC) applies software engineering practices to documentation. Instead of living in disconnected tools, documentation lives next to the code in Git, evolves through pull requests, is versioned with releases, and is validated in CI. Architecture is not just a diagram of the current system, but it is a history of decisions, constraints, and trade-offs.

With DaC, each architectural change should update three things together:

- **Structure**: Context/container/component views and dependency boundaries.
- **Decisions**: ADRs that explain why a solution was chosen.
- **Operations**: Deployment, observability, scalability, and security implications.

A practical DaC setup for Angular architecture work is:

- Keep high-level architecture docs in Markdown (`docs/architecture/`), covering bounded contexts, module boundaries, state strategy, and integration patterns.
- Keep decision records in ADR files (`docs/adr/`) and cross-link them to pull requests and implementation commits.
- Keep API and code-level guidance close to implementation through TSDoc/JSDoc on public services, models, guards, interceptors, and library APIs.
- Generate technical docs with Compodoc and publish them per branch or release as a CI artifact/static site.

Tools like AsciiDoc, Markdown, Docusaurus, and MkDocs are commonly used to write documentation as code. These tools allow for structured, version-controlled documentation that integrates seamlessly with development workflows. For diagrams, use PlantUML with C4-PlantUML for C4 views, Mermaid for sequence, flow, and dependency diagrams directly in Markdown, and Structurizr DSL for model-driven C4 documentation and consistency checks, enable creating and maintaining `Diagrams as Code`.

Publishing DaC can be achieved through various methods. Static site generators like Docusaurus and MkDocs can convert Markdown or AsciiDoc files into interactive documentation websites. These sites can be hosted on platforms like GitHub Pages. For internal documentation, tools like Confluence or self-hosted solutions like Read the Docs can be used. Continuous integration pipelines can automate the generation and deployment of documentation, ensuring it stays up-to-date with the codebase.

3.11 Final Remarks

Throughout this chapter, we explored foundational principles and practical approaches to designing Angular applications at scale. We examined how core software design principles guide architectural decisions across multiple levels, from component composition and state management to service integration and deployment strategies. The unifying theme across all perspectives is that effective architecture emerges from understanding constraints, making deliberate choices, and establishing clear boundaries between concerns.

The specific technologies and patterns available for frontend teams serve different contexts. Rather than searching for a universal "correct" architecture, the most sustainable approach is to start with simplicity, follow established principles, and evolve the architecture as application and team needs grow. This measured, incremental strategy reduces unnecessary complexity while maintaining the flexibility to adapt as requirements change.

4 Testing Angular Applications

4.1 Overview

As applications grow in size and complexity, manual validation alone is no longer enough to catch regressions and ensure release confidence. Software testing establishes whether the system behavior matches expectations across functional, integration, and user-level scenarios.

Delivering high-quality software applications requires proper testing. Effective testing improves the quality of the support and services provided to customers. In addition, a well-tested product results in lower maintenance costs and delivers more accurate, consistent, and reliable outcomes. The cost of fixing a bug in production is significantly higher than the cost of detecting it through an automated test suite. When defects are identified early in the development process, developers can resolve them more efficiently and with less effort.

Modern web applications also run across diverse browsers, devices, operating systems, and network conditions. That variability makes systematic testing essential for compatibility, accessibility, and consistent user experience between development and production environments.

This chapter covers the core testing layers for Angular applications, practical test strategy tradeoffs, and implementation patterns that help teams build fast feedback loops while maintaining high product quality over time. On the tooling side, it discusses Vitest and Jest for unit and integration tests, Playwright for functional and visual tests, and Cucumber for Behaviour-Driven Development.

4.2 Software Development Process

4.2.1 Software Development Life Cycle

Software Development Process is the practice of organizing software work into defined phases to improve planning, delivery, and quality. It is often described as a Software Development Life Cycle (SDLC), which provides a structured path from idea to production operation.

The primary goal of SDLC is to deliver high-quality software that meets user needs within expected time and budget constraints. Typical SDLC activities include:

- **Planning**: Define goals, constraints, scope, and acceptance criteria;
- **Design**: Define system architecture, data flow, interfaces, and technical decisions;
- **Implementation**: Build the solution according to the approved design and requirements;
- **Testing**: Validate behavior, quality, and reliability, and detect defects early;
- **Documentation**: Maintain technical and functional documentation throughout the lifecycle;
- **Deployment**: Release the software to target environments in a controlled way;
- **Maintenance**: Monitor production behavior, fix defects, and implement iterative improvements.

Common software delivery approaches include Waterfall and Agile. Within Agile, Scrum and Kanban are among the most widely used frameworks.

4.2.2 Behavior-Driven Development

Behavior-Driven Development (BDD) is a methodology for developing software through continuous example-based communication between developers, QAs and POs. It describes a cycle of interactions with well-defined outputs, resulting in the delivery of working, tested software that matters.

The process looks something like this:

1. Product Owner (PO) and business analyst define features in Gherkin;
2. PO and business analyst together with the team enrich features with scenarios;
3. Team agrees on a user story by its feature definition;
4. Feature definition is run against a tool parsing and executing the Gherkin and will fail;
5. The features are built and test steps implemented so that the feature passes.

The goal of BDD is a business-readable and domain-specific language that allows teams to describe a system's behavior without explaining how that behavior is implemented. Tests are written in the form of plain-text feature descriptions with scenarios typically written before anything else and verified by non-technical stakeholders. No development skills are required to create tests, as they are written in a universal language. Business Analysts can actively participate in the automated test case review process and give their feedback to enhance it.

4.2.3 Domain-Driven Design

Domain-Driven Design (DDD) is a software design approach that places the business domain at the center of the model. Instead of organizing code around technical layers alone, DDD encourages teams to express the system using the language of the problem space. This makes the code easier to understand, easier to change, and easier to test because the domain concepts remain explicit and consistent.

The core concepts in DDD can be summarized as:

- **Ubiquitous Language**: Use the same terminology in code, tests, and conversations that the business uses to describe the domain;
- **Bounded Contexts**: Separate large systems into explicit boundaries where terms, models, and rules are consistent within each context;
- **Entities**: Represent domain concepts that have identity and continuity over time;
- **Value Objects**: Represent descriptive concepts without identity, usually immutable and compared by value;
- **Aggregates**: Group related entities and value objects under a consistency boundary, with external access through the aggregate root;
- **Domain Services**: Capture important domain behavior that does not naturally belong to a single entity or value object;
- **Domain Events**: Represent meaningful business events that occurred in the domain and can trigger reactions across contexts.

DDD helps us focus on business behavior instead of implementation detail. Domain rules can be tested directly through entities, services, and value objects, while integration tests verify that boundaries between contexts and infrastructure behave correctly.

4.2.4 Spec-Driven Development

Spec-Driven Development (SDD) is an approach where the specification becomes the primary source of truth for implementation. The goal is to describe expected behavior clearly before the code is written, then use that specification to guide development and verification. This reduces ambiguity, aligns stakeholders, and makes it easier to keep requirements and tests in sync.

The process typically looks like this:

1. **Idea**: Identify the problem, user need, or opportunity that should be addressed;
2. **Spec**: Write a clear specification with examples, edge cases, and acceptance criteria;
3. **Review**: Align the specification with stakeholders, developers, and QAs before implementation;
4. **Tests**: Turn the approved specification into executable tests or reviewable test cases;
5. **Code**: Implement the smallest amount of code needed to satisfy the specification;
6. **Validation**: Run the test suite and confirm the implementation behaves as specified.

In testing terms, spec-driven development is useful because it gives the team a concrete contract to validate against. It works especially well when paired with BDD, acceptance criteria, and automated tests that verify behavior from the user's perspective.

4.2.5 Test-Driven Development

Test-Driven Development (TDD) is a testing methodology or a programming practice implemented from the developer's perspective. The test is developed which specifies and validates what the code will do. In simple terms, test cases are created before code is written. The purpose of TDD is to make the code clearer, simple and bug-free. The simple concept of TDD is to write and correct the failed tests before writing new code. This helps to avoid duplication of code as we write a small amount of code at a time in order to pass tests.

Following steps define how to perform TDD:

- Add a test.
- Run all tests and see if any new test fails.
- Write some code.
- Run tests and see all pass.
- Refactor code.
- Repeat.

4.2.6 Acceptance Test-Driven Development

In Acceptance Test-Driven Development (ATDD) technique, a single acceptance test is written from the user's perspective. It mainly focuses on satisfying the functional behavior of the system. This technique attempts to answer the question: *Is the code working as expected?* ATDD is very similar to BDD. However, a key difference between them is that BDD focuses more on the behavior of the feature, whereas ATDD focuses on capturing the accurate requirements.

This technique enhances collaboration among developers, users, and QAs with a common focus on defining the acceptance criteria. The following are some of the key practices in ATDD:

1. Analyzing and discussing the real-world scenarios;
2. Deciding the acceptance criteria for those scenarios;
3. Automating the acceptance test cases;
4. Focusing on the development of those requirement cases.

There are key differences between TDD, BDD and ATDD. Understanding how these methods work can help developers and other individuals involved in software development figure out which strategy works best to achieve their goals.

4.3 Testing Types

Software Testing Type is a classification of different testing activities into categories, each having, a defined test objective, test strategy, and test deliverables. We can define more than 100 types of Software Testing Types. However, all testing types are not used in all types of projects.

Tests can be manual or automated. Both approaches are widely used and are paramount for the creation of a highly functional and robust application. By definition:

- *Manual testing* presupposes that developers/testers manually check the software, comparing the expected and actual results and detecting issues. Testers use the application simulating the real end users' behavior, watching how it is working from their side.
- *Automated testing* is performed by the automated program algorithms that repeat preset actions. In this way, the program compares expected and actual outcomes and gives signals of possible defects.

In general, the most important automated test types for web applications are:

- **Unit Tests**: Testing of individual functions or classes by supplying input and making sure the output is as expected;
- **Integration Tests**: Verifying that multiple components, services, or modules work together correctly, including their interactions and side effects;
- **Functional Tests**: Testing scenarios on the product itself, by controlling the browser or the website, regardless of the internal structure to ensure expected behavior;
- **Visual Tests**: Detecting and reviewing visual changes to ensure the websites and applications always look exactly as intended.

Each type of test has an important part to play. Unit tests for fast developer feedback, integration tests to cover all the corner cases of component integrations, and functional tests to make sure everything works right for the end users. Black Box Testing and White Box Testing are complementary software testing approaches that validate a system's functionality from the user's perspective and verify its internal code logic and structure, respectively.

4.4 Testing Pyramid

Testing Pyramid is a metaphor that groups software tests into layers of different granularity. It also provides guidance on how many tests we should have in each layer. Essentially, the testing pyramid, also referred to as the Test Automation Pyramid, lays out the types of tests that should be included in an automated test suite. It also outlines the sequence and frequency of these tests. The goal is to provide immediate feedback and ensure that code changes do not disrupt existing features.

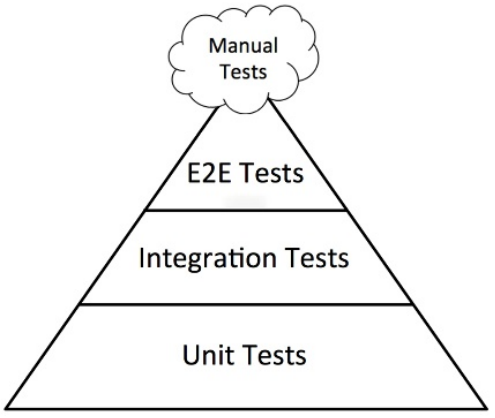


Fig. 1: Testing Pyramid

Unit tests form the base of the testing pyramid. They test individual components or functions to validate that they work as expected in isolation. It is important to cover several scenarios in unit tests, for example, happy paths and error handling. Since this is the largest subset, the unit test suite should run as quickly as possible. Integration tests are the second layer of the testing pyramid. Fundamentally, they verify how a feature communicates with external dependencies. They run more slowly than unit tests. At the top of the pyramid are end-to-end tests. These ensure that the entire application functions as required. End-to-end tests sit at the top of the pyramid because they usually take the longest to run. They can also be fragile because they must cover a wide variety of user scenarios.

As we move up the Testing Pyramid, tests become more expensive because they are harder to create and maintain, and they are often more flaky because they rely on fragile interfaces. At the very top are manual tests, which still have their place, even in the modern age of automation and DevOps, because validating a new feature for the first time requires a human to decide whether it meets the acceptance criteria (typically validated during User Acceptance Testing, UAT).

The testing pyramid helps address the problem of over-reliance on long-running UI tests. However, there are two known anti-patterns. These test suites look like an "ice cream cone" and an "hourglass". An ice cream cone has very few unit tests, some UI tests, lots of QA tests, and many manual tests. An hourglass has many unit and UI tests but few integration tests. Both patterns are problematic because heavy UI coverage makes the regression suite slow. This affects the productivity of the whole team.

In an Angular context, there are several types of tests: isolated and shallow unit tests, integration tests between components, and e2e tests, which can be split into functional and visual tests. Based on Complete Guide to Angular Testing, a common recommendation is that 80% of tests should be unit tests, 15% integration tests, and 5% UI tests. These percentages may vary, but the ratio is usually similar. However, other opinions recommend thinking in scales instead of pyramids. Rather than prescribing fixed amounts of each test type, we should ask, "What risks exist in my system? How can my system break?" and then build the coverage needed in each area to address those risks.

4.5 Testing Tools

Choosing the right testing tools depends on the type of feedback we need, the level of confidence we want to achieve, and how quickly the test suite must run. In Angular applications,

the tooling typically spans unit and integration testing, test-data and dependency mocking, and full end-to-end browser automation. Angular recommends Vitest testing framework for unit and integration tests. For e2e tests, Playwright is recommended. However, we also have other available tools with higher popularity.

In the next table we have a combination of Angular recommendations and the strongest tools:

Type of Tests	Type of Solutions	Tools
Unit and Integration Tests	Recommended	jest rxjs-marbles
	Alternative	vitest
	Boilerplate code	ng-mocks @ngneat/spectator ts-mockery
	API mocking	angular-in-memory-web-api
	Reports	istanbul-reports jest-html-reporters
Functional and Visual Tests	Recommended	playwright cucumber
	Alternative	cypress puppeteer jest-image-snapshot
	API mocking	ng-apimock
	Reports	allure-playwright playwright-html-reporter

In unit and integration tests, we can choose Jest or Vitest testing framework. Both solutions are good choices, but we have other JavaScript test frameworks. Testing with Mocha and Chai is an excellent alternative. Another solution that has been growing is Testing Library. The library provides special tools for different frameworks like React, Preact, React Native, Angular, Vue, and others. It also helps with e2e testing tools like Cypress, Puppeteer, WebdriverIO, and others. For testing observables, marble diagrams are recommended.

To test Angular components, it is important to reduce boilerplate code. ngMocks is indispensable in Angular projects for creating mocks for tests. Spectator is another library to simplify the Angular tests. This library is recommended only for testing templates. It happens that in many cases we test the same thing with functional tests. To avoid duplicate effort, HTML is most often tested with functional tests.

Cypress or Jest with Puppeteer are very popular testing solutions for e2e tests. Cucumber is a framework that runs automated acceptance tests written in a BDD style. Cypress is quite different technology for different tasks from Playwright. It works well for testing components of a web application, but it cannot replace Playwright. This means that a better combination is Jest + Cucumber + Playwright. Popular solutions for API mocking are ng-apimock, json-server and mockserver.

The previous table includes both recommended and alternative solutions. Libraries from both proposals can be combined, but equivalent libraries should not be mixed within the same project. For example, if Jest is selected for unit testing, it should also be used for integration testing. Avoid using Jest for one test suite and, for example, Vitest for another when they fulfill the same role.

4.6 Unit Tests

4.6.1 Fundamentals

Unit Testing is a software testing method in which individual units or components of a software application (such as functions, methods, or classes) are tested in isolation to verify that they work correctly as expected.

In the testing pyramid, unit tests are the base layer and should provide the fastest feedback loop for developers. In practical Angular work, a unit can be a method in a service, a pipe transform, a validator, or a focused behavior in a component class. The main goal is to verify one behavior at a time with deterministic inputs and outputs. To keep this feedback fast and deterministic, unit tests should isolate the code under test from external dependencies. A component under test does not need real services, HTTP calls, timers, or browser integrations. Instead, we use simplified objects that mimic production behavior only where needed. This reduces complexity, avoids flaky tests, and makes failures easier to diagnose. A Test Double is the generic term for these simplified objects. Although test doubles come in many flavors, teams often use the term "mock" informally to refer to most of them.

There are five types of doubles:

- **Dummy Object**: Dummy objects are passed around but never actually used. Usually they are just used to fill parameter lists.

- **Fake Object**: Fakes are objects that have working implementations, but not the same as the production implementation. Usually they take some shortcut and have simplified version of production code.
- **Test Stub**: Stubs provide canned answers to calls made during the test, usually not responding at all to anything outside what's programmed in for the test.
- **Test Spy**: Spies are stubs that also record some information based on how they were called. One form of this might be an email service that records how many messages it was sent.
- **Mock Object**: Mocks are pre-programmed with expectations which form a specification of the calls they are expected to receive. They can throw an exception if they receive a call they don't expect and are checked during verification to ensure they got all the calls they were expecting.

Vitest has test double functions called spies. A spy can stub any function and track calls to it and all arguments. A spy only exists in the `describe` or `it` block in which it is defined and will be removed after each test. There are special matchers for interacting with spies. Jest and Vitest provide similar resources for mocking. In both frameworks, mocks are used to stub out functionality with mock data.

Angular has been designed with testability as a primary goal. In practice, unit testing in Angular combines framework helpers with general testing capabilities from Jest or Vitest. This foundation leads directly into the next sections, where we cover isolated tests, shallow tests, asynchronous behavior, and other patterns used in day-to-day Angular projects.

To learn more about Angular unit tests, Angular Testing, Testing Angular Applications and Angular Test-Driven Development books, Testing Angular online book, and articles such as The Complete Guide to Angular Testing, Angular Testing Recipes, and Three Ways to Test Angular Components are good references.

4.6.2 Isolated Tests

Isolated tests are used to test a unit in isolation. The unit might contain some business logic that needs to be tested in isolation. When writing isolated unit tests all external dependencies should be mocked. The most common examples of isolated tests are tests of functions and class methods. In Angular applications, we should prefer isolated tests to test Services and Pipes.
We can use isolated tests or shallow tests to test Angular components, whichever makes the most sense. With isolated tests, we only will test the class of the component. It is often useful to

test complex components without rendering them. To see how it can be done, see the following component:

```
@Component({
  selector: 'cmn-view-header',
  template: `
    <h1>My Component</h1>
    <cmn-button type="primary-button" (clicked)="onClick()"></cmn-button>
  `,
})
export class ViewHeaderComponent {
  constructor(private readonly sharedService: SharedService) {}

  onClick(): void {
    this.sharedService.copyToClipBoard();
  }
}
```

In the example, we have a component that calls a service when we click on the button. An example of an isolated test:

```
describe('ViewHeaderComponent', () => {
  let component: ViewHeaderComponent;
  let sharedService: SharedService;

  beforeEach(() => {
    sharedService = {copyToClipBoard: jest.fn()} as unknown as SharedService;
    component = new ViewHeaderComponent(sharedService);
  });

  it('#onClick should call #copyToClipBoard', () => {
    component.onClick();

    expect(sharedService.copyToClipBoard).toHaveBeenCalled();
  });
});
```

As we can see, testing Angular components in isolation is no different from testing any other JavaScript object. This approach is preferable when other types of tests are used to interact with the DOM.

4.6.3 Shallow Tests

Testing component classes without rendering their templates works in certain scenarios, but not in all of them. Sometimes, we can only write a meaningful test if we render a component's template. However, we want to keep the test isolated. In this case, we only need to render the template without rendering the child components. This is what is colloquially known as shallow test.

An example of a shallow test:

```
describe('ViewHeaderComponent', () => {
  let button: ButtonComponent;
  let component: ViewHeaderComponent;
  let fixture: ComponentFixture<ViewHeaderComponent>;
  let sharedService: SharedService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [ViewHeaderComponent, MockComponent(ButtonComponent)],
      providers: [
        { provide: SharedService, useValue: { copyToClipBoard: jest.fn() } }
      ],
    }).compileComponents();

    sharedService = TestBed.inject(CommonSharedService);
    fixture = TestBed.createComponent(ViewHeaderComponent);
    component = fixture.componentInstance;
  });

  it('should have <h1> with "My Component"', () => {
    const h1 = fixture.nativeElement.querySelector('h1');

    expect(h1.textContent).toEqual('My Component');
  });

  it('#onClick should call #copyToClipBoard', () => {
    button = fixture.debugElement.query(By.directive(ButtonComponent))
      .componentInstance;
    button.clicked.emit();

    expect(sharedService.copyToClipBoard).toHaveBeenCalled();
  });
});
```

In this example, all external dependencies are mocked (services and components). To mock components, we can use the ng-mocks library. Unlike NO_ERRORS_SCHEMA, which can hide certain issues and does not help with configuring inputs and outputs, ng-mocks automatically mocks component inputs, outputs, and other bindings for us. The NO_ERRORS_SCHEMA tells the Angular compiler to ignore unrecognized elements and attributes. The NO_ERRORS_SCHEMA also prevents the compiler from telling us about the missing components and attributes that we omitted inadvertently or misspelled. We could waste hours chasing phantom bugs that the compiler would have caught in an instant.

The most common issue in shallow testing comes from the amount of boilerplate code. We can create helpers and factory functions to reduce the boilerplate code. Alternatively, we can use Spectator to simplify Angular tests.

4.6.4 Snapshot Tests

Snapshot tests are a valuable way to ensure that a UI does not change unexpectedly. In a typical snapshot test for a web application, a UI component is rendered, a snapshot is captured, and the result is compared against a reference snapshot stored alongside the test. The test fails if the snapshots differ, indicating either an unintended change to the UI or that the reference snapshot should be updated to reflect the component's new expected state.

Snapshot testing are the best alternative to the DOM testing. Rather than asserting values obtained from specific `HTMLElement` instances, we can validate the entire component fixture against a snapshot. This approach requires updating our tests to validate components against snapshots rather than relying on traditional DOM assertions.

An example of a snapshot test:

```
describe("ViewHeaderComponent", () => {
  let component: ViewHeaderComponent;
  let fixture: ComponentFixture<ViewHeaderComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [ViewHeaderComponent, MockComponent(ButtonComponent)],
      providers: [{ provide: SharedService, useValue: {} }],
    }).compileComponents();

    fixture = TestBed.createComponent(ViewHeaderComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should render <h1> with "My Component"', () => {
    expect(fixture).toMatchSnapshot();
  });
});
```

Note that in the initial run (when there aren't any snapshots to test against), Jest or Vitest will treat the first output as the main reference and will save the output in a ***snapshots*** directory in the same directory as the test file. Next example shows the *.snap* file generated by Jest:

```
exports[`ViewHeaderComponent should render <h1> with "My Component" 1`] = `
<cmn-view-header>
  <h1>
    My Component
  </h1><cmn-button
    type="primary-button"
  />
</cmn-view-header>
`;
```

Snapshot testing is not limited to HTML templates. It can also be used to validate JavaScript objects, function outputs, JSON data, configuration files, and many other types of content.

Snapshot tests once promised a revolution, but today they are often criticized for being fragile, creating a false sense of safety, and losing value over time. Even small code changes can cause snapshot failures. At first, teams review those failures carefully and update snapshots when needed. Over time, however, people may start updating snapshots automatically without reviewing them, assuming everything is fine. When that happens, snapshots stop providing meaningful protection, and real regressions can slip into production. Snapshot testing is not inherently bad, but it often provides limited value for template-level UI verification. It is usually more useful for stable outputs such as JSON payloads, configuration files, and large JavaScript objects.

4.6.5 Marble Tests

We can test asynchronous RxJS code synchronously and deterministically by virtualizing time with the TestScheduler. When we need to test Observables, there are two main patterns: the *subscribe and assert pattern* and the *marble testing pattern* based on diagrams. A marble diagram is a domain-specific language in RxJS that helps us model the interactions and values of one or more observables in a test. It is a visual representation, like a schema, that describes what is happening in an observable stream.

We have helper methods that have been added to make creating marble tests easier:

- `hot(marbles: string, values?: object, error?: any)` creates a "hot" observable (a subject) that will behave as though it's already "running" when the test begins. An interesting difference is that *hot* marbles allow a ^ character to signal where the "zero frame" is. That is the point at which the subscription to observables being tested begins.
- `cold(marbles: string, values?: object, error?: any)` creates a "cold" observable whose subscription starts when the test begins.
- `expectObservable(actual: Observable).toBe(marbles: string, values?: object, error?: any)` schedules an assertion for when the *TestScheduler* flushes. The *TestScheduler* will automatically flush at the end of *it* block.
- `expectSubscriptions(actual: SubscriptionLog[]).toBe(marbles: string)` like `expectObservable` schedules an assertion for when the *testScheduler* flushes. Both `cold()` and `hot()` return an observable with a property `subscriptions` of type `SubscriptionLog[]`. Give subscriptions as parameter to

`expectSubscriptions` to assert whether it matches the subscriptions marbles diagram given in `toBe()`. Subscription marble diagrams are slightly different than Observable marble diagrams.

To write a test with marble diagrams we will need to stick to a convention of characters that will help visualize the observable stream:

- ' ' the whitespace is a unique character that will not be interpreted;
- '-' represents a frame of virtual time passing;
- '|' this sign illustrates the completion of an observable;
- '#' signifies an error;
- [a-z] an alphanumeric character represents a value which is emitted by the Observable;
- '()' used to group events in the same frame. This can be used to group values, errors, and completion;
- '^' this sign illustrates the subscription point and is used when we are dealing with hot observables;
- '!' this sign indicates the end of a subscription point.

Most marble testing documentation is available in the official RxJS docs. There are also different libraries that help us write marble tests. For Jest-based projects, the most common choices are "jest-marbles" and "rxjs-marbles". RxJS itself also provides testing utilities, and these libraries are convenience wrappers around them. This syntax gives us the power to create and manipulate observable streams.
For example, for the following observable:

```
export const source: Observable<number> = of(1, 2, 3).pipe(map(x => x * 2));
```

The marble test can be written as follows:

```
describe('Observable', () => {
  it('should multiply by "2" each value emitted', () => {
    const expected = cold('(abc|)', { a: 2, b: 4, c: 6 });

    expect(source).toBeObservable(expected);
  });
});
```

Notice the *source* and *expected* variable declarations. In marble testing, we use this pattern frequently. With this pattern, we can see when values (a, b, c) are emitted, when the stream completes (|), and compare the actual output directly with what is *expected*.

4.6.6 Asynchronous Tests

JavaScript code frequently runs asynchronously. Whether we are fetching data, reading files, or waiting on timers, the framework needs to know when the code it is testing has completed before moving on to the next test. Here are the patterns we will use most often. Testing asynchronous code involves testing Observables, Promises and Timers.

The native timer functions (i.e., *setTimeout*, *setInterval*, *clearTimeout*, *clearInterval*) are less than ideal for a testing environment since they depend on real time to elapse. Vitest timers and Jest Timer Mocks can swap out timers with functions that allow us to control the passage of time. To see how it can be done, see the following class:

```
export class TimerService {
  started = false;

  startTimer(): void {
    setTimeout(() => { this.started = true; }, 100);
  }
}
```

The unit test can be written as follows:

```
describe('TimerService', () => {
  let service: TimerService;

  beforeEach(() => {
    service = new TimerService();
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.useRealTimers();
  });

  it('should update #started after timeout', () => {
    service.startTimer();
    jest.advanceTimersByTime(100);

    expect(service.started).toBe(true);
  });
});
```

Alternatively, Angular provides helper functions *fakeAsync* and *tick* to handle asynchronous tests, which essentially gives us an easy way to run asynchronous code before the assertions. When a test is running within a *fakeAsync* zone, we can use two functions called *flushMicrotasks* and *tick*. The *tick* function will advance time by a specified number of milliseconds, so *tick(100)* would

execute any asynchronous tasks that would occur within 100ms. The *flushMicrotasks* function will clear any "microtasks" that are currently in the queue.

The *fakeAsync* utility function has a few limitations. It won't work if the test body makes an *XMLHttpRequest* (XHR) call. XHR calls within a test are rare so we can generally stick with *fakeAsync*. But if we ever do need to call *XMLHttpRequest*, we'll use the *waitForAsync* utility function.

When we need to test RxJS asynchronous code, marble testing can be an elegant way to test it. RxJS provides the TestScheduler which has a `run` callback. The *TestScheduler* class is part of the `rxjs/testing` module, and enables synchronous testing of asynchronous, or synchronous, Observables.

In the next example, we can see how *TestScheduler* works:

```
export const source: Observable<number> = of(1, 2, 3).pipe(
    delay(100),
    map(x => x + 1),
);
```

The marble test can be written as follows:

```
describe('TestScheduler', () => {
  let scheduler: TestScheduler;

  beforeEach(() => {
    scheduler = new TestScheduler((actual, expected) => {
      expect(actual).toEqual(expected);
    });
  });

  it('should wait 100ms before emitting values', () => {
    scheduler.run(({ expectObservable }) => {
      expectObservable(source).toBe('100ms (abc|)', { a: 2, b: 3, c: 4 });
    });
  });
});
```

In marble tests, time should be expressed using the *TestScheduler*, not real milliseconds.

4.6.7 Code Coverage

Code coverage is a measurement of the percentage of source code executed during automated tests. Test coverage, on the other hand, measures the effectiveness of automated test cases in a qualitative manner.

We can configure minimum coverage thresholds. Thresholds specified as positive numbers represent the minimum required percentage. Anything below these thresholds will cause the test suite to fail with an exit code of 1. Thresholds specified as negative numbers represent the maximum number of allowed uncovered entities. Thresholds can be specified as "global" if we want them to be applied to every file in the project, or as a path or glob if we only want to apply them to certain files.

For example, the following configuration causes test failure if global coverage falls below 80% for statements, lines, branches, and functions:

```
{
  thresholds: {
    global: {
      statements: 80,
      lines: 80,
      branches: 80,
      functions: 80
    }
  }
}
```

This configuration works with Vitest and Jest. For projects with Jest, more details are available in the coverageThreshold section. For Vitest, coverage configuration is available in the Vitest coverage documentation.

While line and branch coverage indicate which code paths are exercised, they do not prove that tests actually detect incorrect behavior. Mutation testing addresses this gap by automatically introducing small changes, called mutants, into the production code and then verifying that at least one test fails for each change. A mutant that is not caught by any test is called a survivor and represents a gap in the test suite. The ratio of killed mutants to total mutants is the mutation score, which is a more meaningful quality indicator than line coverage alone.

Stryker Mutator is the leading mutation testing tool for JavaScript and TypeScript projects, with dedicated support for Angular via @stryker-mutator/vitest-runner and @stryker-mutator/jest-runner. Running mutation tests on critical business logic and domain services helps identify weak assertions and untested edge cases that coverage metrics alone would not surface.

4.6.8 Test Reporters

It's very useful to know which parts of the code are not covered by unit tests. In Angular projects, coverage data can be captured and reported by passing the *--code-coverage* flag. Vitest and Jest rely on Istanbul to generate code coverage reports. By default, these tools report in multiple

formats (*json*, *lcov*, *text*, *clover*), but we can configure them to use any valid Istanbul reporter. Running coverage with an *html* report creates a visual overview of coverage in HTML format. We can view results for the entire app broken down per file and examine individual files to see how often each line was executed. Using the *lcov* report as part of the CI pipeline enables us to track coverage trends across builds.

4.6.9 Debugging

Relying only on *console.log* is rarely enough when a test fails intermittently or only in CI. A stronger approach is to debug tests with breakpoints, source maps, and deterministic test execution.

For Angular tests running in a browser-like environment, the first step is usually to run a single spec in watch mode and attach debugger tools. This allows us to inspect component state, dependency injections, async queues, and DOM updates frame by frame. The Angular docs provide a practical baseline in Debugging Tests.

A common workflow is to debug one test file in a single worker, then attach Node DevTools. The following npm script shows the default script for Jest:

```
{
  "scripts": {
    "test:debug": "node --inspect-brk node_modules/jest/bin/jest.js -i"
  }
}
```

An equivalent npm script can be used for Vitest. In both frameworks, `debugger` statements inside tests or production code under test pause execution when the inspector is attached.

In day-to-day development, it is usually better to prefer IDE test plugins over raw terminal commands for run/debug loops. In VS Code, extensions such as Vitest Extension and Jest Extension integrate with the Testing panel, where we can run a single test, debug directly from `it/describe` blocks, inspect stack traces, re-run failed tests, and navigate quickly between failures and source code. This shortens feedback cycles and reduces context switching.

4.6.10 Best Practices

The following practices summarize how to design unit tests that are deterministic, readable, and cost-effective to maintain:

- ☐ **Follow conventions**: A style guide is a set of standards for how code should be written and organized. Good projects have clear, consistent coding conventions with automated

enforcement. Conventions must be dynamic and adaptable for each team and project, and each team defines its convention. Some guides such as A guide to unit testing in JavaScript and Node.js & JavaScript Testing Best Practices, and Frontend testing standards and style guidelines can be used as a reference.

- ☐ **Follow Design Principles**: The key to good unit testing is writing testable code. Use good naming and comments, avoid duplication, follow single responsibility, keep one behaviour in one test, ensure test order does not affect other tests, minimize dependencies, use design patterns, and avoid global mutable state.

- ☐ **Follow the Arrange-Act-Assert pattern**: The Arrange-Act-Assert (AAA) pattern separates test setup, test action, and test assertion. Arrange creates objects and mocks and sets expectations, Act invokes the method, and Assert verifies outcomes. AAA is well known and recommended, and explicit comments for each section are usually unnecessary. See next example in Jasmine style:

```
describe('Array', () => {
  describe('.push(x)', () => {
    let initialArray;

    beforeEach(() => {
      initialArray = [1]; // Arrange
      initialArray.push(2); // Act
    });

    it('appends x to the end of the Array', () => {
      expect(initialArray).toEqual([1, 2]); // Assert
    });
  });
});
```

- ☐ **Follow Test-Driven Development**: Test-Driven Development is a robust way to design units so behaviour is specified through tests. Consider writing tests first in `red-green-refactor` style. Test-first improves testable design, keeps code minimal, supports safe refactoring, gives fast feedback, and increases confidence to add features, fix bugs, and explore design changes.

- ☐ **Follow the FIRST Principles**: FIRST means tests should be Fast, Isolated, Repeatable, Self-validating, and Timely. Fast tests avoid slowing development/deployment, isolated tests avoid dependence on each other, repeatable tests produce consistent results, self-validating tests require no manual interpretation, and timely tests can be written early and often.

- ☐ **Use lint rules**: Linters catch severe issues and enforce consistency/readability. Beyond base ESLint style rules, include specialized plugins such as promise/security/jest/jasmine plugins for missing resolves, risky regex, missing assertions, and skipped tests.
- ☐ **Describe expectations in a human language**: Name `describe` and `it` blocks so intention is clear in human language. The full test name is the `it` title plus parent `describe` titles, so wording matters. Prefer assertive verbs and avoid vague terms like "should." This makes test suites useful documentation and produces meaningful failure messages.
- ☐ **Name the tests properly**: Test names should be concise, explicit, descriptive, and written in correct English. Read the spec runner output and ensure it is understandable for others. Test names can become live documentation for the codebase.
- ☐ **Structure the tests properly**: Nest suites when needed to structure tests logically into subsets. This helps tests build from least to most specific, makes extension/refactor easier, reduces repetition in branch testing, and encapsulates tests by common denominator. See next example:

```
describe('A set of functionalities', () => {
  describe('A subset of functionalities', () => {
    it('does something great', () => {});

    it('does something awesome', () => {});
  });

  describe('Another subset of functionalities', () => {
    it('also do something great', () => {});
  });
});
```

- ☐ **Categorize tests under at least 2 levels**: Structure suites with at least two `describe` levels: one for the unit under test and one for scenario/category. This improves report readability and navigation in larger suites.
- ☐ **Create new tests for every defect**: Add regression tests for bug fixes so defects do not recur. Start by creating a test that reproduces the issue before changing code, then apply TDD to fix it. This improves quality assurance and reduces post-release glitches.
- ☐ **Mock Dates and Timers**: When production code uses `new Date()` or `Date.now()`, mock time for deterministic tests. Also, Timers (`setTimeout`, `setInterval`, etc.) run normally if we don't mock them.
- ☐ **Mock out all external services and state**: Unit tests should be stateless and should not modify external resources. Otherwise shared external state can make tests influence

each other. Mocking external services also makes tests run much faster than using real-time dependencies.

- ☐ **Do not test external libraries**: Do not test library internals, but rely on library test coverage. Testing thin wrappers that only call library functions usually adds little value. Prefer testing behaviour from the user interaction perspective.
- ☐ **Do not mock everything**: Unit tests should mock external side effects, but not every dependency must be mocked. Real objects are acceptable when setup stays simple, no shared state appears, no AJAX/API/reload side effects happen, and execution speed remains within limits.
- ☐ **Do not test multiple concerns**: If a method has multiple end results, test each result separately. Tests combining concerns (for example with "AND" or "OR" in names) are a smell. Single-concern tests improve failure diagnosis. See next bad example:

```
it('sends the profile to the server and updates the profile view', ()
=> {
  // expect(...).to(...);
  // expect(...).to(...);
});
```

Where is better changing to:

```
it('sends the profile to the server', () => {
  // expect(...).to(...);
});

it('updates the profile view', () => {
  // expect(...).to(...);
});
```

- ☐ **Do not skip unit tests**: Do not comment tests out because they are slow, complex, or flaky. Make tests fast, simple, and trustworthy, otherwise remove them. Avoid `xdescribe/xit` and enforce `no-disabled-tests` and `no-focused-tests` lint rules.
- ☐ **Do not test configuration settings**: Configuration values are not usually a unit of business logic. Keep only one or two tests for configuration loading itself instead of many copy-paste tests for each setting. Snapshot testing can verify full configuration content efficiently.
- ☐ **Do not overuse variables**: Extracting too many values into variables can bloat tests and make failures harder to trace. In short tests, repeated literals may be clearer than excessive indirection.

- ☐ **Do not use static access**: Static access is an anti-pattern because it hides dependencies and side effects and harms testability. Prefer non-static methods, instantiate classes, and inject dependencies via constructor so tests can swap mocks or have alternative implementations.
- ☐ **Do not reuse production code**: Tests should verify production code, not reuse it heavily. Reusing production helpers in tests can hide bugs in those helpers. Prefer clear input-output assertions with hard-coded expected values.
- ☐ **Do not catch errors but expect them**: Avoid verbose try-catch assertions for expected failures. Prefer dedicated one-line error assertions like `toThrow`. Also validate useful error properties or types so failure handling is actionable. See next example:

```
it('adds a user in memory', () => {
  expect(userManager.addUser()).toThrow(new Error());
});
```

- ☐ **Do not test complex user interactions**: Some scenarios are too broad for unit tests, such as multi-step drag-and-drop flows or gallery navigation across loaded data. Test these at higher levels (functional/integration). Keep unit tests focused on simple user actions and event-to-method behaviour.
- ☐ **Do not Repeat Yourself**: Use `before/after` hooks to DRY repeated setup, teardown, and actions. This keeps suites concise/readable and centralizes change points.
- ☐ **Avoid toBeTruthy or toBeFalsy**: Avoid broad truthy/falsy matchers because they can create weak or false-positive tests (`null vs false`, etc.). Prefer specific assertions for exact values and types.
- ☐ **Prefer toBe over toEqual for primitive values**: `toBe` uses `Object.is` and is generally faster and more precise for primitives. Use `toEqual` when deep comparison of objects/arrays is required.
- ☐ **Avoid the All's**: Prefer `beforeEach/afterEach` over `beforeAll/afterAll`. `All` hooks are not reset between tests and can leak state. `Each` hooks enforce independence and make execution flow clearer.
- ☐ **Avoid mocks in favor spies**: Stub dependencies that create side effects (network, external IO) and use spies to verify interactions. This keeps tests isolated and focused while avoiding unnecessary heavy mocking.
- ☐ **Avoid global test data**: Create data per test and clean up after each test to guarantee a clean environment. This prevents inter-test dependencies. If setup cost is high, simplify test data rather than sharing mutable fixtures.

- ☐ **Avoid logic in the tests**: Tests should avoid loops, switches, and conditionals because logic inside tests creates additional bug paths and weakens one-concern clarity.
- ☐ **Cover the general case and the edge cases**: Strange behaviour often appears at edges. Tests should cover common flows and edge cases, and coverage/test-coverage metrics should be used to evaluate effectiveness of exercised paths.
- ☐ **Test only public methods**: Testing internals creates large maintenance overhead with little payoff. Verifying public behaviour implicitly tests private implementation and keeps tests resilient unless outputs are wrong.
- ☐ **Test the behaviour and not the implementation**: Assert outcomes of operations, not internal state details. This allows internal refactoring without forcing test rewrites and preserves behavioural confidence.
- ☐ **Test many input combinations**: Real systems receive many permutations of input. Avoid only happy-path samples and use random or property-based combinations (for example fast-check or jsverify) to expose hidden failures.
- ☐ **Use realistic input data**: Unrealistic test inputs miss production-like bugs. Use realistic and varied data to increase defect detection. Libraries like Faker can generate pseudo-real data similar to production forms and distributions.
- ☐ **Use factory functions**: Factory functions simplify object creation in tests, reduce setup duplication, and improve readability by expressing setup as a single call. They also provide flexible instance initialization and clean dependency injection patterns.
- ☐ **Use the prefixes "actual" and "expected"**: When using variables in equality assertions, prefix them with `actual` and `expected`. This improves readability, clarifies intent, and reduces accidental mix-ups in assertions.
- ☐ **Design for lean testing**: Test code is different from production code. Keep it simple, short, flat, and easy to read. Keep only as much testing abstraction as needed and prioritize agility and clarity.
- ☐ **Shorten the feedback loop**: Testing effectiveness depends on quick iterative loops: try, get feedback, refactor. Faster feedback enables more improvement cycles. Running CI-like checks locally can accelerate this loop significantly.
- ☐ **Tag the tests**: Different scenarios require different subsets of tests (smoke, IO-less, commit-time, PR-time full E2E). Tag tests with keywords so tooling can grep and run only relevant subsets, avoiding slow full runs for small changes.

- ☐ **Focus on test coverage**: Coverage indicates how many lines/branches/statements are reached by tests. Very low coverage is risky, but 100% can be costly and may overemphasize edge corners over critical paths. A common rule of thumb is to set a limit of around 80%.
- ☐ **Analyse the test coverage report**: Coverage reports help reveal under-tested code areas where surprising behaviour can hide. If we do not know what is untested, we do not know where issues may originate.
- ☐ **Avoid duplicate coverage**: Do not duplicate feature coverage across test layers unnecessarily. If functionality is well-covered in unit tests, avoid repeating it in expensive UI tests. Keep UI tests for big-picture user stories spanning components/views.
- ☐ **Measure logical coverage**: Traditional line coverage can mislead by showing visited code without proving behaviour correctness. Use mutation testing to validate whether tests actually detect logical changes, not just execute lines.
- ☐ **Parallelize test execution**: Run multiple tests simultaneously to shorten total feedback time. Parallelism reduces waiting and helps teams resolve issues faster than strictly serial runs.
- ☐ **Know the testing framework API**: Strong knowledge of the testing framework API reduces test size and complexity and improves readability. Using built-in helpers (for example concise spy APIs) often produces clearer tests than verbose low-level assertions.

4.7 Integration Tests

Integration testing is when we test two or more components together. This makes sense for parts of the applications where the integration between components is important to test, such as a container/smart component with container/dumb components. With integration testing, we simply import the implementation of the dependencies of the unit under test even though we still should mock out HTTPClient.

An example of integration testing is:

```
describe('ViewHeaderComponent', () => {
  let buttonElement: HTMLElement;
  let fixture: ComponentFixture<ViewHeaderComponent>;
  let sharedService: SharedService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [ViewHeaderComponent, ButtonComponent],
```

```
      providers: [
        { provide: SharedService, useValue: { copyToClipBoard: jest.fn() } }
      ],
    }).compileComponents();

    sharedService = TestBed.inject(SharedService);
    fixture = TestBed.createComponent(ViewHeaderComponent);
    buttonElement = fixture.nativeElement.querySelector('button');
  });

  it('should call #copyToClipBoard when the button is clicked', () => {
    buttonElement.click();

    expect(sharedService.copyToClipBoard).toHaveBeenCalled();
  });
});
```

In this example, when a user clicks the button component, an event is emitted to the parent component. We cannot mock the *ButtonComponent* because its event listeners are defined within it. In shallow tests, all child components are mocked and events are emitted by calling *@Output()* directly. In this example, the event originates from the *ButtonComponent*.

Although both shallow and integration tests render components, they differ significantly in nature. In the shallow test, we mocked every dependency of the component. In the integration test, we mocked only the *SharedService*. Shallow tests are isolated and, as a result, can be used to drive component design. Integration tests serve to verify correctness.

As discussed, integration tests verify that groups of components and services work together. Other common scenarios for integration tests include testing integrations with Angular Router, NgRx Effects, or Redux Beacon. However, it is recommended to avoid testing complex behaviors using only unit and integration tests. Complex user interactions should be tested with functional tests.

4.8 Functional Tests

4.8.1 Fundamentals

Functional testing, also known as end-to-end (e2e) testing in the Angular ecosystem, verifies the behavior of the application as a whole from the user's perspective. Unlike unit and integration tests, which validate isolated pieces of logic or component interactions, functional tests drive a real browser through real user scenarios, interacting with the application as a human would. They sit at the top of the Testing Pyramid and are the most expensive to write and maintain, but

they provide the highest level of confidence that the product works correctly in a production-like environment.

Playwright is an open-source framework developed by Microsoft for automating web browsers to test web applications. It simulates user interactions such as clicking, filling out forms, navigating between routes, and validating rendered state. Its features include headless execution, cross-browser support (Chromium, Firefox, and WebKit), mobile emulation, network interception, and advanced debugging tools, making it ideal for web applications.
The next example shows a purchase-order-style scenario where we validate the total number of promotions in the first section and the name of the first promotion.

```
describe('Promotional Sections', () => {
  beforeEach(async ({ page }) => {
    await page.goto('/promotions');
  });

  it('should display promotions in the first section', async
({ promotionalSectionPO }) => {
    const firstSection = promotionalSectionPO.getAllSections().first();
    const promotions = promotionalSectionPO.getAllPromotions(firstSection);

    await expect(promotions).toHaveCount(2);
    await expect(promotionalSectionPO.getPromotionName(promotions.first()))
           .toHaveText('Summer Sale');
  });
});
```

In Angular, functional tests are typically run against the compiled application served in a browser, covering flows that span multiple components, routes, services, and network calls. They are the right tool to validate critical user journeys such as login, checkout, or multi-step submissions.

4.8.2 Browser Interaction

Functional tests automate real browser interactions, namely, navigating to URLs, clicking elements, filling in forms, selecting options, scrolling, hovering, pressing keyboard shortcuts, and verifying visible content or application state. The test framework controls the browser through browser automation protocols. Playwright does not use the WebDriver standard for its core automation model, but it uses the Chrome DevTools Protocol for Chromium and Playwright's own browser-specific transport layers for Firefox and WebKit.

Playwright controls browser navigation using methods like `page.goto(url)`. To interact with elements in the browser, tests must first locate them. Playwright offers different locator strategies:

- **CSS selectors**: Select elements using standard CSS syntax, e.g. `.btn-primary` or `input[name="email"]`. Fast and widely supported.
- **Text locators**: Select elements by their visible text content. Playwright provides `page.getByText()` for human-readable text queries.
- **Test IDs**: Select elements by a dedicated `data-testid` attribute, e.g. `page.getByTestId('submit-btn')`. This decouples tests from markup structure and styling.
- **ARIA roles**: Select elements by their semantic role and accessible name, e.g. `page.getByRole('button', { name: 'Submit' })`. This approach promotes accessibility-first development.
- **XPath**: A powerful but fragile and verbose querying language. Its use is strongly discouraged because it is tightly coupled to DOM structure and is hard to maintain.

The recommended approach is to prefer accessible locators (role, label, text) and `data-testid` attributes over structural selectors that leak implementation details.

4.8.3 Auto-Waiting

One of the most common sources of flakiness in functional tests is poor synchronization between the test and the application state. In practice, the default strategy should be auto-waiting. Actions such as `click`, `fill`, and `press`, and Playwright assertions such as `expect(locator).toBeVisible()`, already wait for element readiness and stable conditions. This is why fixed delays such as `waitForTimeout()` are a bad default because they slow down the suite, hide real timing problems, and still fail under variable load.

Explicit waiting is still necessary in specific scenarios, but it should be used intentionally and with clear responsibility. In other words, when auto-waiting is not enough, the test author is responsible for choosing a concrete and meaningful synchronization signal. Typical examples include waiting for navigation (`waitForURL`), waiting for a specific network response (`waitForResponse`), waiting for a loading indicator to disappear, or waiting for an asynchronous backend process to finish.

A practical rule is to use auto-waiting by default and add explicit waits only for domain-specific events that the framework cannot infer on its own. This keeps tests faster, clearer, and more reliable.

4.8.4 Mocking APIs

Functional tests can intercept and mock network requests to control the data the application receives without depending on a live backend. In Playwright's API Mocking, `page.route()` can block a request completely and return fixture data with `route.fulfill()`, which is useful when the scenario needs full control over backend responses. For more details, see next example:

```
describe('Promotions', () => {
  beforeEach(async ({ page }) => {
    await page.route('*/**/api/v1/promotions', async route => {
      const json = [{ name: 'Summer Sale', id: 21 }];

      await route.fulfill({ json });
    });
  });
});
```

When the test should keep most of the real response but adjust only a small part of it, `route.fetch()` can retrieve the original payload and `route.fulfill()` can return a patched version.

Alternatives such as ng-apimock integrate the same idea into the Angular testing workflow and are often preferable in Angular-heavy projects because they centralize mocks outside the Playwright specs. This keeps functional tests focused on user behavior instead of polluting test code with inline API-mocking setup. API mocking still makes tests faster and more reliable, avoids polluting real databases with test data, and lets the team validate edge cases that may be difficult to reproduce on demand.

When the behavior under test depends not on HTTP but on browser-provided capabilities, Playwright also supports mocking browser APIs, typically by installing the mock before page load with `page.addInitScript()`.

4.8.5 Mocking Clock

Time-dependent features such as inactivity logout, countdowns, throttling windows, and scheduled refreshes should use a mocked clock so tests stay deterministic and fast.

Playwright provides a dedicated Clock API. Utilizing `Clock` functionality allows developers to manipulate and control time within tests. The recommended approach is to use `setFixedTime` to set the time to a specific value. If that doesn't work for our use case, we can use `install` which allows us to pause time later on, fast forward it, tick it, etc. `setSystemTime` is only recommended for advanced use cases.

To avoid polluting test cases with repeated time setup, keep clock configuration in fixtures or `beforeEach` blocks and let the test body focus on user behavior and expected outcomes.

4.8.6 Page Objects

Page Objects, also called the Page Object Model (POM), use an object-oriented structure to represent the UI. In this model, each page or major UI component has a corresponding class that centralizes locators and exposes reusable interaction methods. This model is especially useful in medium and large automation suites where many tests depend on the same screens and page states.

The main characteristics are:

- **Structure**: Object-oriented classes, usually one page object class per file;
- **Focus**: UI representation (elements, states, and page-specific actions);
- **Pros**: Centralizes locators, reduces duplication, and improves readability and maintainability;
- **Cons**: Can become bloated or over-engineered in small projects or in UIs that change very quickly.

The pattern usually follows some practical rules:

- Provide an interface that is easy to use from tests;
- Encapsulate locator and interaction details inside the page object;
- Model meaningful pages/components instead of creating classes for every tiny fragment;
- Return another page object when a method navigates to a different page;
- Keep assertions in test files, not in page object classes;
- Keep one page object class per file.

Since elements are encapsulated in these objects, UI changes are generally fixed in one place rather than across many tests.

In the example below, we can see a page object for promotional sections:

```
export class PromotionalSectionPO {
  constructor(private readonly page: Page) { }

  getAllSections(): Locator {
    return this.page.locator('section');
  }
```

```
  getAllPromotions(section: Locator): Locator {
    return section.locator('promotion');
  }

  getPromotionName(promotion: Locator): Locator {
    return promotion.locator('.title');
  }
}
```

Each section contains a list of promotions. Each promotion contains its own details. When we need to execute some assertions for a specific promotion, instead of creating a new method, we can reuse a generic method that receives a promotion element.

4.8.7 Helper Functions

Helper functions use a functional structure and focus on behavior and workflows rather than on page structure. They are useful for multi-step operations such as `loginUser`, `checkoutItem`, data setup, API preparation, and repeated domain actions.

The main characteristics are:

- **Structure**: Functional modules and composable functions;
- **Focus**: Reusable workflows and behavior-oriented operations;
- **Pros**: Fast to write for simple scenarios, reusable across suites, and no class hierarchy overhead;
- **Cons**: If helpers also own locators, UI changes may force updates in many functions unless locator definitions are centralized.

Unlike a Page Object, a Helper should not model a specific screen. Its responsibility is to orchestrate reusable operations that may span pages, API calls, fixtures, and test data.

4.8.8 Flaky Tests

As the test suite grows, a small amount of instability can quickly turn the regression signal into noise. A flaky test is one that passes and fails without a code change, usually because it depends on timing, shared state, brittle selectors, an unstable environment, browser automation limits, or test-order coupling. Flaky tests are worse than no tests because they reduce trust in the suite and make it harder to tell whether a failure represents a real regression or only a temporary failure in the test itself.

The right response is to make the test deterministic, isolate the dependency that is introducing nondeterminism, or remove the unstable scenario from the lower-level suite until it can be stabilized. Temporarily disabling a flaky test can be a pragmatic short-term step, but it should be treated as technical debt rather than a permanent solution.

When a test still needs a safety net, retries can help separate transient failures from stable regressions. By default a test is not retried, but when retries are enabled, a test that passes on retry is marked as flaky rather than healthy, which makes the issue visible in the HTML reporter. In practice, retries should help us gather evidence and reduce noise while we fix the root cause, not hide instability behind a passing build.

4.8.9 Test Reporters

Test reporters for Playwright Test are responsible for turning raw execution results into feedback that different audiences can consume. In practice, teams combine lightweight console reporters such as list, line, and dot for quick local runs with structured outputs such as json, junit, and blob for CI ingestion, trend analysis, and cross-shard merge workflows.

For human-readable diagnostics, the built-in HTML reporter is usually the central artifact because it links test status to retries, errors, traces, screenshots, videos, and timing in one place, which makes triage significantly faster in larger suites. Many teams extend this baseline with ecosystem reporters such as allure-playwright for richer historical dashboards and test metadata, or playwright-html-reporter for alternative presentation and distribution needs, while keeping reporting strategy focused on signal quality, failure diagnosis speed, and long-term visibility of stability and flakiness trends.

4.8.10 Debugging

End-to-end tests can be difficult to debug because they depend on an entire system, may depend on prior actions (such as log-in), and may change the state of the application they're testing.

For daily debugging, prefer IDE integrations over command-line-only workflows. In VS Code, the Playwright Test for VSCode extension and the Testing panel make it easier to run a single scenario, debug with breakpoints, inspect traces, and re-run only failed tests. This reduces context switching and speeds up investigation loops.

For deeper diagnosis, combine Playwright Inspector with browser developer tools to inspect network calls, console errors, DOM state, and timing behavior during each test step. In the Debugging section of Playwright documentation, we have procedures to debug functional tests.

4.8.11 Best Practices

The following practices focus on writing functional tests that are stable, maintainable, and aligned with real user behavior:

- ☐ **Have simple project structure**: A good project structure helps everyone catch up quickly, even on large-scale codebases. Keep an `e2e/` folder at the top level for end-to-end tests and test-specific configuration files. In multi-project workspaces, keep app-specific end-to-end tests in `projects/project-name/e2e/`.
- ☐ **Follow the Given-When-Then pattern**: The Given-When-Then formula is a template intended to guide the writing of functional tests.
- ☐ **Make use of lint rules**: Linters can catch severe issues like incorrectly handled errors and lost information. Use plugins designed for test code patterns, such as eslint-plugin-playwright for waiting and locator reliability.
- ☐ **Do not duplicate test coverage**: Do not create functional tests for areas already covered well by lower-level tests. If a UI component can be covered by a unit test, prefer that approach. Unit tests are easier to maintain, less flaky, and cheaper in CI, while functional tests should focus on big-picture user stories across many components and views.
- ☐ **Do not test everything**: Maximize coverage with fewer tests and avoid excessive granularity that creates maintenance burden. It is recommended to verify functional outcomes rather than internal implementation details.
- ☐ **Do not ignore flaky tests**: Tests depend on external systems and can fail randomly due to network conditions, system load, and other non-deterministic factors. Each flaky test needs targeted stabilization (better waits, setup fixes, etc.) or to be removed. Metrics such as execution time and failure/success trends can help identify root causes.
- ☐ **Do not add logic to the tests**: Avoid `if` statements and `loops` in test bodies. Extra logic can let tests pass without asserting real behavior and can slow execution. Tests should stay one-concern and straightforward.
- ☐ **Do not use XPath locator**: XPath is often the slowest and most brittle locator strategy, and it tends to break when markup changes. Expressions are hard to read and debug, and maintenance cost is high. Prefer alternative locator strategies whenever possible.
- ☐ **Do not use IDs**: IDs are often assumed stable but can change, and frameworks may generate or rely on them internally. Avoid adding IDs only for testing. Prefer user-facing locators and dedicated testing attributes like `data-testid`, and use IDs only when no better option exists.

- ☐ **Prefer user-facing and test-id locators**: In Playwright, use resilient locators such as `getByRole`, `getByLabel`, `getByPlaceholder`, and `getByTestId`. Avoid brittle selectors tied to implementation details, like deep CSS chains and dynamic class names. Use `getByText` only when text is intentional and stable.
- ☐ **Use page objects**: Page Object is a key design pattern for cleaner tests. It encapsulates page element knowledge, supports reuse across tests, and centralizes updates when UI templates change. This significantly improves maintainability for automation teams.
- ☐ **Test case only call page object's methods**: For reusability and maintainability, test cases should call page object methods instead of interacting with controls directly. Even simple clicks should be wrapped so requirement and UI changes require refactoring in one place only.
- ☐ **One file, one-page object class**: Define one page object class per file and use user-friendly names for files and classes. This structure improves project recognizability and makes page objects easier to locate in both small and large projects.
- ☐ **Do not place assertions inside page objects**: Keep page objects focused on interaction abstraction and avoid verification points inside them. Putting assertions in page object classes violates single responsibility and makes test scripts harder to understand and maintain.
- ☐ **Do not place assertions inside of try/catch blocks**: Assertions are meant to fail loudly and notify the test engine when expectations are not met. Wrapping assertions in try/catch can hide failures and reduce build feedback quality. Expect errors directly when an error is the intended outcome.
- ☐ **Make tests independent at the file level**: Playwright can run files in parallel across workers and browsers, and Selenium Grid supports similar parallel cross-browser execution. File-level independence is critical because execution order is not guaranteed and isolated runs should remain reliable.
- ☐ **Navigate to the page under test before each test**: Start each test from a clean page state by navigating before execution. Also maintain route-navigation suites for major app paths to verify app connectivity, user navigation flow, and permission behaviors.
- ☐ **Use async & await**: Prefer `async` and `await` to avoid Callback Hell and Pyramid of Doom style code. With Promise-based tools like Playwright and Puppeteer, async/await keeps test flows flatter, clearer, and easier to maintain.
- ☐ **Mock all API calls**: This is controversial, with different strategies across teams. Some prefer mocks for all tests to avoid network dependencies, but others prefer full-system

realism. A practical compromise is maintaining smoke tests in production-like environments with real data.

- ☐ **Avoid hard waits**: Fixed delays like `waitForTimeout()` are fragile and create slow feedback loops. Increasing timeout values only adds latency and masks real synchronization issues. Prefer Playwright auto-waiting and web-first assertions like visibility and URL expectations.

4.9 Visual Tests

4.9.1 Fundamentals

Visual testing is necessary to make sure that style issues don't pop up and the web application continues to look just as good as it works. Visual testing mostly focuses on visual aspects of the user interface. It allows us to pinpoint every pixel at the UI level so that HTML, CSS, and JavaScript differences don't affect the user experience. Verification of visual aspects can be made manually or with the help of automation.

To make this practical, visual regression follows a simple cycle. First, it is important to create a baseline. It is the first screenshot capture of each step. Second, run another round of tests to check whether implemented changes break the application UI. When the test run is finished, we need to compare the latest test run with the baseline to verify whether any changes appeared. If so, we need to check whether the difference is something the development team intended to do. In other words, we need to check if changes appeared for the right reason. If it turns out that the development team did not intend to implement any of them, then they are called visual bugs. In the last phase, the baseline should be updated with new changes to validate new regression iterations properly. This cycle is represented in the next image.

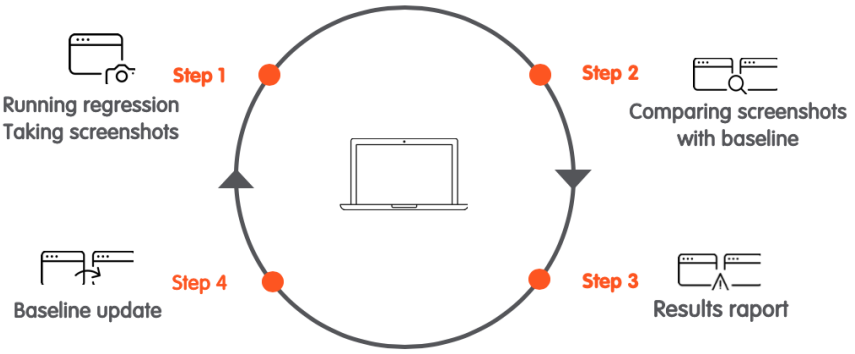


Fig. 2: Visual regression

In practice, there are many tools and libraries that help testers and developers in building test scenarios, but all are focused on:

- Properly rendered and displayed graphics on the page;
- Components of a page displayed correctly, e.g. buttons with proper color and proper shape;
- Layout of a page e.g. each component is in right place on a page;
- Validation of component, if it does not overlay each other.

By comparison, in functional tests we take the following aspects of the application into consideration:

- Validation of input fields;
- Page navigation check;
- Business logic check, e.g. checking CRUD operation on a page;
- Check if hyperlinks are working.

Taken together, functional and visual tests provide the development team with a strong tool for verifying changes during the software development life cycle.

4.9.2 Screenshot Tests

Tests are written to prevent mistakes from becoming user-facing issues. So far, we have focused only on testing application logic. However, the visual appearance of a web app is also important. One way to verify the app's appearance is to use a test that fails when a screenshot changes.

Automated tests that verify visual appearance can be hard to build. Still, we can create a simple screenshot test that fails when major, unintended visual changes appear. We can use Playwright snapshots or jest-image-snapshot to compare a browser screenshot against a reference image. However, comparing images is more complex than comparing text. Several factors can affect screenshot testing, such as the screen resolution on the machine running the tests. Fonts can also cause issues because they may render differently across machines and browsers.

Taking screenshots in Playwright is straightforward. Below, we have an example using Playwright for visual comparison. We can see that we are capturing a screenshot and comparing it to a baseline snapshot.

```
describe('Promotional Sections', () => {
  beforeEach(async ({ page }) => {
    await page.goto('/promotions');
  });

  it('should compare successfully with a baseline', async ({ page }) => {
    await expect(page).toHaveScreenshot('example-page.png');
  });
});
```

In short, this test quickly tells us if the page starts looking different by mistake.

4.9.3 Best Practices

Most best practices for functional testing also apply to visual testing. The following practices focus on creating visual tests that are stable, reviewable, and scalable over time:

- ☐ **Name the screenshots properly**: Initially, screenshots are given default names that are not too descriptive, so include a name so we know what we are actually looking at.
- ☐ **Do not try to test everything**: Try to maximize coverage with fewer tests. This is difficult because we want to know everything that is going on, but the more granular our tests are, the easier it is to track down the actual bugs causing the problem.
- ☐ **Avoid full-page screenshots**: Keep the tests modular or it will get annoying. Also, bigger screenshots mean longer processing time.
- ☐ **Freeze dynamic content**: Mock or control clocks, time zones, and locale-sensitive formatting so date/time labels do not change between runs. This prevents false positives caused by environment-dependent rendering.
- ☐ **Control viewport and browser settings**: Standardize viewport size, device scale factor, browser version, and color scheme in the visual test configuration. Snapshot consistency depends on a predictable rendering environment.
- ☐ **Stabilize fonts and assets**: Use packaged fonts and wait for font loading before taking screenshots. Missing fonts or late-loading assets can shift layout and cause misleading visual diffs.
- ☐ **Mask volatile UI areas**: Exclude or mask unstable UI areas such as advertisements, animated banners, rotating carousels, and user-specific avatars. Focus the comparison on areas that represent product quality.
- ☐ **Disable animations and transitions**: Turn off CSS animations and transitions during snapshot capture. Motion between frames can make deterministic comparison impossible even when the UI is correct.

- ☐ **Use component-level and page-level coverage**: Combine targeted component snapshots with a smaller set of page snapshots. This balances fast diagnosis with confidence in integrated layouts.
- ☐ **Define explicit diff thresholds and policy**: Set clear per-suite tolerance rules and review criteria for acceptable pixel differences. Teams should agree when to accept, reject, or rebaseline a change.
- ☐ **Review snapshots in pull requests**: Make visual diffs part of code review with clear ownership. Human approval of changed baselines helps prevent accidental acceptance of regressions.
- ☐ **Version and document baseline updates**: Record why baseline images changed and link them to intended UI changes. This creates traceability and reduces confusion during future regressions.
- ☐ **Plan for scaling visual testing**: If we are using git to store the baseline images and the project becomes increasingly larger, check out git storage tools like Git Annex or Git Media to prevent the repo from growing out of control.
- ☐ **Use the pattern library**: If the team has a component library, map tests to individual components and their states. This makes it easier to localize failures and identify bugs by validating one component state at a time.
- ☐ **Run the tests frequently**: Build a continuous testing pipeline where the suite runs frequently, such as every day or on each repository change. The earlier issues are found, the lower the cost to fix them.

4.10 Other Tests

4.10.1 Accessibility Testing

Accessibility Testing is a type of testing which determines the usability of a product to the people having disabilities (deaf, blind, mentally disabled etc.). People with disabilities use assistive technology which helps them in operating a software product. Automated tools can quickly detect common issues such as missing labels, low color contrast, incorrect heading hierarchy, and invalid ARIA attributes.

4.10.2 Cross Browser Testing

This testing ensures that software applications perform well on different hardware, operating systems, and network environments. The goal is to verify consistent behavior, not pixel-perfect similarity. Core user journeys should be executed in all supported browsers, while full regression can run in one primary browser plus a smaller compatibility suite in the others. Special attention is needed for CSS layout differences, date/time input behavior, file uploads, font rendering, and browser-specific security restrictions.

4.10.3 Smoke Testing

Smoke testing is a lightweight verification step that checks whether the most critical parts of an application work after a new build or deployment. A smoke test usually confirms that the app starts correctly, the main routes load, essential APIs respond, and a few business-critical flows such as login or navigation are available. The goal is not to validate every detail, but to detect major failures quickly so that obviously broken builds do not move further in the delivery pipeline.

4.10.4 Monkey Testing

Monkey testing is performed by a tester in a random manner, assuming that if the monkey uses the application, then how a monkey enters random values without any knowledge or understanding of the application. The most valuable outcome is not pass/fail status, but identifying unexpected crashes, frozen UI states, and unhandled exceptions that should be converted into deterministic regression tests. There are no special test cases written for monkey testing.

4.10.5 Performance Testing

Performance testing evaluates the application's performance and ensures that it meets expected benchmark requirements. Performance must be measured, monitored, and refined continuously. In addition, the growing complexity of the web creates new challenges for tracking metrics, since results can vary significantly depending on device, browser, protocol, network type, and latency.

4.10.6 Security Testing

Security testing for web applications should cover authentication, authorization, data protection, and dependency risks. Typical checks include role-based access control validation, token handling and expiration behavior, secure cookie settings, protection against common attacks such as XSS and CSRF, and safe handling of secrets in configuration files and logs. Automated

dependency and container scans can complement security testing by identifying known vulnerabilities early in the delivery cycle.

4.11 Gherkin

4.11.1 Fundamentals

Gherkin is a domain-specific language, just like SQL and HTML. Each line is called a step, starts with a keyword, and ends with terminal punctuation. Tabs or spaces are used for indentation, and a comment can be added anywhere, but it should start with a # sign.

```
Feature: Guess the word

  # This scenario has three steps
  Scenario: Breaker joins a game
    Given the Maker has started a game with the word "silky"
    When the Breaker joins the Maker's game
    Then the Breaker must guess a word with 5 characters
```

Each non-blank line must start with a Gherkin keyword, followed by free-form text. The only exceptions are the feature and scenario descriptions.

Based on Gherkin Reference, the primary keywords are:

- **Feature**: The purpose of the *Feature* keyword is to provide a high-level description of a software feature, and to group related scenarios. The first primary keyword in a Gherkin document must always be *Feature*, followed by a ":" and a short text that describes the feature.
- **Rule**: The purpose of the Rule keyword is to represent one business rule that should be implemented. It provides additional information for a feature. A Rule is used to group together several scenarios that belong to this business rule.
- **Scenario (or Example)**: This is a concrete example that illustrates a business rule. It consists of a list of steps. Scenarios follow the pattern Given-When-Then.
- **Given, When, Then, And, But (steps)**: Steps are executed one at a time, in the sequence we've written them in. *Given* steps are used to describe the initial context of the system. *When* steps are used to describe an event, or an action. *Then* steps are used to describe an expected outcome, or result. *And* and *But* are used to avoid successive *Given's*, *When's*, or *Then's*.
- **Background**: Occasionally we are repeating the same *Given* steps in all the scenarios in a *Feature*. A *Background* allows us to add some context to the scenarios that follow it. It

can contain one or more *Given* steps, which are run before each scenario, but after any *Before* hooks.

- **Scenario Outline (or Scenario Template)**: The *Scenario Outline* keyword can be used to run the same *Scenario* multiple times, with different combinations of values.
- **Examples**: A Scenario Outline must contain an *Examples* (or *Scenarios*) section(s). Its steps are interpreted as a template which is never directly run. Instead, the *Scenario Outline* is run once for each row in the *Examples* section beneath it (not counting the first header row).

There are a few secondary keywords as well:

```
""" (Doc Strings)
| (Data Tables)
@ (Tags)
# (Comments)
```

Functional tests can be written in Gherkin and executed with Cucumber + Playwright. Gherkin provides a business-readable format, Cucumber maps scenarios to test steps, and Playwright runs them in real browsers. Scenarios can include steps that sound very similar or even identical. It is recommended to look for opportunities to generalize steps and reuse them. The goal is to build a reusable library of steps so that adding new features requires less effort over time. Reusing steps is recommended whenever possible.

4.11.2 Jira + Xray

Jira is widely used to manage user stories and delivery workflows, while Xray extends Jira with full test management and traceability capabilities. Together, they can keep user stories, BDD scenarios, and execution evidence connected in one flow.

A practical approach to integrate Jira and Xray is:

1. Write or refine acceptance criteria in the Jira user story using business language;
2. Create an Xray Test issue (or use Xray's Cucumber/BDD Test type) linked to that story;
3. Add the Gherkin scenario directly in the Xray test so product owners, QAs, and developers can review it collaboratively;
4. Tag scenarios with issue keys (for example `@TEST-1234`) to preserve traceability between automated tests and Jira artifacts;

5. Execute the automated suite in CI workflow and import Cucumber JSON results back to Xray so test status, history, and evidence are visible in Jira dashboards and test plans.

This workflow improves requirement clarity, keeps BDD tests as living documentation, and provides end-to-end traceability from user story to automated execution.

4.11.3 Cucumber + Playwright

Cucumber combined with Playwright (via playwright-bdd) is a powerful combination for functional testing that bridges the gap between technical and non-technical team members. Cucumber allows test scenarios to be written in Gherkin, a plain-language format that non-technical stakeholders, business analysts, and product owners can read, understand, and validate. Playwright serves as the browser automation engine underneath, providing fast, reliable, and cross-browser testing capabilities.

Below is a practical example of how to configure and use Cucumber with Playwright:

Step 1: Create a configuration file (`playwright.config.js`):

```
const testDir = defineBddConfig({
  features: 'features/**/*.feature',
  steps: 'steps/**/*.js',
});

export default defineConfig({
  testDir,
  reporter: 'html',
});
```

Step 2: Create a feature file (`features/login.feature`):

```
Feature: User Authentication

  Scenario: User can login with valid credentials
    Given I am on the login page
    When I enter email "user@example.com"
    And I enter password "password123"
    And I click the login button
    Then I should see the dashboard
    And URL should contain "dashboard"
```

Step 3: Implement step definitions (`steps/auth.js`):

```
Given('I am on the login page', async ({ page }) => {
  await page.goto('https://app.example.com/login');
});
```

```
When('I enter email {string}', async ({ page }, email) => {
  await page.fill('input[type="email"]', email);
});

And('I enter password {string}', async ({ page }, password) => {
  await page.fill('input[type="password"]', password);
});

And('I click the login button', async ({ page }) => {
  await page.click('button:has-text("Login")');
});

Then('I should see the dashboard', async ({ page }) => {
  await expect(page).toHaveTitle(/Dashboard/);
});

And('URL should contain {string}', ({ page }, urlPart) => {
  expect(page.url()).toContain(urlPart);
});
```

Step 4: Run the tests:

To execute the functional tests, use the following npm script:

```
{
  "scripts": {
    "e2e": "bddgen -c playwright.config.ts && playwright test -c
playwright.config.ts"
  }
}
```

This setup provides several benefits, namely, feature files serve as living documentation, non-technical team members can contribute to test scenarios, Playwright's debugging and reporting tools are fully available, and the test results are automatically generated and can be viewed in the HTML report.

4.11.4 AI + BDD

In the AI era, teams can accelerate Behavior-Driven Development without sacrificing quality by treating Gherkin as the shared language for both acceptance criteria and executable tests. A practical approach is to capture user-story acceptance criteria directly in Gherkin from day one, then evolve those same scenarios into automated tests with Cucumber and Playwright. This keeps requirements, implementation, and validation aligned in a single artifact.

A practical flow is:

1. **Capture acceptance criteria in Gherkin**: Write acceptance criteria as executable Gherkin scenarios, so they become living documentation that product owners, QAs, and developers can review and understand together.
2. **Apply Gherkin-first TDD**: Write the scenario first (or reuse the acceptance criteria), let it fail, implement only what is needed to make it pass, then refactor without breaking it.
3. **Automate with Cucumber + Playwright**: Map approved scenarios to reusable step definitions, then run them in real browsers with Playwright. Keep steps declarative and reuse existing definitions whenever possible to avoid duplication.

This approach keeps feature files readable, improves cross-functional collaboration, and avoids unnecessary growth of duplicated test code. AI can further improve testing quality by drafting scenarios from requirements, suggesting edge and negative cases, highlighting ambiguity in acceptance criteria, generating realistic test data variations, and identifying opportunities to consolidate repetitive steps.

When using AI assistance in automation, Playwright MCP can accelerate browser-test development by combining language-guided generation with real browser execution through Playwright. In practice, this improves selector quality, synchronization decisions, flaky-test triage, and debugging loops.

4.11.5 Best Practices

The following best practices help teams create Gherkin scenarios that remain clear, maintainable, and reusable over time:

- ☐ **Define the actor of the system**: Be explicit about who the user is instead of mixing first-person and third-person language. Clear actor naming makes multi-user behaviour much easier to understand.
- ☐ **Use native language to write scenarios**: Gherkin supports multiple languages, so choose the one that best matches the team and product context. Readability matters more than forcing English when another language is clearer.
- ☐ **Write declarative scenarios**: Focus on behaviour and business value, not on clicks, keystrokes, or other implementation details. Avoid situations where each step is a precise instruction. Declarative steps make scenarios easier to read and less brittle when the UI or flow changes.

- ☐ **Use meaningful scenario names**: Give scenarios names that summarize the main idea in one line. Clear titles make reports easier to scan and failures easier to understand. Try to summarize the gist of a scenario in a single line.
- ☐ **Add good scenario descriptions**: Use descriptions when the title is not enough to explain the context or intent. A short explanation below the feature or scenario helps future readers understand why the scenario exists.
- ☐ **Write the scenario before writing the code**: Drafting scenarios first encourages product, QA, and development to align on behaviour before implementation starts. This reduces ambiguity and surfaces disagreements earlier.
- ☐ **Do not write scenarios in isolation**: Include the whole team when creating scenarios so the result stays automatable and reflects the real domain. Collaboration helps remove unnecessary detail and improves clarity.
- ☐ **Make scenarios independent and deterministic**: Each scenario should run the same way every time, regardless of execution order. Avoid hidden state, shared setup, or data that leaks across scenarios.
- ☐ **Remove unneeded scenarios**: Remove trivial scenarios once the behaviour is already covered by more valuable tests. Keeping them around often adds noise without helping the team understand the product.
- ☐ **Use backgrounds to reduce the number of steps**: Move repeated setup into a `Background` when several scenarios start the same way. This keeps the scenarios shorter and makes the common precondition obvious.
- ☐ **Avoid overuse of scenario outline**: `Scenario Outline` are useful for data-driven cases, but they can quickly multiply test count and execution time. Use them carefully, especially for slow UI-driven flows.
- ☐ **Limit the number of scenarios per feature**: Split large feature files when they start to feel crowded or hard to scan. Smaller files are easier to navigate and keep related cases grouped by purpose.
- ☐ **Limit the number of steps per scenario**: Keep scenarios focused and avoid long procedural chains. If a flow becomes too long, look for a more declarative step, a table, or a better feature split. Consider decreasing their number to some reasonable value, say, 2-3 "Ands" for one `Given`, `When`, or `Then` step.
- ☐ **Avoid using conjunctive steps**: Prefer one action per step when possible, because it keeps steps modular and reusable. Conjunctive steps can be acceptable, but they should

be the exception rather than the default. The next bad example shows two actions connected with the keyword `and`:

```
Given I am on the login page
When I sign in with valid credentials
Then I see the "Welcome User" message and the logout button
```

- ☐ **Use And and But keywords**: Use `And` and `But` to keep the structure of a scenario readable without repeating `Given`, `When`, or `Then`. This makes the flow easier to follow while preserving the intent. See the following example:

```
Given I am on the login page
When I sign in with valid credentials
Then I see the "Welcome User" message
And I see the logout button
But I not see "You are not logged in" message
```

- ☐ **Refactor and reuse step definitions**: Scenarios can include steps that sound very similar or even the same. Reuse common step definitions when they express the same domain concept. A shared step library keeps the suite smaller and makes new scenarios cheaper to add.
- ☐ **Use tags to organize features and scenarios**: Tag scenarios by purpose, level, or execution frequency so they can be filtered and grouped cleanly. Tags make it easier to run smoke, integration, or other subsets on demand.
- ☐ **Use lint rules to check Gherkin syntax**: Linting helps keep scenario structure consistent across authors and catches style problems early. It is especially useful when multiple people contribute to the same suite. Use Gherkin lint to force a consistent style.
- ☐ **Avoid testing through the UI**: Prefer business-facing scenarios over button-click scripts, because UI details are slower and more brittle. Gherkin should describe intent, not merely mirror the interface.
- ☐ **Features are not user stories**: A feature file may collect several related stories, so do not force a one-to-one mapping. Keep the file focused on the behaviour the team needs to understand, then add scenarios only when they add value.

4.12 Testing Suites

Running unit tests should be fast to keep the feedback cycle short, especially in TDD-style development. In Angular projects, component template rendering is often the main execution bottleneck. For this reason, it's recommended to use the *.i.spec.ts* postfix for integration tests

and keep *.spec.ts* for unit tests. For e2e tests, it's recommended to split tests into functional and visual suites.

To make execution simple and consistent, expose each suite through npm scripts. This allows teams to run *npm run test*, *npm run test:unit*, *npm run test:integration*, *npm run e2e*, *npm run functional*, and *npm run visual*, with additional watch-mode variations. For more details, see next npm scripts:

```
{
  "scripts": {
    "ng": "ng",
    "start": "ng serve",
    "test": "npm-run-all -s test:unit test:integration",
    "test:unit": "ng test -c unit --code-coverage",
    "test:unit:watch": "ng test -c unit --code-coverage --watch",
    "test:integration": "ng test -c int --code-coverage",
    "test:integration:watch": "ng test -c int --code-coverage --watch",
    "e2e": "playwright test -c playwright.config.ts",
    "functional": "playwright test -c playwright.config.ts --grep @func",
    "functional:watch": "npm-run-all -p start playwright:watch",
    "visual": "playwright test -c playwright.config.ts --grep @visual",
    "visual:watch": "npm-run-all -p start playwright:watch",
    "playwright": "playwright test -c e2e/playwright.watch.config.ts",
    "playwright:watch": "watch \"npm run playwright\" e2e/test --wait=1"
  }
}
```

In Angular's *angular.json*, we can define separate test configurations through the *test.configurations* property for all tests, unit tests, and integration tests, including watch-mode variations. We can follow the same structure for e2e tests, also with watch-mode variations. The *npm run e2e* script runs all suites, while *npm run functional* and *npm run visual* target functional and visual suites, respectively.

For interactive watch workflows, auxiliary scripts can call testing frameworks using npm-run-all and watch.

4.13 Final Remarks

In this chapter, we covered the main testing layers for Angular applications: isolated and shallow unit tests, integration tests, and end-to-end testing through functional and visual suites.

We looked at a pragmatic strategy that balances confidence, speed, and maintenance cost, with practical guidance on test design, tooling (Vitest/Jest, Playwright, and Cucumber), and when each test type adds the highest value. We also saw how to keep feedback loops fast by separating

unit and integration suites with naming conventions, organizing npm scripts for targeted execution, and identifying slow tests that should be promoted to higher-level suites.

5 Automating Software Delivery

5.1 Overview

Modern software delivery moved from infrequent waterfall releases to continuous workflows shaped by Agile and DevOps practices. As release frequency increases, delivery quality depends on standardized pipelines that provide fast feedback and reliable automation.

In Angular projects, teams commonly use tools such as GitHub Actions or Jenkins to implement Continuous Integration and Continuous Delivery. Automated testing is a critical part of this flow because it reduces repetitive manual effort and validates changes consistently across environments.

A practical delivery concern is the deployment architecture. Angular production builds generate static HTML, CSS, and JavaScript artifacts, so applications can be deployed through static hosting patterns rather than requiring a dedicated Node.js runtime for serving the application itself.

This chapter focuses on automation and delivery practices that help teams ship faster with confidence, including CI/CD workflows, testing automation, deployment patterns, and continuous automation strategies that scale across multiple projects.

5.2 Continuous Testing

5.2.1 Definition

Continuous Testing is the process of executing automated tests as part of the software delivery pipeline to obtain immediate feedback on the business risks associated with a software release candidate. It evolves and extends Test Automation to address the increased complexity and pace of development and delivery of modern applications.

Based on the Tricentis definition, Continuous Testing has 14 key points:

1. Continuous Testing's primary goal is assessing business risk coverage;
2. It provides instant insight into whether a release candidate is too risky to proceed through the delivery pipeline;
3. It establishes a safety net that helps teams protect the user experience in accelerated development processes and avoid software failure headlines;
4. It expects testing to be embedded within the development process, not tacked on at the end;
5. It is seamlessly integrated into the software delivery pipeline and DevOps toolchain;
6. It expects a stable test environment with valid test data to be available for each and every test run;
7. It embraces everything from "shift left" (unit and component coverage) to "shift right" (monitoring/APM, testing in production);
8. It involves executing the right set of tests at the right stage of the delivery pipeline, without creating a bottleneck;
9. It delivers actionable feedback appropriate for each stage of the delivery pipeline;
10. It evaluates each layer of a modern architecture at the appropriate stage of the delivery pipeline;
11. It includes end-to-end tests that realistically assess the end-user experience across all associated technologies (frontend and backend);
12. Tests must be broad enough to detect when an application change inadvertently impacts functionality that users rely on;
13. It reduces false positives by prioritizing robust, flexible modern test frameworks over brittle scripts;
14. It involves continuously reviewing and optimizing the test suite to eliminate redundancy and maximize business risk coverage.

5.2.2 Test Automation

In a traditional development model, testing happens mainly at the end of the cycle. When Test Automation is integrated into day-to-day development, each incremental code change can be validated automatically. In practice, every time a developer commits code to version control, the CI server triggers an application build and executes the automated test suite.

To determine whether the run passed or failed, the Continuous Integration system checks the process exit code and, in some cases, the output logs (for example, text such as "TEST FAILED"). Typically, an exit code of `0` indicates success, while a non-zero code (commonly `1`) indicates failure. After each run, team members receive build notifications that include logs and test reports.

A well-designed test automation system is now essential for modern software delivery. Running tests early and frequently helps teams detect defects sooner and reduce rework, cost, and delivery risk. In mature setups, tests run in parallel across projects, environments, or devices to shorten feedback time. This also frees developers to focus on improving test coverage and participating more actively in code review. Common test types that are strong candidates for automation include Unit Testing, Integration Testing, Functional Testing, Non-functional Testing, Smoke Testing, Security Testing, and Performance Testing.

To be effective, test automation must be defined clearly and optimized from the start. Ambiguity about responsibilities, tooling, or scope usually slows delivery and weakens outcomes. Teams should explicitly decide what to automate, ensure the required skills and tools are available, improve visibility across the organization, and define measurable goals. With these foundations in place, automation can provide consistent and reliable value.

5.2.3 Cross Browser Testing

Cross Browser Testing involves comparing and analyzing the behavior of a web app in different browser environments. It helps ensure that the web app delivers an optimal user experience, independent of the browser used to access it. We need to test in different browsers because, for example, font size can appear differently from browser to browser, JavaScript behavior can differ across web browsers, CSS and HTML may behave differently, image orientation may not be proper, or there may be browser incompatibility with the OS. Also, when we need to support older browsers, Cross Browser Testing is very important.

Cross Browser Testing is a type of non-functional testing that lets us check whether the web app works as intended when accessed through:

- **Different Browser-OS**: i.e., on popular browsers like Firefox, Chrome, Edge, and Safari on any of the popular operating systems like Windows, macOS, iOS and Android.
- **Different devices**: i.e., users can view and interact with the web app on popular devices such as smartphones, tablets, desktops and laptops.
- **Assistive Tools**: i.e., the web app is compatible with assistive technologies like screen readers for people with disabilities.

The sheer number of browsers, devices, and operating systems out there make it impossible to build for and test on every browser-OS combination that may exist. A more realistic goal is to focus testing efforts on maximizing the reach of the web app within the target market. The decision of which browsers and platforms to test on is usually in the hands of Business and Marketing teams (or the client).
Playwright is an end-to-end test framework for modern web applications, including Angular apps. It uses browser automation engines directly (Chromium, Firefox and WebKit), which means we can run tests across browsers. To run tests on multiple browsers, Playwright uses *projects* defined as an array of objects. For more details, see next partial example of *playwright.config.ts* file:

```
{
  "projects": [
    { "name": "chromium", "use": { "browserName": "chromium" } },
    { "name": "firefox", "use": { "browserName": "firefox" } },
    { "name": "webkit", "use": { "browserName": "webkit" } }
  ]
}
```

Playwright can run tests in parallel across workers and projects. We can also define browser-specific options in each project's *use* block. When designing automation test suites, the ability to run tests in parallel is a key feature for decreasing execution time and increasing test efficiency. With Playwright, this is configured in *playwright.config.ts* using *workers* for parallelism and *fullyParallel* when file-level parallel execution is safe.

```
{
  "workers": 4,
  "fullyParallel": true
}
```

Isolating tests is very important to prevent the propagation of faults, facilitate the debugging of errors or flakiness, and eliminate the need to worry about the order of parallel execution.

5.2.4 Mobile Testing

Users expect the application to run smoothly on mobile devices, regardless of operating system, screen size, permissions, or network conditions. Mobile application testing validates that software built for handheld devices behaves correctly, is usable, and remains consistent across environments. In practice, mobile apps are usually grouped into three categories, namely, Mobile Web Apps, Native Apps, and Hybrid Apps.

Playwright is the recommended testing tool for mobile Angular applications. It ships with a catalogue of built-in device emulation profiles that replicate the viewport, user-agent, and pixel ratio of real handsets, so no physical device or additional infrastructure is required for browser-level tests. It is excellent for mobile web, PWAs, and hybrid apps, but cannot natively automate fully native mobile apps.
The following example configures two mobile emulation projects, namely, one for a Pixel 10 (Android/Chrome) and other for an iPhone 17 (iOS/Safari):

```
export default defineConfig({
  use: {
    baseURL: 'http://localhost:4200'
  },
  projects: [
    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 10'] }
    },
    {
      name: 'Mobile Safari',
      use: { ...devices['iPhone 17'] }
    }
  ]
});
```

To simplify the execution across mobile emulation projects, we can define the following scripts.

```
"scripts": {
  "functional": "npm-run-all -s -c android:test ios:test",
  "android:test": "playwright test --project='Mobile Chrome'",
  "ios:test": "playwright test --project='Mobile Safari'"
}
```

Appium is an open-source automation framework for testing native, hybrid, and mobile web applications on Android and iOS, and it can also be used to automate desktop applications through platform-specific drivers. In practice, Appium works by implementing the WebDriver protocol—the same standard used by Selenium. This means tests can be written using familiar Selenium WebDriver APIs.

A typical Selenium + Appium flow for native mobile testing is:

1. Start an Appium server (locally, in CI, or via cloud device providers);
2. Define desired capabilities (or Appium options), such as platform name, platform version, device name, and app/bundle identifier;
3. Create a WebDriver session that targets Android or iOS;
4. Execute element interactions and assertions using the same WebDriver concepts used in browser automation;
5. Run tests in parallel across devices/emulators to reduce feedback time.

The following example shows the idea using Selenium with Appium for Android:

```
const driver: WebDriver = await new Builder()
  .usingServer('http://127.0.0.1:4723/')
  .withCapabilities({
    platformName: 'Android',
    'appium:automationName': 'UiAutomator2',
    'appium:deviceName': 'Android Emulator',
    'appium:appPackage': 'com.example.app',
    'appium:appActivity': '.MainActivity'
  })
  .build();
```

For iOS, replace the capabilities with XCUITest-based values (`platformName: 'iOS'`, `appium:automationName: 'XCUITest'`) and the corresponding app bundle identifier (`appium:bundleId`). The rest of the session lifecycle (element interactions, assertions, and teardown) is identical across platforms.

Testing native applications is outside the scope of this chapter. For that, Espresso, UI Automator, or XCTest can be used.

5.2.5 Selenium Grid

5.2.5.1 What is Selenium Grid?

Selenium Grid is a testing tool that allows us to run test cases in different machines across different platforms. Selenium Grid is a smart proxy server that makes it easy to run tests in parallel on multiple machines. This is done by routing commands to remote instances of web browsers, where a server acts as a hub, routing test commands to multiple registered Grid nodes.

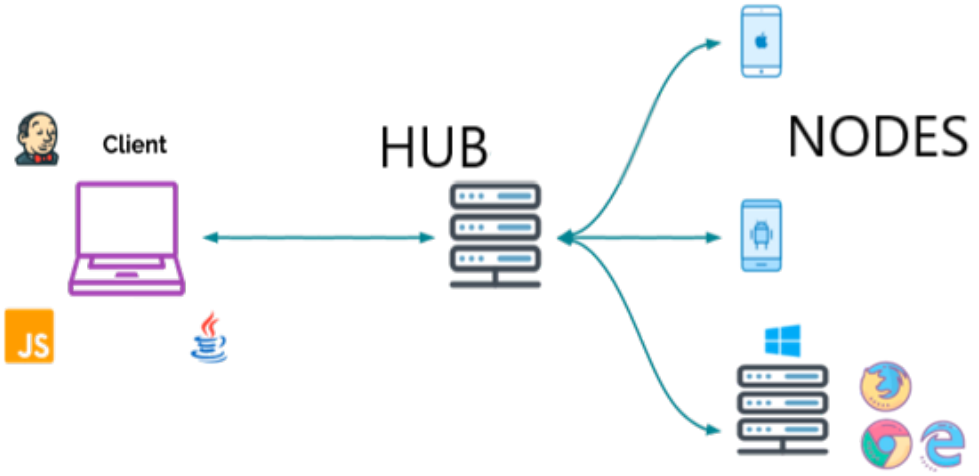


Fig. 1: Components of Selenium Grid

The two major components of Selenium Grid are:

- Hub is a server that accepts requests from the WebDriver client, routing commands to remote browser drivers on nodes. It takes instructions from the client and executes them remotely on multiple nodes in parallel.
- Node is a remote device that consists of a native OS and a remote WebDriver. It receives requests from the hub in the form of JSON test commands and executes them using WebDriver.

In Selenium, Grid uses a modern distributed architecture internally (Router, Distributor, Session Queue, Session Map, and Nodes). In daily usage, we still commonly run it in standalone mode or in hub-and-nodes mode. The main reasons why we might want to use Selenium Grid are the possibility of running tests against multiple browsers, multiple browser versions, and browsers running on different operating systems, while reducing the time it takes for the test suite to complete. To use Selenium Grid, we need to maintain our own infrastructure from within, and since this can be a cumbersome and time-intensive effort, many organizations use IaaS providers or Docker-based setups.

Playwright can connect to Selenium Grid Hub that runs Selenium to launch Google Chrome or Microsoft Edge browser, instead of running the browser on the local machine. In practice, we usually choose between two deployment models, namely, standalone mode for simple local or CI usage, and hub-and-nodes mode for distributed execution across multiple machines.

5.2.5.2 Standalone mode

Standalone mode is the simplest way to run Selenium Grid. In this model, the Grid router and the browser node run inside the same Selenium container or process. In other words, there is only one endpoint to start, monitor, and connect to. This is a good fit for local development, small CI jobs, proof-of-concept setups, and teams that only need one browser host at a time.

The following command starts Selenium in standalone mode with Chromium support:

```
docker run -d -p 4444:4444 --shm-size="2g" \
       -e SE_NODE_GRID_URL="http://localhost:4444" \
       selenium/standalone-chromium:latest
```

Then run functional tests with Playwright by pointing to that single endpoint:

```
SELENIUM_REMOTE_URL=http://localhost:4444 npm run functional
```

If this setup is sufficient for the team, there is no need to use hub-and-nodes mode.

5.2.5.3 Hub and nodes mode

Hub-and-nodes mode is the distributed version of Selenium Grid. In this model, the hub is the central entry point that receives session requests, while one or more nodes register themselves with the hub and advertise the browsers they can run. This is the right choice when we need to scale test execution across multiple machines, isolate browser workloads, or mix different operating systems and browser versions.

Next example shows how to run a Selenium hub and a single Selenium node, and connecting Playwright to the hub. First start the hub container and one or more node containers:

```
docker run -d -p 4442-4444:4442-4444 selenium/hub:latest
docker run -d -p 5555:5555 --shm-size="2g" \
       -e SE_EVENT_BUS_HOST=<selenium-hub-ip> \
       -e SE_EVENT_BUS_PUBLISH_PORT=4442 \
       -e SE_EVENT_BUS_SUBSCRIBE_PORT=4443 \
       -e SE_NODE_GRID_URL="http://<selenium-hub-ip>:4444" \
        selenium/node-chromium:latest
```

Note that the hub and node have different IPs, so we pass `SE_NODE_GRID_URL` pointing to the hub when starting the node container. Then run functional tests with Playwright against the hub URL. For troubleshooting, enable detailed Playwright connection logs:

```
DEBUG=pw:browser* SELENIUM_REMOTE_URL=http://<selenium-hub-ip>:4444 \
       npm run functional
```

Full details are available in the official Selenium Grid documentation and Playwright Selenium Grid guide.

5.2.6 Testcontainers

Testcontainers is an open-source library for providing throwaway, lightweight instances of databases, message brokers, web browsers, or just about anything that can run in a Docker container. Instead of mocking HTTP calls or maintaining a shared staging environment, Testcontainers spins up real Docker containers before test execution and tears them down afterwards. This provides high confidence that the frontend integrates correctly with the real backend stack in an isolated and reproducible environment.

Angular e2e tests commonly stub or mock backend API calls. While that is fast, it gives limited assurance that the real API contract is honoured. A common alternative is to point tests at a shared staging server, which introduces flakiness due to shared state and network dependency. Testcontainers offers a middle ground, because a real, local, ephemeral backend stack that exists only while the tests are running.

Typical use cases for Testcontainers include:

- Starting a real REST API container (e.g. a Quarkus or Node.js service) for Playwright e2e tests to call;
- Seeding a containerised database (PostgreSQL, MongoDB, etc.) with known data so test assertions are deterministic;
- Running the Angular dev server or a built Nginx container alongside the API container and exercising the full stack;
- Replacing mock servers or MSW stubs for contract-level tests.

A backend service that is already containerised can be started directly from its Docker image. The following example starts a custom REST API, waits until the health endpoint responds, and exposes the mapped port to the test process:

```
let apiContainer: StartedTestContainer;
let apiBaseUrl: string;

beforeAll(async () => {
  apiContainer = await new GenericContainer('my-org/my-api:latest')
    .withExposedPorts(8080)
    .withWaitStrategy(Wait.forHttp('/health', 8080))
    .start();
```

```
  apiBaseUrl = `http://localhost:${apiContainer.getMappedPort(8080)}`;
});

afterAll(async () => {
  await apiContainer.stop();
});
```

The `withWaitStrategy` call prevents tests from running until the container is truly ready. `Wait.forHttp` polls the given HTTP path and waits for a `2xx` response. Other strategies include `Wait.forLogMessage` (wait for a log line) and `Wait.forHealthCheck` (use the Docker health check).

By default, each container runs on the Docker bridge network. When both the Angular dev server and the API container run on the same host they can communicate through `localhost` with the mapped port. If services need to communicate with each other inside Docker (e.g. the API container connecting to the database container by hostname), use a Testcontainers network.

5.2.7 Best Practices

The following practices describe how to operationalize continuous testing with reliable feedback loops and scalable automation:

- ☐ **Identify tests that can be automated**: Not every test should be automated. Prioritize high-frequency and high-value scenarios, while preserving manual checks where they are more effective. Revisit this prioritization regularly as product risk, architecture, and user behavior evolve.
- ☐ **Prioritize coverage by business risk**: Use test effort to protect the workflows, integrations, and failure modes that matter most to users and the business. Risk-based prioritization prevents teams from spending too much time on low-impact checks while critical paths remain under-tested.
- ☐ **Embed testing into daily development**: Run automated checks continuously as part of local development, pull requests, and CI instead of leaving validation until the end. Earlier feedback reduces defect escape and shortens the cost of fixing issues.
- ☐ **Run the fastest tests early**: Execute unit and other quick checks before long-running suites to fail fast and avoid wasting pipeline resources. Early failures also shorten feedback loops for developers and reduce overall queue time in shared CI runners.
- ☐ **Use a balanced test strategy across layers**: Place most checks in fast unit/component tests, complement with focused integration tests, and keep end-to-end tests targeted to

critical business journeys. This balance preserves broad risk coverage without making the pipeline slow or brittle.

- ☐ **Keep test environments stable and production-like**: Provide consistent environments, valid configuration, and representative dependencies so test results remain trustworthy. Match key runtime characteristics closely enough that release decisions made from test outcomes remain meaningful.
- ☐ **Keep test data deterministic and resettable**: Use seeded datasets, isolated fixtures, and repeatable setup/teardown so tests start from known state every run. Deterministic test data improves trust in failures and reduces random pipeline instability.
- ☐ **Isolate tests from each other**: Ensure tests can run independently and in any order without shared mutable state leaking between them. Good isolation reduces flakiness, simplifies debugging, and makes parallel execution safer.
- ☐ **Run tests in parallel**: Use parallel workers and distributed infrastructure to reduce feedback time, while monitoring resource contention on shared runners. Group tests by stability and runtime profile so parallelism improves throughput without increasing flakiness.
- ☐ **Run tests locally before pushing**: Developers should execute the same test commands locally that CI executes, reducing broken builds and integration noise. A single, consistent command for local and CI execution helps avoid "works on my machine" drift.
- ☐ **Avoid false negatives and false positives**: Stabilize test environments and data, eliminate flaky scripts, and continuously harden assertions to keep tests reliable. Reliable tests are essential for release decisions, especially when deployments are frequent.
- ☐ **Treat flaky tests as defects**: Track intermittent failures explicitly, quarantine them when necessary, and assign ownership for fixing them. A flaky suite erodes confidence in automation and encourages unsafe reruns or ignored failures.
- ☐ **Validate realistic end-user journeys across the full stack**: Keep a focused set of end-to-end tests that exercise the frontend together with real backend behavior or close substitutes. These tests should confirm that the user experience still works across the actual architecture, not just isolated units.
- ☐ **Cover target browsers, devices, and accessibility paths deliberately**: Base browser, mobile, and assistive-technology coverage on the application audience instead of testing every possible combination. Targeted coverage gives better confidence than broad but shallow execution.

- ☐ **Treat test code as production code**: Apply the same review standards, refactoring discipline, and coding conventions to test suites as to application code. Well-structured test code lowers maintenance cost and keeps feedback reliable as the product evolves.
- ☐ **Deliver actionable feedback from every test stage**: Test output should make failures easy to diagnose with clear logs, reports, screenshots, traces, and ownership. Fast feedback is only useful when teams can understand and act on it quickly.
- ☐ **Check test reports deeply**: Analyze root causes instead of only pass/fail counts, and manually validate suspicious failures so real defects are not masked. Trend analysis across builds helps identify recurring instability, hidden regressions, and fragile test areas.
- ☐ **Use selective execution plus scheduled full runs**: Run impacted tests quickly on each commit or pull request, and execute full regression suites on a regular schedule (for example nightly). This keeps feedback fast while still detecting cross-feature regressions that selective runs may miss.
- ☐ **Continuously review and trim the test suite**: Remove redundant, obsolete, or low-value tests and strengthen coverage where risks have increased. A smaller, sharper suite usually provides better feedback than a large suite full of overlap.
- ☐ **Publish build and test results**: Make the latest pipeline status visible via dashboards, reports, and notifications so failures are acted on quickly. Transparent outcomes strengthen team ownership and reduce time-to-recovery after failed runs.
- ☐ **Stop the pipeline when a stage fails**: Enforce fail-fast behavior so broken changes do not progress to downstream stages. This prevents cascading errors and keeps debugging focused on the first meaningful failure.
- ☐ **Keep test pipelines fast**: Continuously optimize test suites, remove low-value or unstable checks, and scale infrastructure to protect delivery velocity. Fast pipelines encourage frequent commits and reduce the chance that developers bypass quality gates.
- ☐ **Monitor test pipelines continuously**: Track failure trends, flaky tests, queue times, and agent health to prevent persistent delivery bottlenecks. Define clear service-level targets for pipeline duration and stability so deterioration is detected early.
- ☐ **Use smoke-test deployments**: Run fast, high-value end-to-end checks immediately after deployment to confirm critical user flows. Keep smoke suites shallow but broad so they quickly detect outages without delaying the pipeline.

- ☐ **Use production monitoring to improve tests**: Feed incidents, customer-reported defects, and observability data back into the automated test strategy. Production feedback helps teams add missing checks for the failures that actually matter in real usage.
- ☐ **Choose the right automation tools**: Select server and test tools based on team maturity, budget, scalability needs, and ecosystem fit. Standardize tool usage patterns across projects to reduce maintenance overhead and training time.
- ☐ **Involve relevant stakeholders**: Include developers, QA, and operations in automation decisions so test coverage, environments, and tooling reflect real delivery risks. Shared ownership improves prioritization and prevents critical quality concerns from being overlooked.

5.3 Continuous Integration

5.3.1 Definition

Continuous Integration (CI) is the practice of merging code changes from multiple contributors into a shared project and automatically verifying each change before it is accepted. In practice, CI usually runs build, test, and quality checks as soon as code is pushed to version control. This reduces integration problems, keeps the main branch stable, and helps teams move faster with less rework. The term “Continuous Integration” originated with Kent Beck's Extreme Programming development process, as one of its original twelve practices.

Based on Martin Fowler, CI relies on the following practices:

- Put everything in a version controlled mainline
- Automate the build
- Make the build self-testing
- Everyone pushes commits to the mainline every day
- Every push to mainline should trigger a build
- Fix broken builds immediately
- Keep the build fast
- Hide work-in-progress
- Test in a clone of the production environment
- Everyone can see what's happening

- Automate deployment

CI depends on a version control system (VCS). Common VCS options include Git, Mercurial, and Subversion. Popular hosting platforms include Bitbucket, Github, and Gitlab. Once version control is in place, teams usually add merge checks such as automated tests, syntax checkers, code formatters, or dependency vulnerability scans. Automated tests are the most valuable of these checks, because they catch problems before changes reach the main branch.

CI is now a standard part of modern software delivery. Its benefits extend beyond engineering because it improves visibility, coordination, and release confidence across the whole team. Common tools include GitHub Workflows, Bitbucket Pipelines, Jenkins, Bamboo, TeamCity, and CircleCI.

5.3.2 GitHub Workflows

5.3.2.1 What is GitHub Actions?

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows us to automate build, test, and deployment pipelines. We can create workflows that build and test every pull request of the repository, or deploy merged pull requests to production. GitHub Actions are integrated directly into the GitHub repository, making it easy to set up and maintain without requiring additional infrastructure or third-party tools.

GitHub Actions uses Workflows, which are automated processes defined in YAML format and stored in the *.github/workflows* directory of the repository. Workflows are triggered by specific events, such as a push to a branch, creation of a pull request, or on a schedule. Each workflow consists of one or more jobs, and each job contains a series of steps that execute commands or actions.

Creating a workflow file and committing it to the repository provides several benefits:

- Workflows are version controlled and tracked in the same repository as the source code;
- Workflows can be reviewed as part of pull request reviews, ensuring transparency and team consensus;
- Workflows are easily accessible and editable by all team members with repository access;
- GitHub provides a visual interface to view workflow runs, logs, and artifacts;
- Built-in notifications and status checks integrate seamlessly with GitHub pull requests and issues.

5.3.2.2 Workflow Syntax and Structure

A GitHub Actions Workflow is a YAML file that defines the automation process. The basic structure includes triggers, jobs, and steps. The following example shows a simple workflow triggered on every push to the repository:

```yaml
name: Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6

      - uses: actions/setup-node@v6
        with:
          node-version: 22

      - name: Install Dependencies
        run: npm ci
```

The `on` keyword specifies the events that trigger the workflow. Common events include `push`, `pull_request`, `schedule`, and `workflow_dispatch` (manual trigger). The `jobs` section defines one or more jobs that run in parallel by default. The `runs-on` keyword specifies the type of machine to run the job on, with `ubuntu-latest` being a common choice for Node.js projects. Each `step` either runs a shell command with the `run` keyword or uses a pre-built action from the GitHub Marketplace with the `uses` keyword.

5.3.2.3 Declarative GitHub Actions Workflow

Angular CLI require Node.js for build applications, npm for dependency management, and a web browser for testing. GitHub provides hosted runners with Node.js pre-installed, eliminating the need to configure a dedicated machine. However, for more complex scenarios or custom environments, we can use Docker containers within the workflow.

The following partial workflow for an Angular project performs all the necessary steps to build, test, and release an application:

```yaml
jobs:
  Pipeline:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22

      - name: Install Dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Unit Tests
        run: npm run test

      - name: E2E Tests
        run: npm run e2e

      - name: Code Quality
        run: npm run sonar

      - name: Audit
        run: npm audit --production

      - name: Build and Push Docker Image
        run: |
           npm run build
           docker build -t app .
           docker push app
```

The first step, after the initial runner setup, uses *npm* to pull all the required Node.js package dependencies. `npm ci` (clean install) is used instead of `npm install` for faster, more reliable, and consistent dependency installations in automated environments, Docker builds, and production. It strictly adheres to the `package-lock.json` file, skips user-oriented features, and ensures every environment installs the exact same dependencies, avoiding "it works on my machine" issues. The next step is to check TypeScript code using a linter. Notice the `npm run lint` command doesn't pass the *--fix* option to ESLint since we want the command fail if there are any sort of errors. We do not want CI automatically fix lint errors. The next steps are unit tests and e2e tests. We need *test* and *e2e* defined as lifecycle scripts in *package.json*. For both, we can publish HTML reports. These scripts were purposely omitted. SonarQube is a code quality and security tool for detecting bugs and vulnerabilities in applications with thousands of automated static code analysis rules. It supports several programming languages and is recommended for Angular projects. The Audit step is used for auditing package dependencies

for security vulnerabilities. Running *npm audit* will produce a report of security vulnerabilities. We can create a custom script to export an HTML report with npm-audit-html. At last, we are at the actual build step. In this step, the output is the *dist* or *build* directory containing the finished build output. Then, the Docker image is built from the *Dockerfile* recipe.

These steps are recommended for a Continuous Integration and Continuous Delivery process. We can always include other steps or remove some of them. Other steps include project documentation, publishing libraries and deployment. To avoid long runs, we can run some of these steps in parallel. Another improvement includes build the project once. By default, Angular CLI compiles and serves static files for e2e tests. Instead, we can build the project and the Docker image before e2e tests.

5.3.2.4 GitHub Workflows Features

GitHub Actions Workflows provide several powerful features that enhance automation capabilities:

- **Caching**: Dependencies can be cached between workflow runs to improve performance. For Node.js projects, caching *npm* dependencies significantly reduces installation time. The `key` is generated based on the operating system and package lock file hash, ensuring cache invalidation when dependencies change:

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ hashFiles('package-lock.json') }}
```

- **Artifacts**: Build outputs and test reports can be uploaded as artifacts for inspection or downstream jobs. Artifacts are retained for a configurable period and can be downloaded from the workflow run page:

```
- name: Upload Coverage Reports
  uses: actions/upload-artifact@v4
  with:
    name: coverage-reports
    path: coverage/
```

- **Matrix Strategy**: Running the same job across multiple configurations (browsers, Node.js versions, operating systems) in parallel improves test coverage efficiency:

```
strategy:
  matrix:
    node-version: [18, 20, 22]
    os: [ubuntu-latest, macos-latest, windows-latest]

runs-on: ${{ matrix.os }}
```

```
steps:
  - uses: actions/setup-node@v6
    with:
      node-version: ${{ matrix.node-version }}
```

- **Conditional Execution**: Steps and jobs can execute conditionally based on previous results or external conditions:

```
- name: Run Tests
  run: npm run test

- name: Upload Coverage
  if: success()
  run: npm run coverage
```

- **Secrets and Environment Variables**: Sensitive data like API keys and credentials are stored as repository secrets and accessed securely within workflows:

```
env:
  NODE_ENV: production

steps:
  - name: Deploy
    run: deploy.sh
    env:
      DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

- **Context and Expressions**: Dynamic values can be accessed using GitHub's context system and expressions:

```
- name: Display Information
  run: |
    echo "Branch: ${{ github.ref }}"
    echo "Commit: ${{ github.sha }}"
    echo "Actor: ${{ github.actor }}"
```

- **Job Dependencies and Outputs**: Jobs can depend on other jobs and share outputs between them:

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.version.outputs.tag }}
    steps:
      - id: version
        run: echo "tag=v1.0.0" >> $GITHUB_OUTPUT

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying ${{ needs.build.outputs.version }}"
```

- **Status Checks and Notifications**: Workflows automatically report their status to pull requests and can trigger notifications on failure, helping teams maintain code quality standards.

5.3.3 Jenkins

5.3.3.1 What is Jenkins?

Jenkins is a free, open-source automation server that helps automate key stages of the software development lifecycle, including building, testing, and deploying applications. It supports Continuous Integration (CI) and Continuous Delivery (CD), enabling teams to deliver software more efficiently and reliably. Jenkins is a self-contained, Java-based application that can run out of the box on Windows, macOS, and other Unix-like operating systems. Builds can be triggered in several ways, such as through commits to a version control system, scheduled jobs using a cron-like mechanism, or requests to a specific build URL. Jenkins can also initiate builds automatically after dependent jobs have completed. One of Jenkins' greatest strengths is its extensibility. Jenkins offer a rich ecosystem of plugins allows it to integrate with a wide range of development, testing, and deployment tools.

The following command starts Jenkins from the official Jenkins Docker image:

```
docker run -d -p 8080:8080 -p 50000:50000 \
       -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Inside the container, Jenkins stores its state in */var/jenkins_home*. `The -v jenkins_home:/var/jenkins_home` mapping persists that data in a Docker named volume, so jobs, plugins, and configuration survive container recreation.

After downloading, installing, and running Jenkins using the procedure above, the post-installation setup wizard begins. This setup wizard takes us through a few quick "one-off" steps to unlock Jenkins, customize it with plugins, and create the first administrator user through which we can continue accessing Jenkins. It's recommended to follow Jenkins Best Practices to ensure a correct and reliable configuration.

5.3.3.2 Pipelines Syntax and Structure

Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins. Jenkins Pipeline provides an extensible set of tools for modeling simple-to-complex delivery pipelines "as code" via the Pipeline DSL. The definition of a Jenkins Pipeline is written into a text file (called a *Jenkinsfile*) which in turn can be committed to a

project's source control repository. This is the foundation of "Pipeline-as-Code", that is, treating the CI/CD pipeline a part of the project to be versioned and reviewed like any other code.

Creating a *Jenkinsfile* and committing it to source control provides several benefits:

- Automatically creates a pipeline build process for all branches and pull requests;
- Code review/iteration on the pipeline (along with the remaining source code);
- Audit trail for the pipeline;
- Single source of truth for the pipeline, which can be viewed and edited by multiple members of the project.

Jenkins Pipeline is designed to easily use Docker images as the execution environment for a single stage or the entire pipeline. This approach allows teams to use practically any tool that can be packaged in a Docker container:

```
pipeline {
  agent {
    docker 'node:16-alpine'
  }
}
```

For projects which require a more customized execution environment, Jenkins Pipeline also supports building and running a container from a *Dockerfile* in the source repository. More details are available on the Using Docker with Pipeline page. Also, we have a list of Pipeline Best Practices available in the official documentation.

5.3.3.3 Declarative Jenkins Docker Pipeline

A Declarative Jenkins Pipeline using Docker is a structured, opinionated way to define a CI/CD pipeline in Jenkins using a human-readable syntax written in a `Jenkinsfile`. It follows a predefined syntax that organizes the pipeline into sections such as pipeline, agent, stages, and steps.

The next focused pipeline is a clean example for Angular projects, with steps to build, test, and release an application:

```
pipeline {
  agent {
    docker 'circleci/node:16-stretch-browsers'
  }
  stages {
    stage('Install Dependencies') {
      steps {
```

```
        sh 'npm ci --no-cache'
      }
    }
    stage('Lint') {
      steps {
        sh 'npm run lint'
      }
    }
    stage('Unit Tests') {
      steps {
        sh 'npm run test'
      }
    }
    stage('E2E Tests') {
      steps {
        sh 'npm run e2e'
      }
    }
    stage('Sonar') {
      steps {
        sh 'npm run sonar'
      }
    }
    stage('Audit') {
      steps {
        sh 'npm audit --production'
      }
    }
    stage('Build and Push Docker Image') {
      steps {
        sh 'npm run build'
        sh 'docker build -t app .'
        sh 'docker push app'
      }
    }
  }
}
```

First, the agent for this pipeline is defined as a Docker image from CircleCI with Node.js, *npm*, and web browsers. Jenkins will pull the image, launch a container on any slave that has available executors, bind the workspace, and then execute the step commands within that container. The remaining stages are exactly the same as those explained for GitHub Actions Workflow.

Jenkins traditionally uses a web UI for job configuration, which separates CI/CD logic from application code. With Pipeline as Code, pipelines are defined in a *Jenkinsfile* stored in source control, enabling Jenkins to automatically discover and run branch-specific pipelines through multibranch pipelines.

5.3.3.4 Jenkins as Code

Setting up Jenkins can become complex because Jenkins core and plugins expose many settings through the web UI. Historically, teams relied on Groovy init scripts to customize Jenkins and enforce desired state. Those scripts are powerful because they call Jenkins APIs directly, but they also require deep Jenkins internals knowledge and careful security handling. The Configuration as Code Plugin (JCasC) provides a more structured approach, letting teams define Jenkins controller configuration in human-readable YAML files that can be versioned and reviewed.

Many classic Jenkins jobs are stored internally as XML on the controller. Editing that XML directly is hard to maintain and error-prone. For modern Jenkins usage, the recommended baseline is using the controller configuration in JCasC and job logic in *Jenkinsfile* using Pipeline as Code. For teams that still manage large numbers of classic freestyle jobs, Jenkins Job Builder can generate and update jobs from YAML definitions, which helps move job definitions into version control.

To install Jenkins Job Builder from PyPI use:

```
pip install --user jenkins-job-builder
```

Configure a `jenkins_jobs.ini` file to connect with the Jenkins instance:

```
[jenkins]
user=USERNAME
password=API_TOKEN
url=JENKINS_URL
```

Where *user* is the Jenkins username, *password* is typically a Jenkins API token, and *url* is the Jenkins URL. After writing YAML job definitions in a *jobs* folder, update Jenkins with:

```
jenkins-jobs --conf etc/jenkins_jobs.ini update jobs
```

Other common operations include testing a job definition, deleting jobs and providing plugins. More details are available in the official Jenkins Job Builder documentation.

When we have many Jenkins jobs crafted manually, Jenkins Job Wrecker can help bootstrap migration to JJB by exporting existing jobs to YAML. We can install a released version from PyPI:

```
pip install jenkins-job-wrecker
```

Jenkins Job Wrecker supports querying a live Jenkins server dynamically. To translate every job on the server:

```
JJW_USERNAME=<user> JJW_PASSWORD=<password> jjwrecker -a -s <JENKINS_URL>
```

Jenkins Job Wrecker will iterate through all jobs and create *.yml* files in an *output* folder. If authentication is required, set credentials as environment variables. *JENKINS_URL* is the Jenkins instance URL. As with any generated migration output, review and clean up YAML definitions before applying them in production.

5.3.4 Notifications

CI/CD notifications are essential to shorten feedback loops and keep delivery flow healthy. When a build, test, security scan, or deployment fails, the team should be alerted immediately in the channels they already use (for example, Microsoft Teams, Slack, or email). Fast visibility reduces mean time to detect (MTTD), helps teams fix broken pipelines quickly, and prevents unnoticed failures from blocking releases.

In GitHub Actions, notifications are commonly implemented as conditional steps using `if: failure()`, `if: success()`, or `if: always()`, combined with secrets for webhook URLs. The following example sends a message to Microsoft Teams only when the workflow fails:

```
- name: Notify MS Teams
  if: failure()
  run: |
    curl -H "Content-Type: application/json" -d '{"text": "❌ Tests
Failed."}' ${{ secrets.TEAMS_WEBHOOK_URL }}
```

If we also need success notifications (for example, production deployment completed), we can add a separate step with `if: success()` and a different message.

5.3.5 Best Practices

The following practices summarize how to keep CI pipelines secure, fast, and dependable from commit to merge readiness:

- ☐ **Maintain a central source repository**: Keep all source code, pipeline definitions, and build-related configuration in version control as the single source of truth. A fresh checkout should build without hidden dependencies or manual setup. This improves traceability, reproducibility, and team alignment.
- ☐ **Keep builds reproducible and deterministic**: Pin runtime and toolchain versions (for example Node.js, package manager, Angular CLI), commit lockfiles, and prefer immutable container images for CI runners. Version pinning reduces supply-chain risk and prevents surprise breakages.

- ☐ **Use clean, ephemeral runner environments**: Prefer short-lived runners or containers that start from a known baseline for each run. Clean environments reduce hidden state, improve reproducibility, and make failures easier to diagnose.
- ☐ **Commit frequently and integrate daily**: Small, frequent commits reduce integration pain and surface conflicts earlier than large delayed merges. Encourage merging to the baseline at least daily when changes are ready. Avoid committing unfinished work to shared branches and never normalize broken builds.
- ☐ **Minimize long-lived branches**: Integrate to the main shared branch early and often to reduce late-stage merge complexity. Keep branch count and branch lifetime small to preserve CI feedback quality. Short-lived branches help the baseline stay close to production reality.
- ☐ **Every commit should trigger CI pipeline**: Configure CI to trigger automatically on every relevant push or pull request update. This ensures each change is validated in a clean, standardized environment. Consistent per-commit validation prevents defects from accumulating silently.
- ☐ **Follow pipeline as code**: Version pipeline definitions like application code, with reusable modules and automated validation. Review pipeline changes through pull requests using the same standards as application code.
- ☐ **Protect the default branch**: Require passing CI checks before merge, and block direct pushes to protected branches. Branch protection policies ensure integration standards are enforced consistently rather than optionally.
- ☐ **Make the build self-testing**: Build stages should execute automated checks such as linting, unit tests, integration tests, and required quality gates. A successful build should mean both compilation and essential validation passed. This turns CI into an objective quality signal, not just a packaging step.
- ☐ **Run the fastest tests early**: Detect failures quickly by running short tests first, then longer suites. A common order is unit tests, then integration tests, and finally end-to-end tests. Early failures shorten feedback loops and reduce wasted compute.
- ☐ **Run pipeline steps in parallel**: Use parallel jobs for independent tasks to shorten feedback loops and improve throughput. Group tasks by runtime profile and dependencies so parallelism remains stable. Monitor resource usage and queue pressure to verify real gains.
- ☐ **Use matrix builds deliberately**: Run the same pipeline across the supported Node.js versions, operating systems, or browser targets only where that coverage adds real value.

A focused matrix improves compatibility confidence without multiplying CI time unnecessarily.

- ☐ **Cache dependencies and tool downloads**: Cache package-manager data, browser binaries, and other expensive downloads to reduce run time, but scope cache keys so stale artifacts do not hide problems. Cache strategy should optimize speed without weakening reproducibility.
- ☐ **Cancel superseded pipeline runs**: When a newer commit arrives on the same branch or pull request, cancel older in-flight runs automatically. This prevents wasted CI capacity and keeps feedback focused on the latest code state.
- ☐ **Set explicit timeouts and resource limits**: Define sensible job timeouts, retry policies, and resource boundaries so stuck or runaway jobs do not exhaust shared runners. Controlled limits keep feedback predictable and protect CI capacity.
- ☐ **Keep pipelines as fast as possible**: Slow pipelines reduce delivery velocity and discourage frequent integration. Improve speed by scaling CI resources, optimizing test design, and removing low-value or flaky checks. Track pipeline duration trends and treat regressions as operational incidents.
- ☐ **Stop the pipeline when a step fails**: Enforce fail-fast behavior so broken changes do not progress into downstream stages. This prevents cascading errors and noisy failures that hide root causes.
- ☐ **Fix broken builds immediately**: A failing baseline should be treated as top priority for the team. Delayed fixes block other work and reduce trust in CI feedback. Clear ownership and response expectations help restore pipeline health quickly.
- ☐ **Upload artifacts and diagnostics from relevant runs**: Preserve build outputs, coverage reports, screenshots, traces, and logs so teams can inspect failures without rerunning the pipeline immediately. Good artifacts shorten mean time to diagnose.
- ☐ **Notify the team with actionable failure context**: Send CI failures to the channels the team already uses, including enough context to identify the branch, stage, and failing check quickly. Alerts should accelerate recovery, not create extra investigation work.
- ☐ **Build once and promote the artifact**: Avoid recompiling the same application multiple times inside one pipeline when the same output can be passed to later test or packaging steps. This shortens runs and reduces inconsistencies between stages of the same CI execution.
- ☐ **Track artifact provenance and generate SBOMs**: Attach metadata (commit SHA, build ID, dependency manifest) to published artifacts and generate a Software Bill of Materials.

Provenance and SBOM data improve incident response, compliance, and supply-chain transparency.

- ☐ **Do all substantive work within explicit stages**: Place meaningful pipeline activity inside named stages so responsibilities are clear and observable. A minimum structure is separate Build, Test, and Deploy stages.
- ☐ **Avoid complex scripts in pipelines**: Prefer short, focused scripts over large multi-purpose ones. Simpler scripts are easier to maintain, consume fewer resources, and fail in more diagnosable ways. If scripts grow, split them into reusable tools with tests.
- ☐ **Wrap manual inputs in timeouts**: Add timeouts to approval gates and any step waiting for human action. This prevents stuck pipelines and ensures predictable cleanup when no response arrives. Timeouts should be long enough but finite by default.
- ☐ **Shift security checks left in CI**: Include dependency vulnerability scanning, secret detection, and static analysis as first-class CI gates. Security checks should run on every relevant change so risky code is blocked before release stages.
- ☐ **Isolate and secure the CI/CD environment**: Treat CI/CD environment as critical infrastructure because it has access to source code and deployment credentials. Host it on protected internal networks and restrict access using VPNs, least-privilege policies, and strong identity controls. Security hardening should include regular credential rotation and audit logging.
- ☐ **Monitor pipelines continuously**: Observe pipeline reliability, flaky tests, queue times, and agent health as ongoing operational signals. Define service-level targets and response playbooks for recurring failures. Continuous monitoring prevents hidden bottlenecks from stalling delivery.

5.4 Continuous Delivery

5.4.1 Definition

Continuous Delivery is a software development discipline that ensures software can be released to production at any time. In addition to automated testing, the release process itself is automated, so the application can be deployed whenever needed with a single action.

In practice, Continuous Delivery starts with continuous integration, where the team builds versioned executables and runs automated tests to catch problems early. Those executables are then promoted through increasingly production-like environments so the same artifact can be

validated before it reaches production. This is what makes frequent deployments possible, even when the business chooses to release less often.

Continuous Delivery extends automation from source control through production and organizes that flow into a deployment pipeline. A Continuous Delivery Pipeline is the ordered set of build, test, approval, and deployment steps that code changes must pass through before they are released. Each stage uses software gates to automatically promote or reject versioned artifacts, and if the release protocol is not met the pipeline stops, alerts are raised, and the team is notified.

In SAFe, the Continuous Delivery Pipeline is described through four connected activities: continuous exploration, continuous integration, continuous deployment, and release on demand. Together, they show how organizations can evolve their delivery process incrementally while still focusing on faster feedback and reliable value delivery.

5.4.2 GitHub Workflows

GitHub Actions is a practical choice for release pipelines because the workflow lives next to the source code, uses the same pull request and branch model as the team, and can be versioned and reviewed like any other change. In a release context, a workflow usually runs after a merge to the default branch or when a version tag is created, then performs a predictable sequence: install dependencies, build artifacts, run quality checks, publish deliverables, and finally create or update release metadata.

The following partial workflow for an Angular project performs all the necessary steps to build, and release an Angular library:

```yaml
jobs:
  Pipeline:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: 22

      - name: Install Dependencies
        run: npm ci

      - name: Build Library
        run: npm run build

      - name: Publish Library
        run: npm run publish
```

```
      - name: Add Git Tag
        run: |
           git tag "$(jq -r '.version' package.json)"
           git push origin --tags
```

The first step, after the initial runner setup, runs `npm ci` to install the packages from the lockfile. Once dependencies are ready, `npm run build` compiles the Angular library and produces the distributable artifacts, typically in an output directory such as `dist/`. Publishing and version traceability complete the flow. `npm run publish` sends the built package to the target registry, whether that is npm registry, GitHub Packages, or other artifact repository. After publishing, the job creates and pushes a Git tag based on the package version, which creates a clear link between the released artifact and the exact source revision that produced it.

5.4.3 GoCD

GoCD is an open-source build and release tool that helps teams automate Continuous Delivery. It supports plugin-based extensibility for integrations such as authentication providers, version control systems, build tools, cloud platforms, and notification channels. GoCD has two runtime components, namely, the **GoCD Server** and one or more **GoCD Agents**. The server hosts the UI, pipeline configuration, and scheduling logic, while agents execute build and deployment tasks. In practice, we need at least one connected agent to run any job.

The following example starts GoCD server with persistent storage and exposes both required ports:

```
docker run -d -p 8153:8153 -p 8154:8154 -v gocd-server-data:/godata \
       gocd/gocd-server:v25.4.0
```

Port 8153 serves the web UI (HTTPS by default) and port 8154 is used for server-agent communication. Then, start an agent and point it to the server URL on port 8154:

```
docker run -d -e GO_SERVER_URL=https://host.docker.internal:8154/go \
       gocd/gocd-agent-alpine:v25.4.0
```

After startup, open `https://localhost:8153/go` to access the UI. On first run, retrieve the admin password from the server container logs. New agents must be enabled in the GoCD UI (or auto-registered with keys) before they can pick up jobs. Then, we can create delivery pipelines. More details are available in the official documentation and on the GoCD blog.

5.4.4 Delivery Artifacts

The delivery artifact is the packaged output we promote, publish, or deploy after the build is complete. The exact form depends on what we are shipping, for example, a reusable library, a browser application, a micro-frontend, a mobile or desktop wrapper, or an infrastructure-ready deployable for enterprise platforms. Choosing the right artifact format is important because it affects versioning, distribution, deployment automation, rollback strategy, and how other teams consume it.

The most common delivery possibilities for Angular are:

- **Angular library package**: A versioned package published to an artifact repository so other applications can consume it as a dependency. Examples include npm registry, GitHub Packages, Nexus and others.
- **Static web assets**: A compiled browser application distributed as HTML, CSS, JavaScript, and media files. Examples include the contents of the `dist/` folder deployed to GitHub Pages, Firebase Hosting, or a CDN-backed object store.
- **Container image**: A deployable image that packages the built Angular application together with a web server or runtime configuration. Examples include Docker images based on Nginx, or a hardened enterprise base image.
- **Micro-frontend artifact**: A frontend unit packaged to be composed into a larger platform at runtime or build time. Examples include a Module Federation remote entry, a Web Components bundle, or a single-spa micro-frontend published to a CDN or artifact repository.
- **Progressive Web App package**: A browser application prepared for offline installation and distribution, using service workers and web application manifests. Examples include Angular PWAs distributed as static assets through a CDN or app-like deployments with runtime caching and offline support.
- **Mobile or desktop wrapper**: An Angular application packaged for native-like distribution through platform-specific wrappers. Examples include Android and iOS apps built with Capacitor, Ionic, and desktop applications packaged with Electron.
- **Compressed release bundle**: A portable archive generated for controlled handoff, manual deployment, or legacy enterprise environments. Examples include ZIP or TAR archives containing the built application, environment templates, deployment scripts, and release notes.
- **Kubernetes or platform deployment bundle**: A release package that combines the Angular artifact with the deployment descriptors required by the target platform.

Examples include a Helm chart referencing an Angular container image, Kubernetes manifests, OpenShift templates, or GitOps-ready deployment repositories.

Additional deployment guidance is available in the official Angular deployment documentation.

5.4.5 Artifact Repository

5.4.5.1 NPM Registry

npm is the default package manager for Node.js and the standard client used by Angular projects to consume and publish JavaScript packages. It is recommended to install Node.js through nvm so that teams can standardize and switch Node versions easily.

In enterprise environments, *npm* is usually not pointed directly at the public registry. Instead, it is configured to use an internal artifact registry, which becomes the controlled entry point for downloading open-source packages and publishing private packages. Using *npm* together with an artifact registry gives teams a central place to proxy public dependencies, host internal packages, cache downloads, and apply access control. This improves reproducibility in local development and CI, reduces external network dependency, and lets teams treat frontend packages like any other governed build artifact. *npm* is configured to use registry.npmjs.org by default. We can configure *npm* to use any compatible registry we like, and even run a private registry.

We can configure the registry used by *npm* in the *.npmrc* file or using next command:

```
npm config set registry <REPOSITORY_URL>
```

If the registry requires authentication, *npm* must also be configured with credentials or tokens appropriate for the artifact repository.

The same registry flow also supports publishing. When a team produces a shared Angular library or internal package, *npm* can publish it to the hosted *npm* repository using *npm publish*. Versioning should follow semantic versioning so that downstream applications can adopt updates predictably.

Finally, *npm* remains useful for dependency governance when used with an artifact registry. *npm outdated* helps identify newer package versions available through the configured registry, while *npm audit* helps detect vulnerable dependencies. Together, these commands support the artifact lifecycle from consumption and caching to publishing, versioning, and security review.

5.4.5.2 GitHub Packages

GitHub Packages can host *npm* packages directly in GitHub, making it a practical artifact registry when source code and CI/CD already live there. For enterprise projects, this is especially useful for publishing shared libraries, design systems, and internal UI components under the same access model used for repositories.

To publish and consume packages, use a scoped package name (for example, `@my-org/ui-library`) and configure *npm* to route that scope to GitHub Packages:

```
@my-org:registry=https://npm.pkg.github.com
//npm.pkg.github.com/:_authToken=${NPM_TOKEN}
always-auth=true
```

This configuration can live in a project-level *.npmrc* (recommended for consistency) or in the user-level *.npmrc*. For libraries, the *package.json* metadata should include a valid scoped name and, optionally, explicit publish settings:

```
{
    "name": "@my-org/ui-library",
    "version": "1.0.0",
    "publishConfig": {
        "registry": "https://npm.pkg.github.com"
    }
}
```

After building the library output (for example with `ng build my-lib`), publish from the generated package folder using `npm publish`. To consume the package in another Angular project, keep the same scope mapping in *.npmrc* and install it normally with `npm i @my-org/ui-library`.

With this setup, GitHub Packages becomes the organization artifact registry for Angular packages, while *npm* remains the standard client for install, publish, versioning, and audit workflows.

5.4.5.3 Alternative tools

The two most popular package managers in the JavaScript ecosystem are npm, which is an acronym for Node Package Manager, and Yarn, an alternative created by Facebook. However, we have more alternatives such as pnpm, a fast, disk-space-efficient package manager. For artifact repositories, GitHub Packages is a good option for code repositories stored in GitHub. The best alternatives are Nexus and Artifactory. Alternatively, for JavaScript projects, we can use Verdaccio, a lightweight open-source private *npm* proxy registry.

5.4.6 Feature Flags

Feature Flags (also called feature toggles) allow teams to decouple deployment from release by shipping code to production in a disabled state and enabling it later for selected users, tenants, regions, or traffic percentages. This reduces rollout risk and supports progressive delivery practices such as canary releases and A/B testing. In Continuous Delivery, flags are especially useful when a large feature spans multiple pull requests but should only become visible when all parts are ready.

In Angular applications, teams usually implement flags through a shared service that reads runtime configuration (for example from `/assets/feature-flags.json` or an API), enabling route guards, UI toggles, and service-level switching without rebuilds. This model also supports targeting via commercial feature flags platforms. A simpler alternative is storing flags in `environment.ts` and `environment.prod.ts` files, but those values are fixed at build time, so any flag change requires a new build and deployment rather than an immediate runtime rollout or rollback.

Every flag should have a removal plan once fully rolled out, and automated checks should prevent stale flags from accumulating technical debt. It is also recommended to validate both states in tests. Additional guidance is available in the Angular deployment documentation, Martin Fowler's Feature Toggles, and the OpenFeature specification for vendor-neutral flag evaluation patterns.

5.4.7 Best Practices

The following practices summarize the key principles for safe, reliable, and scalable delivery pipelines:

- ☐ **Automate build and deployment steps**: Automate compilation, quality checks, packaging, and environment promotion wherever practical. Automation reduces manual variance and increases release repeatability. Teams can still use approval gates when business or compliance constraints require them.
- ☐ **Build once and promote the same artifact**: Compile/package code only once in a clean environment and reuse that exact artifact across all stages. Rebuilding in later environments can inject subtle differences from tooling, dependencies, or runtime context. Artifact promotion improves confidence that production receives what was already validated.

- ☐ **Decouple deployments from releases**: Deploying code and exposing features to users should be treated as separate decisions. Use mechanisms such as feature toggles, blue-green deployments, canary releases, and A/B testing to control user impact. This approach enables safer, progressive rollout strategies.
- ☐ **Release early and release often**: Frequent releases create tighter feedback loops and reduce the blast radius of each change. Smaller releases are easier to test, diagnose, and roll back if needed. This cadence improves team learning and end-user value delivery.
- ☐ **Make the pipeline the only way to deploy**: Every change should go through the same tested and governed pipeline path. Bypassing the pipeline removes quality gates, traceability, and policy enforcement when risk is highest. Even urgent fixes should follow a controlled fast-track workflow.
- ☐ **Smoke-test every deployment**: Run a fast, high-value set of end-to-end checks immediately after deployment to confirm critical user journeys. Smoke tests should be broad enough to detect obvious outages but short enough to preserve delivery speed. Regular smoke execution can also detect runtime drift after initial rollout.
- ☐ **Deploy the same way to all environments**: Use one automated deployment mechanism for development, staging, and production environment to avoid procedural drift. Consistent deployment mechanics reduce surprises during promotion. Differences between environments should be intentional, documented, and minimal.
- ☐ **Validate in a production-like environment**: Use a pre-production environment that closely mirrors production topology, runtime versions, and key dependencies. This final validation step catches mismatches before customer impact. Production-like testing also improves confidence in performance and scalability behavior.
- ☐ **Monitor production post-deployment**: Treat deployment as the start of the monitoring phase, not the end of the delivery cycle. Emit and observe alerts for errors, latency spikes, and behavioral deviations after each release. Feed observability signals back into the delivery pipeline to improve coverage and risk assessment for future releases.
- ☐ **Use backward-compatible database changes**: Plan schema migrations so old and new application versions can run safely during rollout windows. Prefer expand-and-contract migration patterns, and separate destructive cleanup into later releases. This reduces outage risk during phased deployments and rollbacks.
- ☐ **Externalize environment configuration and secrets**: Keep environment-specific values outside the build artifact and inject them at deploy/runtime through managed

configuration and secret stores. This avoids rebuilding artifacts per environment and improves security posture.

- ☐ **Enable instant propagation across stages**: Successful completion of one stage should trigger the next stage automatically, without unnecessary manual handoffs. This keeps feedback tight and prevents avoidable queue delays. Trigger rules must remain deterministic and observable.
- ☐ **Use progressive rollout guardrails with automated rollback**: Define objective health thresholds (error rate, latency, saturation, business KPIs) for canary or phased rollouts. If thresholds are breached, the pipeline should automatically pause or rollback. Automated guardrails limit blast radius and speed incident response.
- ☐ **Stop the pipeline on failure**: When a stage fails, halt promotion immediately to prevent cascading issues downstream. Investigate and fix the first meaningful failure before resuming. Fail-fast behavior protects production and preserves debugging clarity.
- ☐ **Prepare emergency deployments and rollbacks**: Define rollback and hotfix procedures in advance, and rehearse them periodically. Recovery plans should include who acts, what is reverted, and how validation happens after rollback. Fast, predictable recovery is a core delivery capability.
- ☐ **Use version control for rollbacks**: Rollbacks should be traceable and executable through versioned code, manifests, and artifacts. This avoids ad hoc recovery actions that are hard to audit or reproduce. Versioned rollback paths reduce downtime and release uncertainty.
- ☐ **Keep deployment actions idempotent and immutable**: Deployment steps should be safe to re-run without creating inconsistent state, and runtime artifacts should not be modified in place. Idempotent, immutable delivery behavior improves reliability in retries and recovery scenarios.
- ☐ **Support one-click or low-friction promotions**: Promote validated artifacts between environments with a single controlled action whenever possible. Reducing operational friction lowers manual error risk and shortens lead time. One-click promotion should still preserve approvals, logging, and auditability.
- ☐ **Publish artifacts to a governed repository**: Publish versioned build outputs (npm packages, container images, compiled assets) to a controlled artifact registry such as npm registry, GitHub Packages, Nexus, or Artifactory. A central registry improves traceability, access control, reproducibility, and dependency governance across teams.

- ☐ **Apply semantic versioning for releases**: Version packages and release artifacts following semantic versioning conventions so downstream consumers can adopt updates predictably. Link each release version to a Git tag and build metadata to create a clear and auditable chain between the artifact and the source revision that produced it.
- ☐ **Manage feature flag lifecycle**: Every feature flag should have a defined removal plan once it is fully rolled out, and both the enabled and disabled states should be validated in automated tests. Use runtime-configurable flags rather than build-time flags to allow immediate rollout or rollback without a new build.
- ☐ **Track delivery outcomes and improve continuously**: Measure lead time, deployment frequency, change failure rate, and mean time to recovery to identify pipeline bottlenecks and reliability gaps. Use these metrics to drive small, frequent improvements rather than periodic large rewrites.

5.5 CI vs CD vs CD

5.5.1 Overview

A pipeline is a sequence of processes such as procedures or commands that are executed in series, one after the other. Sometimes the output of one step serves as the input for the next. First, code changes made locally are committed and pushed to a central repository. Next, an automated system checks out the code and builds the project into artifacts that browsers can understand. Tests are then run to verify that recent changes have not broken anything. Finally, the build is deployed to its target environment, where end users can access it. These steps form the core of CI/CD and together make up the software delivery pipeline.

Continuous Integration is a practice where developers frequently merge code changes into a shared repository. Whenever we commit and push changes to the repository, the CI system rebuilds the branch and runs all related test cases to verify that the existing application has not been broken. **Continuous Delivery** is an extension of Continuous Integration that ensures new changes can be released to customers quickly and sustainably. This means that, in addition to automated build and test, the release process itself is automated, allowing the application to be deployed at any point in time by clicking a button. In theory, with Continuous Delivery, teams can choose to release daily, weekly, fortnightly, or on whatever cadence suits their business requirements. **Continuous Deployment** takes this a step further by building on top of Continuous Delivery. With this practice, every change that passes all stages of the production pipeline is

automatically released to customers. There is no human intervention required. Only a failed test will prevent a new change from being deployed to production.

To visualize the whole process and the core differences between them, we can look at the following flow diagram.

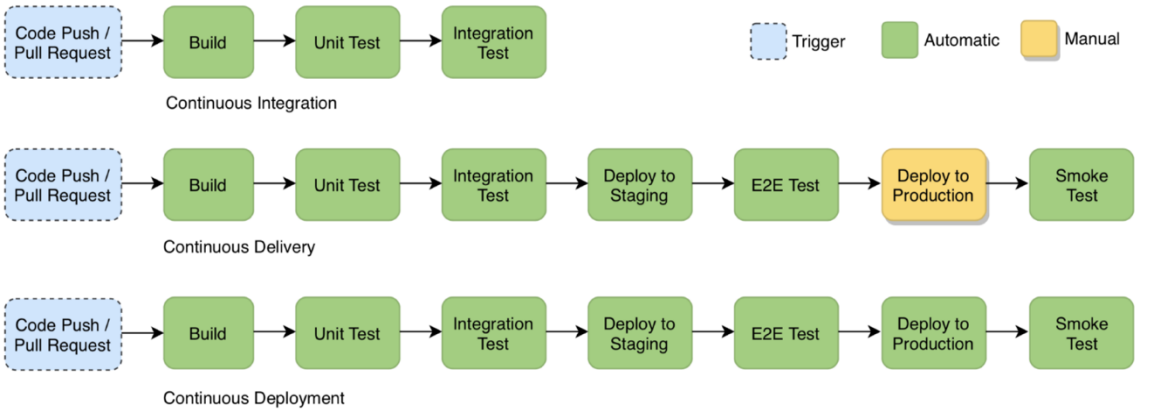


Fig. 2: Continuous Integration vs Continuous Delivery vs Continuous Deployment

To put it simply, Continuous Integration is a part of both Continuous Delivery and Continuous Deployment. Continuous Deployment is like Continuous Delivery, except that releases to the customers happen automatically without manual approval.

5.5.2 Continuous Workflow

Continuous software development methodologies are based on automating the execution of scripts to minimise the chance of introducing errors during development. They require little human intervention (or none at all) from the moment new code is written through to its deployment. This involves continuously building, testing, and deploying code changes in small iterations, reducing the risk of building on top of buggy or failed previous versions.

Consider a scenario where a code change has been discussed in an issue and implemented locally. Once commits are pushed to a feature branch in a remote repository, the CI/CD pipeline configured for the project is triggered. The pipeline then runs automated scripts (sequentially or in parallel) to build and test the application. Once the implementation has been reviewed and approved, the feature branch is merged into the default branch. The changes are then deployed to a production environment, either manually or automatically. If something goes wrong, the team can roll back the changes with ease. The following image illustrates this workflow.

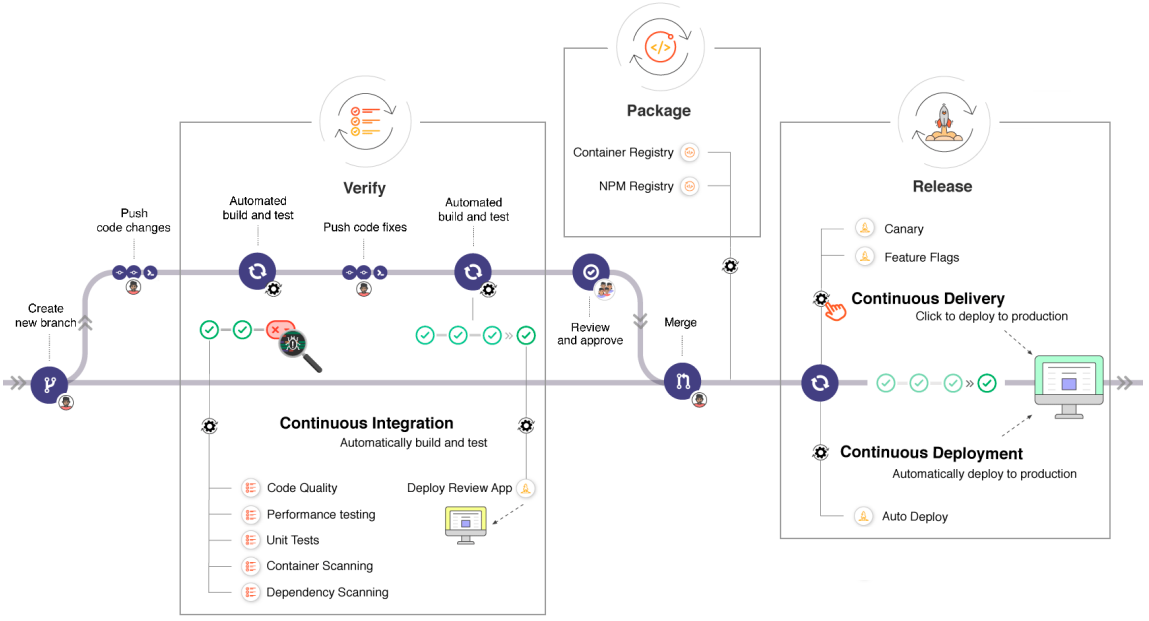


Fig. 3: CI/CD standard workflow

Looking more closely at the workflow, we can identify each stage of the DevOps lifecycle. In the *Verify* stage, Continuous Integration is used to automatically build and test the application, analyse source code quality, run a series of tests, and, if necessary, deploy the changes in a preview environment. In the *Package* stage, Docker images and *npm* packages are stored in an artifact repository. Finally, in the *Release* stage, Continuous Delivery requires a manual trigger to deploy the application to production, whereas Continuous Deployment releases it automatically.

5.5.3 Application Deployment

When we are ready to deploy an Angular application to a remote server, we have various options for deployment:

- **Angular CLI + static hosting:** Run *ng build* (or *ng build --configuration production*) and publish files from *dist/* to a static host such as GitHub Pages or Firebase Hosting. This is the fastest path for small apps.
- **Managed CLI deployment:** use *ng add [package]* to install a deployment builder and run *ng deploy*. This is useful when a platform provides an official or community Angular deploy integration and we want repeatable one-command releases.

- **CI/CD-driven deployment:** Build once in CI, store versioned artifacts, and promote the same build through dev/staging/production with environment-specific configuration and approval gates.
- **Advanced platform setups:** package the app in Docker, serve via HTTP server/CDN, and release with strategies such as rolling, blue-green, or canary. At larger scale, teams manage infrastructure as code (for example with Ansible or Terraform) and often adopt GitOps-based delivery.

Once the application is deployed to production, the monitoring phase begins. Alerts should be raised whenever deviations or errors are detected.

5.5.4 Deployment Strategies

5.5.4.1 "Big Bang" Deployment

The big bang deployment updates all or large parts of an application in a single release. This strategy dates back to the days when software was distributed on physical media and installed by the customer. Big bang deployments required extensive development and testing before release and were often associated with the "waterfall model" of large, sequential releases. Modern applications benefit from regular and often automated updates on both the client side and the server side. As a result, the big bang approach is slower and less agile for modern teams. It is generally a poor fit for modern applications because the risks are too high for public-facing or business-critical systems, where outages can result in major financial loss. Rollbacks are often costly, time-consuming, or even impossible.

5.5.4.2 Rolling Deployment

In a rolling deployment, a new application version gradually replaces the old one. In practice, this means slowly replacing the currently running instances with updated ones over a period of time. During that period, the old and new versions coexist without disrupting functionality or the user experience. This approach makes it easier to roll back components that prove incompatible with the rest of the system. An application suite upgrade is a typical example of a rolling deployment. If the original applications are deployed in containers, the upgrade can be performed one container at a time. Each container is updated to download the latest image from the vendor. If a compatibility issue appears in one of the applications, the container can be recreated from the older image. In this way, the old and new versions of the suite coexist until every application has been upgraded.

5.5.4.3 Blue-Green Deployment

A blue-green deployment requires running two production environments at the same time. The goal is to keep the environments as close to identical as possible, while ensuring that only one of them is live at any given moment. This allows us to switch traffic between environments safely. For the sake of this discussion, the live environment is blue and the staging environment is green. Once testing of the application updates in the green environment is successful, green becomes the live environment and blue becomes the new staging environment, ready for the next iteration of changes. If something goes wrong soon after green goes live, we can easily roll back to blue because we know it was production-ready before the switch.

5.5.4.4 Canary Deployment

Canary deployment is similar to blue-green deployment, but it is even more risk-averse. Instead of switching all traffic from blue to green in a single step, it uses a phased rollout. With canary deployment, new application code is first deployed to a small part of the production infrastructure. Once the release is approved, only a small percentage of users are routed to the new version, which minimizes potential impact. As confidence in the new version grows, it can be released to more servers and exposed to more users. If no errors are reported, the new version can gradually roll out across the rest of the infrastructure. One of the main advantages of a canary release is that it allows teams to perform capacity testing in a real production environment while retaining a safe rollback strategy if issues are found.

5.6 Continuous Automation

5.6.1 Definition

Continuous Automation is the practice of automating the full application lifecycle so software and operational changes can be delivered quickly, consistently, and safely. It brings together infrastructure, application, security, and compliance automation, treating those elements as code so teams can version them, test them under different conditions, detect and correct issues earlier, and apply change reliably at scale. In practice, it extends automation beyond build and deployment into the ongoing operation and governance of software systems.

5.6.2 GitOps

GitOps is an approach to implementing Continuous Deployment for cloud-native applications. It emphasizes a developer-centric way of operating infrastructure by using tools developers are already familiar with, including Git and Continuous Deployment platforms.

The core idea of GitOps is to maintain a Git repository that always contains declarative descriptions of the infrastructure desired in the production environment, together with an automated process that makes the actual production environment match that declared state. To deploy a new application or update an existing one, we only need to update the repository, and the automated process handles everything else.

GitOps organizes the deployment process around code repositories as the central source of truth. There are at least two repositories, namely, the application repository and the environment configuration repository. The application repository contains the application source code and the deployment manifests used to deploy it. The environment configuration repository contains all deployment manifests that describe the desired state of a given environment. It defines which applications and infrastructure services, such as a message broker, service mesh, or monitoring tool, should run there, along with their required configuration and versions.

There are two ways to implement GitOps deployment, namely, Push-based Deployments and Pull-based Deployments. The difference between them lies in how the deployment environment is kept aligned with the desired infrastructure state. When possible, the pull-based approach should be preferred, because it is generally considered the more secure and better practice for implementing GitOps.

5.6.3 GitHub Workflows

GitHub Actions can implement GitOps using a push-based deployment model. In this approach, the CI/CD workflow runs outside the cluster, usually after a merge to the default branch or after a version tag is created. The workflow builds the Angular application, publishes the immutable delivery artifact (for example a Docker image), updates the Kubernetes manifests or Helm values stored in a Git repository, and then pushes those manifest changes toward the target environment. In other words, Git still stores the desired state, but the actual deployment action is initiated by the pipeline runner rather than by an in-cluster controller.

This model works well when the team wants GitHub to orchestrate the full release flow and the infrastructure footprint is still modest. It becomes especially effective when combined with protected branches, required reviews, signed commits, environment-specific folders, and

reusable workflows. Additional guidance is available in GitHub Actions documentation, GitOps principles, and in OpenGitOps principles page.

5.6.4 Argo CD

Argo CD is the most common tool for pull-based deployments in Kubernetes. In the pull-based deployment model, the deployment agent runs inside or near the cluster, continuously watches the Git repository that describes the desired state, compares that desired state with the live cluster state, and reconciles any difference. Instead of a CI server pushing manifests into the cluster, the cluster pulls and applies the declared changes itself.

In practice, Argo CD improves security and operability because the reconciliation loop lives within the platform boundary. The CI system only needs permission to publish artifacts and update Git repositories, while Argo CD is the only component applying manifests to the cluster. This separation keeps Git as the auditable source of truth, makes configuration drift visible, and allows teams to push the same manifest change from the development environment to the staging environment and then to the production environment through common pull requests. Additional guidance is available in Argo CD documentation, CNCF Argo project, and in OpenGitOps principles page.

5.6.5 Best Practices

The following practices focus on safe, scalable automation across infrastructure and application delivery:

- ☐ **Implement GitOps approach**: GitOps uses Git as the source of truth for declarative application and infrastructure state. Delivery changes are promoted by pull requests, giving teams an auditable trail and reviewable rollout intent. This model reduces configuration drift across environments.
- ☐ **Prefer pull-based deployments**: When possible, use in-cluster controllers such as Argo CD to continuously reconcile live state to Git. Pull-based reconciliation narrows the CI system blast radius and keeps deployment permissions closer to the platform boundary. This approach usually improves both security and operational reliability.
- ☐ **Always have backups for data**: Backups do not prevent failures, but they make recovery possible after data loss, corruption, or operator mistakes. Define backup frequency by business criticality and recovery objectives. Restoration drills are as important as backup creation.

- ☐ **Use version control for everything**: Source code, configuration, infrastructure definitions, scripts, and database changes should all be version controlled. This is the foundation of *X as code* practices, such as Infrastructure as Code and Pipeline as Code. If a change is not in version control, it is outside the automation system.
- ☐ **Protect repositories and branches**: Require pull requests, mandatory reviewers, status checks, and branch protection on repositories that define pipelines and environment manifests. Use signed commits where feasible for stronger provenance. Guardrails at the repository layer prevent unsafe or unreviewed production changes.
- ☐ **Use Infrastructure as Code**: Infrastructure as Code provisions and manages environments through code and software engineering practices. IaC removes snowflake environments and improves repeatability across teams and stages. It also enables safer reviews, testing, and rollback of infrastructure changes.
- ☐ **Pin immutable artifacts and deployment inputs**: Deploy container images by digest and pin critical action/plugin versions in automated workflows. Immutable references improve reproducibility and reduce accidental drift from floating tags. This makes rollbacks and incident analysis far more deterministic.
- ☐ **Automate policy and compliance as code**: Encode security, compliance, and operational guardrails as machine-evaluated policies in the pipeline. Policy checks should gate changes before they reach sensitive environments. This keeps governance consistent and auditable at scale.
- ☐ **Test infrastructure code in non-production first**: Validate IaC changes in controlled environments (dev/test/staging) before production rollout. Use the same pipeline path so production behavior is predictable. Staged validation reduces downtime risk and catches configuration errors early.
- ☐ **Peer-review infrastructure and automation code**: Infrastructure code can be as critical as application code and deserves the same review rigor. Reviews should cover security, resilience, cost impact, and rollback feasibility. This practice reduces hidden risk in automation logic.
- ☐ **Take a security-first approach**: Treat CI/CD and automation platforms as high-value targets with strict access controls. Isolate systems, enforce least privilege, and require strong authentication and auditability. Security controls should be built into automation, not added only after incidents.
- ☐ **Automate secret lifecycle management**: Store secrets in dedicated secret managers, rotate them regularly, and avoid hard-coded credentials in repositories or pipeline

definitions. Access should be short-lived, least-privilege, and fully audited. Automated rotation reduces long-term credential exposure.

- ☐ **Run automation from limited accounts**: Execute playbooks and scripts with low-privilege accounts by default, escalating only for specific operations that require it. This follows least-privilege principles and reduces catastrophic change risk. Privileged actions should be explicit, auditable, and time-bounded.
- ☐ **Use scoped, just-in-time privilege escalation**: When elevated access is required, grant only the minimum scope and duration needed for that task. Avoid standing administrative access and make all privileged actions explicitly approved and auditable. Time-bounded elevation reduces lateral movement and blast radius.
- ☐ **Roll out infrastructure changes in stages**: Avoid running the same high-impact change on all machines simultaneously. Progressive rollout helps detect issues under real load before full deployment. Stage-based automation enables safe pause, rollback, and verification points.
- ☐ **Require plan, drift detection, and controlled apply**: For infrastructure changes, generate and review execution plans before apply, detect drift continuously, and fail fast when live state diverges unexpectedly. Plan/apply discipline prevents accidental changes and improves operational predictability.
- ☐ **Separate build and deployment responsibilities**: CI systems should build, test, and publish artifacts, while deployment controllers reconcile environments from Git. This separation simplifies permissions and creates cleaner audit boundaries between software production and software release. It also reduces the chance of direct cluster mutation from ad-hoc pipelines.
- ☐ **Adopt Chaos Engineering principles**: Chaos Engineering helps validate resilience by testing controlled failure scenarios. Start only after incident response, observability, and baseline stability are mature enough. Experiments should be scoped, reversible, and measurable.
- ☐ **Use temporary or ephemeral environments**: Create short-lived environments for automated verification and destroy them after use. Ephemeral environments reduce drift, improve reproducibility, and limit long-lived configuration debt. They are especially useful for pull request validation.
- ☐ **Implement continuous environment testing**: Continuously validate that provisioned environments remain healthy, secure, and policy-compliant over time. Environment

testing should include connectivity, secrets handling, network policies, and service readiness. This turns environment quality into a continuously checked signal.

- ☐ **Design automation to be idempotent**: Idempotent is a property of certain operations or functions meaning they can be applied multiple times without changing the result beyond the initial application. Automation steps should be safe to rerun after partial failures without corrupting state or duplicating resources.
- ☐ **Gate promotions with health checks and automated rollback**: Promote changes between environments only after objective health checks pass (readiness, error rate, latency, dependency checks). Define automatic rollback criteria for failed rollouts. Promotion gates turn reliability signals into enforceable release policy.
- ☐ **Release often with small changes**: Frequent, smaller releases reduce blast radius and improve learning speed. They depend on strong automation, production-like validation, and reliable rollback. Smaller batches are easier to diagnose and recover when problems occur.
- ☐ **Practice redundancy for critical services**: Redundant components and failover paths increase availability when individual systems fail. Redundancy can include multiple instances, zones, regions, or data centers based on criticality. Automation should test failover behavior, not just nominal paths.
- ☐ **Implement application monitoring by default**: Monitoring should provide actionable visibility into service health, performance, and dependencies. Combine metrics, logs, and traces so incidents can be detected and diagnosed quickly. Instrumentation must evolve with system changes.
- ☐ **Review metrics regularly**: Alert thresholds and dashboards should be reviewed as traffic and architecture evolve. Too many false alerts create fatigue, while weak thresholds miss real incidents. Regular metric reviews keep observability aligned with operational reality.
- ☐ **Maintain an effective alerting system**: Alerts should route the right signal to the right responders with clear severity and escalation paths. Fast, accurate alerting reduces mean time to detect and recover. Alert rules should prioritize actionable events over noisy telemetry.
- ☐ **Secure automation scripts**: Standardize strict shell/script modes, input validation, and pre-flight checks for destructive operations. Require explicit safeguards for high-risk actions and fail fast on invalid state. Script-level guardrails prevent avoidable outages caused by operator or variable errors.

- ☐ **Have project documentation**: Keep automation code, naming conventions, and execution workflows well documented to support maintenance and onboarding. Good documentation reduces tribal knowledge and helps teams scale testing practices consistently.

5.7 Final Remarks

Throughout this chapter, we explored how DevOps teams can move from isolated tasks to a complete automated delivery pipeline. We covered Continuous Testing, Continuous Integration, Continuous Delivery, Continuous Deployment, and Continuous Automation, always with the goal of making releases faster, safer, and more repeatable. We examined practical tooling and workflow choices for modern Angular projects, including GitHub Actions, Jenkins, and GoCD for pipeline orchestration, *npm* and GitHub Packages for artifact distribution, and infrastructure-oriented practices that support dependable delivery. We also reviewed notification patterns, production-like staging, build-once-and-promote approaches, and rollout models such as rolling, blue-green, and canary deployments.

The main takeaway is that delivery quality comes from consistency: automate routine checks, keep artifacts and environments predictable, and make each stage of the pipeline provide fast and actionable feedback. When teams combine strong testing discipline with reliable automation and a clear deployment strategy, they can release Angular applications with much greater confidence.

6 Web Infrastructure, Operations and Security

6.1 Overview

Web infrastructure is the foundation that supports websites and web applications on the internet. It consists of servers, networks, databases, software, and security systems that work together to store, process, and deliver web content to users. In essence, web infrastructure provides the foundation that enables websites and web applications to operate efficiently, securely, and continuously for users around the world. Modern web infrastructure often relies on cloud computing, content delivery networks (CDNs), load balancers, monitoring tools, and cybersecurity measures to ensure reliability, scalability, performance, and protection against threats.

For Angular applications, infrastructure provisioning means preparing and configuring the resources required to host and run the application reliably. This includes setting up web servers, networking, and security controls so the application can be deployed safely and reached by users. At the same time, observability gives teams the ability to monitor, measure, and understand application behavior in real time. Also, security must be treated as an ongoing engineering practice, not a final checklist item.

This chapter focuses on the infrastructure decisions that matter most for DevOps teams, namely, hosting, traffic routing, observability, and operational security. It also introduces practical tools such as Docker, Terraform, Prometheus, Alertmanager, Grafana, Jaeger, and others.

6.2 Cloud Providers

Cloud providers deliver on-demand infrastructure and platform services over the internet. Instead of buying and operating physical servers, networking equipment, storage arrays, and data-center facilities directly, teams rent those capabilities from providers such as Amazon Web Services, Google Cloud, and Microsoft Azure. These platforms expose building blocks such as virtual machines, object storage, managed databases, load balancers, DNS, CDN services, observability tooling, and identity management through web consoles, APIs, and infrastructure-as-code workflows.

Cloud strategy also depends on the deployment model. In a **public cloud**, infrastructure is owned and operated by a third-party provider and shared across customers through tenant isolation. Public cloud is the default model for AWS, Azure, and Google Cloud and is usually the fastest path for hosting Angular applications. In a **private cloud**, cloud-style infrastructure is dedicated to a single organization, either in its own data center or in an isolated hosted environment, which can be useful for strict compliance, data residency, or hardware control requirements. In a **hybrid cloud**, an organization combines private infrastructure with public cloud services, for example by serving the Angular frontend from a public CDN while backend systems or sensitive databases remain in a private environment. Hybrid models are common during migration programs and in regulated industries where teams need cloud elasticity without moving every workload into a public platform.

For Angular and other web applications, cloud providers are commonly used to host static assets, run containerized workloads, terminate TLS, distribute traffic globally, and connect the frontend to managed backend services. A simple deployment might upload the build output to object storage and place a CDN in front of it, while a more advanced setup can run the application in containers, integrate with WAF and DDoS protection, and stream logs and metrics into a managed monitoring stack. The main advantage of cloud providers is that teams can provision capacity quickly, automate deployments, and scale traffic handling without owning the underlying hardware.

6.3 Cloud Hosting

6.3.1 Static Files Hosting

Serving an Angular application as static files is the simplest and most cost-effective hosting model available. The Angular `ng build` command produces a folder of HTML, JavaScript, CSS, and assets with no server-side runtime dependency. Those files can be stored in an object storage

service and distributed through a CDN without managing any compute infrastructure. A recommended caching strategy for Angular deployments is to apply a long `Cache-Control: max-age` to versioned assets with hashed filenames while keeping the `index.html` entry point on `Cache-Control: no-cache` so that new deployments are picked up immediately.

Amazon CloudFront

On AWS, static hosting combines Amazon S3 for object storage with Amazon CloudFront as the global CDN layer. The typical setup uploads the Angular build output to an S3 bucket, associates that bucket as a CloudFront origin, configures custom error responses so that 403 and 404 responses from S3 are rewritten to `index.html` with an HTTP 200 status for SPA routing, and attaches an SSL/TLS certificate from AWS Certificate Manager to serve the application over HTTPS on a custom domain. Traffic reaches the nearest CloudFront edge location, where cached responses are served directly to users.

Azure Static Web Apps

On Azure, Azure Static Web Apps is a fully managed service that combines static asset hosting with a globally distributed CDN, SSL certificates, and custom domain configuration. It includes a built-in GitHub Actions or Azure DevOps integration that deploys the Angular build output on every push. A `staticwebapp.config.json` file at the project root controls routing rules, response headers, fallback routes for SPA navigation, and authentication settings.

Firebase Hosting

On Google Cloud, Firebase Hosting is a fully managed static hosting service that serves content over a global CDN, provides SSL certificates, supports custom domains, and includes atomic deploys with one-click rollback. For Angular applications, Firebase Hosting integrates directly with the Angular CLI through the `@angular/fire` library, allowing the team to build and deploy from a single command after the initial project configuration.

6.3.2 Serverless Containers

Serverless containers extend the container model by removing the need to provision or manage the underlying host machines. The application is still packaged as a Docker image, but the infrastructure layer that runs it is abstracted away entirely. Teams define how much virtual CPU and memory each container needs, and the platform handles placement, scaling, networking, and patching transparently. This approach suits Angular applications served by Nginx or another

static web server inside a container when teams want more control over the runtime than static hosting provides but without the operational overhead of managing clusters or virtual machines.

Amazon ECS with Fargate

On AWS, Amazon Elastic Container Service (ECS) combined with AWS Fargate provides a fully serverless container runtime. Teams define a task definition specifying the container image, resource requirements, port mappings, and environment variables, and Fargate provisions the exact compute requested without exposing any underlying EC2 instances. An ECS service keeps the desired number of task replicas running and integrates with an Application Load Balancer to distribute traffic, while Auto Scaling policies adjust the task count based on CPU or memory usage. The container image is typically stored in Amazon ECR (Elastic Container Registry), and is built and pushed from a CI/CD pipeline.

Azure Container Apps

On Azure, Azure Container Apps is the common serverless container platform for production workloads. It provides a fully managed environment with built-in autoscaling from zero to many replicas, traffic splitting between revisions for canary deployments, and health probes. Teams push the container image to Azure Container Registry, define the container app configuration with resource limits and ingress settings, and the platform handles TLS termination, load balancing, and scaling transparently.

Google Cloud Run

On Google Cloud, Cloud Run is a fully managed serverless container platform that scales automatically from zero to thousands of instances based on incoming traffic. Teams push the container image to Artifact Registry and deploy through the `gcloud` CLI or Cloud Build CI/CD platform. Cloud Run integrates with Cloud Load Balancing for custom domains and global traffic management, Cloud CDN for edge caching of static assets, and Cloud IAM for access control.

6.3.3 Kubernetes Cluster

Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and operational management of containerized workloads across a cluster of nodes. Running Angular applications on Kubernetes makes sense when teams need fine-grained control over routing, rolling updates, resource quotas, health probes, multi-environment configuration, or want to co-locate the frontend with backend microservices in a unified platform. Each major cloud provider offers a managed Kubernetes service that removes the responsibility of operating

the control plane while teams manage workloads through standard Kubernetes primitives such as Deployments, Services, Ingress resources, and horizontal pod autoscaling.

Amazon Elastic Kubernetes Service

On AWS, Amazon Elastic Kubernetes Service (EKS) provides a managed Kubernetes control plane integrated with IAM for access control, Amazon ECR for image storage, and the AWS Load Balancer Controller for ALB-backed Ingress resources. EKS also supports Fargate profiles for running pods serverlessly without managing node groups. The deployment manifests are standard Kubernetes YAML. Only cloud-specific annotations for the Ingress controller and the image registry URL differ from other providers.

Azure Kubernetes Service

On Azure, Azure Kubernetes Service (AKS) is the managed Kubernetes offering. Azure handles control plane provisioning, upgrades, and health monitoring, while teams manage only the worker node pools. Container images are stored in Azure Container Registry, and the cluster integrates with Azure Application Gateway for traffic routing, Azure Monitor for observability, and Azure RBAC for access control.

Google Kubernetes Engine

On Google Cloud, Google Kubernetes Engine (GKE) is the managed Kubernetes service. GKE offers two modes: **Standard**, where teams manage node pools and have full control over cluster configuration, and **Autopilot**, where Google manages the nodes entirely and teams only define workloads. Container images are stored in Artifact Registry, and the cluster integrates with Cloud Load Balancing for traffic routing, Cloud Logging and Cloud Monitoring for observability, and Cloud IAM for access control.

6.4 Infrastructure Components

6.4.1 Hosting

In web infrastructure, web hosting is the service layer that makes an application available on the internet and keeps it reliably accessible over time. At the simplest level, static files (HTML, CSS, JavaScript, and images) are uploaded and served directly by a web server.

As requirements grow, teams typically move to managed hosting or virtual servers that add operational capabilities such as TLS, autoscaling, monitoring, and managed networking. At the

advanced end, platforms like Kubernetes orchestrate many containerized services across clusters, providing automated deployments, load balancing, self-healing, and rolling updates for large distributed systems.

For Angular applications, hosting is often static-first because the build output is mostly static assets. A basic setup can serve the generated files from Nginx or Apache, while larger platforms often combine CDN delivery, reverse proxy routing, and containerized workloads for scale and resilience. Choosing the right hosting model depends on expected traffic, latency targets, operational maturity, compliance constraints, and the level of runtime control required by the team.

6.4.2 Domain Name

A domain name gives users a human-readable address to reach a web application. Instead of remembering IP addresses, users access services through domain names such as `example.com`, while the Domain Name System (DNS) resolves that name to the correct infrastructure endpoint.

In practice, a domain is not just branding. It is also part of the routing layer of the system. DNS records can point traffic to web servers, reverse proxies, CDNs, load balancers, email providers, and verification services. Proper domain management therefore affects availability, security, and deployment flexibility.

From an operational perspective, teams should also consider DNS propagation times, certificate management, subdomain strategy, and failover planning.

6.4.3 SPA

A single-page application (SPA) architecture allows teams to move away from monolithic systems and build decoupled frontends.

In a SPA architecture, the browser downloads an initial HTML document together with JavaScript, CSS, and static assets, and then the client application takes responsibility for routing, rendering views, and communicating with backend services. This separation allows frontend teams to evolve the user experience independently from backend implementation details. The frontend can consume REST, GraphQL, or other HTTP APIs without being tightly coupled to server-side page rendering. That makes it easier to adopt headless and composable architectures where multiple services contribute to the final user experience.

SPAs also introduce infrastructure considerations. Because routing happens client-side, the hosting layer must usually rewrite unknown routes back to `index.html`. Teams must also pay close attention to bundle size, caching strategy, SEO requirements, and API latency, since much of the perceived application performance depends on efficient delivery of static assets and fast network calls.

6.4.4 Reverse Proxy

A reverse proxy sits in front of one or more backend services and receives requests on their behalf. It can then forward traffic to application servers, static asset servers, or upstream APIs based on hostnames, paths, headers, or other routing rules. In many production systems, Nginx, Apache, or a cloud-managed gateway performs this role.

Reverse proxies are valuable because they centralize concerns that should not be reimplemented in every application service. For example, they can terminate TLS, compress responses, cache content, enforce security headers, limit request rates, rewrite URLs, and shield internal services from direct public exposure. They are also an effective edge security control point. At this layer, teams can apply WAF policies, bot filtering, IP reputation rules, and request-size or request-rate limits before traffic reaches application services.

For Angular applications, a reverse proxy is commonly used to serve the static assets, route `/api` traffic to backend services, and ensure that SPA routes resolve correctly. It becomes a key control point for both performance and security.

6.4.5 Load Balancer

A load balancer distributes incoming web traffic across multiple servers to ensure efficient resource utilization and prevent server overload.

Load balancing improves both scalability and availability by distributing requests across multiple application instances. Instead of sending all traffic to a single server, the load balancer chooses from a pool of healthy targets, reducing the risk of bottlenecks and improving fault tolerance when one instance becomes slow or unavailable. Modern cloud platforms often provide managed load balancers with built-in health checks, TLS support, autoscaling integration, and traffic policies.

For frontend applications, load balancing is particularly useful when serving Angular applications from multiple nodes or when splitting traffic between several backend APIs. Combined with

health probes and rolling deployments, it allows infrastructure changes to happen with minimal or no downtime.

6.4.6 CDN

A Content Delivery Network (CDN) uses a geographically distributed network of proxy servers that handle incoming requests from nearby users. The main goal of a CDN is to reduce latency and improve reliability by serving cached content from edge locations closer to end users. Static resources such as JavaScript bundles, stylesheets, images, fonts, and videos benefit significantly from CDN delivery because those files can often be cached and reused across many requests.

To keep CDN behavior reliable across releases, teams should define explicit cache keys, TTL policies by content type, and invalidation or purge workflows as part of deployment automation. Without this, stale HTML or mismatched assets can cause hard-to-diagnose frontend regressions. When loading resources from CDNs, use Sub-Resource Integrity (SRI) hashes to ensure the resource has not been tampered with.

For Angular applications, a CDN is especially effective because the build output is largely static and fingerprinted. That makes it safe to apply aggressive cache policies to versioned assets while keeping the main HTML document on a shorter cache lifetime. When configured correctly, this pattern reduces origin load and speeds up rendering for users across different regions.

6.5 Infrastructure Provisioning

6.5.1 Infrastructure as Code

Infrastructure as Code (IaC) is the most reliable way to implement infrastructure provisioning in modern web platforms. Instead of creating resources manually in cloud consoles, teams declare the desired infrastructure in version-controlled files and let tooling plan, apply, and track changes. This approach reduces configuration drift, improves peer review, and makes rollbacks and audits easier because every change is tied to a pull request and a deployment log.

In open-source IaC workflows, teams often start with Terraform because of its broad provider ecosystem and predictable plan/apply lifecycle. OpenTofu follows the same declarative model and HCL-based syntax, offering a community-governed path with strong Terraform compatibility for many existing modules and workflows. Other open-source approaches include Pulumi, which defines infrastructure using general-purpose languages, and Crossplane, which extends

Kubernetes with control-plane-style infrastructure orchestration so cloud resources can be managed as Kubernetes APIs.

Cloud providers offer closer integration with their respective platforms. On AWS, CloudFormation is the native declarative engine for provisioning and updating stacks of resources with built-in drift detection and change sets. On Azure, Bicep provides a concise language that compiles to ARM templates and is optimized for Azure Resource Manager deployments. On Google Cloud, Infrastructure Manager is the current managed Terraform-based provisioning service integrated with Google Cloud governance and IAM controls. In practice, these cloud-native tools are especially useful when teams want first-class access to provider features as soon as they are released and deep integration with platform policy, identity, and audit capabilities.

Provisioning and orchestration tools define and create infrastructure resources, but they do not replace day-2 host and runtime configuration. After networks, clusters, load balancers, and storage are provisioned, teams still need to configure operating systems, packages, services, users, certificates, and application runtime settings consistently. That is where configuration management tools such as Ansible, Chef, and Puppet connect to the IaC workflow. Provisioning creates the infrastructure substrate, and configuration management converges each node or service to the intended operational state.

Cloud Native Computing Foundation (CNCF) provides the governance ecosystem behind key provisioning and operations standards, including projects such as Kubernetes, Prometheus, OpenTelemetry, and Helm that many web platforms depend on.

6.5.2 Ansible

Ansible is an open-source software provisioning, configuration management, and application deployment tool enabling Infrastructure as Code. It runs on many Unix-like systems and can configure both Unix-like systems as well as Microsoft Windows. Unlike provisioning tools that create infrastructure from scratch, Ansible is designed to configure existing machines, for example, installing packages, managing services, and enforcing desired state on servers that are already running. It connects to target hosts over SSH, requires no agent on the managed node, and uses its own declarative language to describe system configuration.

Ansible uses playbooks written in YAML to define the tasks that should be applied to one or more hosts. A playbook describes the desired state of the system, and Ansible ensures that state is reached. To use Ansible, we only need to know minimal YAML structure.

The following playbook demonstrates how to install and start Nginx on an existing machine:

```
---
- name: Install nginx
  hosts: localhost
  connection: local
  become: true

  tasks:
    - name: Install nginx
      apt: name=nginx state=latest
    - name: Start nginx
      service:
        name: nginx
        state: started
```

Assuming we have already set up *passwordless sudo*, we don't need to supply the *-K* or *--ask-become-pass* flags. We can also validate the playbook syntax before running it by adding the *--syntax-check* flag. Then, to execute the playbook run the following command:

```
ansible-playbook ./nginx_install.yml
```

After the command completes, Nginx should be installed and running on the target machine. We can verify by visiting the host IP address in a web browser.

Ansible can be used in a CI/CD workflow to configure the target environment and to then deploy the application on it. The purpose of using Ansible in the pipeline flow is to reuse roles and playbooks for configuration, leaving the CI/CD server only as a process orchestrator instead of a shell script executor. To test playbooks, it is recommended to use virtual machines. To do so, we can use Vagrant, which allows us to quickly set up virtual environments independently from the virtualization backend used on the local system.

Ansible works against multiple managed nodes or "hosts" in the infrastructure at the same time, using a list or group of lists known as inventory. Frequently, we manage passwords, and saving these passwords in plain text is a huge security risk. Ansible Vault is a feature of Ansible that allows us to keep sensitive data such as passwords or keys in encrypted files, rather than as plaintext in Playbooks or Roles.

Finally, it is recommended to take a look at Ansible Best Practices in the official documentation.

6.5.3 Terraform

Terraform is an open-source Infrastructure as Code tool used to provision and manage cloud infrastructure declaratively. Instead of creating resources manually in cloud consoles, Terraform

lets teams define infrastructure in version-controlled files and apply those definitions consistently across environments. This approach improves repeatability, enables peer review and CI/CD automation, and reduces configuration drift.

Terraform uses configuration files written in HCL (HashiCorp Configuration Language). A Terraform configuration describes the desired state of infrastructure, and then Terraform calculates the changes required to reach that state through its `plan` and `apply` workflow.

In Kubernetes-based platforms, Terraform is commonly used to provision shared infrastructure such as resource groups, container registries, clusters, identities, and networking before application deployment. The following Terraform example provisions the core Azure resources needed for Kubernetes, namely, a resource group, an Azure Container Registry, an AKS cluster, and the role assignment that allows AKS nodes to pull container images from ACR.

```
# Create a resource group
resource "azurerm_resource_group" "default" {
  name     = "rg-angular-production"
  location = "eu-west-1"

  tags = {
    environment = "production"
  }
}

# Create a container registry within the resource group
resource "azurerm_container_registry" "default" {
  name                = "private-registry"
  location            = azurerm_resource_group.default.location
  resource_group_name = azurerm_resource_group.default.name
  sku                 = "Standard"
  admin_enabled       = false
}

# Create a kubernetes cluster within the resource group
resource "azurerm_kubernetes_cluster" "default" {
  name                = "aks-angular-production"
  location            = azurerm_resource_group.default.location
  resource_group_name = azurerm_resource_group.default.name
  dns_prefix          = "angular-prod"

  default_node_pool {
    name       = "default"
    node_count = 2
    vm_size    = "Standard_D2_v2"
  }

  identity {
    type = "SystemAssigned"
  }
```

```
  network_profile {
    network_plugin    = "azure"
    load_balancer_sku = "standard"
  }

  tags = {
    environment = "production"
  }
}

# Attaching a container registry to a Kubernetes cluster
resource "azurerm_role_assignment" "default" {
  principal_id         =
azurerm_kubernetes_cluster.default.kubelet_identity[0].object_id
  role_definition_name = "AcrPull"
  scope                = azurerm_container_registry.default.id
}
```

After defining the configuration, the standard execution flow is to initialize the working directory, review the execution plan, and then apply the changes with:

```
terraform apply -var-file="terraform.tfvars" -auto-approve=true
```

After `terraform apply` completes, the Azure resources should be created and linked. We can verify this by checking the resource group, AKS cluster, and ACR integration in the Azure portal or with the Azure CLI. To make this configuration reusable across different environments, define input variables (for names, locations, node sizes, tags, etc.) and use a remote state backend with locking for team-safe collaboration.

Terraform fits naturally into CI/CD pipelines because infrastructure changes can be validated, reviewed, and applied using the same workflow as application code. We can test Terraform code with Terraform tests, enforce consistent style with terraform fmt, catch configuration issues with TFLint, and scan for security/compliance risks with Checkov. These checks help preventing regressions in infrastructure code.

We can use Make to simplify and standardize Terraform work. We can create a `Makefile` that loads environment settings, initializes Terraform, manages workspaces, and gives quick commands for format/lint/test, plan/apply/destroy by environment, plus helper tasks like documentation, security scanning, and cleanup.

Finally, it is recommended to review Terraform Best Practices, Terraform Style Guide and Terraform State in the official documentation.

6.5.4 Helm

Helm is an open-source package manager for Kubernetes that standardizes how application workloads are packaged, configured, and released. Instead of maintaining duplicated raw manifests for each environment, teams define reusable chart templates and environment-specific values files, then apply them consistently across development, staging, and production.

In Kubernetes-based platforms, Helm and Kustomize are commonly used after infrastructure provisioning to install and upgrade runtime components and application workloads. For Angular applications served from containers (for example, with Nginx), templates make it straightforward to parameterize replicas, image repositories and tags, ingress hosts, TLS settings, resource requests and limits, autoscaling behavior, and other environment-specific settings.

A minimal Helm template for an Angular application can be stored as `templates/angular-app.yaml` file:

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: angular-app
  namespace: "{{ .Values.environment.namespace }}"
  labels:
    app: angular-app
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: angular-app
  template:
    metadata:
      labels:
        app: angular-app
    spec:
      securityContext:
        runAsUser: 1000
        runAsGroup: 1000
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: angular-nginx
          image: private-registry.azurecr.io/angular-app:latest
          ports:
            - containerPort: 8080
          securityContext:
            allowPrivilegeEscalation: false
            capabilities:
```

```
            drop:
            - ALL
          resources:
            requests:
              cpu: 250m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            timeoutSeconds: 5
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 15
            timeoutSeconds: 5
          volumeMounts:
            - name: "cache-nginx-volume"
              mountPath: /var/cache/nginx
            - name: "run-volume"
              mountPath: /var/run
            - name: "tmp-volume"
              mountPath: /var/lib/nginx/tmp
      volumes:
        - name: "cache-nginx-volume"
          emptyDir: {}
        - name: "run-volume"
          emptyDir: {}
        - name: "tmp-volume"
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: angular-app-svc
spec:
  selector:
    app: angular-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: angular-app-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
```

```
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - "{{ .Values.ingress.host }}"
      secretName: angular-app-tls
  rules:
    - host: "{{ .Values.ingress.host }}"
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: angular-app-svc
                port:
                  number: 80
```

Unlike a plain Kubernetes manifest, a Helm template is parameterized through `values.yaml`, which is what makes the same chart reusable across environments. A typical production setup also uses an NGINX Ingress Controller with cert-manager to automate TLS certificate provisioning.

To apply the chart, the command below drives an idempotent release workflow:

```
helm upgrade --install --wait app . --values=values.yaml
```

The `upgrade --install` pattern ensures the release is created if missing and upgraded if it already exists, `--wait` blocks until Kubernetes resources become ready, `app` is the release name, `.` points to the chart in the current directory, and `--values=values.yaml` injects environment-specific settings.

This pattern fits CI/CD well because it keeps rollout logic consistent across all environments while still allowing per-environment overrides through values files. Teams usually pair this with Helm lint and KubeLinter for Kubernetes manifest and policy checks, and Helm Unittest for chart unit tests. We can also use helm test for integration or smoke tests against the deployed chart.

Policy controls should be treated as part of provisioning, not an afterthought. Kubernetes policy engines enforce organizational guardrails such as "no latest tags", mandatory resource requests/limits, required labels/annotations, restricted hostPath usage, and approved ingress/security settings. Kyverno is especially practical for application teams because policies are written as Kubernetes-native YAML and can validate, mutate, or generate resources at admission time.

6.5.5 Twelve-Factor App

The Twelve-Factor App methodology is a methodology for building software-as-a-service (SaaS) applications. These best practices are designed to enable applications to be built with portability and resilience when deployed to the web.

The twelve factors are:

- **Codebase**: One codebase tracked in version control can support many deployments, such as development, staging, and production. For Angular projects, that means a single application repository should remain the source of truth, even if it is deployed to multiple environments.
- **Dependencies**: All dependencies should be declared explicitly and isolated from the system runtime. In Angular projects, this is typically handled through `package.json`, lock files, and reproducible install steps in CI/CD.
- **Config**: Configuration that varies between environments should be stored outside the code. API URLs, feature flags, analytics keys, and similar values should be injected per environment rather than hard-coded into the application logic.
- **Backing services**: Databases, message brokers, storage systems, and third-party APIs should be treated as attached resources that can be swapped without major code changes. For frontend systems, this applies especially to backend APIs, identity providers, and observability services.
- **Build, release, run**: The build stage creates an artifact, the release stage combines it with configuration, and the run stage executes it. For Angular applications, this means producing the deployable static bundle first, then promoting that artifact through environments with environment-specific settings handled separately.
- **Processes**: Applications should run as one or more stateless processes. Even when Angular is served as static files, the surrounding runtime components such as Nginx containers or SSR processes should avoid relying on local mutable state.
- **Port binding**: The application should be self-contained and expose its service through a port. This is most relevant when Angular is served through a container, SSR runtime, or edge server process that listens on a defined port rather than depending on a manually managed web server installation.
- **Concurrency**: Scale out by running multiple process instances instead of making a single instance larger. In practice, this means serving the Angular app from multiple containers, nodes, or edge locations behind load balancing when traffic increases.

- **Disposability**: Processes should start quickly and shut down gracefully. Fast startup and clean termination improve rolling deployments, autoscaling, and recovery behavior for the containers or runtimes that deliver the frontend.
- **Dev/prod parity**: Keep development, staging, and production as similar as possible to reduce deployment surprises. Teams should align Node versions, build steps, container images, and infrastructure behavior across environments.
- **Logs**: Logs should be treated as event streams and emitted to standard output, then collected by the platform. For frontend systems, this usually means streaming web server, CDN, SSR, and platform logs into centralized observability tooling instead of storing them locally.
- **Admin processes**: One-off administrative tasks should run as short-lived processes in the same environment as the application. Examples include running database migrations for backend dependencies, validating deployment configuration, or executing operational scripts from the same container image or platform context.

Many of these practices have become standard for modern platforms such as containers and orchestration systems. As a result, many organizations view the Twelve-Factor App as a strong foundation and complement it with newer cloud-native practices.

6.5.6 Well-Architected Frameworks

Well-Architected Frameworks provide a practical decision model for defining infrastructure that is reliable, secure, cost-aware, and operationally sustainable over time. In infrastructure provisioning, they help teams move from ad hoc choices to explicit architecture principles that can be validated during design reviews, encoded in Infrastructure as Code, and continuously checked in CI/CD. Instead of asking only whether an environment can be provisioned, these frameworks guide teams to ask whether it can be operated safely at scale, recovered after failures, evolved without disruptive migrations, and governed consistently across environments.

For web applications, this translates into concrete provisioning decisions, namely, selecting managed services where they reduce operational risk, defining network and identity boundaries early, standardizing logging and metrics during provisioning, planning backup and disaster recovery paths, and embedding cost and performance trade-offs into architecture reviews. Well-Architected guidance also improves repeatability by encouraging tagged resources, documented ownership, policy-as-code controls, and measurable service objectives, all of which reduce configuration drift and make platform behavior more predictable in production.

Major cloud providers publish their own versions of this guidance, with similar goals but provider-specific implementation patterns. The AWS Well-Architected Framework focuses on pillars such as operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. The Azure Well-Architected Framework emphasizes architecture quality through reliability, security, cost optimization, operational excellence, and performance efficiency in Azure-native services. Lastly, the Google Cloud Well-Architected Framework provides design and governance recommendations across operations, security, reliability, performance, cost, and sustainability. Using any of these frameworks as a recurring review baseline helps teams define better infrastructure from the start and continuously improve it as requirements evolve.

6.6 Infrastructure Costs

6.6.1 Cloud Cost Estimates

Estimating cloud spend should happen before provisioning, not after the invoice arrives. For Angular applications, this usually means estimating static hosting, CDN traffic, load balancing, container runtime, observability, and data transfer costs early in design and again during delivery.

Each major cloud provider offers native tooling for early estimates, namely, AWS Pricing Calculator, Azure Pricing Calculator, and Google Cloud Pricing Calculator. These calculators are useful for architecture planning and what-if analysis, but they are mostly manual.

For Infrastructure as Code workflows, Infracost complements cloud provider calculators by reading Terraform code directly and producing cost estimates as part of CI/CD. To use Infracost, create a free API key and set `INFRACOST_API_KEY` so the CLI can retrieve up-to-date pricing data.

To show a breakdown of costs, use the following command:

```
infracost breakdown --path .
```

Internally, *Infracost* parses Terraform HCL directly, so cloud credentials are typically not required just to estimate planned infrastructure.

Infracost is often used in addition to budget alerts and cost management reports to provide an addition layer of communication and protection in CI/CD. Cost policies allow DevOps and FinOps teams to define budget guardrails in pull requests and catch high-impact cost changes before merge.

6.6.2 FinOps

FinOps is an operational framework for managing cloud spending collaboratively across Engineering, Finance, and Product so that teams can make faster decisions with a clear understanding of cost, usage, and business value. The FinOps Foundation describes FinOps as a practice for maximizing the business value of the cloud, not simply reducing spend. In practice, this means cloud costs are treated as an engineering concern alongside performance, reliability, and security. Teams do not wait for an end-of-month invoice to discover waste. Instead, they use near-real-time usage and pricing data to understand where money is going, who owns that spend, and whether the resulting architecture is delivering appropriate value.

The FinOps lifecycle is often summarized as `inform`, `optimize`, and `operate`. First, teams build visibility through tagging strategies, account structures, dashboards, budgets, and allocation rules so they can attribute spend to products, environments, and business units. Next, they optimize by rightsizing compute, selecting appropriate storage tiers, reducing idle resources, improving autoscaling behavior, and making better use of discounts such as Reserved Instances, Savings Plans, or committed-use contracts. Finally, they operationalize those practices by reviewing forecasts regularly, embedding cost checks into delivery pipelines, and making cloud efficiency part of normal engineering planning rather than a one-off cleanup exercise.

The FinOps Framework and cloud guidance such as AWS Cloud Financial Management, the Azure FinOps guide, and Google Cloud FinOps all emphasize that cloud cost management works best when accountability is distributed to the teams creating the spend.

6.6.3 Best Practices

The following practices outline how to build a repeatable FinOps operating model with clear ownership, visibility, and optimization workflows:

- ☐ **Build accurate cost visibility**: Create near-real-time dashboards that show cost, usage, and trends by account, service, workload, and team. Include variance views (day/week/month) so spend drivers are easy to spot.
- ☐ **Map all spend to the business**: Map cloud costs to cost centers, products, and business units using a strict tagging standard plus account/subscription structure. Define a fallback method for untagged and untaggable resources.

- ☐ **Use showback or chargeback with shared-cost rules**: Make teams accountable for their total cost by publishing showback (or chargeback) reports and allocating shared platform/support costs with a transparent formula.
- ☐ **Set budgets, forecasts, and guardrails**: Maintain budgets and rolling forecasts per team/workload. Add proactive alerts for threshold breaches and anomalies to catch overspend early.
- ☐ **Define ownership, roles, and operating cadence**: Assign clear ownership for every major workload and spending area. Align Engineering, Finance, and Product on a regular FinOps review cadence with explicit responsibilities.
- ☐ **Govern with policy and automation**: Enforce tagging, provisioning, and lifecycle policies through automation, not manual review. Block or remediate non-compliant resources where possible.
- ☐ **Optimize utilization continuously**: Track utilization and rightsize compute, storage, and databases. Remove idle or redundant resources, and automate cleanup of temporary/dev environments.
- ☐ **Scale and schedule intelligently**: Use auto-scaling, scheduled start/stop windows, and environment TTLs to match capacity with demand and avoid paying for inactive resources.
- ☐ **Optimize commitment and purchasing strategy**: Actively manage Reserved Instances, Savings Plans, and CUDs, including coverage, utilization, and expirations. Use spot or preemptible capacity for fault-tolerant workloads.
- ☐ **Optimize architecture and data economics**: Choose cost-appropriate services and storage classes, enforce retention policies, and minimize data transfer/egress costs through better workload placement and design.
- ☐ **Manage vendors and commercial contracts**: Review provider pricing changes, support plans, and contracts regularly. Re-negotiate commitments when usage patterns change.
- ☐ **Deliver spend insights quickly to stakeholders**: Publish daily or weekly cost reports to engineering, product, and finance so teams can react while changes are still inexpensive to correct.
- ☐ **Integrate optimization into engineering workflows**: Push rightsizing and waste-reduction recommendations into sprint planning and backlog tools, so cost improvements are tracked like any other delivery work.

- ☐ **Control software license spend**: Audit marketplace and third-party license usage, remove unused subscriptions, and evaluate open-source alternatives where they meet reliability and security requirements.
- ☐ **Eliminate shadow cloud usage**: Route provisioning through approved platforms and guardrails to prevent unmanaged cloud accounts or ad hoc resources from creating hidden risk and untracked spend.
- ☐ **Shift cost optimization left in the SDLC**: Evaluate cost impact during design and implementation (not only in operations) so architecture and feature decisions include performance, reliability, and cost trade-offs from the start.
- ☐ **Benchmark efficiency externally**: Compare normalized cost and utilization metrics against industry peers to identify whether internal optimization results are competitive or lagging.
- ☐ **Measure outcomes, not just savings**: Track KPIs that connect cost to business value (for example: unit cost, budget variance, commitment coverage/utilization, and optimization realization rate). Use scorecards to drive continuous improvement.

6.7 Dockerizing Angular Applications

6.7.1 Nginx

Nginx is a lightweight, high-performance web server, reverse proxy, and load balancer that is widely used in frontend delivery stacks. It is designed to handle many concurrent connections efficiently, which makes it a common choice for serving static assets, terminating TLS, and routing requests to upstream APIs or application services.

Nginx is often used for Angular applications because it is simple to deploy, easy to configure, and well suited to serving the compiled output of a single-page application. Nginx is usually responsible for serving the built files with good cache behavior and rewriting unknown routes back to `index.html` so the Angular Router can handle client-side navigation.

The following minimal configuration serves an Angular build output, adds a cache-friendly route for static assets, and rewrites application routes to the SPA entry point:

```
server {
  listen 8080 default_server;

  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff2)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    try_files $uri =404;
  }
}
```

In a containerized Angular deployment, this configuration is commonly paired with a Docker image that copies the build output into Nginx's default HTML directory and exposes the application on a non-privileged port. From there, Nginx can also be extended to add security headers, compression, request limits, or an `/api` reverse-proxy route when the frontend and backend share the same origin.

6.7.2 Docker Image

A deployable image is the most common delivery possibility for Angular applications. That packages the built Angular application together with a web server or runtime configuration. Examples include Docker images based on Nginx, or a hardened enterprise base image stored in a private registry, Docker Hub, or a cloud registry.

A Dockerfile is a recipe for building a Docker image. Next *Dockerfile* uses Nginx Alpine as base image:

```
FROM nginx:1.25.3-alpine

COPY config/nginx/nginx.conf /etc/nginx/nginx.conf
COPY dist/browser /usr/share/nginx/html

EXPOSE 8080

CMD ["nginx", "-g", "daemon off;"]
```

This Dockerfile defines a recipe for packaging an Angular application into a self-contained, deployable container. It starts from a lightweight Nginx image, adds a custom server configuration and the compiled Angular application files via `ng build` command, declares the port the server will listen on, and specifies the command that keeps the server running when the container starts.

We can build the Docker image using the following command:

```
docker build -f Dockerfile -t angular-app:latest .
```

The resulting image can be deployed anywhere to serve the Angular application. Note that a Nginx configuration is included which contains rules required for redirecting all URLs to *index.html*, where the built-in Angular Router can take over. For more information, see the Nginx documentation and Angular's Server configuration guidance.

6.7.3 Best Practices

The following practices provide a practical checklist for building secure, reproducible, and production-ready Docker images for Angular applications:

- ☐ **Use official images**: Start from trusted, maintained bases like the official Node.js image and Docker Official Images instead of building runtimes from scratch.
- ☐ **Use a private registry**: Reduce supply-chain risk with trusted private registries. Alternatives include Nexus and Artifactory.
- ☐ **Use specific image versions**: Avoid mutable `latest`. Pin versions (typically `major.minor`) or use digests for reproducibility with an update strategy.
- ☐ **Use small-sized images**: Consider using a minimal Linux base image like Alpine or distroless images from Google's distroless project.
- ☐ **Update images frequently**: Rebuild on top of regularly updated base images and follow stable/LTS versioning to capture security fixes.
- ☐ **Scan images regularly**: Shift security left by scanning images in CI with Docker image scanning, Docker and Snyk, Clair, Trivy, or Docker Bench for Security.
- ☐ **Sign and verify images**: Enforce provenance and integrity with Sigstore Cosign. Sign images in CI and verify signatures before pull/run.
- ☐ **Use metadata labels**: Add useful image metadata (owner, license, project) through `LABEL` to improve discoverability and automation.
- ☐ **Exclude unnecessary packages**: Keep only runtime requirements by copying minimal outputs, using production-only dependency install, and multi-stage builds. Use `.dockerignore` file for unnecessary files.
- ☐ **Use multi-stage builds**: Split build and runtime stages with Docker multi-stage builds to produce smaller and cleaner images.

- ☐ **Use layer caching**: Order Dockerfile steps from least to most frequently changing to maximize image layer reuse and faster builds.
- ☐ **Build one image for all environments**: Build once in CI and promote the same artifact across dev/test/prod using externalized configuration.
- ☐ **Avoid privileged containers**: Avoid running containers as root. Do not use `--privileged` unless unavoidable. Prefer minimal capabilities via --cap-drop or via Security Context in Kubernetes.
- ☐ **Do not map any ports below 1024**: Avoid privileged host ports unless explicitly needed. Expose only required ports.
- ☐ **Prevent confidential data leaks**: Never bake secrets into layers. Inject at runtime/build-time and use orchestrator secret stores like Docker Swarm or Kubernetes.
- ☐ **Include health/liveness checks**: Add HEALTHCHECK for Docker/Swarm or Liveness/Readiness for Kubernetes.
- ☐ **Configure logging mechanisms**: Log to `stdout`/`stderr` when possible. Prefer structured JSON logs, and use sidecar collection for file-based legacy apps.
- ☐ **Use stateless and immutable containers**: Containers are designed to be stateless and immutable. Stateless means that any state (persistent data of any kind) is stored outside of a container. Immutable means that a container won't be modified during its life.
- ☐ **Limit container resources**: It's a good practice to limit the memory and CPU usage of the containers, especially if we are running multiple containers on a single machine. This can prevent any of the containers from using all available resources and consequently hindering the others.
- ☐ **Use a linter**: Add Dockerfile linting in CI with tools like hadolint to catch anti-patterns early.

6.8 Observability and Monitoring

6.8.1 Fundamentals

Observability and Monitoring are closely related, but they are not the same thing.

Monitoring focuses on tracking predefined signals to determine whether a system is healthy. It answers questions such as whether latency is rising, error rates crossed a threshold, or a service

is unavailable. Continuous Monitoring extends this idea into an operational practice where systems and infrastructure are watched continuously through dashboards, alerts, and status checks so teams can detect issues quickly and respond before they become outages.

Observability goes further. It is the ability to understand the internal state of a system by examining the signals it emits. In practice, this means collecting and correlating logs, metrics, traces, events, dashboards, and alerts so teams can investigate why a problem is happening, not only whether it is happening. Observability is critical for platform engineering, Site Reliability Engineering (SRE), and DevOps because modern applications rarely fail in simple and isolated ways.

For Angular and other web applications, the distinction matters. Monitoring is useful for detecting symptoms such as high response times, increased JavaScript error rates, failed health checks, or sudden drops in traffic. Observability is what helps teams connect those symptoms to underlying causes across browsers, APIs, CDNs, reverse proxies, containers, and cloud services. In distributed architectures, where a single user action may involve many components, both capabilities are necessary.

Gathering metrics, monitoring components, and configuring alerts is an essential part of setting up and managing production infrastructure. Observability should be built into the platform from the start rather than added only after failures occur. Once instrumentation is in place, teams can define meaningful service objectives, detect regressions earlier, and respond faster when incidents affect users.

OpenTelemetry is the open standard most teams use to implement observability consistently. It provides shared conventions for collecting the three pillars of observability, logs, metrics, and traces, across browsers, backend services, and infrastructure, while keeping the telemetry pipeline independent from any single vendor.

6.8.2 Logging

6.8.2.1 Definition

Logging records discrete events that happened in the system and gives teams the context needed to investigate failures, security incidents, performance degradation, and unexpected user behavior.

For Angular applications, logging is important at two levels, namely, client-side and server-side. In the browser, client-side logs help capture runtime exceptions, failed HTTP requests, routing

problems, feature-flag issues, and other errors that may never appear in server logs. In containerized environments, logging is also important because a container can be rescheduled, replaced, or terminated at any time.

Effective logging in Angular and container platforms usually follows a few practical rules:

- Use structured logs, preferably JSON, so events can be filtered, correlated, and queried reliably;
- Include useful context such as environment, service name, version, request path, trace or correlation ID, user session identifier, and severity level;
- Avoid writing secrets, tokens, personal data, or other sensitive information into logs;
- Aggregate logs centrally so teams can investigate incidents across browsers, CDNs, reverse proxies, containers, and backend APIs in one place.

When logs are collected consistently across the frontend and the infrastructure, teams can connect a user-visible problem to the underlying technical cause much faster. A JavaScript error in the Angular client can be correlated with an API timeout, a Nginx upstream failure, or a container restart, which makes logging a foundational part of operating modern web applications.

Following the Twelve-Factor recommendation, containerized applications and delivery layers such as Nginx should write logs to standard output and standard error, allowing the platform to collect, enrich, and forward them to a centralized logging system such as Loki, Elasticsearch, Amazon CloudWatch, Azure Monitor, or Google Cloud Logging.

6.8.2.2 Logs for Nginx

Nginx provides flexible logging capabilities that can be configured to capture request and response data in structured or custom formats. Nginx writes access logs to a file specified by the `access_log` directive. The `log_format` directive defines a custom format template that controls which fields are captured and how they are presented.

The following example defines a standard log format called `main` that captures essential request and response data:

```
log_format main '$remote_addr - $remote_user [$time_local] $status '
  '"$request" $body_bytes_sent "$http_referer" '
  '"$http_user_agent" "$http_x_forwarded_for"';
```

A more structured JSON format is often preferred because it enables parsing and filtering in log aggregation systems. The next example captures the same data, but displays it in JSON format:

```
log_format json escape=json '{'
  '"time_local": "$time_local",'
  '"remote_addr": "$remote_addr",'
  '"remote_user": "$remote_user",'
  '"request": "$request",'
  '"status": $status,'
  '"body_bytes_sent": $body_bytes_sent,'
  '"request_time": $request_time,'
  '"http_referer": "$http_referer",'
  '"http_user_agent": "$http_user_agent",'
  '"http_x_forwarded_for": "$http_x_forwarded_for"'
'}';
```

Once a format is defined, apply it to the `access_log` directive in the main `http` block, a server block, or a location block. By structuring logs with consistent formats and including trace IDs or request correlation headers, teams can correlate frontend events with backend logs and infrastructure telemetry, enabling faster incident investigation and root-cause analysis.

For containerized environments like Docker or other container orchestration tool, it is recommended to adjust the `access_log` and `error_log` directives in the Nginx configuration to log to `stdout` and `stderr`. This approach helps capture logs in the standard output and error streams, allowing the platform to collect and forward them centrally.

6.8.3 Metrics

6.8.3.1 Definition

Metrics are numerical measurements collected over time that describe how a system behaves. They are essential because they make trends, saturation, regressions, and service health visible at a glance. Unlike logs, which explain individual events, metrics help answer questions such as whether request latency is increasing, whether error rates are rising after a deployment, whether CPU or memory limits are being approached, or whether traffic patterns changed significantly during a release.

For Angular applications, the most useful metrics are often user and delivery-oriented rather than purely server-oriented. Teams commonly track frontend performance indicators such as page load times, Core Web Vitals, JavaScript error rates, API response times, cache hit ratios, and request volumes. In containerized applications, metrics are also critical for understanding platform behavior. Common infrastructure metrics include CPU usage, memory consumption,

container restarts, network throughput, disk I/O, request rate, error rate, latency, and health-check status. These metrics help connect technical behavior to user experience.

Collecting metrics in containerized environments usually involves several layers:

- The application exposes its own metrics endpoint, often at a path such as `/metrics`, with counters, gauges, and histograms for request rate, duration, and failures;
- The container runtime and host provide infrastructure metrics about CPU, memory, filesystem, and network usage;
- Orchestrators such as Kubernetes add workload-level signals such as pod state, restarts, scheduling failures, and replica health;
- Reverse proxies, load balancers, CDNs, and managed cloud services emit their own operational metrics, which should be collected alongside application telemetry.

To make metrics useful, teams should focus on a small set of meaningful indicators instead of instrumenting everything indiscriminately. For web applications, the RED Method (Rate, Errors, and Duration) is often a good starting point. Combined with resource metrics and a few frontend performance measures, this gives a reliable baseline for understanding whether the Angular application, its delivery layer, and its runtime platform are healthy.

In managed cloud platforms, metrics can be sent to services such as Amazon CloudWatch, Azure Monitor, or Google Cloud Monitoring and then visualized or federated into a broader observability platform. Alternatively, Prometheus is a common way to collect these signals. Prometheus scrapes HTTP endpoints on a schedule, stores the resulting time-series data, and makes it available for dashboards and alerts.

6.8.3.2 Metrics for Nginx

Nginx can expose operational counters with the built-in http_stub_status_module, and this is usually the default starting point in production because it is simple, lightweight, and available in standard builds.

The following example adds the `access_log` directive to a dedicated location block:

```
location /metrics {
  access_log off;
  stub_status;
}
```

For richer telemetry, Nginx Plus exposes additional metrics through its API module and live activity views. In practice, Prometheus does not scrape `stub_status` directly. A common pattern is to expose `stub_status` on an internal endpoint and use the Nginx Prometheus Exporter, which reads that endpoint and republishes the values in Prometheus format. This gives a stable and widely adopted integration path without custom parsing logic in application code.

If exposing Prometheus metrics directly from Nginx is preferred, a custom endpoint pattern also works where full control over output format is required. See the following example:

```
location /metrics {
  access_log off;
  add_header Content-Type text/plain;
  set $metrics '# TYPE connections_active gauge\n';
  set $metrics '${metrics}connections_active $connections_active\n';
  set $metrics '${metrics}# TYPE connections_reading gauge\n';
  set $metrics '${metrics}connections_reading $connections_reading\n';
  set $metrics '${metrics}# TYPE connections_writing gauge\n';
  set $metrics '${metrics}connections_writing $connections_writing\n';
  set $metrics '${metrics}# TYPE connections_waiting gauge\n';
  set $metrics '${metrics}connections_waiting $connections_waiting';
  return 200 $metrics;
}
```

When teams need custom business or request-path metrics inside Nginx itself, the lua-nginx-module provides a practical solution to increment counters in Lua during request processing and expose them on `/metrics` in Prometheus exposition format. This is more flexible than `stub_status` and avoids an external application just for metric shaping, while still integrating cleanly with Prometheus.

Operationally, the recommended order is to start with `stub_status` or custom endpoint for Prometheus as the default path, move to Nginx Plus when advanced platform telemetry is needed, or adopt lua-nginx-module when domain-specific metrics generated at the edge layer are needed.

6.8.4 Tracing

6.8.4.1 Definition

Tracing records the lifecycle of a request as it moves through a system. It captures a chain of spans, where each span represents a unit of work such as a browser action, an API call, a database query, or an external service request. While metrics show aggregate behavior and logs capture discrete events, traces reveal how a single transaction actually flowed end to end.

For Angular and other web applications, tracing is especially useful in distributed architectures where one user action can involve many components. In practice, tracing should connect browser telemetry with backend and infrastructure spans by propagating context across service boundaries. Once this instrumentation is in place, teams can correlate traces with logs and metrics, speed up root-cause analysis, and prioritize performance improvements based on real request paths rather than assumptions.

Practical action points for tracing in Angular and container platforms:

- Instrument both frontend and backend paths so a single user action can be followed end-to-end;
- Propagate trace context headers across gateways, APIs, and async boundaries to preserve span relationships;
- Capture trace attributes that improve debugging context, such as environment, service name, version, route, and error status;
- Correlate trace IDs with logs and key metrics in dashboards and alerts to accelerate incident response.

Cloud providers offer managed tracing services that eliminate the need to operate backend infrastructure. On AWS, AWS X-Ray can record trace data from SDKs in application code, automatically capture calls from AWS services, and create service dependency diagrams that show latency and error rates at each hop. On Azure, Application Insights provides distributed tracing with automatic instrumentation for Node.js applications, along with integration with Azure services. On Google Cloud, Cloud Trace integrates with Google Cloud services to collect latency data and generate span distributions and service maps.

For teams that prefer open-source tooling or require multi-cloud tracing, Jaeger is the de facto standard for distributed tracing. Built on the OpenTelemetry standard and the OpenTracing API, Jaeger provides trace collection, storage, and visualization without lock-in to any cloud provider.

6.8.4.2 Jaeger

Jaeger is a CNCF-backed open-source project for collecting and visualizing traces from distributed applications. It follows the OpenTelemetry specification, making it compatible with instrumentation libraries for Node.js, Python, Java, Go, and other languages. Jaeger consists of a collector that receives spans from instrumented code, a backend storage layer that can use Elasticsearch, Cassandra, or BadgerDB, and a UI for querying and visualizing traces.

A Jaeger deployment typically includes the collector (which receives spans), the query service (which provides the UI), and a backend storage system. For local development or testing, the fastest way to start is using the all-in-one image:

```
docker run -d -p 16686:16686 jaegertracing/all-in-one:latest
```

Then, the Jaeger UI is accessible at port `16686`. Teams can search for traces by service name, tags, duration, or error status, then drill into individual traces to see the call graph, latency breakdown, and any error messages or logs associated with each span. In production, Jaeger can be scaled horizontally with a distributed collector fleet and a backend like Elasticsearch, allowing it to handle millions of spans per second from large microservice deployments. Jaeger also supports sampling policies, which can reduce storage and network costs by keeping only traces that match certain criteria, such as errors or slow requests above a threshold.

6.8.4.3 Tracing for Angular

Tracing in Angular usually starts at the browser edge, where a user interaction triggers one or more HTTP requests to backend services. The main goal is to preserve enough request context that teams can follow a user-visible problem across the Angular client, reverse proxies, APIs, and downstream services.

The simplest approach is to attach a trace ID or correlation header to each outgoing request from the web application. In Angular, this is commonly implemented in an `HttpInterceptor` so every outgoing request receives a generated correlation value consistently:

```
@Injectable()
export class ApiInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler):
Observable<HttpEvent<any>> {
    const clone = req.clone({ setHeaders: { 'X-Correlation-Id': uuid() } });

    return next.handle(clone);
  }
}
```

This approach works well when the primary requirement is operational correlation rather than full span visualization. It is especially useful for teams that already centralize logs because the shared request identifier becomes the join key between frontend and backend events.

The more advanced approach is full distributed tracing with OpenTelemetry and a tracing backend such as Jaeger. In that model, the Angular application emits spans for browser interactions and HTTP requests, propagates trace context headers downstream, and lets instrumented backend services continue the same trace. OpenTelemetry provides the vendor-

neutral instrumentation model and context propagation rules, while Jaeger provides collection, storage, and visualization of the resulting trace data. This gives teams a full request timeline rather than just a shared identifier, which makes it much easier to identify where latency was introduced, which dependency failed, and how a single user action traversed the system.

6.8.5 Health Checks

6.8.5.1 Definition

Health checks are lightweight signals used to verify whether an application instance can run correctly and serve traffic. They help platforms decide when to route requests, restart unhealthy processes, or wait for slow-starting services. In modern delivery systems, health checks are foundational for availability and safe rollouts.

For Angular and other web applications, health checks are usually implemented in the web server or the application container. The three core probe types remain essential, namely, liveness confirms the process is alive, readiness confirms it can accept traffic, and startup prevents premature restarts during initialization.

Practical action points for health checks in Angular and container platforms:

- Expose dedicated endpoints such as `/health`, `/ready`, and `/live` with clear and fast responses;
- Keep liveness checks shallow and readiness checks dependency-aware to avoid false restarts;
- Configure probe intervals and timeouts to match application startup and recovery behavior;
- Monitor probe failures as first-class operational signals and correlate them with logs and alerts.

In practice, health checks are usually exposed as a simple HTTP endpoint such as `/health`, `/ready`, or `/live`, and then evaluated by infrastructure components such as Docker `HEALTHCHECK`, Kubernetes Probes, or external uptime tools such as Prometheus Blackbox Exporter.

6.8.5.2 Health Checks for Nginx

In Nginx, a health check can be implemented as a small dedicated endpoint such as `/health`.

```
location /health {
  access_log off;
  default_type application/json;
  return 200 '{"status": "UP"}';
}
```

The `access_log off;` directive avoids noisy probe logs, `application/json` declares the response type, and `{"status": "UP"}` returns a fast success response. This is a good fit when the goal is to confirm that the web server itself is alive.

6.8.6 Alerts

6.8.6.1 Definition

Alerts turn observability data into operational response. Metrics, logs, and traces describe system behavior, while alerting defines when a condition requires human or automated action. A useful alert rule combines a signal, a condition, and an evaluation window, then routes notifications to the appropriate channel.

For Angular and other web applications, effective alerts should be aligned with user impact and service objectives, not raw infrastructure noise. Typical high-value signals include sustained increases in CPU and memory utilization, HTTP 5xx errors, latency degradation, failing readiness checks, repeated container restarts, certificate expiration, CDN origin failures, and abrupt traffic drops. High-quality alerts also provide clear context, including service, environment, current value, threshold, and links to dashboards or runbooks.

Practical action points for alerting in Angular and container platforms:

- Prioritize user-facing service indicators first, especially error rate, latency, and availability;
- Define thresholds and evaluation windows that reduce flapping while still detecting real incidents quickly;
- Route alerts by severity and ownership, with escalation paths for unresolved critical events;
- Include actionable context in every alert payload, such as service, environment, current value, threshold, and dashboard link.

Cloud platforms include built-in tooling for alerting. On AWS, Amazon CloudWatch can route alarms through Amazon SNS. On Azure, Azure Monitor evaluates metric and log conditions and dispatches notifications through Action Groups. On Google Cloud, Cloud Monitoring supports threshold, query-based, and uptime alert policies.

In Kubernetes and self-managed platforms, Prometheus and Alertmanager provide evaluation, deduplication, silencing, escalation, and routing.

6.8.6.2 Prometheus

Prometheus is an open-source monitoring system with a dimensional data model, flexible query language, efficient time series database, and modern alerting approach. Prometheus scrapes metrics from a client (target) over HTTP and places the data into its time series database, which can be queried using its own DSL. Prometheus uses "exporters" that are installed/configured on the clients to convert and expose their metrics in a Prometheus-compatible format.

The Prometheus ecosystem consists of multiple components, many of which are optional. The Prometheus server scrapes and stores time series data. By default, it comes with a UI out of the box. Another key component is the "Service discovery" feature, which allows Prometheus to auto-discover targets in dynamic environments, such as distributed systems or microservice architectures. This eliminates the need to manually update a static list of addresses. Prometheus uses a separate component for alerting called the Alertmanager. It receives metrics from the Prometheus server, groups and processes them, and forwards alerts to the chosen notification system. Users can also build dashboards and integrate visualization tools such as Grafana or Kibana.

We can instantiate Prometheus from Docker image and connect it to the exported metrics of the Docker daemon. Prometheus needs to be configured appropriately so that it knows where to fetch the data from. Such configuration takes place via the *prometheus.yml* file. Below is an example configuration:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    monitor: 'codelab-monitor'
rule_files:
  - 'alert.rules'
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'docker'
    static_configs:
      - targets: ['localhost:9323']
```

In this configuration, Prometheus scrapes targets every 15 seconds. The `scrape_configs` section defines two `job_name` entries, namely, one for the Prometheus server at port 9090 and

another for the Docker engine at port 9323. Detailed instructions for configuring the Docker daemon as a Prometheus target can be found in the Collect Docker metrics with Prometheus page.

To start a Prometheus service using previous configuration, run the following command:

```
docker run -d -p 9090:9090 \
  -v $PWD/prometheus.yml:/etc/prometheus/prometheus.yml \
  -v $PWD/alert.rules:/etc/prometheus/alert.rules \
  prom/prometheus
```

After a few seconds, we can access the Prometheus web interface at http://localhost:9090. The custom Prometheus configuration and alert rules are optional and serve to demonstrate the concepts described.

6.8.6.3 Alertmanager

Alertmanager is the alert handling component in the Prometheus ecosystem. Prometheus evaluates alerting expressions and sends firing alerts to Alertmanager, which then groups, deduplicates, silences, and routes notifications to channels such as email, Microsoft Teams, or Slack.

Prometheus alerting rules need to be added to an *alert.rules* file. Alerting rules allow us to define alerts based on metric conditions. When a rule fires, Prometheus forwards that alert to Alertmanager for notification routing. Below is an example of a simple *alert.rules* file:

```
groups:
  - name: example
    rules:
      - alert: InstanceDown
        expr: up == 0
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Instance {{ $labels.instance }} is down"
          description: "{{ $labels.instance }} has been unreachable for more
than 5 minutes."
```

In this example, the `InstanceDown` alert triggers when the `up` metric equals 0 for 5 minutes, indicating that an instance is down. The `labels` section categorizes the alert, and the `annotations` section provides additional context for the alert notification.

We can find several alert rule examples on the Awesome Prometheus Alerts Rules site.

6.8.7 Dashboards

6.8.7.1 Definition

Dashboards are the operational view layer for observability. They turn metrics, logs, and traces into visual context so teams can quickly understand system health, detect anomalies, and prioritize response. Effective dashboards summarize the current state while allowing fast drill-down during incidents.

For Angular and other web applications, dashboards should combine user-experience and platform signals in one place. Typical views include traffic, error rate, latency, Core Web Vitals, API performance, cache behavior, container health, and resource saturation. Bringing these layers together helps teams connect frontend symptoms to backend and infrastructure causes.

Practical action points for dashboards in Angular and container platforms:

- Start with a small set of service-level indicators (availability, latency, error rate, and traffic) before adding deeper panels;
- Use consistent labels, variables, and naming conventions so dashboards are reusable across environments and services;
- Add links from dashboard panels to logs, traces, and runbooks to speed up incident investigation;
- Store dashboards as code so changes are reviewable, versioned, and reproducible across environments.

Cloud platforms provide managed dashboard services integrated with native monitoring stacks. Amazon CloudWatch, Azure Monitor, and Cloud Monitoring support custom visualizations, shared dashboards, and environment-scoped filtering for production operations.

For teams that prefer open-source tooling or need multi-cloud visibility, Grafana is the de facto standard for dashboard creation and visualization.

6.8.7.2 Grafana

Grafana is a multi-platform open-source analytics and interactive visualization web application. It provides charts, graphs, and alerts for the web when connected to supported data sources. It is expandable through a plug-in system. End users can create complex monitoring dashboards using interactive query builders.

Grafana transforms multiple feeds from Prometheus into metrics in a centralized dashboard. To run Grafana in Docker and integrate it with Prometheus, start with the following command:

```
docker run -d -p 3000:3000 grafana/grafana
```

Open browser to port 3000 and login with default admin credentials. Once logged in, click on the Grafana icon in the top left corner to show the main menu. Select *Data Sources* to navigate to the data source list page. Click *Add New* in the top navigation bar and fill in the form with the values for Prometheus. Using the *Save & Test* option will verify that we can connect to the Prometheus instance.

With data sources configured and checked, we can now import dashboards for Grafana. We can either import it from Grafana Dashboards or we can use the JSON file option.

6.8.7.3 Dashboards for Nginx

To build a useful Grafana dashboard for Nginx in containerized environments, first ensure Nginx containers expose metrics and that Prometheus scrapes both Nginx and container-level metrics.

In Grafana, create a dashboard scoped by labels such as `container`, `pod`, `namespace`, or `service` to filter per environment, and add panels for request rate, 4xx/5xx error rate, request latency (`p50/p95/p99`), active connections, upstream response time, and traffic volume (bytes in/out). It is also recommended to include resource and reliability context in the same view, such as container CPU/memory usage, restart count, and basic availability checks, because these help explain spikes in latency or errors.

To accelerate adoption, download and import community dashboards from Grafana Dashboards and then adapt variables and thresholds to team-specific labels and SLOs.

6.8.8 Best Practices

Understanding the state of infrastructure and systems is essential for ensuring the reliability and stability of services. One of the best ways to gain this insight is with a robust monitoring system that gathers metrics, visualizes data, and alerts operators when things appear to be broken. Here is a proposal list of best practices for Monitoring and Alerting:

- ☐ **Define monitoring scope and system boundaries**: We need to estimate the scope of the Continuous Monitoring solution, including the systems in the infrastructure. In addition, we should also estimate the systems that should be a part of the monitoring processes.

- ☐ **Perform risk evaluation and asset classification**: We should implement a risk assessment for different assets. In addition, we can also classify assets based on risk and the estimated impact of service failure or data breach.
- ☐ **Prioritize critical services, metrics, and alerts**: Before establishing threshold and notification settings, we should first determine which are the most important. It´s recommend listing these in descending order, starting with what could potentially put the job at risk and finishing with minor issues.
- ☐ **Configure monitoring and log aggregation tools**: The configuration is essential to obtain data from applications. These tools incorporate a feature called log aggregation that collects log data from different applications deployed on the infrastructure.
- ☐ **Apply the RED Method for service monitoring**: There are countless metrics that can be chosen for monitoring, and it can be overwhelming to separate the signal from the noise coming in. The RED Method provides a general framework for monitoring the health of a request-based service via three metrics: Rate, Errors and Duration.
- ☐ **Define SLIs, SLOs, SLAs and error budgets**: Service-level indicators and objectives should be explicit for user-critical journeys. Use error budgets to balance reliability work and feature delivery, and align alert thresholds to service-level violations.
- ☐ **Monitor availability with uptime checks**: We can use uptime monitoring tools. This gives us a dashboard of the uptime checks for the site, whether the checks succeeded, and how long they've taken, plus statistics about service uptime.
- ☐ **Use synthetic monitoring for critical user journeys**: Monitor key paths such as login, checkout, and API dependencies from multiple locations. Synthetic tests detect regressions that basic endpoint checks may miss.
- ☐ **Implement Application Performance Monitoring**: Application Performance Monitoring (APM) can provide request and response information, database connection information, remote profiling and tracing for slow spots, and other metrics related to the health of services.
- ☐ **Correlate logs, metrics, and traces with shared context**: Propagate trace or correlation IDs across frontend, edge, and backend components so teams can investigate incidents across systems using a single identifier.
- ☐ **Monitor logs with actionable event standards**: Logs are generated all the time and provide more information than metrics. The first key to good logging is to make logs useful. To do that, only log actionable events that machines or humans can use.

- ☐ **Standardize telemetry naming and labels**: Use consistent metric names, dimensions, severity levels, and environment labels across services to improve dashboard reuse, alert accuracy, and cross-team troubleshooting.
- ☐ **Define retention, privacy, and sampling policies**: Establish how long logs, metrics, and traces are stored, what sensitive data must be redacted, and where sampling is required to control cost without losing incident visibility.
- ☐ **Create clear and actionable dashboards**: Dashboards should combine user-experience and platform signals in one place. Make sure that dashboards include a clear and organized visual representation of the health of the system.
- ☐ **Use vendor support and managed expertise**: Make the most of support offered by software vendor. Recommended when applicable.
- ☐ **Align alerting channels with the target audience**: Monitoring tools that provide a wide range of alerting methods will ensure that, when that alert comes there will be someone there to hear it.
- ☐ **Maintain runbooks and escalation paths for alerts**: Every high-severity alert should map to an owner, a clear runbook, and a defined escalation route to reduce mean time to acknowledge and resolve incidents.
- ☐ **Measure alert quality and reduce noise**: Track false positives, duplicate alerts, and flapping rules. Regularly tune conditions and windows so alerts remain actionable and do not create fatigue.
- ☐ **Monitor the monitoring system itself**: Do not assume that no alerts mean nothing is wrong. There could be a problem with the monitor itself, or the settings.
- ☐ **Prefer buying monitoring solutions over building**: This tricky one attracts contention and debate. While it's tempting to avoid the cost and risk of being locked in with a vendor by building in-house, modern IT systems are complex, so it's far safer to buy IT monitoring tools.
- ☐ **Review metrics and thresholds regularly**: The thresholds we establish might be too high or too low. If we're receiving too many alerts or we aren't notified when a legitimate issue occurs, it's time to review the threshold settings.
- ☐ **Test alerting workflows proactively**: We shouldn't find out whether the alert system is effective only once a critical issue has arisen. Conduct tests so the robustness and effectiveness of the system is certified before an issue occurs.

- ☐ **Run post-incident reviews**: After incidents, identify missing telemetry, unclear dashboards, and weak alerts. Convert findings into tracked actions so the monitoring system improves over time.
- ☐ **Run Continuous Auditing regularly**: The continuous audit is one of the supporting continuous monitoring of best practices that improve the overall process. We should evaluate the constantly emerging and centralized information every day on a consistent basis.
- ☐ **Document the monitoring system**: It is crucial to document exactly how the monitoring tool has been set up and why. Make sure that this documentation can be editable in real-time and easily accessible for all team members.

6.9 Web Security

6.9.1 Fundamentals

Web security encompasses a range of solutions and policies that organizations rely on to protect their web applications, APIs, users, and supporting infrastructure against unauthorized access, data exposure, tampering, service disruption, and abuse. In practice, it is not a single feature but a combination of controls applied across the browser, application code, API layer, hosting platform, and delivery pipeline.

Many of the most damaging weaknesses in modern systems come from broken access control, insecure defaults, excessive data exposure, weak secret handling, vulnerable dependencies, and misconfigured infrastructure rather than from a single dramatic exploit. This is why security work must cover both application design and operational hardening, namely, encrypt traffic with TLS, limit privileges, isolate services, keep software patched, verify build artifacts, scan dependencies, and monitor logs and telemetry for signs of abuse or compromise.

Web security should therefore be treated as a continuous engineering practice rather than a final review step before release. Threats change over time, dependencies evolve, cloud configurations drift, and new features create new trust boundaries. Effective teams design security requirements early, verify them during development with code review and automated testing, enforce them in CI/CD with dependency scanning and policy checks, and validate them in production through logging, monitoring, alerting, and periodic security assessments.

For Angular applications and other SPA architectures, one of the main concerns is that the browser is an untrusted execution environment. Application code runs on devices the team does

not control, communicates over networks that may be observed or manipulated if transport security is weak, and depends on content delivered through CDNs, APIs, and third-party scripts. This makes browser-side threats such as Cross-Site Scripting, Cross-Site Request Forgery, clickjacking, insecure client-side storage, and token theft especially important. At the same time, a frontend application cannot enforce security on its own. Client-side validation improves usability, but real protection still depends on server-side authentication, authorization, input validation, secure session management, and correctly configured HTTP protections such as Content Security Policy, HTTP Strict Transport Security, cookie attributes, and strict CORS rules.

Standards and guidance such as the OWASP Top 10, OWASP ASVS, OWASP Cheat Sheet Series, OWASP API Security Top 10, web.dev secure, and browser-focused documentation from MDN Web Docs provide a useful vocabulary for identifying the main concerns and choosing proportionate controls.

6.9.2 OWASP Top 10

The OWASP Top 10 is the most widely used awareness document for common and high-impact web application security risks. It is not a complete security standard and it does not replace a formal verification framework such as OWASP ASVS, but it gives teams a practical way to recognize the classes of weaknesses that appear most often in real systems.

For DevSecOps teams, the value of the OWASP Top 10 is not limited to backend code review. It helps shape frontend architecture, API design, reverse proxy configuration, session handling, logging, dependency governance, hosting platforms, and CI/CD security controls. It is best treated as a risk taxonomy that highlights where to look first rather than as a checklist that can be ticked off once and forgotten.

The OWASP Top 10:2025 categories are:

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures

9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

Several of these categories directly correspond to concerns addressed in this chapter. Broken access control and authentication failures affect how APIs authorize users and how tokens, sessions, and roles are managed. Cryptographic failures relate to TLS, key handling, and protection of data in transit and at rest. Injection remains relevant not only for SQL and command execution on the server, but also for browser-facing risks such as Cross-Site Scripting when untrusted data reaches unsafe rendering paths. Security misconfiguration appears at the infrastructure layer through overly permissive CORS rules, missing security headers, exposed admin endpoints, weak container defaults, or incorrect CDN and reverse proxy behavior.

In practice, the most effective use of the OWASP Top 10 is to translate each category into concrete engineering controls. That means enforcing server-side authorization for every protected action, using secure defaults in reverse proxies and cloud services, validating and encoding all untrusted input, scanning dependencies, protecting build and release integrity, centralizing security-relevant logs, and testing regularly for both application and infrastructure weaknesses.

6.9.3 Best Practices

The following recommendations are widely accepted web security best practices for websites, web applications, and APIs:

- ☐ **Use Multi-Factor Authentication**: Require MFA for administrators and privileged users, and strongly encourage it for all user accounts. Prefer phishing-resistant factors (for example, FIDO2/WebAuthn security keys) over SMS whenever possible.
- ☐ **Implement strong password policies**: Enforce long passphrases, block breached/common passwords, and rate-limit login attempts. Store password hashes using modern algorithms such as Argon2id, bcrypt, or scrypt with per-password salts and appropriate work factors.
- ☐ **Apply the Principle of Least Privilege**: Grant only the minimum permissions required for each user, service account, and runtime identity. Review and remove unused permissions regularly to reduce blast radius.
- ☐ **Use Role-Based or Attribute-Based Access Control**: Start with RBAC for clear entitlement boundaries and adopt ABAC where context-aware rules are needed. Always enforce authorization server-side, not only in frontend route guards.

- ☐ **Use HTTPS everywhere**: Encrypt all traffic with TLS, redirect HTTP to HTTPS, and enable HSTS with suitable max-age and subdomain coverage. Decommission weak ciphers and old protocol versions.
- ☐ **Encrypt sensitive data**: Encrypt data in transit and at rest with strong, current algorithms. Classify data first so encryption requirements match sensitivity and compliance obligations.
- ☐ **Implement robust key management**: Keep keys in managed KMS/HSM systems, rotate them regularly, and restrict key usage by policy. Never hardcode encryption keys in source code or container images.
- ☐ **Protect application secrets**: Store API keys, tokens, and credentials in secret-management systems and inject them at runtime. Prevent secret exposure with repository scanning, least-privilege access, and short-lived credentials.
- ☐ **Avoid unsafe deserialization**: Avoid deserializing untrusted data formats unless absolutely necessary. If required, validate payload structure strictly and use safe serializers with constrained type handling.
- ☐ **Validate and sanitize all inputs**: Treat every external input as untrusted, validate against strict allowlists, and enforce schema validation at API boundaries. Apply output encoding per sink to reduce injection risk.
- ☐ **Prevent Cross-Site Scripting**: Use framework-safe rendering patterns, contextual output encoding, and sanitization for untrusted HTML. Add CSP and Trusted Types as defense-in-depth against DOM XSS.
- ☐ **Prevent Cross-Site Request Forgery**: Use anti-CSRF tokens (or double-submit cookie patterns), set cookies with proper SameSite policies, and verify origin/referer where appropriate. Keep CSRF protection enabled on all state-changing endpoints.
- ☐ **Implement secure file uploads**: Restrict file type, size, and content. Validate MIME and magic bytes, and scan uploads for malware. Store uploads outside executable paths and serve them from isolated domains or storage.
- ☐ **Set secure cookie attributes**: Mark sensitive cookies as `Secure`, `HttpOnly`, and `SameSite` (`Lax` or `Strict` as feasible). Scope cookie domain/path narrowly and avoid exposing session tokens to JavaScript.
- ☐ **Use secure session management**: Use short-lived session identifiers or access tokens, regenerate session IDs on authentication events, and revoke sessions on logout or suspicion of compromise. Enforce idle timeout and absolute session lifetime.

- ☐ **Use Strong Authentication for APIs**: Prefer OAuth 2.0/OIDC with scoped tokens and well-defined audiences. Validate token issuer, audience, signature, and expiry on every protected endpoint.
- ☐ **Protect sensitive API endpoints**: Enforce authentication and authorization checks consistently at every sensitive API boundary. Add object-level and function-level access control checks to prevent horizontal and vertical privilege escalation.
- ☐ **Implement rate limiting**: Apply per-IP, per-user, and per-token throttling, with stricter controls for auth and high-risk endpoints. Combine rate limiting with bot detection and anomaly rules.
- ☐ **Configure strict CORS**: Use explicit origin allowlists, methods, and headers, especially for credentialed requests. Avoid wildcard CORS on authenticated APIs.
- ☐ **Use Security Headers**: Configure `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`, and `Referrer-Policy`. Validate headers continuously in staging and production.
- ☐ **Segment networks and isolate services**: Separate public ingress from internal services and sensitive data stores. Enforce deny-by-default network policies and reduce lateral movement paths.
- ☐ **Use a Web Application Firewall**: Use WAF controls to block common attacks such as XSS, injection, and protocol abuse before they reach the app. Keep signatures, managed rules, and custom detections updated.
- ☐ **Use API Gateways for security management**: Place APIs behind gateways to centralize authentication, authorization, throttling, request validation, and observability. This reduces direct exposure of backend services.
- ☐ **Enable comprehensive logging and monitoring**: Log security-relevant events such as authentication failures, privilege changes, sensitive data access, and suspicious API calls. Centralize logs in SIEM/observability tools and wire actionable alerts.
- ☐ **Protect log integrity and retention**: Prevent log tampering with append-only storage, signatures, or immutable retention controls. Define retention and access policies aligned with legal and incident-response needs.
- ☐ **Perform regular vulnerability scanning**: Run authenticated infrastructure and application scans continuously, including container, host, and cloud configuration checks. Triage findings by exploitability and business impact.

- ☐ **Conduct Penetration Testing**: Perform periodic manual penetration tests and targeted red-team exercises for critical systems. Feed findings into backlog remediation with clear ownership and due dates.
- ☐ **Use SAST, DAST, IAST, and SCA**: Combine static, dynamic, interactive, and dependency analysis to improve vulnerability coverage. Integrate these checks into CI/CD with quality gates for critical issues.
- ☐ **Secure third-party libraries and dependencies**: Use trusted, actively maintained packages, keep dependencies current, and remove unused libraries. Continuously monitor CVE feeds and advisories for transitive dependencies.
- ☐ **Strengthen supply chain security**: Generate SBOMs, verify package provenance/signatures, and use trusted registries. Defend against dependency confusion and typosquatting with scoped registries and repository policy controls.
- ☐ **Automate security in CI/CD pipelines**: Shift security left with policy-as-code, secret scanning, dependency checks, IaC scanning, and container image scanning. Protect pipelines with least-privilege identities, branch protections, and artifact signing.
- ☐ **Integrate DevSecOps practices**: Embed security checks and approval gates throughout planning, build, release, and operations workflows. Keep feedback loops short so teams fix issues before production.
- ☐ **Implement a secure SDLC**: Define security requirements at design time, apply secure coding standards, and enforce security reviews at each delivery stage. Treat security as a release criterion, not an optional activity.
- ☐ **Practice threat modeling continuously**: Identify assets, trust boundaries, abuse cases, and likely attacker paths early. Revisit threat models when architecture, integrations, or business flows change.
- ☐ **Practice OWASP Top 10 and ASVS mapping**: Translate OWASP risk categories into concrete engineering controls and test cases. Use ASVS levels to drive consistent verification depth.
- ☐ **Create and test an incident response plan**: Define detection, containment, eradication, recovery, and communication procedures for security incidents. Run tabletop and live exercises to validate operational readiness.
- ☐ **Perform regular Risk Assessments**: Evaluate technical and business risk on a recurring cadence and whenever major changes ship. Prioritize mitigation using likelihood, impact, and exploitability.

- ☐ **Meet compliance and audit requirements**: Map controls to applicable standards (for example, ISO 27001, GDPR, NIST). Maintain evidence through continuous controls monitoring and periodic audits.
- ☐ **Provide security awareness training**: Train developers, operators, and reviewers on secure coding, API security, threat modeling, and incident response. Reinforce learning with hands-on exercises and security-focused peer reviews.
- ☐ **Design for security from the start**: Incorporate security requirements into architecture, implementation, testing, and operations from day one. Security-by-design reduces rework and lowers long-term risk.

6.10 Angular-Specific Security Strategies

6.10.1 Preventing Cross-Site Scripting

6.10.1.1 Cross-Site Scripting Security Model

Cross-Site Scripting (XSS) allows attackers to inject malicious code into web pages in order to steal data or impersonate users. Angular reduces this risk by treating template-bound values as untrusted by default, escaping and sanitizing them before they reach the DOM, and pairing those protections with platform controls such as Content Security Policy (CSP) and Trusted Types. See Preventing cross-site scripting and Angular's cross-site scripting security model for more details.

Angular-specific guidance for XSS prevention:

- **Keep templates static and trusted**: Treat template source as trusted code and bound values as untrusted data. Never generate templates by concatenating user input with template syntax, because that creates a code-injection path.
- **Validate browser messaging**: Verify the `origin` of incoming `postMessage` events against a strict allowlist and set an explicit `targetOrigin` when sending messages. Treat `postMessage` channels as untrusted input boundaries equivalent to HTTP endpoints.
- **Harden web worker boundaries**: Treat data exchanged with Web Workers through `postMessage` as a serialization boundary. Validate inputs inside the worker and avoid passing sensitive tokens into worker contexts.

6.10.1.2 Sanitization and Security Contexts

Sanitization inspects an untrusted value and turns it into a value safe for insertion into the DOM. Angular sanitizes values according to their context and provides APIs for cases where automatic sanitization cannot be used. Angular defines four security contexts, namely, `HTML` (used when binding to `innerHTML`), `Style` (used when binding CSS into `style`), `URL` (used for URL properties such as `<a href>`), and `Resource URL` (a URL loaded and executed as code, such as `<script src>`). Angular sanitizes untrusted values for HTML and URLs automatically. Resource URLs cannot be sanitized because they contain arbitrary code. See Sanitization and security contexts for more details.

6.10.1.3 Direct DOM APIs and Explicit Sanitization

Unless Trusted Types are enforced, built-in browser DOM APIs such as `document`, the node available through ElementRef, and third-party libraries do not protect against XSS automatically. Avoid direct DOM interaction and prefer Angular templates. When unavoidable, sanitize with DomSanitizer.sanitize() and the appropriate SecurityContext. See Direct use of the DOM APIs and explicit sanitization calls for more details.

6.10.1.4 Trusting Safe Values

When an application genuinely needs to include executable code, display an `<iframe>` from a URL, or construct a potentially dangerous URL, automatic sanitization must be bypassed explicitly. These calls should be treated as security-sensitive code paths and kept as rare, documented exceptions. To bypass sanitization, inject DomSanitizer and call `bypassSecurityTrustHtml`, `bypassSecurityTrustStyle`, `bypassSecurityTrustUrl`, `bypassSecurityTrustScript`, or `bypassSecurityTrustResourceUrl`. These methods should only be used for values already verified as safe. See Trusting safe values for more details.

6.10.1.5 Content Security Policy

Content Security Policy (CSP) is a platform-level control that provides defense-in-depth against XSS at the browser level, independent of Angular's own sanitization. It restricts which scripts the browser allows, complementing Angular's output encoding. See Content security policy for more details.

Recommendations for setting up and enforcing CSP in Angular applications:

- **Configure CSP headers**: Configure the web server to return a `Content-Security-Policy` header. The minimal policy for a new Angular application is `default-src 'self'; style-src 'self' 'nonce-randomNonceGoesHere'; script-src 'self' 'nonce-randomNonceGoesHere';`. Provide the nonce to Angular using the `autoCsp` workspace option, the `ngCspNonce` attribute on the root element, or the CSP_NONCE injection token. Always use unique, unpredictable nonces per request. When behind a CDN, generate nonces at the edge layer rather than the origin.
- **Control third-party script loading at runtime**: Establish a governance process for onboarding external scripts that includes security review, CSP policy update, and Sub-Resource Integrity (SRI) hashes. Load third-party scripts asynchronously and in sandboxed contexts when possible.
- **Apply Permissions Policy**: Use the `Permissions-Policy` HTTP header (formerly `Feature-Policy`) to disable unused browser features such as `camera`, `microphone`, `geolocation`, `payment`, or `usb`, limiting capabilities available to injected or third-party scripts.

6.10.1.6 Enforcing Trusted Types

Trusted Types provide an additional layer of XSS protection at the DOM level by restricting which DOM sinks the browser allows, complementing Angular's output encoding. Angular defines several policies for enabling Trusted Types, namely, `angular` (required for Angular to function under enforcement), `angular#bundler` (used by the CLI for lazy chunk files), and `angular#unsafe-bypass` (required when `bypassSecurityTrust*` methods are used). Configure the corresponding `Content-Security-Policy: trusted-types` header in production serving infrastructure. See Enforcing Trusted Types for more details.

6.10.1.7 Using AOT Compilation

The AOT compiler prevents template injection vulnerabilities by compiling templates at build time, and greatly improves application performance. The AOT template compiler is the default compiler used by Angular CLI applications, and we should use it in all production deployments. See Use the AOT template compiler for more details.

An alternative to the AOT compiler is the JIT compiler which compiles templates to executable template code within the browser at runtime. Angular trusts template code, so dynamically generating templates and compiling them, in particular templates containing user data,

circumvents Angular's built-in protections. This is a security anti-pattern. For information about dynamically constructing forms in a safe way, see the Dynamic Forms guide.

6.10.1.8 Securing Server-Side Rendering

Server-side Rendering (SSR) introduces unique security concerns because HTML is constructed on the server and user input can reach the rendering pipeline. Injecting template code into an Angular application gives the attacker full control over it. Do not create Angular templates on the server using a templating language, as this introduces template-injection vulnerabilities. Keep all dynamic content in data bindings and never construct templates dynamically from user-supplied data in the SSR request handler. See Server-side XSS protection for more details.

Additional SSR-specific hardening measures:

- **Protect transfer state**: Angular escapes `TransferState` data embedded in `<script id="ng-state">` by default. Keep custom serialization logic from bypassing that protection, and do not place raw user input into transfer state without validation.
- **Generate CSP nonces per request**: Each SSR response should carry a unique, cryptographically random nonce injected into the `Content-Security-Policy` header and into Angular through the `CSP_NONCE` token or the `ngCspNonce` attribute. Do not reuse nonces across requests.
- **Prevent cross-request data leakage**: Avoid global singletons and module-scoped mutable state that can leak data between requests. Scope all user-specific state through Angular's dependency injection and use `REQUEST`/`RESPONSE` tokens from `@angular/ssr`.
- **Apply response cache isolation for authenticated content**: Set `Cache-Control: private, no-store` on SSR responses that contain personalized content. Use `Vary` headers to separate cache entries by authentication state and never cache user-specific HTML without explicit cache-key segmentation.

6.10.2 Preventing HTTP-level Vulnerabilities

6.10.2.1 Preventing Cross-Site Request Forgery

Cross-site request forgery (CSRF/XSRF) tricks authenticated users into unknowingly submitting requests that the server cannot distinguish from legitimate ones. The standard defense is a cookie-to-header token pattern because the server sets a randomly generated token in a cookie, the client reads it and sends it as a custom request header, and the server rejects requests where

the values do not match. This works because same-origin policy prevents attacker-controlled pages from reading cookies or setting custom headers for another domain. See Cross-site request forgery for more details.

Angular provides built-in client-side support for CSRF protection through its HttpClient:

- **HttpClient XSRF/CSRF security**: Angular's HttpClient includes an interceptor that reads a token from a cookie (by default `XSRF-TOKEN`) and sets it as an HTTP header (`X-XSRF-TOKEN`) on all mutating requests to relative and same-origin URLs. Then, the backend must set the token cookie and validate the header on each eligible request.
- **Configure custom cookie/header names**: Use withXsrfConfiguration when backend conventions differ from the defaults. To prevent collisions when multiple Angular applications share the same domain or subdomain, give each application a unique cookie name.

6.10.2.2 Preventing Cross-Site Script Inclusion

Cross-Site Script Inclusion (XSSI) allows an attacker's website to read data from a JSON API by overriding built-in JavaScript object constructors and including the API URL with a `<script>` tag. The attack succeeds only if the returned JSON is executable as JavaScript. Servers prevent this by prefixing all JSON responses with the well-known string `")]}',\n"` to make them non-executable. Angular's HttpClient recognizes this convention and strips the prefix automatically before parsing. Enable server-side JSON prefixing for sensitive endpoints and treat XSSI protection as defense-in-depth alongside CORS, authentication, and authorization.

6.10.2.3 Authentication, Authorization, and Session Security

Authentication, authorization, and session management are frequently under-specified in SPA architecture documents, yet they are among the most important controls in Angular platforms. This is especially true for how tokens are stored and transmitted, which is a common source of frontend security incidents. The following practices reduce the attack surface across identity, sessions, and credential storage:

- **Use standards-based authentication**: Prefer OAuth 2.1/OIDC with Authorization Code + PKCE for browser applications instead of custom login or token flows.
- **Keep authorization server-side**: Route guards improve UX but are not security boundaries. Enforce role- and permission-based authorization in backend APIs.

- **Apply least privilege**: Request only required scopes and permissions, and separate admin capabilities from standard user paths.
- **Harden session lifecycle**: Implement idle timeout, absolute session expiration, and token revocation for compromised sessions.
- **Session tokens in cookies**: Prefer HttpOnly, secure cookies when the architecture allows it, reducing JavaScript access to credentials.
- **Long-lived token storage**: Avoid storing long-lived tokens in `localStorage` for sensitive applications, because any successful XSS can read them.
- **Token lifetime strategy**: Use short-lived access tokens and rotating refresh tokens to limit replay windows.
- **Protect cookies correctly**: Set `Secure`, `HttpOnly`, and `SameSite` attributes, and use HTTPS end-to-end.
- **Token logging**: Do not log tokens in browser logs, network error messages, or observability payloads.
- **Prevent open redirect through Angular Router**: Validate redirect target URLs before navigating. Reject absolute URLs pointing to external domains unless they appear in an explicit allowlist, and prefer relative paths with `Router.navigate()` or `Router.navigateByUrl()`.
- **Harden service worker lifecycle**: Restrict service worker scope to the minimum necessary path, serve the script with `Cache-Control: no-cache`, and implement a kill-switch to force-unregister compromised versions. Ensure the precache manifest only includes expected application assets.

6.10.2.4 Securing APIs and Configuring CORS

Angular security also depends on API boundary controls, especially in headless and microservice architectures where the frontend communicates with multiple backend services. Proper configuration at the API layer prevents unauthorized access and data leakage:

- **Use strict CORS allowlists**: Allow only trusted origins, methods, and headers. Avoid wildcard origins for authenticated endpoints.
- **Validate all input on the server**: Client-side validation improves usability but does not protect backend resources.
- **Apply API protections**: Use schema validation, rate limiting, request size limits, and consistent error handling that avoids leaking internal details.

- **Use TLS everywhere**: Use TLS between browser, edge, and service tiers to prevent token and data exposure in transit.

6.10.2.5 HTTP Security Headers and Browser Isolation

HTTP security headers harden delivery at the reverse proxy layer and reduce classes of client-side attacks. The Nginx example below combines SPA routing with a practical production baseline that covers transport security, content-type enforcement, clickjacking prevention, and referrer control.

```
server {
  location / {
    try_files $uri $uri/ /index.html;

    add_header Content-Security-Policy "default-src 'self'; script-src 'self';
style-src 'self'; img-src 'self' data:; font-src 'self' data:" always;
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
  }
}
```

Beyond the baseline described above, additional browser isolation measures can further reduce the remaining client-side attack surface. These include preventing framing with `frame-ancestors 'none'` in the CSP and `X-Frame-Options: DENY`, adopting cross-origin isolation headers where applicable, restricting embedded third-party content, and aligning these controls with CSP and Trusted Types.

6.10.3 Preventing Server-Side Request Forgery

Angular validates the `Host`, `X-Forwarded-Host`, `X-Forwarded-Proto`, `X-Forwarded-Prefix`, and `X-Forwarded-Port` headers strictly in the SSR/hybrid rendering request pipeline to prevent header-based SSRF. Invalid headers trigger an error log, and requests with unrecognized hostnames receive a `400 Bad Request` response. See Preventing Server-Side Request Forgery (SSRF) for more details.

Angular-specific settings for SSRF protection:

- **Configuring allowed hosts**: Define explicit allowed hostnames in `angular.json` under `security.allowedHosts` or when initializing AngularAppEngine / AngularNodeAppEngine. Wildcard patterns such as `*.example.com` are supported. For

Node.js deployments, the `NG_ALLOWED_HOSTS` environment variable (comma-separated) provides a runtime alternative. Using `*` to allow all hostnames is strongly discouraged and should only be used when host validation is enforced at a separate layer such as a load balancer or reverse proxy.

- **Configuring trusted proxy headers**: By default, Angular ignores all `X-Forwarded-*` headers. Enable `trustProxyHeaders` only when the application runs behind a trusted reverse proxy that strictly validates or overwrites these headers. Configure trusted headers when initializing AngularAppEngine or AngularNodeAppEngine, either as a list of specific headers or as `true` to trust all `X-Forwarded-*` headers.

6.10.4 Application Security Governance

6.10.4.1 Secure SDLC, Dependency Security, and Supply Chain

Dependency and supply chain controls reduce the risk of compromised packages reaching production. Angular applications rely on a deep dependency tree, making proactive management essential:

- **Establish dependency update policy**: Use regular patch windows and emergency vulnerability response runbooks.
- **Pin and verify CI/CD actions and build tooling**: Reduce pipeline compromise risk by pinning and validating build pipeline dependencies. Pin build tool versions, audit build dependencies separately from runtime dependencies, and restrict network access to only the required data sources.
- **Generate and track SBOMs**: Improve vulnerability response and compliance reporting across the dependency tree.
- **Sign build artifacts and container images**: Verify signatures during deployment to enforce integrity and provenance.
- **Prevent build-time secret leakage**: Ensure that environment variables, API keys, and service tokens used during pre-rendering are never serialized into the generated HTML or JavaScript bundles. Use CI/CD secret scanning tools to audit build output before publishing.
- **Review pre-rendered content for data sensitivity**: Verify that pre-rendered pages do not contain role-specific content, draft records, or user-specific data. Content that requires authorization should be loaded at runtime through authenticated API calls, not baked into static pages.

- **Validate SRI hashes for CDN-delivered assets**: Generate Sub-Resource Integrity (SRI) hashes for JavaScript and CSS bundles at build time and include them in `integrity` attributes so browsers reject tampered assets served from CDN or intermediate caches.

6.10.4.2 Security Auditing and Continuous Verification

Angular applications must follow the same security principles as regular web applications and should be audited accordingly:

- **Add static and dynamic security testing** (SAST/DAST) in CI/CD for both Angular code and deployed endpoints.
- **Conduct regular security audits**: Review Angular-specific patterns including `bypassSecurityTrust*` usage, direct DOM manipulation, dynamic template generation, and third-party library integrations.
- **Automate dependency vulnerability scanning**: Integrate tools such as `npm audit`, Dependabot, or Snyk into CI/CD pipelines to catch known vulnerabilities before deployment.
- **Verify security controls in production**: Periodically validate that CSP headers, Trusted Types, CORS policies, and other security configurations remain correctly applied across all environments.

6.11 Final Remarks

This chapter covered the core elements required to deliver and operate Angular applications in production, spanning hosting and cloud architecture, automation, and Infrastructure as Code. The key themes (observability, security, and cost management) are not afterthoughts but fundamental to reliable operations.

Building effective operations demands sustained investment in tooling, process, and culture. Cloud providers and open-source ecosystems offer mature solutions for most challenges, but teams must configure them thoughtfully, establish clear ownership, document decisions, and cultivate a culture in which infrastructure work is valued on an equal footing with application development. Infrastructure is not a fixed destination but an evolving practice shaped by business needs and technological change. By combining technical depth with sound governance and a commitment to continuous improvement, teams can build Angular applications that are resilient, secure, and ready to adapt to emerging challenges.

7 Web Application Performance

7.1 Overview

Web performance combines objective measurements with perceived user experience during load and runtime. Users perceive performance through waiting time, responsiveness, and visual smoothness, not through technical metrics alone. A web application that blocks the main thread, shifts layout unexpectedly, or delays feedback after a click feels unreliable even if it eventually completes the task.

This user perception maps to practical time thresholds that shape whether an experience feels instant, continuous, or frustrating. The table below summarizes these perception windows and explains why frontend performance work must be tied to user-centered outcomes.

Time window	User perception
0 to 100 ms	Respond to user actions within this time window and users feel like the result is immediate. Any longer, and the connection between action and reaction is broken.
100 to 1000 ms	Within this window, things feel part of a natural and continuous progression of tasks. For most users on the web, loading pages or changing views represents a task.
1000 ms or more	Beyond 1000 milliseconds (1 second), users lose focus on the task they are performing.
10000 ms or more	Beyond 10000 milliseconds (10 seconds), users are frustrated and are likely to abandon tasks. They may or may not come back later.

To evaluate and improve user experience, teams rely on frameworks such as Core Web Vitals, Lighthouse, and RAIL model.

This chapter introduces the concepts and techniques required to build high-performing Angular applications. The following sections connect browser fundamentals, Angular runtime mechanics, and architectural delivery choices to practical outcomes in responsiveness, stability, and Core Web Vitals.

7.2 Fundamentals

7.2.1 Core Web Vitals

Core Web Vitals are Google's user-centric performance metrics for page load speed, interaction responsiveness, and visual stability. They are measured from real users in the field and should be treated as product quality signals, not just technical KPIs.

The three Core Web Vitals metrics are:

- **Largest Contentful Paint (LCP)**: Measures loading performance by tracking when the largest visible element in the viewport is rendered (often an image, heading, or large content block). A good target is **2.5 seconds or less** at the 75th percentile.
- **Interaction to Next Paint (INP)**: Measures responsiveness by capturing how long it takes from a user interaction (click, tap, key press) to the next visual update. A good target is **200 ms or less** at the 75th percentile.
- **Cumulative Layout Shift (CLS)**: Measures visual stability by quantifying unexpected layout movement during the page lifecycle. A good target is **0.1 or less** at the 75th percentile.

These metrics directly impact user satisfaction and can influence search visibility. Optimizing Core Web Vitals usually improves both perceived quality and business outcomes.

7.2.2 Performance Metrics

7.2.2.1 Time to First Byte

Time to First Byte (TTFB) measures how quickly the first byte of the server response reaches the browser after a request starts. It reflects server processing, edge/CDN behavior, and network

latency before rendering can begin. High TTFB usually indicates backend, origin, or transport bottlenecks rather than frontend rendering issues.

7.2.2.2 First Contentful Paint

First Contentful Paint (FCP) measures when the browser first renders any content from the DOM (text, image, canvas, or SVG). It is a key signal that the page is no longer blank and loading is progressing. Improve FCP by reducing render-blocking resources and prioritizing critical rendering-path assets.

7.2.2.3 Largest Contentful Paint

Largest Contentful Paint (LCP) reports when the largest visible content element in the viewport is rendered. This is one of the most important user-centric load metrics and a Core Web Vital. Optimize LCP with fast server responses, efficient critical CSS, and prioritized delivery of media and primary content.

7.2.2.4 Interaction to Next Paint

Interaction to Next Paint (INP) measures the latency between a user interaction (click, tap, key press) and the next visual update. INP captures responsiveness across the full page lifecycle, not just at load time. Lower INP by reducing main-thread contention, long tasks, and unnecessary change detection work.

7.2.2.5 Cumulative Layout Shift

Cumulative Layout Shift (CLS) measures visual stability by quantifying unexpected layout movement during the page lifecycle. A high CLS creates poor UX through content jumps and accidental interactions. Prevent CLS by reserving space for images/embeds, avoiding late DOM insertions above visible content, and stabilizing font rendering.

7.2.2.6 Total Blocking Time

Total Blocking Time (TBT) is a metric that sums the portions of long tasks (over 50 ms) between FCP and Time to Interactive. It indicates how much time the main thread is blocked and unable to respond quickly to input. TBT is strongly correlated with interaction quality and is useful for diagnosing heavy JavaScript execution.

7.2.2.7 Time to Interactive

Time to Interactive (TTI) estimates when a page becomes reliably interactive, meaning it has rendered useful content and can respond to user input without major delay. While less emphasized than Core Web Vitals today, TTI still helps evaluate bootstrapping and JavaScript execution cost. Improve TTI by reducing startup JavaScript and minimizing long tasks during initialization.

7.2.2.8 Speed Index

Speed Index measures how quickly visible parts of the page are populated during load, based on visual progression over time. It complements milestone metrics like FCP and LCP by representing the overall rendering trajectory. A better Speed Index typically comes from progressive rendering and early delivery of above-the-fold content.

7.2.2.9 Page Weight

Page Weight represents the total transferred bytes required to load a page, including HTML, CSS, JavaScript, images, fonts, and third-party resources. Larger payloads increase download and parse/execute time, especially on mobile networks. Control page weight with compression, code splitting, image optimization, and dependency hygiene.

7.2.2.10 DOM Content Loaded

DOM Content Loaded (DCL) is the time it takes for the full HTML code of a webpage to be loaded. The faster this happens, the faster users can see useful functionality. This time doesn't include other resources such as CSS and images.

7.2.2.11 Request Count

Request Count is the total number of network requests required to load a route or page state. Excessive requests can create waterfall delays, connection contention, and higher protocol overhead. Reduce request count by bundling strategically, inlining truly critical assets, and eliminating unnecessary third-party calls.

7.2.2.12 Cache Hit Rate

Cache Hit Rate measures the percentage of requests served from cache (browser, CDN, or edge) instead of the origin. Higher hit rates reduce latency, origin load, and bandwidth costs while

improving repeat-visit experience. Improve this metric with content hashing, `Cache-Control` policies, and cacheable API/data strategies where freshness allows.

7.2.2.13 API Latency

API latency measures how long backend endpoints take to respond, typically tracked across percentiles such as p50, p75, p95, and p99. It directly impacts route activation time, loading states, and perceived responsiveness in Angular applications. Optimize by removing request waterfalls, improving query efficiency, and applying targeted caching.

7.2.2.14 Error Rate

Error rate is the proportion of failed requests or failed user interactions over total attempts in a time window. Even moderate latency looks worse to users when reliability drops, retries increase, or partial failures accumulate. Track error rate alongside latency and Web Vitals to evaluate true end-to-end experience quality.

7.2.3 Performance Insights

Performance Insights provide actionable insights on a website's performance and are available in both the DevTools Performance panel and Lighthouse panel. It provides a guided analysis layer built on top of Performance traces. Instead of only showing raw timeline events, they highlight likely bottlenecks (for example LCP delays, render-blocking requests, layout shifts, long tasks, and expensive selector work) and explain why each one matters for user experience.

Performance Insights is most effective when combined with Angular DevTools. Use Insights to locate browser-level bottlenecks, then use Angular DevTools to verify component-level change detection and rendering behavior behind those bottlenecks.

7.2.4 Performance Patterns

7.2.4.1 Optimize Loading Sequence

Loading sequence optimization is the practice of carefully selecting the order in which resources are requested and executed so that users see meaningful UI as quickly as possible. To achieve this, prioritize critical path assets such as HTML, critical CSS, route-critical JavaScript, and media so they are loaded first. Non-essential scripts, widgets, and below-the-fold assets should be delayed until after the initial render. Maintaining a shallow dependency graph helps avoid long request waterfalls that can delay interactivity.

7.2.4.2 Dynamic Import

Dynamic import enables loading JavaScript modules at runtime using `import()`, rather than shipping all code upfront. This pattern is especially effective for route-level features, admin-only functionality, heavy editors or charts, and locale packs. In Angular, dynamic imports work well in combination with lazy routes and `@defer` blocks to reduce the initial bundle size. Always handle loading and error states gracefully so that deferred chunks do not degrade the user experience, especially on slow networks.

7.2.4.3 Import On Visibility

Import on visibility is a pattern where code is loaded only when a component or section is about to enter the viewport. This can be implemented using `IntersectionObserver` or Angular's `@defer (on viewport)` triggers for content that appears near the fold. Preloading just before visibility helps mask network latency from the user. However, avoid using this pattern for above-the-fold or immediately interactive UI, as it can delay essential content.

7.2.4.4 Import On Interaction

Import on interaction defers code loading until the user initiates an action, such as clicking, focusing, or hovering. This approach is ideal for infrequently used features like advanced filters, help overlays, heavy modals, and analytics panes. Optionally, resources can be prewarmed on `mouseenter` or `focus` events to improve perceived responsiveness. Always provide fallback UI while the chunk is being fetched and initialized to maintain a smooth user experience.

7.2.4.5 Bundle Splitting

Bundle splitting divides a large JavaScript payload into smaller, independently loadable chunks. Splitting bundles by route and by feature ensures that users only download what is necessary for their current journey. It is important to keep chunk boundaries meaningful to avoid code duplication across many small bundles. Regularly review chunk composition with analyzer tools to prevent regressions and maintain optimal performance.

7.2.4.6 PRPL Pattern

The PRPL pattern (Push/Preload, Render, Pre-cache, Lazy-load) organizes resource loading to enable instant first interaction. Preload resources required for the initial route, render the shell and primary content as early as possible, and pre-cache assets for routes that are likely to be

visited next. Everything not required for the first meaningful interaction should be lazy-loaded to optimize both speed and efficiency.

7.2.4.7 Tree Shaking

Tree shaking is the process of removing unused code during the build, ensuring that dead code is not shipped to production. To maximize the benefits of tree shaking, prefer ES modules and design libraries to be free of side effects. Import only required symbols (e.g., `import { fn }`) instead of using broad namespace imports. Always validate the results with bundle analysis tools, as incorrect package metadata can limit the effectiveness of tree shaking.

7.2.4.8 Preload

Preload is a high-priority resource hint for assets that will be needed very soon in the current navigation. Use preload for critical fonts, images, and route-critical JavaScript chunks. However, avoid overusing preload, as too many high-priority requests can starve other essential work. Always align preload usage with actual critical-path measurements rather than assumptions.

7.2.4.9 Prefetch

Prefetch is a low-priority hint that instructs the browser to fetch resources likely to be needed for future navigations. Prefetch is best used for probable next routes, non-critical modules, and upcoming media. Trigger prefetching when the browser is idle or when there is a strong indication of navigation intent. Be selective with prefetching to avoid wasting bandwidth on low-probability paths.

7.2.4.10 Optimize Loading Third-Parties

Third-party optimization focuses on reducing the performance cost of analytics, ads, embeds, chat widgets, and tag managers. Load third-party scripts after core content unless they are strictly business-critical. Use the `async` or `defer` attributes, lazy mount embeds, and remove unused vendors to minimize their impact. Always monitor third-party scripts separately in performance traces, as they often dominate long tasks and can significantly affect user experience.

7.2.4.11 List Virtualization

List virtualization is a technique that renders only the visible rows and a small buffer, rather than the entire dataset. This is especially useful for long tables, feeds, and logs, as it reduces DOM size

and paint/layout costs. Pair virtualization with stable item heights or efficient measurement to ensure smooth scrolling. For very large back-end datasets, combine virtualization with pagination or incremental data fetching.

7.2.4.12 Compressing JavaScript

JavaScript compression reduces the transfer size of assets by using Brotli or Gzip before they are sent over the network. Prefer Brotli for modern browsers, while keeping Gzip as a fallback for compatibility. Serve precompressed assets from the CDN or server and verify the `Content-Encoding` header in production. Remember, compression complements minification and tree shaking, but does not replace them.

7.2.5 Event Loop

JavaScript is single-threaded, meaning it executes one piece of code at a time on a single call stack. However, browsers or Node.js handle network requests, timers, user interactions, and animations concurrently. The mechanism that makes this possible is the Event Loop. Understanding the Event Loop is foundational to web performance because user input, asynchronous callbacks, and visual updates all depend on how the web browser schedules work on the main thread.

At a high level, the Event Loop checks whether the call stack is clear and whether there are pending tasks in the task queue or microtask queue.

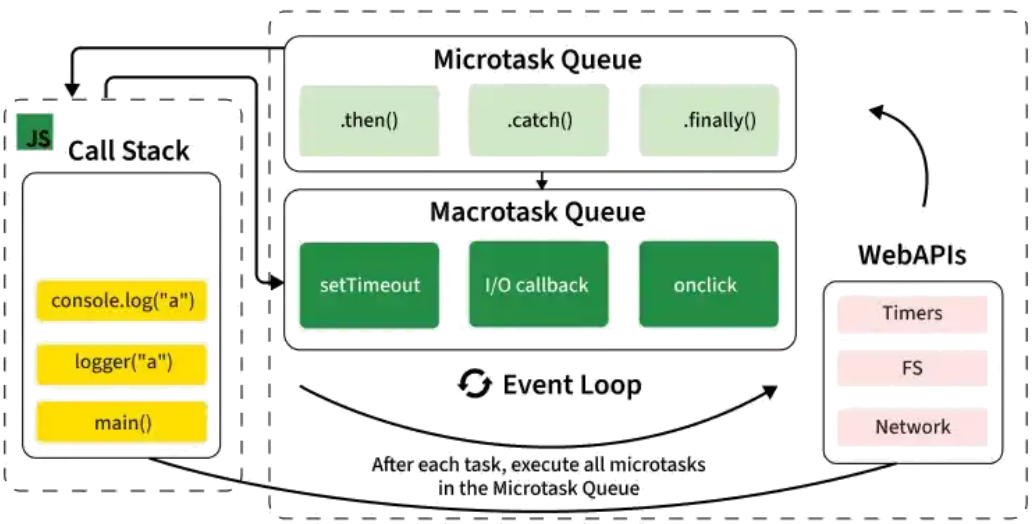


Fig. 1: Event Loop

The Event Loop continuously cycles through the following steps:

1. **Execute the current task** from the task queue (also called the macrotask queue). This includes script execution, `setTimeout/setInterval` callbacks, I/O callbacks, and UI event handlers.
2. **Drain the microtask queue** completely. After each task completes, the engine processes all pending microtasks before moving on. Microtasks include `Promise` callbacks (`.then`, `.catch`, `.finally`), `queueMicrotask()`, and `MutationObserver` callbacks.
3. **Render** (if needed). After the current task and all pending microtasks finish, the browser may take a rendering opportunity if a visual update is due. In that rendering phase, it can run `requestAnimationFrame` callbacks, recalculate styles, perform layout, paint, and composite.
4. **Pick the next task** and repeat.

The distinction between macrotasks and microtasks has direct performance implications:

- **Macrotasks** (`setTimeout`, `setInterval`, script execution, message events, I/O, UI event handlers) are picked one at a time from the queue. Between macrotasks, the browser may have an opportunity to render.
- **Microtasks** (`Promise.then`, `queueMicrotask`, `MutationObserver`) are drained completely before the browser can render or pick the next macrotask. A long chain of microtasks will block rendering just as effectively as a long synchronous function.

The Event Loop is the runtime scheduler that determines when the code runs, when the user sees updates, and when interactions feel responsive.

7.3 Angular-Specific Performance Strategies

7.3.1 Zone.js vs Zoneless

Angular's change detection is triggered and scheduled in response to asynchronous events and application state changes, with rendering occurring as part of Angular's synchronization process. Zone.js is a signaling mechanism that patches asynchronous APIs (`setTimeout`, `Promise`, event listeners) to notify Angular when async operations complete, triggering change detection.

The following options outline practical Angular strategies to improve web application performance:

- **Zoneless**: Angular relies on Signals and explicit scheduling, giving developers finer control over when change detection runs and reducing unnecessary re-renders.
- **`NgZone.runOutsideAngular()`**: Executes code without triggering change detection, which is useful for high-frequency events (mousemove, scroll, animation frames) that do not need to update Angular state on every invocation.

In large Angular applications with heavy interaction, zoneless change detection can reduce unnecessary global change detection cycles. Use zoneless mode when the team is comfortable with explicit reactive patterns. Prefer Signals, the `async` pipe, and explicit state transitions over imperative mutation.

7.3.2 Component Lifecycle

Angular components go through a predictable lifecycle from creation to destruction. Understanding each phase helps place logic in the right hook, avoid unnecessary work during change detection, and prevent memory leaks. Next image shows the lifecycle hooks.

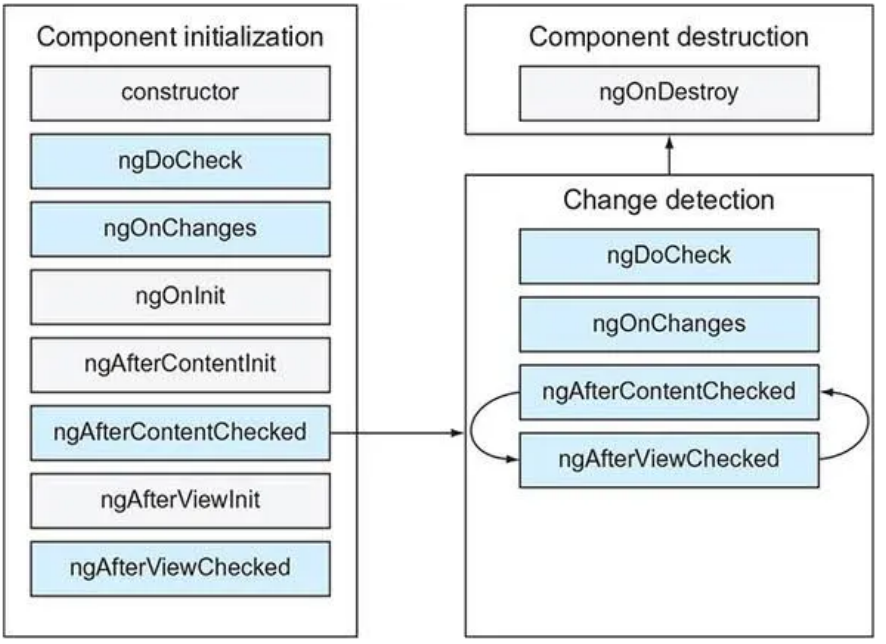


Fig. 2: Lifecycle hooks

Lifecycle hook responsibilities:

- **`constructor`**: Dependency injection and lightweight initialization only. Avoid side effects and expensive logic here.
- **`ngOnChanges`**: React to `@Input()` changes, normalize input data, or trigger recalculations that depend on parent-provided values.
- **`ngOnInit`**: Initialize component state, start data streams, and perform setup that should run once.
- **`ngDoCheck`**: Advanced custom change detection only. Keep this hook extremely cheap because it runs frequently.
- **`ngAfterContentInit/ngAfterContentChecked`**: Work with content projected via ng-`content`.
- **`ngAfterViewInit/ngAfterViewChecked`**: Access `@ViewChild/@ViewChildren` references and run logic that needs the view to be fully initialized.
- **`ngOnDestroy`**: Cleanup phase. Unsubscribe, stop timers, disconnect observers, and release resources.

Performance guidance:

- Prefer `OnPush` and immutable input patterns so hook executions happen only when meaningful state changes.
- Avoid triggering synchronous heavy computations inside view-check hooks and precompute with Signals `computed()` or memoized RxJS streams.
- Use `ngOnChanges` whenever an `@Input()` changes to trigger recalculations that depend on values provided by the parent component.
- Use teardown patterns (`takeUntilDestroyed`, `DestroyRef`, or explicit cleanup in `ngOnDestroy`) to avoid leaks and long-term runtime degradation.
- Keep hot-path hooks (`ngDoCheck`, `ngAfterViewChecked`, `ngAfterContentChecked`) minimal and deterministic to avoid slow computations in these hooks.

7.3.3 Change Detection Strategies

Angular's change detection mechanism is responsible for keeping the DOM in sync with the component state. Understanding and optimizing change detection is one of the most impactful architectural decisions for application performance.

The following example shows how to configure the change detection in components:

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class UserCardComponent {
  @Input() user!: User;
}
```

Angular provides two change detection strategies:

- **Default (or Eager)**: Angular checks the entire component tree on every change detection cycle. Simple but can become expensive in large applications.
- **OnPush**: Angular only checks a component when its inputs change (by reference), or an event is emitted from the component or its children, or an observable subscribed to via `async` pipe emits, or manual detection is triggered. `OnPush` should be the default choice for all presentational components.

7.3.4 Web Workers

Web Workers move JavaScript execution to background threads so CPU-intensive work does not block rendering or input handling on the main thread. In Angular, they are a strong option when expensive computation degrades interaction responsiveness. Web Workers are usually not the right tool for trivial logic, DOM access (not available in workers), or work dominated by network latency. Use workers as a throughput tool, not as a default abstraction.

Angular provides first-class setup via the CLI using `ng generate web-worker <name>`. This scaffolds a worker file and updates project configuration, so the worker is bundled correctly.

Rules to implement the communication model in Angular:

- Use `postMessage` and `onmessage` with small, explicit message contracts (`type`, `payload`, `requestId`) so calls are traceable and debuggable.
- Messages are structured-cloned by default. For large binary data, use Transferable Objects such as `ArrayBuffer` to avoid copy overhead.
- Encapsulate worker interaction in an Angular service and expose request/response streams via RxJS or Signals so components stay declarative.

Workers are ideal for large dataset filtering/sorting, encryption/hashing, media transformation, local indexing/search, and computationally expensive business rules. Rule of thumb: consider a worker when tasks frequently exceed the frame budget (about 16 ms) or create long tasks above 50 ms during user interactions. Validate impact with Chrome Performance traces and Angular DevTools before and after adoption.

7.3.5 Service Workers

Service Workers essentially act as proxy servers that sit between web applications, the browser, and the network (when available). In Angular, the built-in Angular Service Worker is focused on production caching for app-shell assets and controlled data freshness, which improves repeat-load speed, resilience on unstable networks, and offline behavior.

Angular provides first-class setup via the CLI. Add it with ng add @angular/pwa, which installs the package, enables service worker registration in production, creates/updates `ngsw-config.json`, and configures web app manifest assets.

For performance, treat service worker configuration as a cache policy design task:

- Cache critical static assets eagerly so repeat navigations and route transitions avoid network round trips.
- Cache non-critical assets lazily to avoid overfetching during first load.
- Use `dataGroups` with `performance` strategy for read-heavy, latency-sensitive APIs where slightly stale data is acceptable.
- Use `dataGroups` with `freshness` strategy for endpoints that must prioritize up-to-date responses.
- Define conservative `maxAge` and `maxSize` values so caches do not grow unbounded on long-lived clients.

When configured well, Angular service workers can reduce repeat-visit load times, smooth performance on high-latency networks, and improve perceived reliability without changing core UI code.

7.3.6 Lazy Loading

Lazy loading is a design pattern that delays the loading of modules until they are needed. Angular supports lazy loading through the router, where feature modules are loaded on demand when the user navigates to a specific route:

```
const routes: Routes = [
  {
    path: 'feature-a',
    loadChildren: () => import('./features/feature-a/feature-a.module')
      .then(m => m.FeatureAModule)
  },
```

```
  {
    path: 'feature-b',
    loadComponent: () => import('./features/feature-b/feature-b.component')
      .then(c => c.FeatureBComponent)
  }
];
```

With standalone components, `loadComponent` allows lazy loading at the component level, providing even more granular control over bundle splitting. Additional strategies include:

- **Preloading strategies**: Angular provides PreloadAllModules and custom preloading strategies to load lazy modules in the background after the initial application has loaded.
- **Deferred loading**: Angular provides `@defer` blocks that enable template-level lazy loading, loading components and their dependencies only when specific conditions are met (viewport visibility, interaction, idle time, or custom triggers).

7.3.7 Bundle Optimization

Minimizing the JavaScript bundle size directly impacts initial load time and Core Web Vitals. Angular provides several built-in and configurable optimization techniques:

- **Tree shaking**: Angular's production build automatically removes unused code. Ensure libraries are tree-shakable and avoid side effects in module imports.
- **Code splitting**: The Angular CLI automatically splits the bundle at lazy-loaded route boundaries. `@defer` blocks provide sub-route code splitting at the template level.
- **Build optimizer**: Angular's build optimizer removes Angular-specific decorators and applies advanced dead-code elimination.
- **Budget policy**: Define and enforce size thresholds in `angular.json` using Angular budgets so builds fail fast on regressions.
- **Bundle analysis**: Use webpack-bundle-analyzer, source-map-explorer or Bundlephobia to identify large dependencies and optimization opportunities.

7.3.8 Runtime Performance

Runtime performance affects the user experience during navigation and interaction. Angular recommends several techniques for optimizing runtime performance:

- **Virtual scrolling**: The Angular CDK's `CdkVirtualScrollViewport` renders only the visible items in long lists, maintaining performance with thousands of items.
- **`@for track`**: When rendering lists, always provide a tracking function so Angular can efficiently update DOM elements instead of recreating them.

- **`@defer blocks`**: Angular deferred views load components lazily based on triggers (viewport, interaction, idle, timer), reducing the initial rendering workload.
- **Pure pipes**: Angular reevaluates a pure pipe only if it detects that its inputs have changed, compared to the previous time Angular called it.
- **Web Workers**: Offload computationally heavy tasks (data processing, encryption, complex calculations) to Web Workers to keep the main thread responsive.
- **`NgOptimizedImage`**: Angular's image directive provides automatic lazy loading, `srcset` generation, and preconnect hints for optimized image delivery.
- **Memoization**: Use `computed()` signals or RxJS `distinctUntilChanged` and `shareReplay` operators to avoid unnecessary recomputation and re-rendering.

7.3.9 Rendering Strategies

Rendering strategy has first-order impact on performance. Angular applications can combine CSR, SSR, SSG, and hydration depending on route and business needs.

- **CSR (Client-Side Rendering)**: Good for highly interactive internal applications where SEO is not critical.
- **SSR (Server-Side Rendering)**: Improves perceived load speed and LCP for content-heavy routes by returning HTML from the server.
- **SSG (Static Site Generation)**: Ideal for mostly static routes (marketing, docs, legal pages) with very fast delivery through CDN caching.
- **Hydration**: Reuses server-rendered HTML and attaches client behavior, reducing expensive full re-rendering on startup.

In practice, use a hybrid strategy, namely, SSR/SSG for first-visit and SEO-sensitive routes, CSR for authenticated dashboards and heavy interaction areas, and `@defer` and lazy loading to progressively activate non-critical UI blocks.

7.3.10 Signals with OnPush

Signals help Angular update only the reactive graph that actually changed. Combined with `OnPush`, they reduce unnecessary checks and improve interaction responsiveness. The following example shows how to configure the change detection:

```
@Component({
  selector: 'app-cart-summary',
  template: `
    <p>Total: {{ total() | currency }}</p>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class CartSummaryComponent {
  readonly items = signal<{ price: number; qty: number }[]>([]);

  readonly total = computed(() =>
    this.items().reduce((sum, item) => sum + item.price * item.qty, 0)
  );
}
```

As a rule of thumb, keep template expressions cheap and deterministic. Note that `OnPush` is the default in recent Angular versions.

7.3.11 Caching Strategies

Client-side caching reduces network requests and improves responsiveness. Common caching strategies for Angular data layers include:

- **In-memory caching**: Store frequently accessed data inside an Angular service, but the cache disappears when the page is refreshed.
- **RxJS-based caching**: Using `shareReplay(1)` in RxJS to cache the latest value of an observable stream and share it among multiple subscribers.
- **Signal-based caching**: Using Angular signals and `resource()` to create reactive, automatically cached data sources.
- **Service Worker caching**: Using Angular's Service Worker with configurable data groups for API caching with performance or freshness strategies.
- **IndexedDB**: For large datasets that need to persist across sessions, IndexedDB provides client-side storage with query capabilities.

7.3.12 API Layer Performance

API performance directly shapes frontend outcomes. Poor API design commonly manifests as request waterfalls between dependent endpoints, duplicate calls from multiple components, oversized payloads, and frequent retry or timeout cycles. These issues increase route activation time, delay data-bound UI updates, and generate extra rendering work as components cycle repeatedly between loading, partial, and error states.

Because UI responsiveness depends heavily on API and state architecture, apply the following strategies:

- Aggregate related requests to eliminate serial waterfall latency.
- Cache idempotent GET responses with `shareReplay` and clear invalidation rules.
- Share observable streams across components to eliminate duplicate HTTP calls.
- Paginate large datasets and combine with virtual scrolling.
- Apply optimistic updates for low-risk user actions to improve perceived speed.

7.4 Web Server Performance Strategies

7.4.1 Performance Concerns

Regardless of how well the Angular application is optimized at the code level, the web server sitting between the build output and the browser has a decisive impact on what the user actually experiences. Misconfigured servers can negate bundle-size wins, add unnecessary round trips, or block the browser from caching and rendering efficiently.

In real-world environments, users often run on constrained mobile networks, high-latency links, and shared Wi-Fi where every extra request or handshake is amplified. For this reason, web server tuning should be treated as part of frontend performance engineering, not as a separate infrastructure concern.

The most common server-level concerns that affect frontend performance are response compression, HTTP caching headers, connection management, CORS preflight overhead, secure transport configuration, redirect behavior, and correct content types. Each of these is controlled outside the Angular codebase, typically in the web server (Nginx, Apache, IIS), the CDN, or the cloud load balancer. Their effects show up in Core Web Vitals and user-perceived speed, especially during initial loading and navigation.

The following sections focus on the practical server strategies that most influence Angular delivery quality. Together, these optimizations reduce bandwidth consumption, improve repeat-load speed, and stabilize user experience across both fast and slow networks.

7.4.2 Fallback Routes

Angular applications using client-side routing need the server to return `index.html` for all deep-link paths that do not map to a real static file. If the server returns a `404` instead, users who refresh or share a URL will see an error page.

Apply the following guidelines to configure fallback routes for Angular applications:

- In *Nginx*, use `try_files $uri $uri/ /index.html;` inside the location block.
- On CDNs and cloud hosting, configure a custom error document or a catch-all rewrite rule that serves the SPA shell.
- Define API proxy or backend locations before the generic fallback so API routes are not accidentally masked by the SPA entry point.
- Ensure missing static files still return proper `404` responses instead of incorrectly returning `index.html`.
- Return the correct `Content-Type` for every response and add `X-Content-Type-Options: nosniff` so browsers do not guess how to interpret an asset.

A correct fallback configuration is not just a correctness concern, it also affects perceived performance because a `404` or redirect chain before the app loads adds wasted round trips and delays *Time to First Byte*.

7.4.3 Caching Headers

HTTP caching headers determine whether the browser can reuse a previously fetched resource or must make a new network request. Incorrect caching is one of the most common server misconfigurations that harms frontend performance.

Apply the following rules to balance repeated loads with safe and predictable caching behavior:

- Serve only content-hashed static assets (`main.abc123.js`, `styles.def456.css`) with long-lived `Cache-Control` values such as `public, max-age=31536000, immutable`.
- Serve `index.html` and other non-hashed entry points with short or zero cache lifetimes (for example `no-cache` or `max-age=0, must-revalidate`) so the browser always fetches the latest version that references the current hashed assets.

- Use `ETag` or `Last-Modified` headers as a validation mechanism for resources that cannot use content hashing, so the server can respond with `304 Not Modified` and save bandwidth.
- Keep service worker metadata (`ngsw.json`, `ngsw-worker.js`) fresh so clients detect updates promptly. Do not give these files the same year-long immutable policy as hashed application assets.

Next `Nginx` configuration block combines SPA fallback behavior with cache policies for HTML and static assets:

```
server {
  etag on;

  location / {
    try_files $uri $uri/ /index.html;
    add_header Cache-Control "no-cache, must-revalidate";
  }

  location ~* \.(?:css|js|json|jpeg|jpg|png|woff|ttf|otf|svg|woff2|eot)$ {
    add_header Cache-Control "public, max-age=31536000, immutable" always;
  }
}
```

What each part means:

- `try_files $uri $uri/ /index.html;` serves real files when they exist, otherwise falls back to `index.html` for Angular client-side routes.
- `Cache-Control "no-cache, must-revalidate"` on `location /` allows the HTML entry document to be stored but requires validation before reuse after a deployment.
- `Cache-Control "public, max-age=31536000"` on static assets allow long-lived browser and intermediary caching for files that rarely change in-place. Unhashed files need a separate, shorter-lived policy (e.g. `ngsw.json` and `ngsw-worker.js`).
- `etag on;` supports conditional validation for resources that are not content-hashed.

Misconfigured caching either forces the browser to re-download unchanged assets on every navigation (hindering performance on repeat visits) or serves stale bundles after a deployment (causing runtime errors when old code references missing chunks).

7.4.4 CDN and Edge Caching

Content Delivery Networks (CDNs) are geographically distributed networks of servers that cache application content close to end users, significantly reducing latency and improving availability. Edge caching refers to caching policies and logic applied at CDN edge locations (the servers closest to users) rather than only at the origin server. This enables cached content to be served from geographically nearby infrastructure.

Apply the following guidelines to optimize CDN and edge caching for web applications:

- **Cache hashed assets**: Content-hashed filenames (`main.abc123.js`) can be cached with `public, max-age=31536000, immutable` because the URL changes whenever the content changes. The `immutable` directive tells clients not to revalidate the asset while it is fresh. A CDN may still evict it before the freshness period ends.
- **Revalidate entry points**: Use `no-cache` or `max-age=0, must-revalidate` for `index.html` so clients and shared caches validate it before reuse. Conditional requests such as `ETag` can save bandwidth when the file has not changed, provided the origin and CDN support them.
- **Set appropriate TTLs (Time-To-Live) per content type**: Cache hashed static images, stylesheets, and scripts for a year. Keep unhashed files and HTML short-lived, and cache dynamic data for seconds to minutes only when freshness, authorization, cookies, and privacy requirements allow it.
- **Implement cache invalidation strategies**: Use URL versioning (content hashing), cache keys with version parameters, or explicit purge APIs to invalidate stale content after deployments without waiting for TTL expiration.
- **Define cache keys deliberately**: Account for query strings, cookies, authorization, `Accept-Language`, and device or image-variant parameters. When the response varies by request origin, send `Vary: Origin` so the CDN does not reuse one CORS response for another origin.

For Angular applications, a CDN can serve as the primary distribution mechanism for HTML, JavaScript, CSS, static assets, and, where appropriate, API responses. Proper CDN configuration is one of the highest-impact performance optimizations because it compounds across millions of requests and can directly improve TTFB, LCP, and cache hit rates for a global audience. For SSR or authenticated applications, do not cache personalized HTML in a CDN.

7.4.5 Server Compression

Minimizing JavaScript bundle size directly improves application performance. If the server, CDN, or reverse proxy is not configured to compress responses, even well-optimized Angular bundles will still travel over the network larger than necessary.

Apply the following guidelines to configure compression correctly across the delivery stack:

- Prefer **Brotli** for text-based responses such as HTML, CSS, JavaScript, JSON, and SVG when supported, or use **Gzip** as the broad-compatibility fallback.
- Do not waste CPU recompressing assets that are already compressed, such as JPEG, PNG, WebP, AVIF, MP4, ZIP, or PDF files.
- Terminate compression at the right layer. In many architectures, the CDN, ingress controller, or reverse proxy should own compression instead of each application instance.
- Avoid compressing the same response more than once.
- Verify the result in production by checking `Content-Encoding`, `Vary: Accept-Encoding`, and actual transferred size in browser DevTools.

Next `Nginx` configuration block enables Gzip response compression for text-based assets before they are sent to the browser:

```
http {
  gzip on;
  gzip_vary on;
  gzip_min_length 1024;
  gzip_static on;

  gzip_types
    application/javascript
    application/json
    application/manifest+json
    font/eot
    font/otf
    font/ttf
    font/woff
    image/svg+xml
    text/css
    text/javascript
    text/plain
    text/xml;
}
```

What each part means:

- `gzip on;` activates Gzip compression globally in the `http` context.
- `gzip_vary on;` ensures the response includes `Vary: Accept-Encoding` so caches separate compressed variants correctly.
- `gzip_min_length 1024;` avoids spending CPU compressing very small payloads that gain little from compression.
- `gzip_static on;` allows serving precompressed files (`.gz`) when they exist, reducing runtime CPU cost.
- `gzip_types ...;` defines which MIME types should be compressed.

Operationally, compression is a tradeoff between CPU cost and transfer size. For most Angular applications, the bandwidth savings are substantial, but teams should still test on real infrastructure to ensure compression levels do not create unnecessary server latency under load. Alternatively, Gzipper can compress files using Deflate, Brotli, Gzip, Zopfli, or Zstandard and can be integrated with common CLI-based build workflows.

7.4.6 HTTP/2, HTTP/3, and TLS Optimization

Transport and connection settings define how quickly bytes can move from server to browser after routing and caching are already correct. At this layer, major performance gains usually come from reducing handshake overhead, reusing connections effectively, and avoiding unnecessary protocol fallback. These choices are implemented in the CDN, load balancer, or reverse proxy, and primarily affect TTFB and loading metrics rather than interaction latency after the page is running.

The following `Nginx` configuration block enables HTTP/2, HTTP/3, TLS, and keep-alive tuning:

```
server {
  listen 443 ssl;
  listen 443 quic reuseport;
  http2 on;
  http3 on;

  ssl_certificate /path/to/fullchain.pem;
  ssl_certificate_key /path/to/privatekey.pem;
  ssl_protocols TLSv1.2 TLSv1.3;
  ssl_session_cache shared:SSL:10m;
  ssl_session_timeout 1d;
  ssl_session_tickets off;

  ssl_stapling on;
  ssl_stapling_verify on;
```

```
  add_header Alt-Svc 'h3=":443"; ma=86400' always;

  keepalive_timeout 65;
  keepalive_requests 1000;
}
```

What each part means:

- `listen 443 ssl;` enables HTTPS on TCP, and `http2 on;` enables HTTP/2 for that server.
- `listen 443 quic reuseport;` enables QUIC transport, while `http3 on;` enables HTTP/3.
- `ssl_certificate` points to the server certificate and intermediate certificate chain, while `ssl_certificate_key` points to the matching private key.
- `ssl_protocols TLSv1.2 TLSv1.3;` allows modern TLS with broad client compatibility. HTTP/3 itself requires TLS 1.3.
- `ssl_session_cache` and `ssl_session_timeout` help handshake reuse for repeat connections.
- `ssl_stapling` reduces certificate validation round trips when OCSP infrastructure is healthy.
- `Alt-Svc` advertises HTTP/3 availability to compatible clients. `always` ensures the header is added to non-success responses as well.
- `keepalive_timeout` and `keepalive_requests` control connection reuse and resource pressure tradeoffs.

Performance optimization for HTTP/3 on NGINX focuses on minimizing latency and maximizing throughput. This can involve tuning the underlying UDP settings for QUIC, adjusting session reuse parameters, and optimizing TLS configuration to reduce handshake times. Monitoring and changing settings based on real-world usage patterns are crucial, as the optimal configuration can vary significantly depending on specific traffic loads, content types, and network conditions.

7.4.7 CORS Optimization

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that controls whether JavaScript code from one origin can access resources hosted on a different origin. When a cross-origin request does not meet the requirements for a "simple" request, the browser sends a preflight `OPTIONS` request to the destination server to verify that the actual request is permitted. This verification happens before the actual request is sent.

A preflight request is triggered when any of the following conditions are true:

- The HTTP method is not `GET`, `HEAD`, or `POST`.
- Request headers are added that are not CORS-safelisted (for example `Authorization`, `X-Trace-Id`, or tenant-specific headers).
- The request's `Content-Type` is not `application/x-www-form-urlencoded`, `multipart/form-data`, or `text/plain`.

Each preflight adds an extra round trip before the actual request, which increases latency, especially on high-latency networks or mobile connections. Additionally, every tier in the request path (API gateway, proxy, load balancer, and origin application) must correctly route and respond to `OPTIONS` requests, making preflight failures a common source of production issues.

In practice, one of the most effective ways to reduce this overhead is to route API requests through the same origin as the Angular application. For example, a CDN or reverse proxy can forward requests from `/api/*` to the backend while the browser continues to communicate with a single origin. This removes the cross-origin boundary from the browser-to-edge request, avoiding CORS preflights on that leg while preserving the separation between the frontend and backend services. Where cross-origin requests are still necessary, allow browsers to cache successful preflight responses with `Access-Control-Max-Age`, use simple requests where appropriate, and define an explicit allowlist of trusted origins.

7.4.8 Server-Side Rendering

Server-Side Rendering is a rendering strategy in which the server generates the initial HTML for a route and sends it to the browser. This allows users to see meaningful content before client-side JavaScript finishes bootstrapping the application.

In Angular, server-side rendering can improve the initial user experience by delivering meaningful content sooner, especially on slower devices or networks. It can also improve how crawlers and link preview systems consume page content and metadata. However, server rendering adds work to the server and can increase TTFB when rendering blocks on slow APIs or expensive computation. Use route-appropriate caching or streaming where possible, and use hydration with transfer state to avoid repeating server-fetched data in the browser.

Angular integrates server-side rendering through the framework and CLI with @angular/ssr. Alternatively, AnalogJS is an Angular meta-framework inspired by the developer experience of frameworks such as Next.js and Nuxt.

7.4.9 Server Scaling

Server scaling distributes incoming requests across multiple application instances to handle growing traffic, reduce the impact of individual server failures, and maintain predictable response times. For Angular applications using Server-Side Rendering, scaling is especially important because server rendering can be CPU-intensive. A single instance can become a bottleneck quickly unless responses are cached or rendering work is otherwise reduced.

A typical scaling architecture employs several strategies:

- **Load balancing**: Route requests across multiple instances using algorithms such as round-robin, least connections, or weighted distribution. Health checks should remove unhealthy instances from the pool automatically.
- **Stateless design**: Store session state externally (for example, in a cache or database) rather than in process memory, so any instance can handle a request. Avoid relying on local files or other instance-specific state, and make instances replaceable.
- **Autoscaling**: Monitor CPU, memory, request latency, throughput, queue depth, and error rate to add or remove instances based on demand. Use percentile-based thresholds (p95/p99) and saturation signals rather than averages alone.
- **Appropriate sizing**: Choose instance sizes based on measured CPU, memory, concurrency, and SSR rendering requirements. Leave enough headroom for traffic spikes, and monitor per-instance metrics to detect imbalance or saturation.

For Angular applications, ensure the deployment pipeline supports rolling or blue-green updates so new code can be deployed with minimal disruption. Connection draining, which allows existing requests to complete before shutdown, prevents abrupt connection drops during instance replacement. Combined with external caching and durable data stores, these patterns support horizontal scaling while keeping application instances stateless and improving resilience during traffic spikes.

7.4.10 API Latency Optimization

API latency is the time elapsed between when a frontend application sends a request and when it receives the complete response from the backend. High API latency can degrade frontend performance and user experience by extending loading states, delaying route activation, and keeping data-dependent UI unavailable for longer.

Apply the following strategies to mitigate API latency impact:

- **Optimize backend query performance**: Reduce database query time, eliminate N+1 queries, add appropriate indexes, and use query-result caching where it is safe. Profiling response times is the foundation of API optimization.
- **Aggregate or batch API calls**: Combine related requests when doing so reduces network round trips without creating oversized or slow responses. For example, fetch a user profile and settings in one call instead of two.
- **Cache API responses strategically**: Share request observables with RxJS `shareReplay` to prevent duplicate calls among active consumers, and cache idempotent `GET` responses only with clear freshness and invalidation rules.
- **Implement optimistic updates**: For low-risk operations, such as liking a post or toggling a preference, update the UI before the API responds and revert the change if the request fails. This improves perceived responsiveness but requires clear failure handling.
- **Paginate and limit payloads**: Fetch only the data needed for the current view rather than entire datasets. Return compact representations and combine pagination with virtual scrolling for large lists.
- **Use edge caching for read-heavy APIs**: Cache stable, public, read-only responses such as configuration, lists, or reference data at the CDN or service-worker layer when their freshness and privacy requirements allow it. Set an appropriate time to live (TTL).

For SSR applications, API latency can block server rendering when the route waits for API data before producing HTML. Slow APIs therefore increase TTFB and can delay LCP. Prioritize backend optimization, parallelize independent requests, and consider rendering a shell or loading non-critical data on the client when the framework and UX allow it.

7.5 Performance Testing

7.5.1 Frontend vs Backend

Performance testing should be automated, repeatable, and tied to clear, user-centered thresholds. In practice, this requires testing both **frontend performance** and **backend performance**, because users experience the combined result rather than either layer in isolation.

Frontend performance testing focuses on what the browser must download, execute, and render, as well as how quickly it responds during user interactions. Typical concerns include bundle size, render-blocking resources, long main-thread tasks, layout stability, interaction latency, and

smooth route transitions. Backend performance testing focuses on the server and data layer under realistic load. Typical concerns include TTFB, API response times, tail latency (p95/p99), error rate, throughput, queue saturation, database contention, and cache hit ratio.

Although these test categories are different, they are tightly connected. Backend behavior directly influences frontend outcomes and Core Web Vitals:

- Slow SSR or delayed API responses increase TTFB and often delay LCP.
- High backend tail latency keeps the UI in loading states longer, degrading perceived responsiveness.
- Unstable endpoints (timeouts/retries) produce spinner flicker, partial renders, and interaction interruptions.
- Chatty or waterfall APIs force extra round trips, delaying route activation and interactive readiness.

For this reason, mature teams treat performance as an end-to-end quality gate. The goal is not only fast JavaScript or fast APIs, but also fast user journeys. With a combined strategy, teams can:

- Improve user experience by finding bottlenecks across network, server, and browser rendering.
- Prepare for unexpected demand by validating behavior beyond expected traffic and concurrency.
- Increase delivery confidence by reducing production risk with repeatable thresholds and regression gates.
- Optimize infrastructure cost by sizing capacity based on measured behavior, not assumptions.

7.5.2 Testing Approaches

Testing approaches should reflect how users experience the system across the browser, network, and backend services. Instead of relying on a single tool, use a set of complementary tests, with each one answering a different question at the appropriate stage of delivery.

Use several test levels so that each test type does what it does best:

- **Pull request level**: Run fast Lighthouse and route-budget checks on critical user paths (such as login, checkout, or dashboards) to catch obvious frontend regressions without slowing development. Builds should fail only for meaningful regressions.

- **Nightly or pre-release level**: Run broader synthetic and end-to-end tests, including scripted user journeys, trace analysis, realistic network conditions, geographic variation, multi-region WebPageTest runs, backend load tests, API latency validation under concurrency, and cache-effectiveness checks.
- **Production level**: Continuously monitor real-user and service telemetry, with alerts for p75/p95 degradations in frontend and backend KPIs such as LCP, INP, CLS, API latency, and error rates. This verifies that improvements measured in other stages translate to real-world performance.

A practical CI policy should reflect real user contexts. Set separate thresholds for desktop and mobile profiles because mobile devices and networks typically have less CPU, memory, and connectivity stability. Effective performance tests also require realistic loads and data in production-like environments. Monitor response times and infrastructure metrics, define clear success criteria, account for third-party dependencies, and retest performance after code changes. A mature performance testing program combines realistic workload modeling, production-like environments, continuous monitoring, and automated regression detection.

7.5.3 Lighthouse

Lighthouse is a tool that evaluates a page under controlled conditions and reports scores plus diagnostics for **Performance**, **Accessibility**, **Best Practices**, and **SEO**. Lighthouse is most useful as a regression gate in CI because it catches common failures early (render-blocking resources, excessive JavaScript, long main-thread tasks, inefficient image delivery) before they reach production.

Use Lighthouse as **controlled lab verification**, not as a replacement for real-user monitoring. Lighthouse helps compare builds consistently and block regressions quickly, while Core Web Vitals in production confirm whether users have actually experienced improvements.

Minimal package setup commonly used for this test style:

```
npm install -D lighthouse chrome-launcher testcontainers
```

See the next example using a containerized workflow:

```
describe('Application', () => {
  let container: StartedTestContainer;
  let baseUrl: string;
  let report: RunnerResult;

  beforeAll(async () => {
```

```
    const config = { extends: 'lighthouse:default' };
    const chrome = await chromeLauncher.launch({ chromeFlags: ['--
headless'] });
    const options = { logLevel: 'info', output: 'json', port: chrome.port };

    container = await new GenericContainer('my-org/my-app:latest')
      .withExposedPorts(4200)
      .start();

    baseUrl = `http://localhost:${container.getMappedPort(4200)}`;

    report = await lighthouse(baseUrl, options, config)
      .finally(() => chrome.kill());
  });

  afterAll(async () => {
    await container.stop();
  });

  it('performance score should be >= 0.90', () => {
    expect(report.lhr.categories['performance'].score)
      .toBeGreaterThanOrEqual(0.9);
  });

  it('accessibility score should be >= 0.90', () => {
    expect(report.lhr.categories['accessibility'].score)
      .toBeGreaterThanOrEqual(0.9);
  });

  it('best-practices score should be >= 0.90', () => {
    expect(report.lhr.categories['best-practices'].score)
      .toBeGreaterThanOrEqual(0.9);
  });

  it('seo score should be >= 0.90', () => {
    expect(report.lhr.categories['seo'].score)
      .toBeGreaterThanOrEqual(0.9);
  });
});
```

How Lighthouse test works in practice:

1. It runs a production-like artifact in a Docker container (`my-org/my-app:latest`) to keep the execution environment stable.
2. It launches headless Chrome and executes Lighthouse programmatically against the mapped container URL.
3. It stores the `RunnerResult` and validates both category thresholds (`>= 0.9`) and concrete audit budgets (for example LCP and TBT).
4. It performs explicit cleanup (`chrome.kill()` and `container.stop()`) to keep test runs reliable in CI.

For reliable measurements, prefer **two execution environments**, with one machine hosting the application and another running the load/audit agent. Running both on the same host can distort metrics because CPU, memory, disk I/O, and network stack contention from one process affects the other.

7.5.4 WebPageTest

WebPageTest provides synthetic tests with realistic network shaping, geographic test locations, repeat-view behavior, and rich artifacts (waterfalls, filmstrips, request-level diagnostics). WebPageTest is a strong complement to Lighthouse when route-level analysis is needed under conditions that resemble real users more closely.

Use WebPageTest as **environment-realistic synthetic verification**, especially for nightly and pre-release checks. Lighthouse remains a fast pull-request gate, while WebPageTest adds deeper validation for caching behavior, regional latency, and third-party impact.

Minimal package setup commonly used for this test style:

```
npm install -D webpagetest
```

See the next example using webpagetest client:

```
describe('Application', () => {
  let report: any;

  beforeAll(async () => {
    const wpt = new WebPageTest('www.webpagetest.org',
process.env['WPT_API_KEY']);
    const options = {
      location: 'Dulles:Chrome',
      connectivity: '4G',
      runs: 1,
      firstViewOnly: true,
      pollResults: 5,
      timeout: 120,
    };
    const testResults = await new Promise((resolve, reject) => {
      wpt.runTestAndWait(process.env['APP_BASE_URL'], options, (error,
result) => error ? reject(error) : resolve(result));
    });

    report = testResults.data.average.firstView;
  });

  it('LCP should be <= 2500 ms', () => {
    expect(report.largestContentfulPaint).toBeLessThanOrEqual(2500);
  });
```

```
  it('TTFB should be <= 800 ms', () => {
    expect(report.timeToFirstByte).toBeLessThanOrEqual(800);
  });

  it('Speed Index should be <= 3000 ms', () => {
    expect(report.speedIndex).toBeLessThanOrEqual(3000);
  });
});
```

How WebPageTest works in practice:

1. It uses the official `webpagetest` client and a real WebPageTest endpoint (`www.webpagetest.org`) authenticated with `WPT_API_KEY`.
2. It runs a route test with explicit profile controls (`location`, `connectivity`, `runs`) so results are comparable over time.
3. It uses `runTestAndWait` to execute and wait for completion, then reads the `firstView` metrics.
4. It validates concrete budgets (for example LCP, TTFB, and Speed Index) instead of relying only on a single aggregate score.

Note that public WebPageTest agents cannot access `localhost` or private CI networks directly. Use a reachable preview/staging URL (`APP_BASE_URL`) or run private agents when testing non-public environments.

7.5.5 k6

k6 is an open-source performance testing tool and cloud service from Grafana Labs. For Angular SSR applications, k6 is particularly effective at verifying server-side load time characteristics using HTTP-level metrics.

Use k6 as **scriptable load time validation** for CI and pre-release pipelines. Server-side rendering pages are rendered on the server before being sent to the browser. Therefore, HTTP-level metrics such as request duration, Time to First Byte (TTFB), and response size are the primary indicators of server-side rendering performance.

k6 provides native packages for Linux, Mac, and Windows. See the k6 installation guide for platform-specific instructions. Alternatively, we can pull the official k6 Docker image:

```
docker pull grafana/k6
```

Before running the test file, ensure that a runnable k6 HTTP script exists in `K6_SCRIPT_PATH` with a default function that issues `http.get` requests against critical SSR routes, and the

target application is reachable through `APP_BASE_URL`. Then, use the following opinionated example to run the performance tests:

```
describe('Application', () => {
  let summary: any;

  beforeAll(() => {
    execFileSync(
      'k6',
      [
        'run',
        '--summary-export', 'dist/perf-summary.json',
        '--summary-trend-stats', 'avg,min,max,p(75),p(90),p(95)',
        '--vus', '10',
        '--duration', '30s',
        process.env['K6_SCRIPT_PATH']!,
      ],
      {
        env: { ...process.env, APP_BASE_URL: process.env['APP_BASE_URL'] },
        stdio: 'inherit',
      }
    );

    summary = JSON.parse(readFileSync('dist/perf-summary.json', 'utf8'));
  });

  it('request duration p95 should be <= 1000 ms', () => {
    expect(summary.metrics.http_req_duration['p(95)'])
      .toBeLessThanOrEqual(1000);
  });

  it('TTFB p95 should be <= 400 ms', () => {
    expect(summary.metrics.http_req_waiting['p(95)'])
      .toBeLessThanOrEqual(400);
  });

  it('error rate should be < 1%', () => {
    expect(summary.metrics.http_req_failed.value).toBeLessThan(0.01);
  });
});
```

How k6 works in practice:

1. The k6 HTTP script issues `http.get` requests against SSR routes, verifying that the server returns fully rendered HTML for each critical page under load.
2. The `execFileSync` function keeps the load test inside the test lifecycle.
3. The run exports a JSON summary (`--summary-export`) and includes `p(95)` in trend stats to capture tail latency, not just averages.
4. JSON summary is used on each `it` assertion per metric: total request duration (p95), TTFB via `http_req_waiting` (p95), and error rate.

k6 is useful not only for load testing but also for browser-level performance verification through the `k6/browser` API.

7.5.6 Best Practices

The checklist below provides practical performance testing practices that help teams establish repeatable, production-relevant testing and prevent regressions over time:

- ☐ **Define clear performance objectives**: Establish measurable targets for response times, throughput, concurrent users, error rates, and resource utilization before testing begins.
- ☐ **Automate performance testing**: Integrate performance tests into CI/CD pipelines to detect regressions early and ensure consistent testing practices.
- ☐ **Test realistic user behavior**: Simulate actual user journeys and workflows, including logins, searches, transactions, and form submissions, rather than isolated requests.
- ☐ **Use production-like environments**: Conduct tests in environments that closely match production infrastructure, database schemas, cache behavior, and network conditions.
- ☐ **Create representative test data**: Use realistic data volumes and distributions to accurately reflect production workloads and uncover data-related performance issues.
- ☐ **Establish performance baselines**: Measure and document current system performance to enable comparison and identify regressions over time.
- ☐ **Perform multiple types of performance tests**: Include load, stress, spike, endurance (soak), and scalability testing to evaluate behavior under different conditions.
- ☐ **Measure end-to-end performance**: Monitor performance across all layers, including the browser, CDN, load balancer, application servers, databases, caches, and external services.
- ☐ **Monitor system resource utilization**: Track CPU, memory, disk I/O, network usage, thread pools, and connection pools during tests to identify bottlenecks.
- ☐ **Analyze backend percentile metrics**: Focus on P50, P90, P95, and P99 response times rather than relying solely on averages, which can mask poor user experiences.
- ☐ **Analyze frontend performance metrics**: Measure user-centric metrics such as Largest Contentful Paint (LCP), First Contentful Paint (FCP), and Time to Interactive (TTI).
- ☐ **Test APIs independently**: Validate API response times, throughput, payload sizes, and authentication overhead separately from UI testing.

- ☐ **Account for third-party dependencies**: Evaluate the impact of external services, APIs, authentication providers, payment gateways, and analytics tools.
- ☐ **Correlate performance and infrastructure metrics**: Link response times and throughput with system metrics, logs, and traces to simplify root-cause analysis.
- ☐ **Monitor performance in production**: Use real-user monitoring (RUM), application performance monitoring (APM), logging, and tracing to validate real-world performance.
- ☐ **Identify and eliminate bottlenecks**: Investigate slow queries, cache misses, lock contention, inefficient code paths, and excessive network calls using evidence-based analysis.
- ☐ **Test cold and warm cache scenarios**: Evaluate performance both with empty caches and fully populated caches to understand real-world behavior.
- ☐ **Validate error handling under load**: Ensure the application degrades gracefully under high load through rate limiting, retries, circuit breakers, and meaningful error responses.
- ☐ **Document and version test scenarios**: Maintain test scripts, datasets, configurations, and results to ensure repeatability and historical comparison.
- ☐ **Continuously optimize and retest**: Treat performance testing as an ongoing process of testing, analyzing, tuning, and validating improvements.

7.6 Performance Monitoring

7.6.1 Profiling Angular Applications

Profiling Angular applications means measuring where time is spent across network activity, JavaScript execution, change detection, rendering, and API wait states. The goal is to find the bottlenecks that most affect user-centric metrics such as LCP, INP, CLS, and route transition time.

The following areas represent the primary targets for frontend performance profiling and optimization:

- **Startup and route activation**: Measure TTFB, FCP, and LCP, along with hydration or bootstrap cost, and determine whether route-level bundles or resolver waterfalls delay the first meaningful paint.
- **Interaction latency (INP)**: Inspect click/type/scroll responsiveness and identify long tasks over 50 ms that block main-thread input handling.

- **Change detection churn**: Detect components that are checked too often, expensive template expressions, unstable list identity (`@for track`), and avoidable global checks.
- **Rendering and layout stability**: Find costly style recalculation, layout thrashing, repaint-heavy animations, and layout shifts from late-loading content.
- **Memory and long-session behavior**: Track detached nodes, subscription leaks, timer or event-listener leaks, and progressive slowdown in long-lived dashboard screens.

A practical profiling workflow is iterative. Use the Chrome DevTools Performance panel to record and analyze real interactions, then use the Angular DevTools Profiler to inspect component-level change detection behavior behind browser trace hotspots. For quick local checks, `console.time` and `console.timeEnd` are useful, but meaningful profiling should be validated in production mode because development mode includes additional framework checks. Treat profiling as a continuous loop, namely, use local profiling for root-cause diagnosis, CI checks for regression prevention, and production telemetry for real-user validation.

7.6.2 Chrome DevTools

Chrome DevTools is a browser-based diagnostics suite for frontend performance analysis. It provides end-to-end visibility into runtime behavior in one place, allowing teams to inspect CPU activity, rendering cost, network waterfalls, and memory behavior during the same user interaction.

The following Chrome DevTools features are the most useful starting points when profiling frontend performance issues:

- Use the Performance panel as the primary profiler to record representative user flows and inspect long tasks, main-thread blocking, scripting, layout, rendering, and paint in one timeline.
- Use the Lighthouse panel to run repeatable lab audits for performance, accessibility, SEO, and related loading and responsiveness metrics. Treat these results as controlled estimates, not a replacement for field data from real users.
- Use the Rendering tools to visualize paint flashing, layout shifts, frame rendering statistics, scrolling performance, and layer borders while diagnosing rendering regressions.
- Use the Network panel together with CPU and network throttling to reproduce slower-device conditions, inspect request waterfalls, verify caching and compression, and determine whether network latency is driving LCP or route delays.

- Use the Sources panel with JavaScript breakpoints, async stack traces, `console.time`, and `debugger` statements to trace expensive execution paths behind long tasks or slow interactions.
- Use the Layers and Paint Profiler to analyze compositing, layer promotion, and paint costs, especially in animation-heavy or scroll-intensive interfaces.
- Use the Application panel to inspect service workers, Cache Storage, IndexedDB, and offline caching behavior when repeat-visit speed or PWA caching is part of the performance problem.
- Use the Recorder panel to script and replay critical user journeys so profiling runs stay comparable across fixes and regressions.
- Use the Coverage panel to detect unused JavaScript and CSS on critical routes.
- Use the Memory panel to investigate leaks involving detached DOM nodes, event listeners, subscriptions, or long-lived references.

A practical workflow is to reproduce one slow interaction, record a focused trace, and isolate tasks longer than 50 ms before changing code. Then map each hotspot to concrete Angular code paths (component logic, template bindings, or third-party script execution), apply one targeted optimization, and profile again under the same conditions. This iterative loop keeps performance work evidence-driven and avoids speculative refactoring.

7.6.3 Angular DevTools

Angular DevTools is a browser extension that provides debugging and profiling capabilities for Angular applications. It exposes Angular-specific runtime behavior that browser traces alone cannot fully explain, including component update propagation, change detection activity, and the component tree involved in a slow interaction.

The following Angular DevTools capabilities are the most useful starting points when profiling Angular-specific performance issues:

- Use the **Components** tab to inspect live component state (`@Input`, local state, Signals, and outputs), identify unexpected updates, and verify how changes propagate through the component tree.
- Use the **Profiler** tab to record Angular activity during real user flows and identify components with frequent checks or disproportionate rendering cost.

- Use the **Injector Tree** tab to inspect dependency-injection scopes and provider resolution paths, helping identify unexpected provider placement or duplicated service instances.
- Use the **Router Tree** tab to inspect route hierarchy, lazy-route boundaries, guards and resolvers, and activation structure when diagnosing slow navigations or request waterfalls.

A practical workflow is to reproduce one slow interaction in production mode, inspect the **Router Tree** for navigation structure, review the **Injector Tree** for provider-scope issues, capture a **Profiler** recording, and identify high-cost components in **Components**. Then map the findings to concrete code paths (template expressions, list rendering, derived state, event handlers, or resolver/guard flow), apply one focused optimization, and profile again under the same conditions. Finally, verify the browser-level impact in Chrome DevTools to confirm that reduced Angular work also improves scripting time, interaction responsiveness, and user-perceived smoothness.

7.6.4 Lighthouse CI

Lighthouse CI provides a system for storing Lighthouse reports, comparing runs across commits, and applying assertions to detect regressions. It has three main parts, namely, the CLI (`lhci`), a server that stores build data, and optional assertions that can fail CI when budgets are exceeded.

The following command provides a minimal setup for running the Lighthouse CI Docker Server:

```
docker run -d -p 9001:9001 -v lhci-data:/data \
  patrickhulce/lhci-server:latest
```

This starts the server at `http://localhost:9001` using its default embedded storage, which is suitable for a local example. In production, use persistent storage and place the service behind a reverse proxy or single sign-on layer. After the server is available, create a project and generate an upload token through the Lighthouse CI project setup flow. Store the token in CI secrets (for example, `LHCI_UPLOAD_TOKEN`) and never commit it to source control. With the server and token ready, add a minimal `.lighthouserc.json` in which `collect` defines the audited URLs and `upload` defines where results are published.

Minimal `.lighthouserc.json` configuration file:

```
{
  "ci": {
    "collect": {
      "numberOfRuns": 3,
      "url": [
        "http://localhost:4200/"
      ]
    },
    "upload": {
      "target": "lhci",
      "serverBaseUrl": "http://localhost:9001",
      "token": "${LHCI_UPLOAD_TOKEN}"
    }
  }
}
```

In this minimal configuration, `numberOfRuns: 3` reduces run-to-run variance by collecting multiple samples. The `url` array can start with one business-critical route and later expand to additional routes, `serverBaseUrl` must point to a reachable Lighthouse CI server, and `token` should be resolved from CI secrets. The `localhost` values in this example must be replaced with reachable application and server URLs when the command runs in CI. Once the application is built and available, run Lighthouse CI with:

```
npx lhci autorun
```

The `autorun` command performs collection, runs any configured assertions, and uploads the results in one execution.

A Lighthouse CI server turns isolated pipeline outputs into a shared performance history. Instead of losing context in per-build artifacts, teams can compare results by commit and branch through a central interface and control publishing with upload tokens. It also provides centralized retention, storage, and access policies, making performance data easier to review consistently over time.

7.6.5 Profiling Node.js Applications

Profiling Node.js applications means measuring CPU, memory, and runtime behavior while a service is running. The goal is to find bottlenecks such as high CPU usage, event-loop blocking, memory leaks, and slow code paths that reduce responsiveness and scalability.

In practice, monitor end-to-end latency (p50, p95, p99), throughput (requests per second), error rate, event-loop delay and utilization, CPU saturation (user and system time), GC frequency and pause time, heap growth over time, and runtime indicators such as open handles and file-descriptor pressure. If metrics degrade sharply under traffic spikes, investigate contention and

blocking hot paths. If memory keeps growing and does not recover after garbage collection, investigate object retention and possible leaks.

For runtime analysis, use V8 profiling and Node's CPU profiling and inspector features. A common workflow is to run Node with the inspector enabled and analyze traces in Chrome DevTools for Node.js, where flame charts show JavaScript execution, GC cost, and I/O wait boundaries. For memory analysis, use heap snapshots and allocation timelines to verify whether objects are retained too long. Profile under representative load because idle profiles can hide contention and scheduling effects. AutoCannon is a practical quick option for local and CI runs, while k6, Artillery, and wrk are strong alternatives.

For visual diagnostics, Clinic.js offers Clinic Doctor for broad bottleneck detection, Clinic Flame for CPU hotspots, and Clinic Bubbleprof for asynchronous and event-loop flow analysis. If deeper comparison is required, alternatively, use GraalVM Node.js tooling and observability integrations under the same workload.

For production monitoring, continuously collect service indicators (latency, traffic, errors, and saturation), runtime metrics (event-loop delay, heap usage, old-space usage, and GC pauses), and system metrics (CPU steal, memory pressure, disk I/O, and network I/O). Export metrics, traces, and logs through OpenTelemetry, store metrics in Prometheus, and use Grafana to query, visualize, and alert on the collected telemetry. This turns profiling from one-off debugging into a repeatable performance engineering practice.

7.7 Best Practices

The following practices summarize the highest-impact techniques for improving frontend loading, rendering, server delivery, and runtime performance:

- ☐ **Apply PRPL pattern**: PRPL (Push, Render, Pre-cache, Lazy-load) describes a pattern for making web pages load and become interactive faster. It is a web performance architecture introduced by Google.
- ☐ **Minify code**: Minification removes unnecessary characters and metadata without changing how the browser processes the code. Minifying JavaScript, CSS, and HTML reduces transfer size and can improve load performance.
- ☐ **Compress bundles**: Compression reduces file size using formats like Gzip or Brotli. The browser indicates supported algorithms through `Accept-Encoding`, and the server declares the selected algorithm via `Content-Encoding`.

- ☐ **Enable tree-shaking**: Tree-shaking, also known as dead-code elimination, removes unused exports from statically analyzable modules by starting from entry points and retaining code that may execute. Bundlers such as webpack, Rollup, and Parcel support this optimization.
- ☐ **Implement code-splitting**: Code-splitting breaks the codebase into chunks loaded on demand. As apps grow, JS and CSS bundles become larger, and a single-minified-file strategy is no longer ideal. This approach is used to improve application performance, especially on initial load.
- ☐ **Implement lazy-loading**: Use Lazy-loading to load only what is needed when it is needed to reduce initial load time. Avoid lazy-loading the default route, split code at logical breakpoints, and defer off-screen images using native lazy loading or Intersection Observer.
- ☐ **Audit third-party dependencies**: Review dependencies and look for smaller or better alternatives to reduce final bundle size. If a package is added for tiny functionality that could be implemented natively or by the framework, it adds avoidable overhead. Use Bundlephobia to identify large dependencies and optimization opportunities.
- ☐ **Optimize LCP resource priority**: Ensure the LCP element is discovered early, not lazily loaded when above the fold, and uses proper priority hints (`fetchpriority="high"` or preload when justified).
- ☐ **Use resource hints**: Resource hints are `rel` values on `link` elements for external resources. Use `dns-prefetch`, `preconnect`, `prefetch`, and `preload` to help user agents decide which origins to connect to and which resources to fetch or pre-process early.
- ☐ **Use service workers**: A Service Worker acts like an HTTP proxy in the browser, intercepting client requests and either handling them or forwarding to the network. Service workers can cache static assets, provide offline fallbacks, handle broken images, support messaging between tabs, and apply request-specific caching strategies.
- ☐ **Use web workers**: Use Web Workers for CPU-heavy code that needs significant execution time. Workers run in background threads and perform processor-intensive tasks without blocking the main thread.
- ☐ **Use OnPush with Signals**: Combine Angular OnPush change detection with Signals for predictable reactivity and fewer unnecessary checks. Keep template expressions cheap and move derived values to `computed()`.

- ☐ **Track items in lists**: Always track stable item identities in repeated templates so Angular can reuse DOM nodes instead of recreating rows on updates. This is especially important for dynamic tables, feeds, and filtered lists.
- ☐ **Defer non-critical elements**: Apply deferred loading to below-the-fold sections and rarely used elements so first render stays focused on route-critical content.
- ☐ **Optimize Zone.js for high-frequency events**: For scroll, mousemove, and animation loops, run handlers with NgZone.runOutsideAngular when updates do not need immediate change detection, then re-enter Angular only when state actually changes.
- ☐ **Prevent memory leaks**: Use teardown-safe patterns like `takeUntilDestroyed` and `DestroyRef` to clean subscriptions, timers, and observers that otherwise degrade runtime performance over time.
- ☐ **Enforce bundle budgets**: Configure Angular build budgets for initial bundles and lazy chunks so regressions fail early during pull requests instead of reaching production.
- ☐ **Use server-side rendering**: Traditional SPAs cannot render until required JavaScript is available. Server-side rendering pre-renders pages on the server and sends rendered markup during initial load. Combining server-side and client-side rendering is recommended and supported by Angular, React, and Vue.
- ☐ **Apply partial hydration**: Hydration makes pre-rendered content interactive. To reduce client JavaScript, send and hydrate only small interactive parts instead of shipping the entire app, especially on mostly static pages.
- ☐ **Prevent layout reflow**: Prevent content reflow to maintain a stable user experience. If content dimensions are unknown, containers can collapse and shift the layout. Reserve space by defining width and height for containers so browsers can paint predictable layouts.
- ☐ **Optimize media and fonts**: Use progressive and compressed images and evaluate modern formats like WebP and AVIF for significant size savings. For PNG and SVG, use optimization tools. Consider placeholder strategies like BlurHash. Prefer efficient video formats where appropriate and apply practical web font loading strategies with WOFF2 and suitable fallbacks.
- ☐ **Eliminate unused CSS and fonts**: CSS can accumulate redundant kilobytes over time and create dead weight that hurts performance and maintainability. Remove unused CSS manually or with tools like PurifyCSS, PurgeCSS and UnCSS. Also remove unused fonts to save additional network bytes.

- ☐ **Optimize CSS selectors**: Avoid overly complex descendant chains and expensive selector patterns on large DOM trees. Prefer simple, class-based selectors over IDs, `!important`, and long selector chains to reduce style recalculation cost as highlighted in CSS selector costs article.
- ☐ **Use system fonts**: Sometimes the best strategy is not loading custom fonts at all and relying on system font stacks. Using default operating-system fonts can boost performance because no extra font files are downloaded.
- ☐ **Minimize HTTP requests**: Each request adds connection, scheduling, and response overhead, although HTTP/2 and HTTP/3 reduce the cost of many parallel requests. More HTTP requests generally slow page load. Inspect the network dependency tree and remove avoidable serial dependencies on the initial path (for example CSS -> JS -> API waterfalls) so first render is not blocked by long chains.
- ☐ **Minimize request chains**: Inspect the network dependency tree and remove avoidable serial dependencies on the initial path (for example CSS -> JS -> API waterfalls) so first render is not blocked by long chains.
- ☐ **Use HTTP/2 or HTTP/3**: HTTP/2 introduces major performance improvements, including multiplexing so multiple requests and responses can be sent simultaneously. Moving to HTTPS and modern HTTP protocols is a valuable investment for overall speed.
- ☐ **Optimize CORS preflights**: Prefer same-origin API routing, use simple requests when appropriate, cache successful preflight responses with `Access-Control-Max-Age`, and maintain an explicit trusted-origin allowlist.
- ☐ **Use modern JavaScript**: Avoid serving unnecessary legacy syntax/polyfills to evergreen browsers to reduce parse and execution cost. A polyfill is an implementation of a web platform feature that websites can use in browsers that do not natively support the feature.
- ☐ **Configure Cache-Control headers**: Cache-Control directives define who may cache a response, under what conditions, and for how long, reducing network round trips for reusable resources. Content-hashed static assets are a common case for long-lived caching with `immutable` directive.
- ☐ **Differentiate asset caching**: Serve content-hashed assets with long-lived immutable caching, while keeping `index.html`, `ngsw.json`, and `ngsw-worker.js` short-lived or revalidated so deployments are discovered without stale-bundle failures.

- ☐ **Use a CDN for caching**: Cache public, versioned assets close to users, deliberately define cache keys and `Vary` behavior, and never cache personalized or authorized HTML/API responses without an explicit privacy model.
- ☐ **Configure mobile viewport**: Include a proper viewport declaration (for example `width=device-width, initial-scale=1`) so mobile input and layout behavior are optimized. See Optimize viewport for mobile article.
- ☐ **Apply async/defer attributes**: Both attributes can prevent classic external scripts from blocking HTML parsing. `async` executes as soon as the script is available and can run before parsing finishes, while `defer` executes after parsing and preserves the order of deferred scripts.
- ☐ **Optimize Event Loop**: Break up long synchronous tasks and microtask chains, avoid blocking the main thread during startup or interaction, and move computation that regularly exceeds the frame budget or creates long tasks to a Web Worker.
- ☐ **Minimize visual rendering cost**: Avoid applying costly effects such as large `box-shadow`, `filter`, `backdrop-filter`, and clipping to many elements. Profile paint and compositing work before using these effects in frequently updated or scroll-heavy views.
- ☐ **Use CSS for visual effects**: Use CSS transitions and animations for straightforward visual changes so the browser can optimize them. Reserve JavaScript-driven animation for interactions that require imperative control or physics.
- ☐ **Minimize DOM access**: DOM access is expensive and rendering can consume significant time. Minimize repeated DOM operations and batch updates to reduce rendering disturbance and avoid slowing the browser.
- ☐ **Reduce DOM size**: Large DOM trees increase style recalculation, layout, paint, and compositing costs. Deep trees or many siblings can make small updates expensive. Keep only necessary elements, remove unnecessary wrappers, paginate or virtualize long lists, and defer hidden sections.
- ☐ **Use virtual scrolling**: Virtual scrolling renders only visible content plus a small buffer, reducing DOM size and layout, paint, and update work. It works best for long lists with predictable item dimensions. Angular projects can use `CdkVirtualScrollViewport` from the Angular CDK.
- ☐ **Throttle and debounce events**: Limiting JavaScript execution frequency can fine-tune performance. Throttling caps call frequency over time, while debouncing delays

execution until activity settles. Use these patterns for clicks, API calls, mouse/touch movement, resize, scroll, and autosave events.

- ☐ **Optimize API interactions**: Remove request waterfalls, parallelize independent calls, share idempotent GET streams with `shareReplay`, define cache invalidation, paginate large payloads, and combine pagination with virtual scrolling where appropriate.
- ☐ **Apply optimistic updates**: Apply optimistic UI only to low-risk actions, provide visible failure handling, and restore the previous state when the server rejects the operation.
- ☐ **Select rendering strategy**: Use CSR for highly interactive authenticated areas, SSR or SSG for content and SEO-sensitive routes, and hydration or deferred activation to avoid downloading and executing non-critical client code twice.
- ☐ **Track relevant metrics**: Track metrics that matter most to the application, such as INP, LCP, CLS, TBT, Speed Index, and CPU time. Core Web Vitals are a practical subset that reflects user-centric, field-measurable experience outcomes.
- ☐ **Monitor production continuously**: Combine RUM, APM, logs, traces, and infrastructure metrics. Alert on regressions in p75 Core Web Vitals, p95/p99 API latency, error rate, throughput, resource saturation, and cache effectiveness. Use lab tools for regression detection rather than as a replacement for field data.

7.8 Final Remarks

Performance is a continuous engineering discipline, not a one-time task. Architectural choices such as rendering strategy, state management, and change detection set the limits of what an application can achieve. Implementation details such as bundle size, template work, caching, list rendering, and image delivery determine how efficiently it reaches those limits. Budgets, profiling, performance tests, and production monitoring then keep the application within those limits as it evolves.

Treat performance as a core product feature: define user-centered targets, measure them in the lab and in the field, and investigate regressions before they become normal. Use the performance checklist to guide that work, but let profiling and real-user evidence determine which changes matter most for the users.

Afterword

I wanted this book to cover the common topics that apply to most Angular projects. Some applications require deeper knowledge in specific areas than others. Some use solutions for state management, micro-frontends, or server-side rendering. Those topics are covered at a high level here to keep the book focused and manageable. Other areas, such as infrastructure and security, are explored in more depth when they are directly relevant to Angular applications and the web domain. I expected to highlight the most important aspects of achieving engineering excellence.

Index

A

Accessibility, 29, 92, 345
Accessibility Testing, 200
Acceptance Test-Driven Development, 167
Animations, 45, 152, 353
Angular, 19, 156, 355
Angular DevTools, 98, 315, 346
Angular Elements, 51
Angular Framework, 71, 91, 110
AOT Compilation, 304
API Latency, 109, 335, 354
Application State Management, 130
Architecture, 20, 144, 349
Architecture Decision Records, 159
Architectural Patterns, 30, 115, 117
Argo CD, 251
Artifact Repository, 238, 240, 247
Artificial Intelligence, 68
Asynchronous Tests, 178
Automation, 19, 220, 310

B

"Big Bang" Deployment, 248
Behavior-Driven Development, 164, 205
Behavioral Patterns, 30, 115
Blue-Green Deployment, 249
Bounded Contexts, 114, 160, 165
Browser APIs, 19, 69, 191
Browser Interaction, 189
Build Configuration, 97
Build Environments, 97
Bundle Optimization, 324
Bundle Splitting, 316, 324

C

C4 Model, 157, 158
Caching, 36, 280, 354
Caching Headers, 155, 327, 328
Caching Strategies, 73, 155, 350
Canary Deployment, 249
CDK, 65, 148, 353
Change Detection Strategies, 321, 322
Chrome DevTools, 67, 345, 349
CI/CD, 53, 250, 343
Clean Architecture, 119, 120, 133
Client Code Generation, 139
Client-Side Rendering, 93, 325, 351
Cloud, 216, 277, 341
Cloud Cost Estimates, 274
Cloud Hosting, 258, 328
Cloud Providers, 258, 274, 310
Code Coverage, 28, 179, 180
Code Quality, 19, 62, 247
Coding Conventions, 33, 181, 222
Component Lifecycle, 320
Component-Based Architecture, 72, 117, 127
Components, 19, 129, 347
Compressing JavaScript, 318
Content Security Policy, 297, 302, 303
Continuous Automation, 211, 249, 255
Continuous Delivery, 57, 237, 255
Continuous Integration, 57, 227, 255
Continuous Testing, 200, 220, 255
Continuous Workflow, 246
Conventional Commits, 59, 60, 96
Core Web Vitals, 64, 292, 354
CORS, 90, 307, 352
Creational Patterns, 30, 115
Cross Browser Testing, 201, 213
CSS, 19, 146, 353
CSS Architecture, 24, 144
CSS Methodologies, 40, 41, 144
Cumulative Layout Shift, 93, 312, 313

D

Data Flow Architecture, 126
Defensive CSS, 151
Deferrable Views, 108, 156, 157
Dependency injection, 29, 114, 321
Dependency Management, 52, 69, 225
Dependency Governance, 55, 244, 297
Dependency Inversion Principle, 113, 114
Dependency Security, 54, 309
Deployment Strategies, 161, 248
Delivery Artifacts, 239
Design Patterns, 30, 115, 182
Design Principles, 111, 161, 182
Design Systems, 45, 66, 241
Design Tools, 65, 66
Development Workspace, 95, 110
Directives, 73, 95, 352
Docker, 140, 250, 347
Docker Image, 140, 250, 341
Documentation as Code, 160
Domain Name, 262
Domain-Driven Design, 114, 129, 165
Don't Repeat Yourself, 112
Dynamic Import, 46, 316

E

Error handling, 65, 110, 344
Error Rate, 64, 293, 354
Event Loop, 318, 319, 353
Event-driven Architecture, 117, 128, 129

F

Fallback Routes, 259, 328
Feature-Based Architecture, 123, 124
Feature Flags, 57, 119, 272
First Contentful Paint, 313, 343
Flaky Tests, 171, 221, 236
Functional Programming, 33, 36, 83
Functional Tests, 28, 191, 218

G

Git, 23, 63, 253
Git Branching Strategies, 57
GitHub Packages, 238, 241, 255
GitHub Workflows, 224, 237, 250
GitOps, 240, 250, 251
GraphQL, 116, 141, 262
Guards & Interceptors, 80
Graceful Degradation, 151

H

Health Checks, 254, 281, 335
Helm, 240, 265, 271
Hexagonal Architecture, 120, 121
HTML, 19, 176, 353
HTTP, 28, 219, 352

I

Incremental Hydration, 157
Infrastructure as Code, 248, 266, 310
Infrastructure Components, 261, 288
Infrastructure Costs, 274
Infrastructure Provisioning, 257, 269, 273
Integration Tests, 28, 170, 234
Interface Segregation Principle, 113
Internationalization, 45, 91, 110
Isolated Tests, 172, 182

J

Jenkins as Code, 232
Jira, 203, 204
JIT Compilation, 98, 304

K

Kubernetes, 239, 267, 290
Kubernetes Cluster, 260, 267, 268

L

Largest Contentful Paint, 93, 312, 343
Layered Frontend Architecture, 122, 124
Lazy loading, 46, 149, 350
Lighthouse, 27, 338, 348
Lighthouse CI, 347, 348
Liskov Substitution Principle, 113
List Virtualization, 317
Load Balancer, 260, 309, 343

M

Marble Tests, 176, 177, 179
Metrics, 48, 291, 354
Micro-Frontends Architecture, 119, 125
Mobile First, 148
Mobile Testing, 215, 216
Mocking APIs, 191
Mocking Clock, 191
Modular Architecture, 124, 145
Modules, 22, 125, 350
Monorepo, 54, 153, 158
Monkey Testing, 201

N

Nginx, 219, 282, 333
NgRx, 26, 130, 188
NgRx SignalStore, 130, 131, 133
NPM Registry, 51, 239, 244

O

Object-Oriented Programming, 30, 32
Observability, 93, 273, 349
OnPush, 77, 322, 350
Open–Closed Principle, 113
OWASP, 22, 298, 301
OWASP Top 10, 22, 298, 301

P

Package Managers, 21, 54, 241
Page Objects, 192, 196
Page Weight, 314
Performance, 19, 281, 354
Performance Insights, 315
Performance Metrics, 312, 343
Performance Monitoring, 294, 344
Performance Patterns, 30, 315
Performance Testing, 149, 337, 344
Perceived Responsiveness, 89, 315, 337
Pipes, 73, 107, 325
Playwright, 28, 196, 219
PRPL Pattern, 316, 349
Promises, 38, 106, 178
Progressive Enhancement, 150, 151, 152
Progressive Web Apps, 94
Project Documentation, 96, 227, 255
Project Structure, 95, 146, 195

R

Reactive Architecture, 126, 127
Rendering, 22, 262, 355
Request Count, 314
Resilient CSS, 144, 150
Responsive Design, 46, 147, 148
REST, 37, 142, 299
Reverse Proxy, 262, 308, 347
Rolling Deployment, 248
Routing, 26, 156, 352
Runtime Performance, 324, 349, 351
RxJS, 19, 126, 336
RxJS Observables, 129, 141, 142
RxJS Subjects, 131, 132

S

SCSS, 41, 146, 147
Sass, 19, 41, 147
Screenshot Tests, 198

Security, 22, 269, 355
Security Testing, 201, 213, 310
SEO, 28, 155, 354
Selenium Grid, 196, 217, 219
Separation of Concerns, 36, 112, 118
Server Compression, 331
Server Scaling, 335
Server-Sent Events, 48, 143
Server-Side Rendering, 22, 325, 355
Serverless Containers, 259
Service workers, 27, 239, 350
Services, 68, 160, 344
Shallow Tests, 172, 173, 188
Signal-Based Architecture, 129, 130
Signals, 68, 137, 350
Single Responsibility Principle, 106, 112
Smoke Testing, 201, 213
Snapshot Tests, 175, 176
Software Architecture Documentation, 157
Software Development Life Cycle, 164, 198
Source Maps, 91, 99, 181
Spec-Driven Development, 166
Speed Index, 314, 341, 354
State Management, 26, 131, 355
State Machines, 135, 136, 137
State Management Architecture, 130
Static Files Hosting, 258
Static Site Generation, 93, 156, 325
Structural Patterns, 30, 115
Style Guide, 33, 90, 268

T

Terraform, 248, 266, 274
Test Automation, 168, 212, 213
Test Reporters, 180, 194
Test-Driven Development, 166, 172, 182
Testing, 19, 193, 344
Testing Pyramid, 168, 171, 188
Testing Suites, 208
Testing Tools, 169, 170
Testing Types, 167
Time to Interactive, 313, 314, 343
Total Blocking Time, 313
Tracing, 43, 286, 344
Tree Shaking, 317, 318, 324
Trusting Safe Values, 303
Twelve-Factor App, 272, 273
TypeScript, 19, 73, 226

U

Unit Tests, 63, 183, 271
User Access Control, 90
User Experience, 19, 262, 351

V

Version control, 19, 223, 272
Version Control Systems, 23, 56, 238
Visual Tests, 163, 170, 199

W

Web Application Performance, 311, 333, 354
Web Components, 19, 52, 239
Web Development, 19, 45, 70
Web Security, 296, 298
Web Technologies, 19, 22
Web Workers, 27, 302, 350
WebSockets, 84, 116, 143
Well-Architected Frameworks, 273
Workspace Architecture, 152

Z

Zone.js, 130, 319, 351
Zoneless, 130, 319, 320

Advanced Angular Applications is a guide to work effectively in enterprise contexts with Angular. In this book, you will find guidance on architecture and coding standards, quality assurance and testing strategies, project automation, infrastructure and operational concerns, security, and performance optimization throughout the entire application lifecycle.

André de Sousa, 2026